

React + Snooker Fantasy League Course

Contents

Chapter 1 · Lesson 1 — The Problem (no code yet)	16
Why this lesson exists before any code	17
The pub conversation	17
The 8 fantasy teams	18
The 5 rounds and the 31 matches	18
The scoring rules — this is the heart	19
What the app actually has to do (the user stories)	20
What the app explicitly does NOT do	20
The architecture this implies	21
Two-sentence summary	21
Vibe prompt you would have used (to verify understanding before coding)	21
CHECK YOURSELF	22
Chapter 1 · Lesson 2 — Data Modeling (TypeScript interfaces)	22
Why we model the data before we render it	22
A 60-second TypeScript primer (for total beginners)	23
Modeling Match	23
Why winner? is optional	24
Why score? and seed1? are optional	25
Modeling Team	25
1. r1, r2, qf, sf are arrays of strings — but final is just a string.	26
2. The cosmetics live next to the picks.	26
3. r1Players? and r2Players? exist for a subtle scoring case.	26
The most important rule in the codebase (the invariant)	26
Modeling PlayerInfo (the dictionary, not an array)	27
Modeling the score detail (output of scoring, used by every UI)	28
Putting it all together: lib/types.ts	29
Why the TypeScript compiler is your friend	31
The quick exercise	31
Vibe prompt you would have used	32
CHECK YOURSELF	32
Answer to #1 (because it's a real gotcha)	32

Chapter 1 · Lesson 3 — Pure Functions & Scoring	33
What is a “pure function” and why do we care?	33
Walking through scorePick	34
Branch 1: if (!match !match.winner) return null;	34
Branch 2: if (pick === match.winner) return 3;	35
Branch 3: return 1;	35
The whole function in one breath	35
Walking through calculateTeamScores	36
Round 1 walkthrough	36
Rounds 2 through Final	38
The Final is special	39
The total and the return	40
Why is the return shape so wide?	40
The whole lib/scoring.ts	40
Trying it out (optional)	42
Vibe prompt you would have used	42
CHECK YOURSELF	43
 Chapter 1 — Foundations	 43
What you’ll learn	44
Lessons in this chapter	44
Files you’ll create in this chapter	44
New React/Next.js concepts introduced	44
How long it should take	45
Before you start	45
 Chapter 2 · Lesson 1 — Why Next.js + TypeScript	 45
The interview question	45
The React framework landscape (as of 2026)	46
The Vite vs Next.js decision (the honest one)	47
Vite wins on:	47
Next.js wins on:	47
Why TypeScript (not plain JavaScript)?	48
1. Catches errors at compile time, not at runtime.	48
2. Self-documenting code.	48
3. Editor superpowers.	48
When NOT to use TypeScript	49
Why Tailwind CSS (and why we barely use it)	49
1. To set up globals.css properly.	49
2. To leave the door open for future Tailwind use.	49
3. So next/font and other Next.js features that integrate with Tailwind work cleanly.	50
Why inline styles instead of Tailwind classes?	50
What about CSS Modules, styled-components, emotion, vanilla-extract?	50
Why we don’t need a state management library (Redux, Zustand, Jotai)	51

Why Recharts (when it's time for charts)	51
The decision tree, in one sketch	52
What this lesson did NOT cover	53
Vibe prompt you would have used (to make the framework decision)	53
CHECK YOURSELF	53
Chapter 2 · Lesson 2 — Scaffolding the Project	54
The two ways to start a Next.js project	54
Step 0: Create the folder	54
Step 1: package.json	54
"name", "version", "private"	55
"scripts" — the npm command surface	56
"dependencies" — what ships in production	56
"devDependencies" — what's needed only at dev/build time	57
Step 2: tsconfig.json (TypeScript compiler config)	57
Step 3: next.config.mjs	58
Step 4: tailwind.config.ts	59
Step 5: postcss.config.mjs	59
Step 6: .eslintrc.json	60
Step 7: .prettierrc	60
Step 8: .gitignore	61
Step 9: The app/ directory (Next.js App Router)	61
app/layout.tsx	62
app/page.tsx	63
app/globals.css	64
Step 10: Install dependencies	65
Step 11: Run the dev server	65
What create-next-app would have done for you	66
Vibe prompt you would have used	66
CHECK YOURSELF	67
Chapter 2 · Lesson 3 — Creating the Data Files	67
Why this lesson is "boring" and why that's a feature	67
1. Real apps spend a lot of time on data plumbing.	67
2. Typing the data forces you to feel the invariants.	68
The four files we'll build	68
File 1: lib/matches.ts	68
Why one type import at the top	68
export const ROUND1_MATCHES: Match[]	69
The objects themselves	69
Pending matches look different	69
The four other rounds	70
How the file ends	70
File 2: lib/teams.ts	70
What's in each team	71

How to type 16 picks without going insane	71
Color choices	72
Two optional fields you might see	72
File 3: lib/players.ts	72
Record<string, PlayerInfo> explained	73
Country flag emojis	73
Seeds: top-16 only	73
Why a dictionary, not an array	74
File 4: lib/constants.ts	74
BALL_COLORS – the seven snooker ball colors	75
th and td – shared table cell styles	75
tabStyle – a function that returns a style object	76
Sanity-checking with a quick console smoke test	76
What you’ve built so far	77
Vibe prompt you would have used	77
CHECK YOURSELF	78
Chapter 2 – Project Setup	78
What you’ll learn	78
Lessons in this chapter	79
Files you’ll create in this chapter	79
New React/Next.js concepts introduced	80
How long it should take	80
Before you start	80
Chapter 3 · Lesson 1 – Anatomy of a Component	80
What is a “component”, really?	81
Functional vs class components – the 2026 answer	81
What is JSX?	81
Why does this matter?	82
The full anatomy: a real component, line by line	83
Line 1: imports	84
Lines 3–9: the props interface	84
Line 11: the function declaration	85
Lines 12–32: the return – JSX	86
The inline style attribute	87
Template literals for dynamic colors	88
Self-closing tags	88
Recap: the four parts of every component file	88
Server vs client components – a teaser	89
Vibe prompt you would have used	89
CHECK YOURSELF	90
Chapter 3 · Lesson 2 – Rendering the Standings	90
The smallest version that proves data flows	90

Step 1: stub the orchestrator	91
'use client' at the top — finally	91
Importing from a sub-path	92
Stub style — minimal so the data is the focus	92
Step 2: the v1 StandingsTab	92
What's happening in this file	93
Line 1–2: imports	93
Line 4: the function	94
Line 5–8: derive the data	94
Lines 11–22: the JSX	95
The key prop and why it matters	96
What is React doing?	96
What makes a good key?	97
The classic key bug	97
Embedding expressions inside JSX	97
Returning JSX vs returning null	98
Step 3: visit your browser	98
Why this is “the right pace” for a beginner	99
Vibe prompt you would have used	99
CHECK YOURSELF	100
Chapter 3 · Lesson 3 — Progressive Enhancement	100
What “progressive enhancement” means here	101
v2: extract the table into its own component	101
1. Props for data — teams is now passed in	102
2. An optional callback prop — onTeamClick?	103
3. Style spreading	103
v3: update StandingsTab to use the new LeagueTable	103
v4: medal-color the rank circle	104
IIFE inside JSX (immediately-invoked function expression)	105
A nested ternary for the colors	105
display: 'inline-flex' for centering	106
v5: per-round columns	106
v6: the Final Pick chip	107
The conditional coloring	108
v7: the FormBadge	108
Lucide icons as components	110
Picking a component dynamically	110
Calculating percentages inline	110
v8: stat cards above the table	111
CSS Grid with auto-fit and minmax	113
Computed values inside the component	113
Importing icons by name	113
What v8 looks like	114
What we did NOT do (yet)	114

The discipline you just practiced	114
Vibe prompt you would have used (per feature)	115
CHECK YOURSELF	116
Chapter 3 — Your First React Component	116
What you'll learn	116
Lessons in this chapter	117
Files you'll create in this chapter	117
New React/Next.js concepts introduced	117
How long it should take	117
What you'll see in your browser at the end	118
Before you start	118
Chapter 4 • Lesson 1 — The useState Mental Model	118
What is “state”?	118
What is useState?	119
Anatomy of useState	119
What does the setter do?	120
The re-render cycle	121
The mental loop	121
Hooks: the rules	122
Rule 1: Call hooks at the top level. Never inside loops, conditions, or nested functions.	122
Rule 2: Call hooks only from React function components or other hooks.	122
Hooks are detectable by name	123
Initial value can be expensive — use a function	123
Multiple state values	123
State updates are batched	124
State updates can be functional	124
What state is in <SnookerFantasyLeague>?	125
A small example: the round filter	125
When state belongs in a parent (lifting state up)	126
The hook isn't magic — it's a closure trick	127
Vibe prompt you would have used	127
CHECK YOURSELF	128
Chapter 4 • Lesson 2 — The Orchestrator Pattern	128
What is an “orchestrator” component?	128
Step 1: rewrite SnookerFantasyLeague.tsx as an orchestrator	129
Walking the orchestrator, top to bottom	131
Line 1: 'use client'	131
Lines 3–13: imports	132
Lines 15: a string-union type for tab IDs	132
Lines 18–19: the two state cells	132
Lines 21–24: the memoized derived data	133

Lines 26–32: the tabs configuration	133
Lines 35–80: the JSX	134
The cross-tab interaction	135
The && short-circuit	135
Updating the tab components to receive props	136
Stub the other four tabs	137
Why does state live where it lives?	137
Why this is interview gold	138
Why we DON'T use Context yet	138
Vibe prompt you would have used	139
CHECK YOURSELF	140
Chapter 4 · Lesson 3 — useMemo and Derived Data	140
What is useMemo?	140
What does “across renders” mean?	141
Reference equality vs value equality	141
Why this matters: the cascade	142
When NOT to memoize	142
1. The computation is cheap.	143
2. The result isn't being passed to a child or used in another hook's deps.	143
3. The dependencies change every render anyway.	143
The senior rule	144
What about React.memo?	144
What about useCallback?	145
When the dependency array is wrong	145
Missing a dependency	145
Including a dependency that breaks memoization	146
The right way	146
Why we use useMemo for teamsWithScores	146
Why we DON'T use useMemo for the tabs array	147
A common pitfall: memoizing JSX	147
What about state machines and useReducer?	148
The whole orchestrator's hook list	148
Vibe prompt you would have used	148
The big picture	149
CHECK YOURSELF	149
Chapter 4 — State & Hooks	150
What you'll learn	150
Lessons in this chapter	150
Files you'll create or update	151
New React/Next.js concepts introduced	151
How long it should take	151
Why this chapter is the most important in the course	151
Before you start	152

Chapter 5 · Lesson 1 — When to Split a Component	152
The two failure modes	152
The three real reasons to split	153
Test 1: The copy-paste test	153
Test 2: The “what vs how” test	153
Test 3: The reusability test	154
The case for tiny components	154
When NOT to split	155
Signal 1: The component takes too many props.	155
Signal 2: The component has logic that depends on its parent’s context.	156
Signal 3: Extraction makes the parent harder to read.	156
The composition pattern: parent says what, child knows how	157
Walkthrough: extracting <TeamChip>	157
Walkthrough: when NOT to extract <TwoColumnLayout>	159
The children prop — the most underused React feature	159
A note on file structure	160
Vibe prompt you would have used	161
CHECK YOURSELF	161
 Chapter 5 · Lesson 2 — Prop Drilling vs Context	 162
What is “prop drilling”?	162
The internet’s bad advice	162
When prop drilling is fine (most of the time)	163
In our codebase	163
Why we DON’T use Context for teams	163
When Context IS the right answer	164
1. Theme	164
2. Authenticated user	165
3. Locale / i18n	165
What these have in common	165
Context’s performance footgun	165
What about Zustand, Jotai, and friends?	166
Zustand	166
Jotai	167
Redux	167
The senior decision tree	167
A real-world Context: the dark mode example	168
The “create + Provider + custom hook” trio	169
Default value null + assertion in the hook	169
The Provider pattern	170
What we DO use that’s “globalish”	170
Module-level constants	170
Style helper functions	171
The seven-level drilling test	171
Vibe prompt you would have used	171

CHECK YOURSELF	172
Chapter 5 · Lesson 3 — Building PlayerDetail (Case Study)	172
The problem	172
The component tree	172
Step 1: PlayersTab and the conditional render	173
Local state for the detail view	174
Selected by name, not by object	175
The conditional return	175
Step 2: PlayerCard — the leaf	175
A button, not a div	176
Optional finalPicksCount prop	176
Hover effects via inline onMouseOver / onMouseOut	177
Step 3: PlayerDetail — the big one	177
Why doesn't this need useMemo?	178
The pickAnalyses.map is the most important transform	179
Step 4: MatchPickAnalysis — the per-match section	179
The clickable opponent name	180
The callback chain	180
Step 5: TeamChip and StatTile — the leaves	181
Step 6: PathStep and PathArrow — the readability components	182
What we just learned about composition	182
Vibe prompts for the Players tab	183
CHECK YOURSELF	184
Chapter 5 — Composition	184
What you'll learn	184
Lessons in this chapter	185
Files you'll create or update	185
New React/Next.js concepts introduced	186
How long it should take	186
What you'll see at the end	186
Before you start	186
Chapter 6 · Lesson 1 — Recharts Fundamentals	187
The single insight that makes Recharts click	187
The shape of a Recharts chart	187
The data prop is a flat array of objects	188
The <ResponsiveContainer> is non-negotiable	189
The four "decoration" components	189
<CartesianGrid> — the dotted/dashed grid	189
<XAxis> — the X-axis	190
<YAxis> — the Y-axis	190
<Tooltip> — the hover tooltip	190
<Legend> — the color key below	190

Tree-shakable imports	190
The chart card pattern	191
Custom tooltip styling	192
Bar chart vs line chart vs area chart	193
When NOT to use Recharts	193
A worked example: simplest possible chart	194
A note on <code><RechartsCommonProps></code>	194
Vibe prompt you would have used (for the simplest chart)	194
CHECK YOURSELF	195
Chapter 6 · Lesson 2 — Reshaping Data for Charts	195
The transform pattern	195
Reshape #1 — Stacked bar of points by round	196
Abbreviating long team names	196
The numeric fields are the bar heights	197
The chart that consumes it	197
Reshape #2 — Pick accuracy %	197
<code>.filter()</code> to count hits	198
<code>Math.round((hits / max) * 100)</code>	199
The horizontal-bar chart that consumes it	199
Reshape #3 — The Final pick distribution	199
Counting with <code>.forEach()</code> and a dictionary	200
<code>Object.entries()</code> to convert dict → array	200
<code>.sort()</code> for descending count	201
Reshape #4 — Per-bar colors with <code><Cell></code>	201
What a <code><Cell></code> does	201
Why this exists	202
The <code>label</code> prop on <code><Bar></code>	202
The InsightCard — separating data from presentation	202
The grid layout	203
Recap: the transforms you've learned	204
Vibe prompt you would have used	204
CHECK YOURSELF	204
Lesson 6.3 — Pivots and Internal State	205
1. What is a pivot?	205
Why pivot?	206
2. The question this chart answers	206
3. Plan the data shape first	206
4. Define the type contract	207
Why these types?	208
Why are <code>correct</code> , <code>wrong</code> , <code>p1Name</code> , <code>p1Backers</code> , <code>p2Name</code> , <code>p2Backers</code> all optional?	209
Why export <code>interface</code> ?	209
5. The component shell	209

Read this carefully — there are three big ideas here.	210
6. Build the tab buttons	211
Why a simple button row instead of a <code><select></code> or radio?	211
Why one <code>onClick={() => setActive(r.id)}</code> per button?	212
7. Build the chart	212
Three details worth circling.	213
8. Build the pivot helper in <code>AnalyticsTab.tsx</code>	214
8.1. Why is this a function inside the component, not a <code>lib/</code> helper?	215
8.2. What is <code>pickKey</code> doing?	215
8.3. Why the special case for <code>'final'</code> ?	215
8.4. The label trick.	216
8.5. The vote count.	216
8.6. The implicit invariant.	217
9. Wire it up	217
10. The big question: lift state, or keep it local?	218
When to keep state local	218
When to lift state up	218
The diff is roughly:	218
11. Common mistakes I want you to avoid	219
11.1. Building the data inside the chart	219
11.2. Storing the pivoted data in <code>useState</code>	219
11.3. Lifting <code>active</code> into the orchestrator “just in case”	219
12. Vibe prompt you would have used	220
13. CHECK YOURSELF	220
14. Where you are now	221
What’s next	221
Chapter 6 — Data Visualization	222
What you’ll learn	222
Lessons in this chapter	222
Files you’ll create or update	222
New React/Next.js concepts introduced	223
How long it should take	223
Why charting deserves its own chapter	223
Before you start	223
Lesson 7.1 — Production Polish	224
1. The mindset shift	224
2. Accessibility (<code>a11y</code>) — the part that gets you hired	225
2.1. Use semantic HTML, not <code><div></code> soup.	225
2.2. Every image must have alt text.	225
2.3. Tab interfaces need <code>role="tablist", role="tab", role="tabpanel"</code>	225
2.4. Every interactive element must be keyboard-reachable.	226
2.5. Color contrast: text must be readable.	227
2.6. The five-rule checklist	227

3. Error boundaries — when one component crashes	227
3.1. The class-component reality	228
3.2. Wrap things that might break	229
3.3. The interview question this answers	230
4. Loading states with <code>loading.tsx</code>	230
4.1. When does this fire?	232
4.2. Why <code>aria-live="polite"</code> ?	232
5. Bundle audit with <code>next build</code>	232
5.1. What to look at	232
5.2. The numbers to worry about	233
5.3. Dynamic import in 30 seconds	233
6. Lighthouse — the report card	233
6.1. The three highest-leverage fixes	234
7. Performance: the React-specific wins	234
7.1. The <code>useMemo</code> you already wrote	234
7.2. <code>React.memo</code> for expensive children	234
7.3. The React DevTools Profiler	235
8. Defensive rendering	235
8.1. Guard against missing data	235
8.2. Empty states	235
9. Vibe prompt you would have used	236
10. CHECK YOURSELF	236
11. Where you are now	237
Lesson 7.2 — Deployment and Testing	237
Part A — Deployment	238
1. The deployment story for Next.js	238
2. Connect the repo to Vercel	238
3. Preview deployments	239
4. Environment variables	239
4.1. Local <code>.env.local</code>	240
4.2. Vercel environment variables	240
5. Build-time vs. run-time	240
Part B — Testing	241
6. The testing pyramid (Trophy)	241
7. Install Vitest + React Testing Library	242
8. Unit tests for <code>scorePick</code> and <code>calculateTeamScores</code>	243
What's worth noting	244
9. Integration test for the standings tab	245
10. End-to-end test with Playwright	246
What this test catches	247
11. CI with GitHub Actions	247
Why include <code>npm run build</code> ?	248
Why not run E2E in CI yet?	248
12. The five-test rule	248

13. Vibe prompt you would have used	249
14. CHECK YOURSELF	249
15. Where you are now	250
Lesson 7.3 – The Interview Narrative	250
1. The 90-second walkthrough	250
1.1. The script (memorize this shape, swap your own words)	251
1.2. Why this script works	251
1.3. The variant for non-technical interviewers	252
2. The ten questions you will be asked	252
2.1. <i>“Walk me through your folder structure.”</i>	252
2.2. <i>“How does state management work in your app?”</i>	252
2.3. <i>“How do you decide when to extract a component?”</i>	253
2.4. <i>“How do you handle data fetching?”</i>	253
2.5. <i>“What’s the trade-off between server and client components?”</i>	253
2.6. <i>“How do you optimize performance in React?”</i>	253
2.7. <i>“How do you handle errors?”</i>	254
2.8. <i>“What’s your testing strategy?”</i>	254
2.9. <i>“How would you scale this to 1,000 teams?”</i>	254
2.10. <i>“What would you do differently?”</i>	254
3. The vibe-coding prompt library	255
3.1. The <i>project-shaping</i> prompt	255
3.2. The <i>data-modeling</i> prompt	255
3.3. The <i>pure-logic</i> prompt	255
3.4. The <i>first-component</i> prompt	256
3.5. The <i>progressive-enhancement</i> prompt	256
3.6. The <i>state-and-tabs</i> prompt	256
3.7. The <i>composition</i> prompt	256
3.8. The <i>charting</i> prompt	257
3.9. The <i>production-polish</i> prompt	257
3.10. The <i>testing</i> prompt	257
4. The pattern behind every prompt	257
5. The hardest interview questions, rehearsed	258
5.1. <i>“Show me the worst code in this project. Why is it the worst?”</i>	258
5.2. <i>“Why didn’t you use Redux / Zustand / TanStack Query?”</i>	258
5.3. <i>“How would I know if a feature you added broke something?”</i>	258
5.4. <i>“What’s the most React-specific bug you’ve fixed?”</i>	259
6. Closing the interview strong	259
6.1. About the codebase	259
6.2. About the team	259
6.3. About the role	260
7. The portfolio link	260
8. The final vibe prompt	260
9. CHECK YOURSELF	261
10. Where you are now	261

11. Final words	261
Chapter 7 — Mastery + Interview Prep	262
What you'll learn	262
Lessons in this chapter	263
New concepts introduced	263
How long it should take	263
Why this chapter exists separately	263
Before you start	264
What this chapter is <i>not</i>	264
Learn React + Next.js by Building a Snooker Fantasy League	264
Who this course is for	264
What you'll build	265
What you'll learn (the React inventory)	265
How the course is structured	266
How to use this course	267
The recommended workflow	267
Pace yourself	268
Don't skip Chapter 1	268
Reading vs building	268
What's in the rest of the repo	269
Prerequisites	270
Conventions used in this course	270
A note on AI / vibe coding	270
Ready?	271
Snooker Fantasy League: A Reverse-Engineering Course	271
Confirming the source	271
Lesson 0 — The Problem (no code)	272
Imagine the pub conversation	272
The 8 teams	272
The scoring rules (3 bullets)	273
What the app needs to do	273
Vibe prompt you would have used	273
CHECK YOURSELF	273
Lesson 1 — The Shape of the Data (data modeling, no UI yet)	274
What's a Match?	274
What's a Team?	275
What's PLAYER_INFO and why is it an object keyed by name?	275
The most important rule in the codebase	276
Vibe prompt you would have used	276
CHECK YOURSELF	277
Lesson 2 — The Pure Logic (scoring, before any React)	277
Why scoring lives outside React	277

Walking through scorePick line by line	278
Why correct is null, not false	278
r1 += pts 0 – the running total trick	279
What calculateTeamScores returns	279
Vibe prompt you would have used	280
CHECK YOURSELF	280
Lesson 3 – The First Render (a single component, no tabs yet)	281
v1: just team names and totals	281
v2: per-round columns	282
v3: progressive enhancement of the same column	282
v4: the special “Final Pick” cell	283
v5: the leader highlight	283
v6: the form badge	284
Vibe prompt you would have used (at each step)	284
CHECK YOURSELF	285
Lesson 4 – State & Tabs (introducing useState)	285
Two pieces of state, one home	285
useMemo for the decorated team list	286
Wiring the tabs	287
Why not put selectedTeam inside <PredictionsTab>?	288
Vibe prompt you would have used	289
CHECK YOURSELF	289
Lesson 5 – Composition & Prop Drilling (when to split a component)	289
The chain, explained	289
When to extract: the copy-paste rule	291
Tiny components that exist for readability, not reuse	291
Prop drilling: when it’s fine	292
Vibe prompt you would have used	292
CHECK YOURSELF	293
Lesson 6 – Charts & Data Transforms (Recharts)	293
Reshape #1: stacked-bar totals	293
Reshape #2: per-bar colors with <Cell>	294
Reshape #3: pivot from “team picks” to “match agreement”	295
Internal tab state inside a chart	296
Vibe prompt you would have used	296
CHECK YOURSELF	296
Lesson 7 – The Evolution (what the git log would have shown)	297
Milestone 1 – “Just give me the standings”	297
Milestone 2 – “I want to see the matches themselves”	297
Milestone 3 – “I want to compare predictions side by side”	298
Milestone 4 – “Round 2 happened, the bracket shrunk”	298
Milestone 5 – “Show me a player’s tournament journey”	299
Milestone 6 – “I want pretty charts”	299
Milestone 7 – “Wu Yize won. Crown him.”	299
Putting the timeline together	300

Vibe prompt you would have used (the meta-pattern)	300
CHECK YOURSELF	301
Lesson 8 – The Shape of Your Prompts (meta-lesson on vibe coding)	301
The seven shapes of a good vibe prompt	301
Three bad prompts and why they're bad	302
The prompt-tree of this app	303
Final word	303
CHECK YOURSELF	304
21-Day Study Plan – React + Snooker Fantasy League	304
Week 1 – Foundations (Days 1-7)	304
Day 1 (60 min) – Orientation	304
Day 2 (60 min) – Chapter 1.1	304
Day 3 (60 min) – Chapter 1.2	305
Day 4 (75 min) – Chapter 1.3	305
Day 5 (60 min) – Chapter 2	305
Day 6 (60 min) – Chapter 3.1-3.2	305
Day 7 (60 min) – Chapter 3.3 + review	305
Week 2 – Composition & Hooks (Days 8-14)	306
Day 8 (60 min) – Chapter 4.1	306
Day 9 (60 min) – Chapter 4.2	306
Day 10 (60 min) – Chapter 4.3	306
Day 11 (60 min) – Chapter 5.1	306
Day 12 (60 min) – Chapter 5.2	306
Day 13 (60 min) – Chapter 5.3	307
Day 14 (60 min) – Week 2 review	307
Week 3 – Visualization, Polish, Deployment (Days 15-21)	307
Day 15 (60 min) – Chapter 6.1	307
Day 16 (75 min) – Chapter 6.2	307
Day 17 (75 min) – Chapter 6.3	307
Day 18 (60 min) – Chapter 7.1	308
Day 19 (75 min) – Chapter 7.2	308
Day 20 (60 min) – Chapter 7.3	308
Day 21 (90 min) – Capstone	308
Daily habits (every day, ~10 min on top)	308
What to skip if you're short on time	309
What to add if you have more time	309
When you finish	309

Chapter 1 • Lesson 1 – The Problem (no code yet)

Goal: read this lesson without writing a single line of code, and end with a clear, written-down understanding of what we are about to build and why.

Why this lesson exists before any code

Junior developers open their editor first. Senior developers open a **scratch pad** first. The difference is that seniors know they're going to spend two hours coding the wrong thing if they don't spend ten minutes understanding the *right* thing.

Open a notes file. Or grab a piece of paper. We're going to talk about the app for fifteen minutes before we touch a keyboard, and at the end you should be able to answer a friend asking *"what does the app do?"* in two sentences without hand-waving.

This is the same skill that gets you through interview whiteboarding. The candidate who writes code immediately fails. The candidate who says *"hold on, let me make sure I understand the problem"* and asks two clarifying questions passes.

The pub conversation

Imagine you and seven friends watch the World Snooker Championship every May. Sheffield, the Crucible, two weeks of tense green-baize matches. Someone — let's say it's you — says *"we should make a fantasy league out of this."* Everyone agrees. Pints are raised.

Now what?

Snooker has a feature that makes fantasy especially cheap to build: **the bracket is published before the tournament starts**. All 32 players are seeded, paired up, and the entire knockout structure is fixed. Every match in every round is **known in advance** — only the winners are unknown.

That single fact does an enormous amount of architectural work for us. Read it again: *every match in every round is known in advance, only the winners are unknown*.

What follows from it?

- We can ask each of our 8 friends to **fill out the entire bracket up front** — 16 picks for the round of 32, 8 picks for the round of 16, 4 for the quarter-finals, 2 for the semis, 1 for the final. **31 picks per friend, 248 total**, all locked in before the first ball is potted.
- We can list every match by **bracket position**, not by `matchId`. The matches don't get added or removed during the tournament; only their winner field gets filled in.
- The tournament is **finite**. There's a defined end state — somebody wins the final — at which point every team's score is final too.

- The data is **small**. We're talking ~250 picks, 31 matches, 32 players, 8 teams. That fits in a single JavaScript file. **No database needed.**

That last point is the one that quietly determines the whole stack. If we needed a database, we'd need a backend, which means an API, which means hosting. We don't, so we get to build a static page that anyone can host for free.

Decision dropped out of the problem statement, no opinions required.

The 8 fantasy teams

Real teams from the real app:

Team name	Vibe
Invincibles	"Unstoppable & Unbeatable" ☒
Uncredibles	"Bold predictions, bolder mistakes" ☒
Hopeless	"We tried, we failed, we're charming" ☒
Clueless	"What's a maximum break?" ☒
Break Builders United (BBU)	"Old school grinders" ☒
The Untouchables	"Smooth, calculated, lethal" ☒
Selbies	"Mark Selby fan club, basically" ☒
One Four Sevens	"Maximum vibes only" ☒

Each team is one or two friends sitting at home with a printout of the bracket, picking who they think wins each match.

Notice that the team names are **personality-driven**, not strategy-driven. *Invincibles* isn't a description; it's a brag. *Hopeless* isn't a strategy; it's a shrug. **The data carries the vibe.** That's a pattern: when an app has personality, that personality lives in the data, not the code. The code reads `team.motto`; the data is what makes the motto charming.

The 5 rounds and the 31 matches

The tournament is structured as a single-elimination knockout:

Round 1	(Last 32)	16 matches	Best of 19 frames
Round 2	(Last 16)	8 matches	Best of 25 frames
Quarter-Finals (QF)		4 matches	Best of 25 frames

Semi-Finals (SF)	2 matches	Best of 33 frames
Final	1 match	Best of 35 frames

Add them up: **16 + 8 + 4 + 2 + 1 = 31 matches**. The math works because each match eliminates exactly one player, so going from 32 players to 1 champion takes exactly 31 eliminations.

For our app, that means **5 separate “rounds” of data**, each one half the size of the previous. The shape of the data tells you the shape of the UI: probably a tab or a section per round, probably similar-but-not-identical layouts in each.

The scoring rules — this is the heart

There are exactly three rules. Memorize them, because everything else in the codebase is derived from them.

1. **3 points** if your pick wins their match.
2. **1 point** (a “consolation”) if your pick loses their match.
3. **null** — *not zero, not false* — if the match has not yet been played.

Rule 3 is the one beginners get wrong. Let’s spell it out.

Imagine it’s Day 1 of the tournament. Round 1 is just starting. Out of 16 R1 matches, **zero** are finished. What’s *Invincibles’* Round 1 score?

A naive answer says “zero.” But zero is wrong. Zero implies “we computed the score and it came out to zero — like getting all 16 picks wrong.” Zero would put a team that *will* score 30 points indistinguishable from a team that *did* score 0. That’s a bug.

The correct answer is **the score is undefined yet**. It’s not yet a number. It’s `null` — the absence of a number. As matches finish, picks get scored; the score creeps up from null toward its eventual value. **Pending and zero are different states.**

This three-state thinking — *correct, wrong, pending* — runs through every UI screen in the app. The standings table greys out pending columns. The predictions matrix uses three colors (green, red, gray). The pick-list cards have three border colors. They all come back to: **the underlying scoring function returns 3, 1, or null** — never 0 for a pending pick.

If you only take one thing away from this lesson, take this. The most senior thing you can do in a data model is **respect the difference between absent and zero**.

What the app actually has to do (the user stories)

Once the picks are in and the tournament starts, the friends want to be able to:

1. **See who's leading.** *Standings tab.* A leaderboard with totals and per-round breakdowns.
2. **See match results.** *Matches tab.* Cards per match, marked finished/not-finished, scores visible.
3. **See whose picks were right or wrong.** *Predictions tab.* A matrix comparing all 8 teams' picks side-by-side, color-coded.
4. **Drill into a player.** *Players tab.* Click any of 32 players, see who they were, who picked them, who picked against them, what they earned for their backers.
5. **See trends.** *Analytics tab.* Bar charts of points per round, accuracy by team, who agreed on what.

Five user stories, five tabs. **The UI is determined by the user stories**, not by what looks cool. If a tab can't be traced back to a thing one of the eight friends would actually click for, it doesn't exist.

When you're prepping for an interview, this is the answer to *"how would you decide what to build first?"* — you start with the user stories, count them, name them, and the architecture falls out.

What the app explicitly does NOT do

Equally important. By saying these out loud you save yourself from over-engineering:

- **No login.** Eight friends, one shared web page. No accounts.
- **No database.** Picks are typed into a JavaScript file by hand. The "admin" updating winners is whoever owns the repo (you).
- **No real-time updates.** When a match ends, the file owner edits winner: 'Wu Yize' and pushes. Friends refresh. That's the entire ops story.
- **No mobile app.** Just a web page that looks decent on a phone.
- **No multi-tournament support.** This codebase is for *the 2026 Crucible*. Next year's tournament is a new repo (or a fork).

Each of those "nots" is a feature. Every "no" is a thing you don't have to build, debug, or maintain. **A senior engineer is as proud of what's not in the codebase as what is.**

The architecture this implies

We haven't picked a framework yet, but the problem has already chosen one for us.

- **Single-page app** (one URL, all interaction is client-side).
- **Static deployment** (no backend, no database — just files served from a CDN).
- **Data lives in code** (a `lib/` folder full of TypeScript constants).
- **State is small** (which tab is active, which team or player is selected — that's basically it).

That description matches **Next.js with the App Router, statically rendered**. Or React + Vite. Or even plain HTML + JS. We'll pick Next.js in Chapter 2 because it gives us file-based routing, TypeScript out of the box, and zero-config Vercel deployment. But notice we picked it **after** describing the problem, not before. The problem dictated the requirements; the requirements dictated the framework. **Never the other way around**. That's the difference between a senior and a junior — juniors pick a framework they like and bend the problem to fit; seniors describe the problem and let the framework choose itself.

Two-sentence summary

Try writing this for yourself before reading mine:

An eight-team fantasy snooker league for the 2026 World Championship. Each team locks in 31 picks (16 R1, 8 R2, 4 QF, 2 SF, 1 Final) before the tournament; as matches finish, picks score 3 for a winner and 1 for a loser, and the app shows standings, match results, prediction comparisons, per-player drill-downs, and analytics charts.

If your version says roughly the same thing, you're ready for Lesson 2. If it doesn't, re-read the user stories section.

Vibe prompt you would have used (to verify understanding before coding)

*"I'm about to build a single-page React app for a fantasy snooker league. Eight teams pick winners across five rounds (R1: 16 picks, R2: 8, QF: 4, SF: 2, Final: 1). Scoring: 3 points if the pick wins, 1 if it loses, null while the match is unplayed. No database — bracket and team picks are edited by hand in code. Five tabs: Standings, Matches, Predictions, Players, Analytics. **Don't write any code yet.**"*

Just confirm you understand the problem and ask me anything that’s ambiguous.”

The “don’t write any code yet” line is the single most useful trick in this lesson. It forces the LLM (and yourself) to surface ambiguities. Try it on every problem: *“summarise the problem back to me, ask three clarifying questions, then I’ll tell you to start coding.”*

CHECK YOURSELF

Answer these out loud or in writing. If you can’t answer one, scroll back up.

1. **Why is `null` the right value for a pending pick instead of `0`?** What goes wrong if you use `0`?
2. **Why are there exactly 31 matches in the tournament?** Justify the number from first principles.
3. **The problem statement says “no database.” What three follow-on decisions does that single constraint make for us?** (Hint: hosting, auth, who-can-edit.)
4. **Two friends ask to add a 9th fantasy team next year. What’s the minimum data change required?** (You don’t need to write the code — just say which file/structure you’d touch.)
5. **The five user stories map to five tabs. If the friends asked for a “schedule” tab showing matches in chronological order, would that be a sixth user story or the same as the existing Matches tab?** Argue both sides.

When you’ve answered these, head to [02-data-modeling.md](#). We’ll start translating the problem into TypeScript shapes.

Chapter 1 · Lesson 2 — Data Modeling (TypeScript interfaces)

Goal: translate yesterday’s pub conversation into precise TypeScript types. By the end of this lesson you’ll have written `Lib/types.ts` — the single most important file in the codebase, even though it doesn’t render anything.

Why we model the data before we render it

There’s a saying in engineering: **“Show me your data structures and I’ll know your code.”** It’s attributed to Linus Torvalds, but the idea is older. The shape of your data **dictates the**

shape of your code. Everything else is decoration.

If you walk into a React interview and the interviewer hands you a problem, the first move that separates seniors from juniors is *what you draw on the whiteboard first*. Juniors draw a component tree. Seniors draw the data. Once the data is right, the component tree falls out almost mechanically.

So before we ever import React, we're going to spend this lesson drawing the data — but in TypeScript instead of on a whiteboard.

A 60-second TypeScript primer (for total beginners)

If you've never used TypeScript, two things to know:

1. **TypeScript is JavaScript plus type annotations.** Every TS file is also a valid JS file once the types are erased. Types are a development-time feature; they vanish at runtime.
2. **Types describe shapes.** When we write `interface Match { id: number; winner?: string; }`, we are saying *“any object claiming to be a Match must have an `id` field that is a number, and may optionally have a `winner` field that is a string.”*

That's almost all you need for this lesson. Two more details, then we'll write some types:

- `?` after a field name means **optional** — the field may be present or absent. `winner?: string` means *winner might be a string, or might not exist at all (undefined)*.
- `|` is a **union** — “either this or that”. `pick: string | null` means *pick is either a string or the literal value null*.

That's the whole TypeScript story we need for Chapter 1. Don't overthink it. The compiler is just a really pedantic friend reading over your shoulder.

Modeling Match

A match is a single contest between two players. From Lesson 1 we know it has:

- A position in the bracket (its “id”, really an ordinal — match 1, match 2, ...)
- Two players (let's call them p1 and p2)
- A winner — *but only after the match is played*
- A score — display only, like `'10-7'`

- For Round 1, the seed of player 1 (because R1 pairs are usually shown as “1 vs 32”, “2 vs 31”, etc.)

In TypeScript:

```
export interface Match {
  id: number;
  p1: string;
  p2: string;
  winner?: string;
  score?: string;
  seed1?: number;
}
```

Six fields, three required, three optional. Read each ? carefully — the ? is **load-bearing**. It’s the difference between “*this field always exists*” and “*this field exists only sometimes*.”

Why winner? is optional

This is the fork in the road that powers the whole app. Re-read Lesson 1’s scoring rules: a match that hasn’t been played has null (or, equivalently here, undefined) for every team’s score. The data-level mechanism that produces that behaviour is **the absence of match.winner**.

When the file owner adds a winner, the field appears. Until then, it’s not there at all. There is no winner: null in the data, no winner: ' ', no winner: 'TBD'. There is *just no winner field*. That’s the cleanest way to model “not yet known”: **the field doesn’t exist**.

The TypeScript compiler will then, on its own, force every consumer of match.winner to handle the undefined case:

```
const w = match.winner;           // type is `string | undefined`
const isWin = pick === w;         // works, but isWin is false if w is undefined
if (match.winner) { /* ... */ }  // narrowed to `string` inside this block
```

That’s the type system **doing free safety work for you**. If you’d modeled winner as winner: string (required), every place that called scorePick would happily compare pick === ' ' and silently return *consolation* instead of *pending*. The optional ? is what saves you.

Why score? and seed1? are optional

score is optional because finished matches have one and pending matches don't. Same logic as winner.

seed1 is more interesting. It's only on R1 matches in our data. Why? Because that's the only round where the *bracket position is meaningful enough to display*. By round 2, "seed 1 vs unseeded qualifier" is no longer the point — the point is the matchup itself, "Zhao vs Murphy". So we never bother to add seeds in R2/QF/SF/F objects, and the type accommodates that with ?. Optional fields aren't about "data we forgot to fill in" — they're about **fields that simply don't exist for this row**.

Modeling Team

A team has a name, some cosmetics (color, accent, icon, motto), and **five arrays of picks**:

```
export interface Team {
  name: string;
  color: string;
  accent: string;
  icon: string;
  motto: string;
  r1: string[];
  r2: string[];
  qf: string[];
  sf: string[];
  final: string;
  // Round-2 Lookup tables for handling players whose path traveled through
  // bracket halves than the simple seed pairing implies (used by scoring as
  r1Players?: string[];
  r2Players?: string[];
}
```

Three things to notice:

1. `r1`, `r2`, `qf`, `sf` are arrays of strings — but `final` is just a string.

Lesson 1 reminded us there's only one Final match. So a team only makes one Final pick. We *could* have written `final: string[]` with one element, but that would cost us readability everywhere we touch it: `team.final[0] === 'Wu Yize'` reads worse than `team.final === 'Wu Yize'`. So we accept the asymmetry. The cost is that scoring code has one branch for Final and another for everything else (we'll see this in Lesson 3).

Asymmetry in your data is fine when it matches the real-world asymmetry. “There is one final” is a real-world constraint. The data should reflect it.

2. The cosmetics live next to the picks.

`color`, `accent`, `icon`, `motto` are decorative. They drive every gradient, badge, and chip in the UI. They live on the same object as the picks because **each team is one logical thing** — not two (a “team identity” plus “team picks”). Splitting them would buy nothing and add a level of indirection.

This is a common beginner mistake: over-normalizing. SQL databases want you to factor out shared data into separate tables. **A 250-row JavaScript file does not.** When the data is small and self-contained, denormalize. Optimize for reading the data with your eyes, not for storage efficiency.

3. `r1Players?` and `r2Players?` exist for a subtle scoring case.

These aren't needed for normal scoring — they're a hint used by `calculateTeamScores` when a team's pick *advances* through the round in a non-obvious way (e.g. the team picked someone in their R1 spot, but the R2 spot is for a different bracket position than they expected). For most teams these arrays are unused. They're optional precisely because most teams won't have them.

We'll come back to this in Lesson 3 when we write the scoring function. For now, just know the field exists and is rare.

The most important rule in the codebase (the invariant)

This is the thing that, if you forget it, every score in the app silently goes wrong:

For each round, the *i*-th element of `team.r1` (or `team.r2`, `qf`, `sf`) corresponds to the *i*-th element of `ROUND1_MATCHES` (or `ROUND2_MATCHES`,

QF_MATCHES, SF_MATCHES).

There is no `matchId` field on each pick. There is no key linking a pick to a match. **Order is the contract.** Index 0 of `team.r1` is the team's pick for `ROUND1_MATCHES[0]`. Index 5 is the pick for `ROUND1_MATCHES[5]`. Always.

If you re-order `ROUND1_MATCHES` and forget to re-order all 8 teams' `r1` arrays in lockstep, every Round 1 score is wrong. There is **no warning**, no compile error, no runtime test catching this. It's a convention enforced by you, the developer.

In a "proper" enterprise codebase you'd add a foreign key — each pick would be { `matchId: 5, player: 'Mark Allen'` } and scoring would `find()` the match by id. That's safer but **noisier to read and write**. For an 8-team league with one owner editing the file, the convention is fine. The moment a 9th team or 33rd player enters, you'd reconsider.

This is a recurring tradeoff in software: **safety vs. simplicity**. Senior engineers know when each is right. The answer is rarely "always one or always the other."

Modeling PlayerInfo (the dictionary, not an array)

There are 32 players. Each has a country flag, a seed, a status string, a flag-text label:

```
export interface PlayerInfo {  
  country: string;    // emoji flag, e.g. '🇨🇳'  
  seed?: number;      // 1-16, or undefined for qualifiers  
  status: string;     // 'Defending Champion', 'World No. 1', etc.  
  flag: string;        // 3-letter, e.g. 'CHN'  
}
```

Notice `seed?` is optional — only the top 16 are seeded. Qualifiers don't have a seed.

But the more interesting decision is what *container* we put 32 of these in. Two options:

```
// Option A: array of objects with a 'name' field  
export const PLAYERS: { name: string; country: string; seed?: number; ... }[] = [  
  { name: 'Zhao Xintong', country: '🇨🇳', seed: 1, ... },  
  // ...  
];
```

```
// Option B: object keyed by name
export const PLAYER_INFO: Record<string, PlayerInfo> = {
  'Zhao Xintong': { country: '??', seed: 1, ... },
  // ...
};
```

Both are valid. The codebase picks **Option B**. Why?

Because the most common operation in the app is “given a player’s name (from a match’s p1/p2, or from a team’s pick), get the metadata for that player.” That’s a hash lookup:

```
const info = PLAYER_INFO['Zhao Xintong']; // O(1)
```

If we’d picked Option A, every lookup would be:

```
const info = PLAYERS.find(p => p.name === 'Zhao Xintong'); // O(n)
```

For 32 players, the performance difference is negligible. The **readability** difference, however, is significant — and the readability difference is multiplied by the number of times the lookup appears in the code. In our codebase, it appears in <PlayerDetail>, in <MatchPickAnalysis>, in the standings hover tooltip, in the player card, etc. Probably 10+ places. Each of them reads cleaner with Option B.

This decision — *what’s the lookup key?* — is one you’ll make on every dataset. Ask yourself “*how will I most commonly query this?*” The answer becomes the key.

Modeling the score detail (output of scoring, used by every UI)

When calculateTeamScores runs, it doesn’t just return a number. It returns **detail records** that the UI uses to color cells, write tooltips, count corrects/wrongs. Each detail looks like this:

```
export interface ScoreDetail {
  match: Match;
  pick: string;
  points: number | null; // 3 | 1 | null
  correct: boolean | null; // true | false | null
}
```

Two tri-state fields. `points` is 3 (correct), 1 (wrong), or null (pending). `correct` is true,

false, or null (pending).

You might ask “*why both?*” Aren’t correct and points redundant? Almost, but not quite:

- `points` is what you sum to get a total.
- `correct` is what you count to get a hit-rate (e.g. “*Invincibles got 11 of 16 R1 picks right*”).

Both come up. The UI reads `correct` for stat tiles (“11 of 16”) and `points` for the running totals (“33 / 48”). Storing them both, computed once, saves every UI place from re-deriving one from the other.

Putting it all together: `lib/types.ts`

Here’s the whole file, all in one place. Open `lib/types.ts` in the project root and compare:

```
// lib/types.ts

export interface Match {
  id: number;
  p1: string;
  p2: string;
  winner?: string;
  score?: string;
  seed1?: number;
}

export interface PlayerInfo {
  country: string;
  seed?: number;
  status: string;
  flag: string;
}

export interface ScoreDetail {
  match: Match;
  pick: string;
```

```

    points: number | null;
    correct: boolean | null;
}

export interface TeamScores {
    r1: number;
    r2: number;
    qf: number;
    sf: number;
    f: number;
    total: number;
    r1Details: ScoreDetail[];
    r2Details: ScoreDetail[];
    qfDetails: ScoreDetail[];
    sfDetails: ScoreDetail[];
    fDetails: ScoreDetail[];
}

export interface Team {
    name: string;
    color: string;
    accent: string;
    icon: string;
    motto: string;
    r1: string[];
    r2: string[];
    qf: string[];
    sf: string[];
    final: string;
    r1Players?: string[];
    r2Players?: string[];
}

export interface TeamWithScores extends Team {
    scores: TeamScores;
}

```

```
}
```

Six interfaces. Read each one. **None of them imports React.** None has a JSX angle bracket. This is pure data modeling, and it's the most important file in the project.

The `extends Team` at the bottom is a TypeScript trick: a `TeamWithScores` is a `Team` with one extra field, `scores: TeamScores`. The orchestrator component (Chapter 4) builds `TeamWithScores` objects by spreading the original team and adding scores. Until then, just notice that the type system tracks both.

Why the TypeScript compiler is your friend

If you typed `team.r1[0]` somewhere in this app, TypeScript knows it's a `string`. If you typed `match.winner` somewhere, TypeScript knows it might be undefined and forces you to handle that. If you tried to assign a `Team` to a variable typed `Match`, TypeScript yells at you immediately.

The types are the contract. Once they're written, every piece of code that touches a `Team` or `Match` has to obey them. That's the safety net that lets you refactor confidently — change a type, and the compiler tells you every file that breaks.

A common interview question: *"why use TypeScript instead of plain JavaScript?"* The answer in two parts: (1) **catch errors at compile time** instead of runtime, (2) **document the data shapes** so future-you (or future-collaborator) knows what's going where. The second one is at least as important as the first. Types are documentation that **doesn't drift out of sync with the code**, because the code won't compile if it does.

The quick exercise

Open `lib/types.ts` in the repo. Compare it to what we wrote above. Is there anything different? (Spoiler: no — the file is exactly what's in this lesson, possibly with the order reshuffled. The types in the lesson are the live, working types in the app.)

If you're following along by re-creating the project from scratch, type the file out yourself. Don't copy-paste. Typing it is how the patterns sink in. The whole file is about 50 lines.

Vibe prompt you would have used

"Write me a `lib/types.ts` file with TypeScript interfaces for: (1) `Match` with `id: number`, `p1: string`, `p2: string`, `optional winner?: string`, `optional score?: string`, `optional seed1?: number`; (2) `PlayerInfo` with `country`, `status`, `flag` strings and `optional seed?: number`; (3) `Score-Detail` with `match: Match`, `pick: string`, `points: number | null`, `correct: boolean | null`; (4) `TeamScores` with the 5 round totals plus `total` and 5 `Details` arrays; (5) `Team` with `name`, four cosmetic strings, `picks` arrays for `r1[]`, `r2[]`, `qf[]`, `sf[]`, a single `final: string`, and two optional helper arrays; (6) `TeamWithScores` extends `Team` with one additional `scores: TeamScores` field. Don't write any other code."

Notice how specific every field name is. This is what a good vibe prompt looks like for type generation: each field is named, each type is given, each `?` is intentional. The LLM has nothing to hallucinate. **Specificity in the prompt means specificity in the output.**

CHECK YOURSELF

1. **What's the difference between `winner?: string` and `winner: string | undefined` in TypeScript?** (Subtle — answer at the end of this lesson.)
2. **`team.final` is a string but `team.sf` is a `string[]`. Justify the asymmetry.** What would it cost in code-readability to "fix" it by making `final` an array of one?
3. **`PLAYER_INFO` is an object keyed by name, not an array. Name two other places in the app where this lookup pattern would be cheaper than `.find()`.** (You don't have to know the codebase yet — guess.)
4. **The "match arrays in pick array order" invariant is enforced by convention, not by the type system.** What change to the `Team` interface would let TypeScript enforce it for you? What's the cost?
5. **Open `lib/types.ts` and find one type you don't immediately understand. Read it three times. Write down what every field is for.** (No question, just an exercise.)

Answer to #1 (because it's a real gotcha)

`winner?: string` and `winner: string | undefined` look almost the same, but they differ on **whether the field has to be specified at all** when constructing the object.


```

interface A { winner?: string; }
interface B { winner: string | undefined; }

const a: A = {}; // ✅ valid (field omitted)
const a2: A = { winner: undefined }; // ✅ valid (field present, undefined)

const b: B = {}; // ❌ ERROR: 'winner' is missing
const b2: B = { winner: undefined }; // ✅ valid (field present, undefined)

```

? says “the property may not exist on the object at all.” | undefined says “the property must exist, but its value can be undefined.” For our Match type we want ? — pending matches simply don’t have a winner field, period.

When you’ve got these answered, head to [03-pure-functions-and-scoring.md](#).

Chapter 1 · Lesson 3 — Pure Functions & Scoring

*Goal: write `Lib/scoring.ts` — two pure functions, no React, no JSX, no DOM.
By the end of this lesson the heart of the application is beating, and you haven’t typed `import React` even once.*

What is a “pure function” and why do we care?

A **pure function** has two properties:

1. Given the same inputs, it always returns the same output.
2. It has no side effects — it doesn’t mutate any state outside itself, doesn’t write to disk, doesn’t make network calls, doesn’t `console.log`, doesn’t read a global variable that might change.

```

// Pure
function add(a: number, b: number): number {
  return a + b;
}

// Impure (uses external state)
let counter = 0;

```

```
function increment(): number {
  counter += 1;
  return counter;
}
```

Pure functions are the part of your codebase that **survives every refactor**. UI frameworks come and go — React might be replaced by Solid in three years, you might migrate to Vue, you might one day rewrite the whole thing as a CLI tool. But `scorePick(pick, match)` doesn't change. It's just a rule about snooker scoring expressed in code.

In our app, `scorePick` and `calculateTeamScores` are the **business logic layer**. Everything else — the components, the styles, the gradients — is the **presentation layer**. Mixing them is the most common React mistake. Keeping them separated is the senior move.

Interview tip. When asked “*how do you structure a React app?*”, the seniors say “*separate pure data/logic from rendering.*” If you can point to your own `lib/` folder full of pure functions and your `components/` folder of pure presentation, you've got a working answer with proof.

Walking through `scorePick`

Open `lib/scoring.ts` in the repo. The first function is six lines:

```
import type { Match } from './types';

export function scorePick(pick: string | null, match: Match | undefined): number {
  if (!match || !match.winner) return null;
  if (pick === match.winner) return 3;
  return 1;
}
```

Read it. Three branches:

Branch 1: `if (!match || !match.winner) return null;`

This is the “pending” branch. Two ways the function arrives here:

- **!match** — there's no match object at all. This is defensive. We use it because some teams have *fewer* picks than there are slots in later rounds (e.g. a team's pick lost

in R1, so their R2 pick technically points at a player who doesn't appear in R2). The match lookup returns undefined. We don't want to crash; we want to return null.

Defensive coding without sloppy coding.

- **!match.winner** — the match exists but no one has won yet. The winner field is undefined (recall Lesson 2: it's an *optional* field that's literally absent on pending matches).

Return null. Not 0. Lesson 1 covered this; here's the line of code that enforces it.

Branch 2: if (pick === match.winner) return 3;

If we reach this line, we know match.winner is a string (TypeScript narrowed it from the previous early-return). If the team's pick matches the winner, they get **3 points**.

Notice the === (strict equality). In JavaScript that means *same type, same value*. Both pick and match.winner are strings, so it's a plain string comparison.

This is also why **player names must match exactly**. If a team writes pick: 'Zhao Xintong' but the match record says winner: 'Zhao Xintong' (with two spaces), the comparison silently fails and the team gets the wrong score. This is the kind of bug that hides for days. Most teams discover it once they accidentally save the file with a trailing space and watch their score drop by 3 points.

Branch 3: return 1;

The match has a winner, but the pick wasn't them. The team gets a **consolation point**. Snooker's hard; you tried.

This branch is the implicit else. There's no else keyword needed because the previous two branches both return. Once we get past them, the only remaining case is "match finished, pick wrong."

The whole function in one breath

"If the match isn't finished, return null. Otherwise, return 3 if the pick won, 1 otherwise."

That's the entire scoring rule. **Six lines of code, infinite consequences.**

Walking through calculateTeamScores

This is the workhorse. Open `lib/scoring.ts` and look at the second function. We'll go piece by piece.

```
import { ROUND1_MATCHES, ROUND2_MATCHES, QF_MATCHES, SF_MATCHES, FINAL_MATCHES } from './constants';
import type { Team, TeamScores, ScoreDetail } from './types';

export function calculateTeamScores(team: Team): TeamScores {
  let r1 = 0, r2 = 0, qf = 0, sf = 0, f = 0;
```

The function takes a `Team` and returns a `TeamScores` (the wide return shape we modeled in Lesson 2: 6 numeric totals + 5 detail arrays).

The `let` declarations initialize five running totals to zero. We'll add to them as we walk each round.

Why `let` instead of `const`? Because we're going to mutate these. `let` is "I'll reassign." `const` is "I won't." TypeScript / JS strict-mode lints you on mismatches. **Naming things accurately is part of writing readable code.**

Round 1 walkthrough

```
const r1Details: ScoreDetail[] = team.r1.map((pick, i) => {
  const pts = scorePick(pick, ROUND1_MATCHES[i]);
  r1 += pts || 0;
  return {
    match: ROUND1_MATCHES[i],
    pick,
    points: pts,
    correct: pts === null ? null : pts === 3,
  };
});
```

Five lines, four ideas:

Idea 1: `team.r1.map((pick, i) => ...)` `.map()` is JavaScript's "transform every item in this array into something else." For each `pick` (a player name string) at position `i`,

run the callback and collect the results. The callback receives both the value (`pick`) and the index (`i`).

We need `i` because we're going to look up the *matching* match by position: `ROUND1_MATCHES[i]`.

The invariant from Lesson 2 in action: the *i*-th pick is for the *i*-th match. Without that invariant, this `.map` couldn't be 5 lines — it'd need a `.find()` or a lookup table.

Idea 2: `scorePick(pick, ROUND1_MATCHES[i])` We call our pure function from earlier. It returns `3 | 1 | null`. We assign that to `pts`.

This is the only place in the whole codebase that the scoring rule is applied. **Logic centralized, used everywhere.** If the league rules change next year (e.g. “5 points for picking a final-frame thriller”), there's exactly one function to update.

Idea 3: `r1 += pts || 0;` This is a small but clever line. `pts` might be 3, 1, or `null`. JavaScript's `||` operator returns the first *truthy* value, treating `null` as falsy. So:

- `pts = 3` $\boxtimes$ `r1 += 3` `||` `0` $\boxtimes$ `r1 += 3`
- `pts = 1` $\boxtimes$ `r1 += 1` `||` `0` $\boxtimes$ `r1 += 1`
- `pts = null` $\boxtimes$ `r1 += null` `||` `0` $\boxtimes$ `r1 += 0`

Pending picks add nothing. The total reflects only finished matches. As Round 1 plays out, `r1` creeps up from 0 toward its final value, never spiking incorrectly.

A more “modern” version would use the nullish coalescing operator: `r1 += pts ?? 0`. The `??` only treats `null` and `undefined` as falsy (unlike `||` which also rejects `0`). For our case it doesn't matter — `pts` is never `0`. But `??` is the pedantically correct choice. The codebase uses `||`; either is acceptable.

```
return {
  match: ROUND1_MATCHES[i],
  pick,
  points: pts,
  correct: pts === null ? null : pts === 3,
};
```

Idea 4: the returned detail object Each iteration of `.map` returns one `ScoreDetail`. By the end of the `.map`, `r1Details` is an array of 16 `ScoreDetail` objects, one per Round 1

match.

Notice correct: `pts === null ? null : pts === 3`. This is a **ternary inside a ternary** that produces a tri-state boolean:

- If `pts === null`, correct is `null` (pending).
- Otherwise (`pts` is 3 or 1), correct is `pts === 3` — i.e. true if 3 points, false if 1.

That tri-state is what powers three-color UI cells: green for correct, red for wrong, gray for pending. Without it, “wrong” and “pending” would render the same.

Rounds 2 through Final

The pattern repeats:

```
const r2Details: ScoreDetail[] = team.r2.map((pick, i) => {
  const pts = scorePick(pick, ROUND2_MATCHES[i]);
  r2 += pts || 0;
  return {
    match: ROUND2_MATCHES[i],
    pick,
    points: pts,
    correct: pts === null ? null : pts === 3,
  };
});
// ... and the same shape for qf, sf
```

Five almost-identical blocks, one per round. **Duplication-by-round** is one of the recurring patterns of this codebase.

A more architected version would parameterize over a list of rounds:

```
const ROUNDS = [
  { key: 'r1', matches: ROUND1_MATCHES },
  { key: 'r2', matches: ROUND2_MATCHES },
  // ...
] as const;
ROUNDS.forEach(({ key, matches }) => { /* shared logic */ });
```

Cleaner? Yes. But it has costs:

- The TypeScript types need to be more flexible (not all rounds have the same shape; the Final is special).
- The reader needs to load the abstract loop in their head before they can see what each round does.
- Each round is currently *grep-able* by its name — a debug-friendly property.

The codebase chose copy-paste for **5 rounds in one place**. That's a trade-off, not a mistake. **Copy-paste is forgivable up to about 3 occurrences; beyond that you should consider abstracting.** Five round blocks is right at the edge — you could argue it either way. The author chose readability. Both choices ship.

The Final is special

```
const finalMatch = FINAL_MATCH[0];
const fPts = scorePick(team.final, finalMatch);
f += fPts || 0;
const fDetails: ScoreDetail[] = [{
  match: finalMatch,
  pick: team.final,
  points: fPts,
  correct: fPts === null ? null : fPts === 3,
}];
```

Three reasons it's different:

1. **team.final is a string, not an array.** Recall Lesson 2 — there's only one Final, so we accept the asymmetry.
2. **FINAL_MATCH[0]** — even though there's only one Final match, we kept it in an array for consistency with the other rounds. So we index into it once.
3. **fDetails is wrapped in an array of one** — for consistency with the other *Details arrays. UI code that does `team.scores.fDetails.map(...)` works exactly like `r1Details.map(...)`. **Symmetry where the UI sees it; asymmetry where the data needs it.**

The total and the return

```
const total = r1 + r2 + qf + sf + f;
return { r1, r2, qf, sf, f, total, r1Details, r2Details, qfDetails, sfDetails }
}
```

Sum the five round totals into `total`. Return all 11 fields. **Compute once, render five different ways** — Lesson 2’s “wide return shape” pays off here. Every UI screen in the app will read from this object in different ways. None of them re-runs the scoring.

Why is the return shape so wide?

You might think “*why return both `r1` (the number) and `r1Details` (the array)? Couldn’t we always derive the number from the array?*”

We could:

```
const r1 = r1Details.reduce((sum, d) => sum + (d.points || 0), 0);
```

But:

- The standings table needs the number. Computing it from the array on every render of every row would be wasteful.
- The number is what most callers want; the array is what some callers want. Pre-computing both means **every caller picks what’s cheap**.
- The data is small enough (5 numbers) that the storage cost is zero.

This is **the wide-return-object pattern**: when a function naturally produces several related pieces of data and they’re all going to be consumed somewhere, return them all in one object. Don’t make the caller call you N times.

The whole `lib/scoring.ts`

Open the file in the repo. Compare to what we’ve discussed:

```
import { ROUND1_MATCHES, ROUND2_MATCHES, QF_MATCHES, SF_MATCHES, FINAL_MATCHES } from './types';
import type { Team, TeamScores, ScoreDetail, Match } from './types';

export function scorePick(pick: string | null, match: Match | undefined): number {
  if (!match || !match.winner) return null;
}
```



```

    if (pick === match.winner) return 3;
    return 1;
}

export function calculateTeamScores(team: Team): TeamScores {
    let r1 = 0, r2 = 0, qf = 0, sf = 0, f = 0;

    const r1Details: ScoreDetail[] = team.r1.map((pick, i) => {
        const pts = scorePick(pick, ROUND1_MATCHES[i]);
        r1 += pts || 0;
        return { match: ROUND1_MATCHES[i], pick, points: pts, correct: pts === null ? 0 : 1 };
    });
    const r2Details: ScoreDetail[] = team.r2.map((pick, i) => {
        const pts = scorePick(pick, ROUND2_MATCHES[i]);
        r2 += pts || 0;
        return { match: ROUND2_MATCHES[i], pick, points: pts, correct: pts === null ? 0 : 1 };
    });
    const qfDetails: ScoreDetail[] = team.qf.map((pick, i) => {
        const pts = scorePick(pick, QF_MATCHES[i]);
        qf += pts || 0;
        return { match: QF_MATCHES[i], pick, points: pts, correct: pts === null ? 0 : 1 };
    });
    const sfDetails: ScoreDetail[] = team.sf.map((pick, i) => {
        const pts = scorePick(pick, SF_MATCHES[i]);
        sf += pts || 0;
        return { match: SF_MATCHES[i], pick, points: pts, correct: pts === null ? 0 : 1 };
    });

    const finalMatch = FINAL_MATCH[0];
    const fPts = scorePick(team.final, finalMatch);
    f += fPts || 0;
    const fDetails: ScoreDetail[] = [{
        match: finalMatch,
        pick: team.final,
        points: fPts,
        correct: fPts === null ? 0 : 1
    }];
}

```

```

    correct: fPts === null ? null : fPts === 3,
  }];

  const total = r1 + r2 + qf + sf + f;
  return { r1, r2, qf, sf, f, total, r1Details, r2Details, qfDetails, sfDetails };
}

```

About 40 lines. Five round blocks plus two utility functions. **No React, no JSX, no DOM.** This file would compile and run correctly in Node.js, in Deno, in a browser console, in a Cloudflare Worker, in a serverless function — anywhere JavaScript runs.

That portability is a feature. If next year you decide to write a Discord bot that posts standings hourly, this scoring file goes into the bot unchanged. It's pure logic.

Trying it out (optional)

If you want to feel it work, paste this into a Node REPL:

```

import { scorePick } from './lib/scoring';

scorePick('Wu Yize', { id: 1, p1: 'Wu Yize', p2: 'Murphy', winner: 'Wu Yize' });
scorePick('Murphy', { id: 1, p1: 'Wu Yize', p2: 'Murphy', winner: 'Wu Yize' });
scorePick('Wu Yize', { id: 1, p1: 'Wu Yize', p2: 'Murphy' });

```

That's the entire scoring engine, from a black box. Three calls, three predictable answers.

Vibe prompt you would have used

"Write Lib/scoring.ts. Two pure functions. (1) scorePick(pick: string | null, match: Match | undefined): number | null — return null if !match || !match.winner, else 3 if pick === match.winner, else 1. (2) calculateTeamScores(team: Team): TeamScores — for each of r1/r2/qf/sf use team.rX.map((pick, i) => scorePick(pick, ROUNDx_MATCHES[i])), accumulate the points (treating null as 0), and produce per-pick ScoreDetail records of shape { match, pick, points, correct } where correct is points === null ? null : points === 3. Final is special: team.final is a single string against FINAL_MATCH[0]. Return { r1, r2, qf, sf, f, total,

```
r1Details, r2Details, qfDetails, sfDetails, fDetails }.  
Import Match, Team, TeamScores, ScoreDetail from ./types. No React,  
no side effects."
```

This is the most precise vibe prompt you'll write. Notice every type is named, every shape is given, the asymmetry of Final is called out, the imports are listed. **The LLM has nothing to invent.** A prompt this specific produces the right answer almost regardless of which model you use.

CHECK YOURSELF

1. `r1 += pts || 0` – what would happen if you removed the `|| 0`? Walk through the case `pts = null` step by step.
2. `scorePick` is called from inside `calculateTeamScores.map`. Is `scorePick` still a pure function in that context, even though it's called inside a loop? Why or why not?
3. Why does `calculateTeamScores` return both `r1` (the total) and `r1Details` (the array of details)? Is one redundant? Could the UI live without one of them?
4. The Final case is hard-coded as one extra block at the bottom of the function. Sketch what the function would look like if `team.final` were instead a one-element array (`team.final: string[]`). Does the function become longer or shorter? Easier to read or harder?
5. You're given a new league rule: a team that picks a player who reaches the Final but loses gets +5 bonus points (regardless of whether they were the team's Final pick). Where in this codebase would you add that logic? Is it a change to `scorePick`, to `calculateTeamScores`, or to the data? Why?

When you've answered these, you've finished Chapter 1. Onward to [Chapter 2 – Project Setup](#), where we'll finally open Next.js and turn this pile of types and functions into a running web page.

Chapter 1 – Foundations

Before you write a single line of React, you need to understand the problem and the data. Half of senior interviews are *"how would you model this?"* – and that's a question that's answered before any JSX gets typed.

What you'll learn

- How to **read a problem statement** and identify the data shapes it implies.
- How to **design a data model** in TypeScript — interfaces, unions, optional fields.
- Why **pure functions** are the foundation of every React app, even before React enters the picture.
- The mental difference between *the data* and *the view of the data*.
- Why `null` is a valid value (and so is `undefined`, but they mean different things).

Lessons in this chapter

1. **01-the-problem.md** — A pub-style explanation of the fantasy snooker league. No code. The questions every senior asks before opening their editor.
2. **02-data-modeling.md** — Translating “8 friends, 32 players, 31 matches” into TypeScript interfaces. The most important invariant in the whole codebase.
3. **03-pure-functions-and-scoring.md** — Writing `scorePick` and `calculateTeamScores` with zero React, zero JSX, zero DOM.

Files you'll create in this chapter

- `lib/types.ts` — TypeScript interfaces for `Match`, `Team`, `PlayerInfo`, `ScoreDetail`, `TeamScores`.
- `lib/scoring.ts` — the two pure functions `scorePick` and `calculateTeamScores`.

New React/Next.js concepts introduced

None. That's the point of Chapter 1. You'll use `interface`, `type`, and plain JavaScript functions — no React imports yet.

Why no React yet? Because **the data layer is what survives every UI rewrite.**

If you can't model the problem cleanly without React, adding React won't fix it. Junior devs reach for `useState` on day one and end up with a mess. Seniors model the data first, then ask “where does this *change*?” — and the answer to that question is what becomes state. Chapter 1 is the foundation Chapter 4 builds on.

How long it should take

- 30 min reading per lesson
- 30 min typing the code in lessons 2 and 3
- ~2.5 hours total for the chapter

Before you start

Make sure you have:

- A working Node.js install (`node --version` should print v20 or v22).
- An empty folder where you'll build the project (or use this repo's root if you're following along with the finished version open in another window).
- 90 minutes of uninterrupted time. The lessons reference each other; binge-reading lesson 1 then doing lesson 2 a week later won't stick.

Open `01-the-problem.md` and start there.

Chapter 2 · Lesson 1 — Why Next.js + TypeScript

Goal: understand the framework decision tree well enough to defend “we used Next.js” in an interview without looking nervous. Beginners pick frameworks by vibe; seniors pick by requirements.

The interview question

“So why did you pick Next.js for this project?”

Bad answer: “Because everyone uses it.” Or: “Because the AI suggested it.”

Good answer: “The app is a static, single-page experience with file-based data. I needed TypeScript out of the box, zero-config deployment, and a way to optionally pre-render at build time. Next.js gave me all three with one command, plus image optimization and routing if I ever extended it. Vite would have worked too — Vite is leaner and faster in dev — but Next gave me Vercel deployment as a free side effect, which mattered.”

That's the answer this lesson is preparing you to give. We're going to walk through the decision space.

The React framework landscape (as of 2026)

When you decide to use React, you're not done. You still have to pick a **build tool** or **meta-framework**. The major options:

Tool	What it is	Best for
Next.js	A meta-framework on top of React. File-based routing, server-side rendering (SSR), static site generation (SSG), API routes, image/font optimization, deployment to Vercel.	Apps that need any combination of: SSR, SEO, multi-page routing, server-side data, deployment ease.
Vite + React	A bare-bones build tool. Just bundles and serves React. No router, no SSR, no nothing extra.	Single-page apps where you want minimal magic. Library or component-collection projects.
Create React App (CRA)	The legacy starter. Officially deprecated by the React team in 2025.	Don't pick this. Migration target only.
Remix	A meta-framework like Next.js, but more loyal to web standards (forms, fetch, etc.).	Form-heavy apps, apps wanting to embrace progressive enhancement.
TanStack Start	A new entrant, "Vite + React + a router + SSR" assembled by hand.	Apps that want Next.js features without Next.js opinions.
Plain HTML + script tag	Just a <code><script src="react.js"></code> and a CDN link.	Demos, single-page widgets, learning.

For our snooker app, we need to satisfy these requirements (from Chapter 1):

- Single-page interaction (one URL, all interaction client-side)
- Static deployment (no backend)

- Data lives in code (TypeScript constants)
- Tiny state (which tab, which selected item)

Vite would work. Next.js would also work. Plain HTML + a script tag would work for a smaller version. CRA is dead. Remix is overkill for a no-form app. TanStack Start is too new.

So our real choice is **Vite vs Next.js**.

The Vite vs Next.js decision (the honest one)

For an app **this size**, here's the honest tradeoff:

Vite wins on:

- **Dev server startup speed.** Vite boots in ~200ms; Next takes 2-3 seconds.
- **Less framework magic.** What you see in your code is what runs.
- **Smaller bundle by default.** No SSR overhead, no Next runtime.
- **Easier to migrate to a non-React framework later.** Vite is generic.

Next.js wins on:

- **Vercel deployment with literally one click.** Push to GitHub, import on Vercel, done. Vite needs a hosting target you configure (Netlify, Cloudflare Pages, etc.).
- **TypeScript + Tailwind + ESLint pre-wired.** `npx create-next-app` gives you a working project with all of these. With Vite you assemble them yourself.
- **Image and font optimization** out of the box (`next/font`, `next/image`).
- **Routing if you ever expand.** If next year you add a `/leaderboard/[year]` route, Next has it free; Vite needs `react-router-dom`.
- **Server components for free.** Even though we won't use them heavily, the *option* is there.

For this project, **Next.js wins because of deployment ergonomics**. We will probably push this to Vercel, and Next.js + Vercel is the smoothest pipeline available. The cost is a slightly heavier dev server. We'll pay it.

Interview answer template: *"We used Next.js because [main reason] and [secondary reason]. Vite would also have worked — it's lighter — but [main reason] was the tiebreaker."* Replace `[main reason]` with whichever Next.js feature

mattered most for your project. **Always be ready to defend the alternative.** Interviewers respect “I considered X but picked Y because Z” answers, and distrust “I just used what was popular.”

Why TypeScript (not plain JavaScript)?

This is a quicker decision. Three reasons.

1. Catches errors at compile time, not at runtime.

The example we’ll never get tired of:

```
// JavaScript – silently broken  
const team = teams[0];  
console.log(team.scors.r1); // typo, returns undefined, app shows "undefined"  
  
// TypeScript – error before you save  
const team = teams[0];  
console.log(team.scors.r1); // ERROR: Property 'scors' does not exist on ty
```

You’ll make typos. JavaScript will let them ship. TypeScript won’t. **Compile-time safety is free; runtime debugging is expensive.** That alone pays for TypeScript a hundred times over.

2. Self-documenting code.

When you write `function calculateTeamScores(team: Team): TeamScores`, anyone reading the code knows the input shape, the output shape, and they can Cmd+Click to navigate to either. **Types are documentation that the compiler keeps honest.** Comments drift. Types don’t.

3. Editor superpowers.

VS Code, Cursor, Windsurf — every modern editor turns into a god-tier autocomplete machine when you have types. You type `team.` and it shows you `r1`, `r2`, `qf`, `sf`, `final`, You don’t have to memorize the shape; the editor reminds you.

When NOT to use TypeScript

There's a reasonable counterargument for tiny scripts (under 50 lines), prototypes you'll throw away, and learning exercises where the type ceremony obscures the lesson. For *anything* shipped to production, multi-file, or maintained by more than one person — TypeScript wins.

Interview answer template: *"Compile-time errors, self-documenting types, and editor support. The cost is a small upfront learning curve and slightly more verbose code, but for any project bigger than a 50-line script the win is overwhelming."*

Why Tailwind CSS (and why we barely use it)

This is the one part of the stack that has a small surprise. The codebase uses **inline styles** (e.g. `style={{ background: '#0F5132' }}`) more than Tailwind classes. Then why is Tailwind installed at all?

Three reasons:

1. To set up `globals.css` properly.

`app/globals.css` has these three Tailwind directives at the top:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

`@tailwind base` is the part that matters most: it injects Tailwind's CSS reset (Preflight). That's a curated set of "make all browsers behave the same" rules — `* { box-sizing: border-box }`, `h1 { font-size: inherit }`, etc. **Without a reset, every browser styles your app slightly differently.** With Tailwind installed we get the reset for free, and we don't even have to use any classes.

2. To leave the door open for future Tailwind use.

If you later want to convert one component from inline styles to Tailwind classes, the toolchain's already set up. No re-config needed.

3. So next/font and other Next.js features that integrate with Tailwind work cleanly.

Some Next plugins assume Tailwind is present. Having it costs ~5KB of CSS and zero developer time.

Why inline styles instead of Tailwind classes?

The codebase uses dynamic, computed styles based on team colors, gradients with multiple custom values, and conditional logic that would produce very long class strings if rendered with Tailwind:

```
// Inline (what the codebase uses)
<div style={{
  background: `linear-gradient(135deg, ${team.color} 0%, ${team.accent} 100%`,
  boxShadow: `0 8px 32px ${team.color}50`,
  borderRadius: 16,
}}>

// Tailwind (what it'd look like with arbitrary values)
<div className="rounded-2xl"
  style={{ background: `linear-gradient(135deg, ${team.color} 0%, ${team.
    boxShadow: `0 8px 32px ${team.color}50` }}>
```

Tailwind's strength is **static styling repeated across many elements** (buttons, headings, cards). When every visual is computed from runtime props (team colors), Tailwind doesn't help — and inline styles read more cleanly. **Right tool, right job.**

In an interview: *"We use Tailwind for the reset and global setup; inline styles for component-level dynamic theming. If a piece of styling becomes static and repeated, we'd convert it to Tailwind classes."* That's a defensible answer.

What about CSS Modules, styled-components, emotion, vanilla-extract?

Quick tour of the alternatives:

- **CSS Modules** (.module.css) — built into Next.js. Locally-scoped class names. Great for static styling. We could have used them; we chose inline because of the dynamic theming. Both are valid for production.

- **styled-components / emotion** — CSS-in-JS libraries. Powerful but add bundle size and runtime overhead. Falling out of favor (2023+) because of bundle size and SSR pain. Don't reach for them on new projects.
- **vanilla-extract** — a typed, zero-runtime CSS-in-TS library. Newer, niche. Worth knowing about; not necessary here.

For our app: **inline styles + Tailwind reset** is the simplest, lightest approach that satisfies our constraints. **Senior engineers prefer the simplest thing that works.**

Why we don't need a state management library (Redux, Zustand, Jotai)

We have **two pieces of top-level state**: `activeTab` and `selectedTeam`. That's it.

For two pieces of state, `useState` in the orchestrator component plus prop drilling is **the right answer**. Reach for Redux when:

- A dozen-plus components share the same data,
- The state is updated from many places,
- You need time-travel debugging or middleware (e.g. logging actions),
- You're building a Figma-tier app with operational transforms.

We're building a snooker fantasy league. None of the above apply. **Picking Redux for this app would be over-engineering — and interviewers can tell.** A senior says *"the smallest thing that works."* If the requirements grow, you migrate. You don't pre-migrate.

Interview tip. When asked *"how do you manage state?"*, your answer should always start with the size of the app. *"For our small app, we use `useState` in the parent component and prop drilling. If state coordination grew complex, we'd reach for Zustand first because it's the lightest option that scales beyond `useState`."* That answer signals you've thought about it; it signals you don't reach for libraries you don't need; it signals you know the next step if needed.

Why Recharts (when it's time for charts)

Foreshadowing Chapter 6, but worth noting now.

We use **Recharts** for the analytics tab. Alternatives:

- **Chart.js + react-chartjs-2** — older, larger, imperative API. Works but feels last-decade.

- **Victory** — declarative like Recharts, slightly more flexible, slightly bigger.
- **D3** — the underlying library. Massively powerful but you write more code.
- **Visx** — Airbnb's collection of low-level D3-React components. Power-user choice.
- **Nivo** — beautiful, declarative, larger bundle.

Recharts is the **default React charting library**. It's:

- Declarative (you describe what you want, not how to draw it).
- Built on D3 internally (so you get D3's correctness without D3's complexity).
- Tree-shakeable (only the chart types you import end up in the bundle).
- TypeScript-friendly out of the box.

We're not building Bloomberg Terminal. Recharts is enough. **Don't reach for D3 directly until Recharts can't do what you need.**

The decision tree, in one sketch

If you were starting from scratch tomorrow and had to choose, here's the flowchart in your head:

Do I need a backend / API?

├ Yes —————> Next.js (or full-stack: SvelteKit, Remix, Nuxt)

└ No

|

├ Will I deploy to Vercel? —> Next.js (path of least resistance)

|

├ Single page, no routing? —> Vite + React

|

└ Many pages, static, no SSR? —> Next.js (SSG mode) or Astro

Do I need TypeScript? —————> Yes. Always. (Unless < 50 LOC throwaway)

Do I need a state library? —————> Only if useState + prop drilling can't handle
Try Zustand before Redux.

Do I need a styling library? —————> Tailwind. Or inline styles. Don't pick CSS-in-JS in 2026.

Do I need charts? —————> Recharts. Don't reach for D3 until Recharts

Memorize that flowchart. It's not infallible, but it's the senior default. You'll be wrong sometimes; that's fine. Defending the wrong default is easier than defending no default.

What this lesson did NOT cover

- **Server vs client components in detail** (Chapter 7).
- **The exact package.json contents** (next lesson).
- **How Next.js bundles your code under the hood** (advanced — out of scope).
- **Why Server Actions exist** (Next.js 14+ feature; we don't use them).

These are all worth a Google later if you're curious. For now, the framework decisions are made. Next lesson: we type the commands and create the project.

Vibe prompt you would have used (to make the framework decision)

"I'm building a single-page React app for a fantasy sports league. ~250 hand-typed data records, 5 tabs, no backend, no auth, no real-time, deployment to Vercel. Should I use Next.js or Vite + React? Lay out the tradeoffs honestly. Don't just say 'use the popular one.'"

The "don't just say use the popular one" line is what gets the LLM to actually weigh trade-offs instead of giving you the most upvoted Stack Overflow answer.

CHECK YOURSELF

1. **A friend says "but Next.js has SSR, isn't that wasted on a static page?"** Compose a one-paragraph defense of using Next.js for this project anyway.
2. **Your interviewer says "Vite is faster in dev — why didn't you use it?"** Give the honest two-sentence answer.
3. **Why does the project have Tailwind installed even though most styles are inline?** State three concrete reasons.
4. **Tomorrow you start a new project: a 200-product e-commerce site with checkout.** Walk through the framework decision tree out loud.
5. **Tomorrow you start a different new project: a Figma-style collaborative whiteboard.** Different answer? Why?

When you're ready, head to [02-scaffolding-the-project.md](#) and we'll start typing commands.

Chapter 2 · Lesson 2 — Scaffolding the Project

Goal: from an empty folder to a running Next.js dev server, with every config file explained line by line. By the end of this lesson, `npm run dev` boots a placeholder page at `http://localhost:3000`.

The two ways to start a Next.js project

There are two common starting points:

1. **`npx create-next-app@latest`** — interactive scaffold. Asks you a dozen questions and generates a full project. Recommended for **most** real projects.
2. **Build the files by hand.** Manual scaffold. Slower but you understand every line. Recommended for **learning**.

This lesson does Option 2. You'll scaffold by hand. By the end you'll know what `create-next-app` would have done for you, and why.

Step 0: Create the folder

```
# In PowerShell on Windows
New-Item -ItemType Directory -Path "snooker-fantasy" | Out-Null
Set-Location "snooker-fantasy"
```

(If you're following along inside the existing repo, you can skip this — just imagine the rest of the lesson playing out in an empty folder.)

Step 1: `package.json`

This is where every JavaScript project starts. Create a file called `package.json` in the project root with this content:

```
{
  "name": "snooker-fantasy",
```

```

"version": "0.1.0",
"private": true,
"scripts": {
  "dev": "next dev",
  "build": "next build",
  "start": "next start",
  "lint": "next lint",
  "format": "prettier --write \"**/*.{ts,tsx,md,json}\"",
},
"dependencies": {
  "next": "14.2.15",
  "react": "^18.3.1",
  "react-dom": "^18.3.1",
  "lucide-react": "^0.453.0",
  "recharts": "^2.13.0"
},
"devDependencies": {
  "typescript": "^5.6.2",
  "@types/react": "^18.3.11",
  "@types/react-dom": "^18.3.0",
  "@types/node": "^20.16.10",
  "autoprefixer": "^10.4.20",
  "postcss": "^8.4.47",
  "tailwindcss": "^3.4.13",
  "eslint": "^8.57.1",
  "eslint-config-next": "14.2.15",
  "prettier": "^3.3.3"
}
}

```

Walk through every key:

"name", "version", "private"

- **"name"** is the project name. Used by tools, never by your code.
- **"version"** follows semver (semantic versioning). Doesn't matter for an unpub-

lished app; matters a lot for npm packages.

- **"private": true** is a safety latch. It tells npm *"do not publish this to the public registry."* Don't omit it. Many beginners have accidentally published their unfinished apps because they didn't have this.

"scripts" — the npm command surface

These are the commands you'll type 100 times during development:

- **npm run dev** `next dev` — starts the development server with hot reloading at `http://localhost:3000`.
- **npm run build** `next build` — produces an optimized production build in `.next/`. This is what runs on every CI / Vercel deploy.
- **npm start** `next start` — runs the production build (after build).
- **npm run lint** `next lint` — runs ESLint with Next's preset rules.
- **npm run format** `prettier --write "**/*.{ts,tsx,md,json}"` — auto-formats every file.

You'll mostly live in dev and occasionally build. The others are CI / deploy concerns.

Why does npm run dev need run but npm start doesn't? Historical: a few script names (start, test, install, restart, stop) are built-in shortcuts; the rest need run. It's a wart. Live with it.

"dependencies" — what ships in production

Five packages:

- **next** — the framework itself. Pinned to a specific version (14.2.15) intentionally; we want reproducible builds. The `^` in front of others means "any compatible minor version."
- **react** and **react-dom** — React itself plus the DOM renderer. They're separate so React Native or other renderers can swap react-dom out.
- **lucide-react** — a beautiful, tree-shakeable icon library. We use it for the tab icons (Trophy, Calendar, Target, Users, BarChart3) and a handful of other places.
- **recharts** — declarative charts (covered in Chapter 6).

That's it. Notice what's **not** here: no Redux, no axios, no moment, no lodash, no react-router-dom (Next.js has its own routing). **Senior projects have small dependency lists.** Every package you add is a maintenance debt.

"devDependencies" — what's needed only at dev/build time

These are tooling: TypeScript itself, type definitions for React/Node, Tailwind/PostCSS, ESLint, Prettier. They don't ship in the production bundle.

The split between dependencies and devDependencies is **discipline**. If you accidentally put prettier in dependencies, your production bundle will include Prettier (10MB) for no reason. Always put dev-only tools in devDependencies.

Step 2: tsconfig.json (TypeScript compiler config)

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "plugins": [{ "name": "next" }],
    "paths": { "@/*": ["./*"] }
  },
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
  "exclude": ["node_modules", "_source"]
}
```

The most important keys, in plain English:

- **"target": "ES2022"** — compile down to JavaScript syntax that's at most "ES2022" (modern, well-supported). Used to be "ES5" for IE11 days; that's history.

- **"strict": true** — turn on all strict checks: no implicit any, strict null checks, etc. **Don't disable this.** It's the whole point of TypeScript.
- **"noEmit": true** — TypeScript only checks types; it doesn't output .js files. Next.js handles compilation via SWC (a Rust-based compiler).
- **"jsx": "preserve"** — leave JSX in place; let Next.js compile it. (Other values like "react" would compile JSX to React.createElement calls in TypeScript itself; Next.js prefers to do that step.)
- **"paths": { "@/*": ["./*"] }** — the **path alias**. Lets you write import X from '@/lib/types' instead of '../..../lib/types'. The @/ is a convention; you could pick any prefix (~/, \$/, etc.). Most Next.js projects use @/.
- **"plugins": [{ "name": "next" }]** — enables Next.js-specific TypeScript features (e.g. typed routes).
- **"include"** lists which files TypeScript should check; **"exclude"** explicitly skips _source/ (where the original .jsx file lives) and node_modules.

If you don't fully understand every flag, that's fine. **Most TS configs in the wild are copy-paste from a working project**, including in senior repos. The key flags to actually understand are strict, paths, and noEmit. The rest is plumbing.

Step 3: next.config.mjs

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  reactStrictMode: true,
};
export default nextConfig;
```

That's the entire config. Two notes:

- The **/** @type ... */** comment is JSDoc — it tells your editor (TypeScript or VS Code) what shape this object should have, even though the file is .mjs (no TS). You get autocomplete on nextConfig properties for free.
- **reactStrictMode: true** double-invokes some React lifecycle calls during development to surface bugs (e.g. side effects in render, deprecated APIs). It's free; leave it on.

.mjs vs .js? Next.js wants config files in ESM (import/export) format. The .mjs extension forces Node to treat the file as ESM regardless of "type": "module" in pack-

age.json. Belt and braces.

Step 4: tailwind.config.ts

```
import type { Config } from 'tailwindcss';

const config: Config = {
  content: [
    './app/**/*..{js,ts,jsx,tsx,mdx}',
    './components/**/*..{js,ts,jsx,tsx,mdx}',
  ],
  theme: { extend: {} },
  plugins: [],
};
export default config;
```

The only key that matters here is **content**. It tells Tailwind where to scan for class names so it can generate only the CSS you actually use. If you have `className="bg-red-500"` in `components/Foo.tsx`, Tailwind sees it during the build and includes `bg-red-500` in the output CSS. If you don't reference a class anywhere, it's dropped.

This is the **purge / tree-shaking** that makes Tailwind's "thousands of utilities" not produce a 50MB CSS bundle. The output is usually under 30KB.

Note that we *don't* include `_source/**/*.{js,ts,jsx,tsx,mdx}` in content. The `.jsx` reference file lives there, but we don't want its classes (which we may have changed) leaking into the build.

Step 5: postcss.config.mjs

```
const config = {
  plugins: { tailwindcss: {}, autoprefixer: {} },
};
export default config;
```

PostCSS is the CSS processor that runs Tailwind and adds vendor prefixes (e.g. `-webkit-` for older browsers). You'll never touch this file again after creating it.

Step 6: .eslintrc.json

```
{
  "extends": ["next/core-web-vitals"],
  "rules": {
    "react/no-unescaped-entities": "off"
  }
}
```

Two things:

- **"extends": ["next/core-web-vitals"]** — pulls in Next.js's recommended ESLint rules, which include "Core Web Vitals" performance hints (e.g. "use next/image instead of ").
- **"react/no-unescaped-entities": "off"** — disable the rule that complains about apostrophes (don't) and quotes inside JSX. This rule is more annoying than helpful in practice; we turn it off.

Step 7: .prettierrc

```
{
  "semi": true,
  "singleQuote": true,
  "trailingComma": "es5",
  "printWidth": 100
}
```

Prettier's job is to **end every code-style argument** in your team. With this config:

- Semicolons at the end of statements (;).
- Single quotes for strings ('hello' not "hello").
- Trailing commas where ES5 allows (objects, arrays — not function args).
- Lines wrapped at 100 chars.

Pick a config, commit it, never argue about it again. **Prettier is a productivity multiplier specifically because it removes pointless decisions.**

Step 8: .gitignore

```
node_modules/  
.next/  
out/  
.DS_Store  
*.log  
.env*.local
```

What we **don't** want in git:

- `node_modules/` — these are installed via `npm install`. Massive. ~400MB.
- `.next/` — Next.js's build cache. Regenerated on every build.
- `out/` — static export output, if you use `next export`.
- `.DS_Store` — macOS folder metadata. (You might not need this on Windows, but it's harmless.)
- `*.log` — log files.
- `.env*.local` — local secrets.

Never commit `node_modules`. It's the most common rookie mistake — and it bloats the repo to gigabytes.

Step 9: The `app/` directory (Next.js App Router)

Next.js 13+ uses **file-based routing**: every folder under `app/` becomes a URL route. The mapping is:

```
app/page.tsx      → /  
app/about/page.tsx → /about  
app/blog/[slug]/page.tsx → /blog/anything
```

We have only one route (`/`), so we need:

```
app/  
  layout.tsx    ← wraps every page with the HTML shell  
  page.tsx      ← the home page  
  globals.css   ← global styles
```

app/layout.tsx

```
import type { Metadata } from 'next';
import { Roboto, Roboto_Slab } from 'next/font/google';
import './globals.css';

const roboto = Roboto({
  subsets: ['latin'],
  weight: ['300', '400', '500', '700', '900'],
  variable: '--font-roboto',
  display: 'swap',
});

const robotoSlab = Roboto_Slab({
  subsets: ['latin'],
  weight: ['600', '700', '900'],
  variable: '--font-roboto-slab',
  display: 'swap',
});

export const metadata: Metadata = {
  title: 'Snooker Fantasy League · Crucible 2026',
  description: 'Fantasy league standings for the 2026 World Snooker Champion',
};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en" className={` ${roboto.variable} ${robotoSlab.variable}`} >
      <body>{children}</body>
    </html>
  );
}
```

Three big concepts here:

1. This is a server component (no 'use client' directive). Server components are the App Router's default. They render to HTML on the server (or at build time for static routes), ship zero JavaScript to the browser, and can await server-side data directly.

A server component **cannot use** `useState`, `useEffect`, event handlers like `onClick`, or any browser-only API. If you need any of those, you mark a file with 'use client' at the top and React knows to ship it as JS.

This `layout.tsx` doesn't need any of those, so server component is correct.

```
const roboto = Roboto({ subsets: ['latin'], weight: [...], variable: '--font-
```

2. next/font/google for fonts This downloads the Roboto and Roboto Slab fonts at build time, hosts them on your own server, and exposes them as a CSS variable. **Your fonts never come from Google's servers in production.** That's a privacy + performance win:

- No request to `fonts.googleapis.com` (which can leak user data).
- The font is served from the same domain as your HTML, so there's one less DNS lookup.
- Next.js automatically applies `font-display: swap` and other best practices.

The CSS variable is set on `<html>` via `className={roboto.variable}` and you can use it from CSS (`font-family: var(--font-roboto)`).

```
export const metadata: Metadata = { title: '...', description: '...' };
```

3. The metadata export Next.js reads this and produces the corresponding `<title>` and `<meta name="description">` tags in the HTML head. **No need to manage <head> manually**, no need for `react-helmet`. Just export an object.

This is the "Metadata API" introduced in Next 13, and it's a quiet superpower of the App Router. SEO improvements get easier the more you lean on it.

app/page.tsx

```
import SnookerFantasyLeague from '@components/SnookerFantasyLeague';

export default function Page() {
  return <SnookerFantasyLeague />;
}
```

That's the entire home page. It just renders the orchestrator component. Why?

Because `app/page.tsx` is a **route component** — it's where Next.js looks when someone visits `/`. We want our actual application logic to live in `components/`, not under `app/`. The `app/page.tsx` is just a thin wrapper.

This is the **routing layer is separate from the app layer** pattern. Senior projects keep `app/` minimal and put logic in `components/`. Easier to test, easier to extract.

The `@components/SnookerFantasyLeague` import uses our path alias from `tsconfig.json`. Without the alias, this would be `../../components/SnookerFantasyLeague` or worse.

`app/globals.css`

```
@tailwind base;
@tailwind components;
@tailwind utilities;

:root {
  --background: #fff8e7;
  --foreground: #1f2937;
}

html, body {
  padding: 0;
  margin: 0;
  background: var(--background);
  color: var(--foreground);
  font-family: var(--font-roboto), system-ui, sans-serif;
}
```



```
* {  
  box-sizing: border-box;  
}  
  
button {  
  font-family: inherit;  
}
```

Three Tailwind directives plus some basic resets and a CSS variable theme. The most important line is `font-family: var(--font-roboto), ...` — that’s where the font we set up in `layout.tsx` actually gets applied.

`box-sizing: border-box` on every element means **padding and border are included in width/height calculations**. Otherwise `width: 100px; padding: 10px` produces a 120px-wide element, which is maddening. Always include this.

Step 10: Install dependencies

```
npm install
```

This reads `package.json`, downloads all dependencies, and writes a `package-lock.json` that pins exact versions. First install on a fresh project takes a few minutes.

You’ll see warnings about deprecated transitive dependencies — those are mostly fine for an app project. (Library authors care more.) The build will work.

Step 11: Run the dev server

```
npm run dev
```

Open `http://localhost:3000`. You should see — at this point — an error, because `SnookerFantasyLeague.tsx` doesn’t exist yet. That’s fine. **The error message is your friend** — it confirms the routing is working, the build pipeline is running, and TypeScript is checking.

If instead you see a “page not found” or the dev server failed to start, scroll back through this lesson and verify:

- Every config file is named correctly (.mjs vs .js matters).
- package.json has all five dependencies.
- app/layout.tsx has the right structure.
- You're in the right folder.

What create-next-app would have done for you

You just hand-built what create-next-app does in 30 seconds. Was it worth it?

For learning, **yes**. You now know:

- What every config file is.
- Why tsconfig.json has paths, what strict does, what noEmit means.
- What app/layout.tsx is for.
- What server components are (the default!).
- Why the dev server starts where it does.

For a real project, **use create-next-app** and get back to building. But knowing what it did frees you to *modify* what it did. Most beginners are afraid to touch their tsconfig.json because they don't know what it does. You now do.

Vibe prompt you would have used

If you skipped the manual scaffold:

"Set up a new Next.js 14 project at the root of an empty folder. App Router, TypeScript strict mode, Tailwind CSS (set up but I'll mostly use inline styles), Prettier with single quotes / trailing commas, ESLint with the next/core-web-vitals preset. Add Lucide-react and recharts as dependencies. The app has one route (/) that renders <SnookerFantasyLeague /> from @/components. Configure next/font/google for Roboto and Roboto Slab and apply them via CSS variables. Don't generate any actual app components yet — just the scaffold."

Specific, scoped, names every tool. Notice "Don't generate any actual app components yet" — this is the same trick from Chapter 1 Lesson 1. Get the LLM to stop at scaffolding.

CHECK YOURSELF

1. **Why is next pinned to a specific version while react uses ^18.3.1?** What's the difference between pinned, caret (^), and tilde (~) version specifiers?
2. **You add a `console.log` to `app/page.tsx` and reload — where does the log appear, in your browser console or your terminal? Why?**
3. **A teammate adds `import { useState } from 'react'` to `app/layout.tsx` and gets an error. Why? What's the fix?**
4. **`tsconfig.json` has `"paths": { "@/*": ["./*"] }`. What do you change to add a second alias `~/lib/*` mapping to `./lib/*`?**
5. **`reactStrictMode: true` causes some effects to run twice in dev. Why does this help catch bugs?** (Hint: it's about side effects in render or in `useEffect` cleanup.)

When you're ready, head to [03-creating-data-files.md](#) — the boring but essential lesson where we type out all the match data.

Chapter 2 · Lesson 3 — Creating the Data Files

Goal: fill out `lib/matches.ts`, `lib/teams.ts`, `lib/players.ts`, and `lib/constants.ts`. By the end of this lesson, the data layer is complete and `calculateTeamScores(team)` produces real numbers.

Why this lesson is “boring” and why that’s a feature

You're about to spend an hour typing match results, team picks, and player metadata. There's no React in this lesson. No JSX. No clever logic. Just data.

This is **on purpose**. Two reasons:

1. Real apps spend a lot of time on data plumbing.

In an interview, you'll be asked to build a feature in 45 minutes. **A third of that time is going to be typing fixture data** so you have something to render. Practicing it on a real, larger-than-toy dataset prepares you for that.

2. Typing the data forces you to feel the invariants.

When you write a Team object and have to lay out its 16 R1 picks in the same order as the 16 matches in ROUND1_MATCHES — you internalize Lesson 1.2's invariant in your fingers. That's worth more than reading about it three times.

If you really want to skip the typing, copy the files from the repo. **But read every section even if you copy.** The commentary explains *why* each piece looks the way it does.

The four files we'll build

```
lib/
├── matches.ts      ← all 31 matches across 5 rounds
├── teams.ts        ← all 8 fantasy teams with their picks
├── players.ts      ← player metadata dictionary
└── constants.ts    ← shared style helpers and ball colors
```

These are the **inputs** to the scoring function from Chapter 1 Lesson 3. Without them, calculateTeamScores has nothing to crunch.

File 1: lib/matches.ts

Open lib/matches.ts in the repo. The file starts like this:

```
import type { Match } from './types';

export const ROUND1_MATCHES: Match[] = [
  { id: 1, p1: 'Zhao Xintong', p2: 'Liam Highfield', winner: 'Zhao Xintong' },
  { id: 2, p1: 'Judd Trump', p2: 'Gary Wilson', winner: 'Judd Trump' },
  // ... 14 more
];
```

Walk through the structure:

Why one type import at the top

```
import type { Match } from './types';
```

The `import type` keyword is a TypeScript-only thing. It says “I’m only importing this for type-checking; erase this import in the compiled JS.” The runtime never needs `Match` (it’s just a shape, not a value), so erasing it produces smaller bundles.

If you wrote `import { Match }` instead, the compiler would *also* erase it because it’s an interface — but `import type` makes the intent explicit. Senior projects use `import type` consistently.

```
export const ROUND1_MATCHES: Match[]
```

- **export** so other files (`lib/scoring.ts`, components) can import it.
- **const** because we never reassign the array. (We could mutate the array’s contents, but TypeScript’s readonly modifiers can prevent that too — we don’t bother for an internal data file.)
- **: Match[]** is the type annotation: an array of `Match` objects. **TypeScript will complain** if you forget a required field on any item, or add a typo, or pass a number where a string is expected. That’s the whole point.

The objects themselves

```
{ id: 1, p1: 'Zhao Xintong', p2: 'Liam Highfield', winner: 'Zhao Xintong'
```

Six fields: `id`, `p1`, `p2`, `winner`, `score`, `seed1`. All required for finished R1 matches. Matches the `Match` interface exactly.

The fields are aligned vertically with whitespace for readability — that’s a style choice, not a requirement. Prettier will collapse them on save unless your `printWidth` is wide enough; the codebase uses 100 chars and the alignment fits comfortably.

Pending matches look different

By the end of the tournament every match has a winner. But during the tournament, before any QF matches were finished, the `QF_MATCHES` array would have looked like this:

```
{ id: 1, p1: 'Zhao Xintong', p2: 'Mark Allen' }, // no winner, no score yet
```

Two fields instead of five. **Optional fields can simply be left out.** This is the “absent vs zero” distinction from Lesson 1, expressed in data.

In the *current* state of the codebase, every match has been played and every winner is filled in. But you can imagine going back to a pre-Final state where `FINAL_MATCH[0]` was just `{ id: 1, p1: 'Shaun Murphy', p2: 'Wu Yize' }` — no winner, no score. The scoring function would correctly return `null` for everyone's Final pick until the field is filled.

The four other rounds

```
export const ROUND2_MATCHES: Match[] = [ /* 8 entries */ ];
export const QF_MATCHES: Match[]     = [ /* 4 entries */ ];
export const SF_MATCHES: Match[]     = [ /* 2 entries */ ];
export const FINAL_MATCH: Match[]    = [ /* 1 entry  */ ];
```

Same shape, fewer entries. Note `FINAL_MATCH` is still an **array** even though it has just one element. Why?

Because consistency with the other rounds is more valuable than the trivial savings of `export const FINAL_MATCH: Match`. Code that does `.map()` over rounds can treat them all uniformly:

```
[ROUND1_MATCHES, ROUND2_MATCHES, QF_MATCHES, SF_MATCHES, FINAL_MATCH].forEach(
  round => { /* ... */ }
);
```

If `FINAL_MATCH` were a single `Match` object, the loop above would crash. **Symmetric data shapes simplify the code that reads them.**

How the file ends

The full file is about 90 lines. It exports five constants. The order is **R1 first, Final last** — chronological order from the user's point of view, not bracket-evaluation order. The reader's brain is happier in chronological order.

File 2: `lib/teams.ts`

This is the longest file in the project. Opening it in the repo:

```

import type { Team } from './types';

export const TEAMS: Team[] = [
  {
    name: 'Invincibles',
    color: '#0F5132',
    accent: '#10B981',
    icon: '🏆',
    motto: 'Unstoppable & Unbeatable',
    r1: ['Zhao Xintong', 'Judd Trump', 'John Higgins', 'Mark Williams'], /* ... */
    r2: ['Zhao Xintong', 'Mark Williams'], /* ... */
    qf: ['Zhao Xintong', 'Mark Allen', 'John Higgins', 'Wu Yize'],
    sf: ['Shaun Murphy', 'Wu Yize'],
    final: 'Wu Yize',
  },
  {
    name: 'Uncredibles',
    color: '#DC2626',
    accent: '#F87171',
    // ...
  },
  // ... 6 more teams
];

```

What's in each team

- **Identity:** name, color, accent, icon, motto. The color and accent are the team's primary brand pair, used for gradients in headers and badges.
- **Picks:** r1 (16 strings), r2 (8 strings), qf (4 strings), sf (2 strings), final (1 string).

That's it.

How to type 16 picks without going insane

Realistically you do this with a printout of the bracket on one side and your editor on the other. For each match in ROUND1_MATCHES, ask the team owner “*who do you pick?*” and

write the name in the corresponding slot.

The order is **the same as ROUND1_MATCHES**. So `r1[0]` is the team's pick for `ROUND1_MATCHES[0]` (which is the Zhao vs Highfield match — so the answer is either 'Zhao Xintong' or 'Liam Highfield').

When you're typing the file, having both files open side-by-side in split view is essential. **Visual alignment between two files is the real-world enforcement of the invariant.**

Color choices

Each team has a primary color and an accent. The convention used in this codebase:

- **color** is the darker / saturated shade.
- **accent** is the lighter shade.

They're paired in CSS gradients like:

```
background: `linear-gradient(135deg, ${team.color} 0%, ${team.accent} 100%)`
```

That produces a smooth gradient from dark to light along a 135° diagonal. The visual richness of the team headers comes entirely from this two-color recipe.

When choosing colors:

- Don't use HSL math to pick the accent. Use a tool like Tailwind's color palette (emerald-700 paired with emerald-400, etc.) or coolers.co.
- Avoid pure red/green/blue — they look childish. Stick with curated palettes.
- Make sure the contrast is high enough that white text reads cleanly on top.

Two optional fields you might see

Recall from Lesson 1.2 that Team has two optional helper arrays: `r1Players?` and `r2Players?`. You'll see them on a few teams. They're only used in edge cases for scoring. Most of the eight teams will not have these arrays. Don't worry about them yet; you'll meet them again in Chapter 4 if at all.

File 3: `lib/players.ts`

```
import type { PlayerInfo } from './types';
```



```
export const PLAYER_INFO: Record<string, PlayerInfo> = {
  'Zhao Xintong': { country: '🇨🇳', seed: 1, status: 'Defending Champion', f
  'Judd Trump': { country: '🇬🇧', seed: 2, status: 'World No. 1',
  'Mark Allen': { country: '🇬🇧', seed: 3, status: 'Top Tier',
  // ... 29 more
};
```

This is the **dictionary** we discussed in Lesson 1.2 — keyed by player name, value is meta-data.

Record<string, PlayerInfo> explained

Record<K, V> is TypeScript shorthand for “an object whose keys are of type K and values are of type V.” Record<string, PlayerInfo> means “any string key is allowed; every value must be a PlayerInfo.”

It’s **identical** to { [key: string]: PlayerInfo }. Both forms work. Record reads cleaner.

Country flag emojis

Yes, you can put flag emojis directly in TypeScript strings. UTF-8 encoding handles them natively. Just paste them in:

```
'🇨🇳' // China
'🏴' // Black flag – hopefully not for England
'🏴󠁧󠁢󠁥󠁮󠁧󠁿' // Actual England flag (regional emoji, made of multiple code points)
```

Note: the codebase uses 🏴 (a generic black flag) for English players because the regional emoji 🏴󠁧󠁢󠁥󠁮󠁧󠁿 doesn’t render reliably across browsers/OSes. **Pick what renders** — don’t pick what’s “technically correct” if it looks broken on most devices.

Seeds: top-16 only

```
'Liam Highfield': { country: '🇬🇧', status: 'Qualifier', flag: 'ENG' }, // no
```

Of the 32 players, the top 16 are seeded (1 through 16). The other 16 are qualifiers and have no seed field. The Player Info interface marks seed?: number as optional

precisely for this case.

Why a dictionary, not an array

Already covered in Lesson 1.2. The most common operation is “given a player’s name, get their metadata” – that’s an $O(1)$ hash lookup with a dictionary, or an $O(n)$ `.find()` with an array. The dictionary wins on both performance and readability.

File 4: `lib/constants.ts`

This is the smallest file but pulls its weight. Open it:

```
// lib/constants.ts

export const BALL_COLORS = [
  '#FFFFFF', // white
  '#FBBF24', // yellow
  '#16A34A', // green
  '#92400E', // brown
  '#3B82F6', // blue
  '#EC4899', // pink
  '#000000', // black
];

export const th: React.CSSProperties = {
  padding: '14px 16px',
  textAlign: 'left',
  fontSize: 13,
  fontWeight: 800,
  letterSpacing: '1px',
};

export const td: React.CSSProperties = {
  padding: '12px 8px',
  borderBottom: '1px solid #F3F4F6',
  fontSize: 15,
};
```

```
export function tabStyle(active: boolean): React.CSSProperties {
  return {
    padding: '12px 22px',
    border: 'none',
    cursor: 'pointer',
    borderRadius: 10,
    fontFamily: 'inherit',
    fontWeight: 700,
    fontSize: 15,
    background: active ? '#0F5132' : '#F3F4F6',
    color: active ? '#FBBF24' : '#374151',
    transition: 'all 0.2s',
  };
}
```

Three exports:

BALL_COLORS — the seven snooker ball colors

```
export const BALL_COLORS = ['#FFFFFF', '#FBBF24', '#16A34A', '#92400E', '#3B5998', '#F8766D', '#4F81BD']
```

Used in the hero header to render seven decorative dots in the top-right corner. Pure aesthetic. Lives in `constants.ts` because (1) the colors are a reusable palette and (2) extracting them out of the orchestrator keeps that file readable.

th and td — shared table cell styles

```
export const th: React.CSSProperties = { padding: '14px 16px', /* ... */ };
export const td: React.CSSProperties = { padding: '12px 8px', /* ... */ };
```

These are used by the standings table. They're typed as `React.CSSProperties` — the type that React expects for `inline style={...}` props. By centralizing them here, every place that renders a table cell can pull from the same source:

```
import { th, td } from '@/lib/constants';
```

```
<th style={th}>Name</th>
<td style={td}>Wu Yize</td>
```

tabStyle — a function that returns a style object

```
export function tabStyle(active: boolean): React.CSSProperties { ... }
```

This is a **style helper** — a function that takes a piece of state and returns a style object. The argument decides which colors come back. Useful when many places need the same “tab pill” look but with different active states.

This is what experienced React engineers do *before* reaching for a CSS-in-JS library. **A function returning a style object is the simplest possible style abstraction.** It’s also TypeScript-friendly.

Sanity-checking with a quick console smoke test

Once all four files are in place, you can verify your data by adding a temporary `console.log` in `lib/scoring.ts`:

```
import { TEAMS } from './teams';

const test = TEAMS.map(t => ({ name: t.name, scores: calculateTeamScores(t) })
console.log(test.sort((a, b) => b.scores.total - a.scores.total));
```

This won’t run in the browser yet (we don’t have a UI), but if you boot a Node REPL or write a quick `node test.ts` script with `tsx`, you should see the eight teams sorted by total score with sensible numbers — not all zeros, not all NaN, not all undefined.

If the numbers look wrong:

- **All zeros?** Probably the match arrays don’t have winner filled in, or the picks don’t match player names exactly (typo).
- **NaN total?** Probably a `pts` value somewhere is undefined; check `scorePick` is reaching all branches.
- **Score is correct but unsorted?** That’s fine — sorting happens in the orchestrator (Chapter 4).

Remove the `console.log` after you’ve sanity-checked. Production code shouldn’t `console.log`.

What you’ve built so far

Three lessons in. Your `lib/` folder is complete:

```
lib/
├─ types.ts           ← (Chapter 1.2)
├─ scoring.ts        ← (Chapter 1.3)
├─ matches.ts        ← (this lesson)
├─ teams.ts          ← (this lesson)
├─ players.ts        ← (this lesson)
└─ constants.ts      ← (this lesson)
```

That’s the **entire data layer** of the app. **Zero React**. Zero JSX. Zero DOM. If you handed this folder to someone using a different UI framework — Vue, Svelte, Solid — they could build the same app on top of it without changing a line.

This is the property you want from your data layer. **Framework-independent business logic**. When the React replacement comes along in 2030, your `lib/` folder migrates unchanged.

Vibe prompt you would have used

“Create `lib/matches.ts`, `lib/teams.ts`, and `lib/players.ts` with TypeScript constants. The data is: [paste the bracket and team picks here]. Each match is { `id`, `p1`, `p2`, `winner?`, `score?`, `seed1?` }. Each team has 5 picks arrays in match-index order: `r1[16]`, `r2[8]`, `qf[4]`, `sf[2]`, `final` (single string). `PLAYER_INFO` is a `Record<string, PlayerInfo>` keyed by player name. Use `import type` for the `Match/Team/PlayerInfo` types from `./types`. Do not generate any UI.”

Notice the constraint: “Do not generate any UI.” For data-only tasks, you say so explicitly. **Without that constraint, the LLM will helpfully make you a `<MatchTable>` component you didn’t ask for.**

CHECK YOURSELF

1. **Why is `import type` used instead of `import` for the type imports?** What's actually different in the compiled output?
2. **`FINAL_MATCH` is an array of one element instead of a single `Match` object. Justify the choice.** Would symmetry alone be enough? Are there other reasons?
3. **Of the eight teams, three have ONE typo in their `r1` picks (e.g. `'Zhao Xintong '` with a trailing space).** Walk through what would happen at scoring time. Would the bug be visible immediately? When and how?
4. **A new player joins the bracket as a 17th seed (replacing a withdrawal).** What's the minimum set of files you have to update? Walk through the change.
5. **`PLAYER_INFO` uses `Record<string, PlayerInfo>`. The other type form is `{ [key: string]: PlayerInfo }`. They're equivalent. Pick one and explain to a junior dev why you'd prefer it.**

When you've answered these, you've finished Chapter 2. Onward to [Chapter 3 – Your First Component](#), where we **finally** import React.

Chapter 2 – Project Setup

From an empty folder to a running dev server. No React yet — but by the end you'll have a Next.js + TypeScript + Tailwind app booting at `http://localhost:3000`, with your `lib/` folder full of typed data and pure scoring logic.

What you'll learn

- **Why Next.js** is the right framework for this app (and when it would be the wrong one).
- **What every config file does** — `package.json`, `tsconfig.json`, `tailwind.config.ts`, `next.config.mjs`, `.eslintrc.json`. No more "I just copy-paste these from someone."
- The Next.js **App Router** mental model: file-based routing, `app/` directory layout, what `layout.tsx` is, what `page.tsx` is.
- How to **transcribe data into TypeScript files** without losing your mind to bracket-matching errors.

- The npm install / npm run dev workflow, and what to do when something breaks.

Lessons in this chapter

1. **01-why-nextjs-typescript.md** — The framework decision tree. Next.js vs Vite vs Create-React-App vs plain HTML. Why we picked what we picked, and what would change if the requirements were different.
2. **02-scaffolding-the-project.md** — Run-by-run walkthrough of creating the project from scratch. Every file. Every line. Every config option, explained.
3. **03-creating-data-files.md** — Filling out lib/matches.ts, lib/teams.ts, lib/players.ts with real data. The boring lesson where you'll spend the most keystrokes.

Files you'll create in this chapter

package.json	← dependencies
package-lock.json	← (generated by npm)
tsconfig.json	← TypeScript compiler config
next.config.mjs	← Next.js config
tailwind.config.ts	← Tailwind theme config
postcss.config.mjs	← PostCSS pipeline (used by Tailwind)
.eslintrc.json	← ESLint rules
.prettierrc	← Prettier formatting rules
.gitignore	← what git ignores
README.md	← top-level project README
app/	
layout.tsx	← root HTML wrapper (server component)
page.tsx	← the home route
globals.css	← Tailwind directives + global styles
lib/	
types.ts	← (Chapter 1 Lesson 2)
scoring.ts	← (Chapter 1 Lesson 3)
matches.ts	← all 31 match objects
teams.ts	← all 8 team objects with picks
players.ts	← PLAYER_INFO dictionary
constants.ts	← shared style helpers + ball colors

New React/Next.js concepts introduced

- **App Router** — Next.js 13+ file-based routing system.
- **Server components** (the default) — components that render on the server, no JavaScript shipped to the browser.
- **'use client' directive** — opt a single file into client-side rendering. Foreshadowed here, used in earnest in Chapter 4.
- **TypeScript path aliases** (`@/components/...`) — clean imports without `../../../../`.
- **Tailwind CSS** — utility-first styling. We won't use it heavily (the codebase uses inline styles), but we set it up.

How long it should take

- 30 min reading per lesson
- 1.5–2 hours typing the data into Lesson 3 (the data is real, the file is long)
- ~3.5 hours total

Before you start

You need:

- Node.js 20+ installed (`node --version`)
- A terminal (Windows users: PowerShell is fine, the lessons assume PowerShell commands)
- ~30 minutes for `npm install` to finish on a fresh project (it's about 400 packages)

If you're following along with this repo as the reference, you can `cd` into a *new* empty folder and rebuild the project there, then `diff` against this repo's state. Or just read along and sanity-check against the live files in `package.json`, `app/`, `lib/`.

Open [01-why-nextjs-typescript.md](#) when ready.

Chapter 3 · Lesson 1 — Anatomy of a Component

Goal: by the end of this lesson, you can look at any React component file and identify every part of it — the function, the props, the JSX, the return, the imports — and explain what each one does to a junior developer.

What is a “component”, really?

In React, a component is **just a function**. Specifically, a function that:

1. Takes a single argument called props (an object with whatever data the parent passes in).
2. Returns either a piece of JSX (looks like HTML), a string, a number, an array, or null.

That’s it. There’s no class, no constructor, no extends, no decorators. **A component is a function**. If you can write a function, you can write a React component.

Here’s the smallest possible React component:

```
function Hello() {  
  return <h1>Hello, world.</h1>;  
}
```

Six tokens. One return. That’s a complete, valid, renderable React component. You could put it on a page right now.

Functional vs class components — the 2026 answer

If you’ve read any React tutorial older than 2020, you’ve probably seen `class MyComponent extends React.Component { render() { ... } }`. That’s the **class component** style. It still works, but **you should never write a new class component in 2026**.

The React team has explicitly said functional components + hooks are the path forward. Class components are a legacy style that exists for backwards compatibility. Every new component in this course is a function.

If an interviewer asks you about class components, the answer is: *“I know they exist, I can read them, I’d refactor to functional + hooks if I owned the code.”* That’s enough.

What is JSX?

The thing that looks like HTML inside a component is **JSX** — JavaScript XML. It’s a syntax extension to JavaScript that lets you write tag-like markup directly in your code.

```
return <h1>Hello, world.</h1>;
```

That `<h1>...</h1>` is JSX. It is **not HTML**. It looks like HTML, but it's actually compiled into a JavaScript function call. Specifically, a Next.js / React build pipeline transforms the line above into:

```
return React.createElement('h1', null, 'Hello, world.');
```

`React.createElement(type, props, ...children)` is what JSX is “really” doing under the hood. The compiler does this for you so you can write the more readable JSX form. **Knowing this saves you from a dozen confusing error messages later.**

Why does this matter?

Three implications fall out of “JSX is just function calls”:

1. JSX returns a value. Because it compiles to `React.createElement(...)`, JSX produces a **JavaScript object** (a “React element”) at runtime. You can assign it to a variable:

```
const heading = <h1>Hello</h1>; // heading is an object, not HTML
return <div>{heading}</div>;    // and you can embed it inside more JSX
```

That's why you can store JSX in a variable, return it from a function, pass it as a prop, etc. **JSX is data, not markup.**

2. You can use JavaScript inside JSX with {}. Curly braces inside JSX let you embed any JavaScript expression:

```
const name = 'Wu Yize';
return <h1>Hello, {name}!</h1>;           // "Hello, Wu Yize!"
return <p>{1 + 2}</p>;                     // "3"
return <p>{teams.map(t => <span key={t.name}>{t.name}</span>)}</p>; // a li
```

Anywhere a JS expression is valid, {} lets you drop one in. Statements (like `if` and `for`) are not expressions, so they don't fit; you'll see ternaries (`condition ? a : b`) and `.map()` calls instead.

3. Naming and capitalization matter. JSX distinguishes between **tags** and **components** by the first letter:

```
<h1>...</h1>      // lowercase → HTML element
<Hello />           // uppercase → React component (your function)
```

Compiled output:

```
React.createElement('h1', null, ...)    // string type → DOM element
React.createElement(Hello, null)         // function type → React component
```

If you accidentally name your component function `hello()` instead of function `Hello()`, JSX will treat `<hello />` as the HTML tag `<hello>` (which doesn't exist) and your component will silently not render. **Always capitalize component names.**

The full anatomy: a real component, line by line

Open `components/standings/StatCard.tsx` from the repo. It's a small but complete component:

```
import type { LucideIcon } from 'lucide-react';

interface StatCardProps {
  icon: LucideIcon;
  label: string;
  value: string | number;
  color: string;
  bgColor: string;
}

export default function StatCard({ icon: Icon, label, value, color, bgColor }) {
  return (
    <div
      style={{
        background: `linear-gradient(135deg, ${bgColor} 0%, ${bgColor}DD 100%`,
        color: '#FFFFFF',
        padding: 24,
        borderRadius: 16,
        boxShadow: `0 8px 24px ${bgColor}40`,
      }}
    >
```

```

    >
    <Icon size={28} strokeWidth={2.5} />
    <div style={{ fontSize: 13, letterSpacing: '1.5px', fontWeight: 600, o
      {label}
    </div>
    <div style={{ fontSize: 32, fontWeight: 900, fontFamily: "'Roboto Slab
      {value}
    </div>
  </div>
);
}

```

Let's dissect it from top to bottom.

Line 1: imports

```
import type { LucideIcon } from 'lucide-react';
```

This component imports a TypeScript **type** (not a value) from the `lucide-react` package. `LucideIcon` is the type of any icon component (Trophy, Calendar, Star, etc.). The actual icon will be passed in as a prop — this file doesn't pick a specific one.

`import type` (vs plain `import`) tells TypeScript “this is type-only; erase from runtime output.” Senior projects use it consistently.

Lines 3–9: the props interface

```

interface StatCardProps {
  icon: LucideIcon;
  label: string;
  value: string | number;
  color: string;
  bgColor: string;
}

```

This is the **props contract** for the component. Five fields:

- `icon: LucideIcon` — the Lucide icon component to render (e.g. Trophy, Star).

- `label: string` – the text under the icon (“LEADER”, “MATCHES”, etc.).
- `value: string | number` – the big number/text in the middle. **Union type** – accepts either.
- `color: string` – primary accent color (used in some variants).
- `bgColor: string` – the gradient background base.

By declaring this interface, **the component documents its API**. Any caller using this component must pass props matching this shape, or TypeScript yells at them. Conversely, the *inside* of the component knows exactly what it has to work with.

The convention `<ComponentName>Props` for the interface name is universal in the React community. Stick with it.

Line 11: the function declaration

```
export default function StatCard({ icon: Icon, label, value, color, bgColor
```

Several things going on here:

export default Means *“this is the main export of the file, and a caller can import StatCard from './StatCard' without curly braces.”*

Compare to `export function StatCard()` (a “named export”) which would require `import { StatCard } from './StatCard'` with braces.

Convention in this codebase (and most React projects): **one component per file, exported as default**. The reason is consistency with import syntax – every file you import a component from looks the same. There are religious wars about default vs named exports; pick one and don’t mix.

```
function StatCard({ icon: Icon, label, value, color, bgColor }: StatCardProps
```

Destructuring the props This is JavaScript object destructuring with a TypeScript annotation. It’s equivalent to:

```
function StatCard(props: StatCardProps) {
  const Icon = props.icon;
  const label = props.label;
```

```
// ...  
}
```

But shorter and more idiomatic.

The `icon: Icon` part is a **rename** — we’re pulling out the prop called `icon` and giving it the local name `Icon` (capitalized). Why? Because we’re going to render it as a JSX tag (`<Icon ... />`), and JSX needs uppercase to recognize it as a component. **Without the rename, `<icon />` would compile to a non-existent HTML element.**

This is a small but classic React quirk. Whenever you accept a component as a prop, capitalize it locally.

Lines 12–32: the return — JSX

```
return (  
  <div style={{ ... }}>  
    <Icon size={28} strokeWidth={2.5} />  
    <div style={{ ... }}>{label}</div>  
    <div style={{ ... }}>{value}</div>  
  </div>  
);
```

Three children of a `<div>`:

1. The icon component (e.g. `<Trophy size={28} />`).
2. A `<div>` containing the label text.
3. A `<div>` containing the value (number or string).

Notice the **return (...)** with parens around the JSX. Why?

When JSX spans multiple lines, JavaScript’s automatic semicolon insertion (ASI) can confuse the parser. Wrapping the JSX in `( )` makes it unambiguous that this is one big expression. **Always wrap multi-line JSX in parens.** It’s not strictly required by the language; it’s required by every real codebase to avoid ASI bugs.

The inline style attribute

```
<div
  style={{
    background: `linear-gradient(135deg, ${bgColor} 0%, ${bgColor}DD 100%)`,
    color: '#FFFFFF',
    padding: 24,
    borderRadius: 16,
    boxShadow: `0 8px 24px ${bgColor}40`,
  }}
>
```

Three things happening:

Two layers of {} `style={{...}}` has TWO layers of braces. The outer pair is JSX's "embed an expression here." The inner pair is a JavaScript **object literal**. So it's actually `style={ + {...} + }` — the JSX expression `{...}` evaluates to an object literal that React assigns to the style attribute.

Beginners often forget the second pair and write `style={...}` — that's a syntax error.

CamelCase property names CSS uses kebab-case (`background-color`, `border-radius`). React inline styles use camelCase (`backgroundColor`, `borderRadius`). Why? Because JavaScript object keys can't have hyphens unless quoted, and unquoted keys are nicer to type.

Internally React converts `backgroundColor: 'red'` to `style.backgroundColor = 'red'` on the DOM element. The browser handles the rest.

```
padding: 24,           // OK — React adds 'px' for you
fontSize: 14,          // OK — same
zIndex: 50,            // OK — but no 'px' added (zIndex is unitless)
padding: '24px',       // also OK
padding: '14px 16px',  // string when you need multiple values
```

Numbers vs strings For most CSS properties that accept a length, React assumes pixels if you pass a number. Pass a string when you need a unit other than px (`'2rem'`, `'50%'`)

or multiple values.

Template literals for dynamic colors

```
background: `linear-gradient(135deg, ${bgColor} 0%, ${bgColor}DD 100%)`,
```

This uses a JS **template literal** (backticks + `${...}`). The expression `${bgColor}` is the value of the `bgColor` prop interpolated into the string.

The trailing `DD` is hex shorthand for ~87% opacity (`DD = 221 / 255`). So if `bgColor = '#0F5132'`, the gradient runs from solid `#0F5132` to `#0F5132DD` — same color, slightly transparent. This produces a smooth 1-color gradient that doesn't look flat.

This kind of **dynamic styling based on props** is exactly why we use inline styles instead of Tailwind classes. There's no Tailwind class for "this color, 87% opacity, with a 135° gradient" computed at runtime. Inline wins.

Self-closing tags

```
<Icon size={28} strokeWidth={2.5} />
```

Components with no children use **self-closing syntax**. Same rule as `<img />` and `<br />` in XHTML. A non-self-closing version like `<Icon size={28}></Icon>` would also work, but self-closing is shorter and more conventional for empty elements.

Recap: the four parts of every component file

You'll see this skeleton everywhere:

```
import { ... } from '...'; // 1. imports

interface FooProps {          // 2. props contract (TypeScript)
  bar: string;
}

export default function Foo({ bar }: FooProps) { // 3. function with destructured props
  return (                     // 4. return JSX
    <div>{bar}</div>
```



```
);  
}
```

That's it. **Every component you'll ever write follows this pattern.** Once you see it three times you'll see it everywhere.

Server vs client components — a teaser

You may have noticed: at the top of `components/standings/StatCard.tsx` there's no `'use client'` directive. Neither was there one on `app/layout.tsx`. So what kind of component is this — server or client?

Default is server in the App Router. A component is a server component unless something inside it (or its parent) has `'use client'` declared.

`StatCard` is a server component. So is `app/layout.tsx`. They render to HTML on the server, ship zero JavaScript to the browser. That's fine because they don't use any browser-only features.

`<SnookerFantasyLeague>` (the orchestrator) WILL need `'use client'` because it uses `useState`. Once a parent is marked client, all its descendants are also rendered client-side. **You only need `'use client'` at the boundary** — the highest component in your tree that needs interactivity. Everything below that boundary is automatically client.

Chapter 4 will introduce `'use client'` formally. For now, just know:

- Server components are the default.
- They're free (zero JS shipped to the browser).
- They cannot use `useState`, `useEffect`, `onClick`, `localStorage`, or any browser API.
- `'use client'` opts a file (and its descendants) into client rendering.

Vibe prompt you would have used

"Write me a `StatCard` component in `components/standings/StatCard.tsx`. Props: `icon: LucideIcon`, `label: string`, `value: string | number`, `color: string`, `bgColor: string`. Render a div with a gradient background using `bgColor` (linear-gradient 135deg from full to ~87% opacity), white text, 24px padding, 16px border radius, big shadow tinted with `bgColor`.

Inside: the icon at 28px stroke 2.5, then the label in small letterspaced text, then the value as a big serif heading. Inline styles only. TypeScript, default export.”

The trick is naming **every prop, every style detail, every visual choice**. When the spec is this specific, the LLM has nothing to invent. **Specificity in, predictability out.**

CHECK YOURSELF

1. **Why does the prop destructuring use `{ icon: Icon }` instead of just `{ icon }`?**
What would break if you used the lowercase form?
2. **`style={{ ... }}` has two layers of braces. Walk through what each layer does.**
What error would you get if you wrote `style={ ... }` (one layer)?
3. **`padding: 24` works, but `font-size: '14'` does not. Why?** What’s the difference between unitless numbers and string values in React style objects?
4. **A teammate writes `<icon size={28} />` and gets a console warning about an unknown HTML element. What’s wrong, and what’s the one-character fix?**
5. **`StatCard.tsx` doesn’t have `'use client'` at the top. Is it a server component or a client component? Why?** Could you use `useState` inside it as-is? Could a parent component with `'use client'` use `<StatCard />` inside it?

When you’ve answered these, head to [02-rendering-the-standings.md](#) — we build our first real, data-driven component.

Chapter 3 · Lesson 2 — Rendering the Standings

Goal: build the first real, data-driven React component in this app — `<StandingsTab>` v1. By the end of this lesson, you’ll see a sortable table of 8 fantasy teams in your browser, ranked by total points.

The smallest version that proves data flows

When you start a new feature in React, the first version should be **the smallest thing that proves the data is real**. For the standings, that’s just team names and totals — no styling, no icons, no medals, no per-round columns. Just rows and a number.

We’ll build that v1 first, see it work, then iterate. **This is the way.**

Why? Because if your first version has 200 lines of styling AND data binding AND sorting AND conditional rendering, and something doesn't work, you have to debug *all of it at once*. A 15-line v1 lets you find data-binding bugs in 5 seconds. Then you add styling on top, debug only the styling, and move on. **One concern at a time.**

Step 1: stub the orchestrator

Before the standings tab can render, the page needs *something* to render. Create `components/SnookerFantasyLeague.tsx` with a stub:

```
'use client';

import StandingsTab from '../tabs/StandingsTab';

export default function SnookerFantasyLeague() {
  return (
    <div style={{ padding: 32, fontFamily: 'sans-serif' }}>
      <h1>Snooker Fantasy League</h1>
      <StandingsTab />
    </div>
  );
}
```

Five things to notice:

'use client' at the top — finally

This is the directive we've been foreshadowing. Putting `'use client'` as the first line of a file marks the file (and everything it imports that's not also marked server) as a **client component**. That means it gets shipped to the browser as JavaScript and can use `useState`, `useEffect`, event handlers, etc.

Why does `<SnookerFantasyLeague>` need to be a client component? Because in the next chapter we'll add `useState` to track which tab is active. State requires client-side hydration. Pre-marking it now saves a refactor later.

Does this make EVERYTHING client-side now? Yes — but only the `SnookerFantasyLeague` subtree, which is most of the app. The `app/layout.tsx` and `app/page.tsx`

are still server components and still render to HTML on the server. That HTML wraps the client JS bundle. So the page is still **statically prerenderable** (no real-time data) and benefits from server rendering for SEO, while still allowing client-side interactivity.

This is the App Router's superpower: **mix server and client at any boundary**. The boundary here is `<SnookerFantasyLeague>`. Above it (in `layout.tsx` and `page.tsx`) — server. Inside it — client.

Importing from a sub-path

```
import StandingsTab from './tabs/StandingsTab';
```

The relative import `./tabs/StandingsTab` works because we're inside `components/` and `StandingsTab.tsx` is at `components/tabs/StandingsTab.tsx`.

We *could* also use the path alias: `import StandingsTab from '@/components/tabs/StandingsTab'`. Both work. The codebase uses relative imports for *intra-folder* references and `@/` aliases for *cross-folder* references (e.g. `@/lib/teams`). It's a soft convention that keeps imports local-feeling.

Stub style — minimal so the data is the focus

```
<div style={{ padding: 32, fontFamily: 'sans-serif' }}>
```

This wrapper exists purely so the page isn't visually flush against the browser edge. We'll replace it with the gradient hero in Chapter 4.

Step 2: the v1 StandingsTab

Create `components/tabs/StandingsTab.tsx`:

```
import { TEAMS } from '@/lib/teams';
import { calculateTeamScores } from '@/lib/scoring';

export default function StandingsTab() {
  const teamsWithScores = TEAMS.map(t => ({
    ...t,
    scores: calculateTeamScores(t),
  }));
```

```

    })).sort((a, b) => b.scores.total - a.scores.total);

    return (
      <table>
        <thead>
          <tr>
            <th>Rank</th>
            <th>Team</th>
            <th>Total</th>
          </tr>
        </thead>
        <tbody>
          {teamsWithScores.map((team, i) => (
            <tr key={team.name}>
              <td>{i + 1}</td>
              <td>{team.name}</td>
              <td>{team.scores.total}</td>
            </tr>
          ))}
        </tbody>
      </table>
    );
  }
}

```

Run `npm run dev` and visit `http://localhost:3000`. You should see a table with 8 rows showing team names ranked by total points.

It looks ugly. **That's fine.** It works.

What's happening in this file

Walk through each piece.

Line 1–2: imports

```
import { TEAMS } from '@lib/teams';
import { calculateTeamScores } from '@lib/scoring';
```

We're pulling the data and the scoring function from the `lib/` folder we built in Chapter 2. The `@/` is the path alias from `tsconfig.json`. **Notice there's no `import React` from `'react'` at the top** — modern JSX (since React 17) doesn't require it. The compiler imports React automatically when needed. You'll only import `{ useState, useMemo }` from `'react'` for hooks.

Line 4: the function

```
export default function StandingsTab() {
```

No props. The component uses imported data directly, with no parent involvement. That's fine for a leaf component that owns its data.

Line 5–8: derive the data

```
const teamsWithScores = TEAMS.map(t => ({
  ...t,
  scores: calculateTeamScores(t),
})).sort((a, b) => b.scores.total - a.scores.total);
```

This is the most important line in the file. Read it twice.

Step 1: `TEAMS.map(t => ({ ...t, scores: calculateTeamScores(t) })))` For each team, produce a new object that: - Spreads the team's existing fields (`name`, `color`, `r1`, `r2`, ...): `...t` - Adds a new `scores` field computed from `calculateTeamScores(t)`.

The result is an array of “team-with-scores” objects — the original team plus a `scores` object containing all the round totals and detail arrays.

The `...t` **spread** is JavaScript shorthand for “copy all of `t`'s fields into the new object.” Without it we'd have to type out every field manually:

```
{ name: t.name, color: t.color, accent: t.accent, /* etc */ scores: calculat
```

Spread is shorter and more maintainable. If a new field is added to Team later, the spread automatically picks it up.

Step 2: `.sort((a, b) => b.scores.total - a.scores.total)` Sort by `scores.total` descending. Because `b - a` is positive when `b` is bigger than `a`, `.sort()` puts bigger values first. **This is the standard “sort descending by a number” recipe.**

If you wanted ascending instead, you’d write `a - b`.

Why is sorting OK here? `.sort()` mutates the original array. That’s a footgun in React if you’re not careful — sorting a prop array would mutate the parent’s data. But here, **`TEAMS.map(...)` produces a brand new array** that we then sort. The original `TEAMS` is untouched. **Mutating an array you just made is fine.** Mutating an array someone else gave you is not.

Why is this in the function body, not at the module level? We could move the `teamsWithScores` calculation outside the component:

```
const teamsWithScores = TEAMS.map(...).sort(...); // module-level

export default function StandingsTab() {
  return ( /* ... */ );
}
```

It would still work. It would even be slightly faster (computed once, not per-render). But:

- For Chapter 4, we’ll need this calculation inside the component so we can pass `teamsWithScores` to other tabs from the orchestrator.
- For now, having it inside is fine — `calculateTeamScores` for 8 teams is microseconds.
- In Chapter 4 we’ll wrap it in `useMemo` to make the per-render cost negligible.

We’re optimizing for **readability and refactor-ability**, not raw speed.

Lines 11–22: the JSX

```

return (
  <table>
    <thead>
      <tr><th>Rank</th><th>Team</th><th>Total</th></tr>
    </thead>
    <tbody>
      {teamsWithScores.map((team, i) => (
        <tr key={team.name}>
          <td>{i + 1}</td>
          <td>{team.name}</td>
          <td>{team.scores.total}</td>
        </tr>
      ))}
    </tbody>
  </table>
);

```

Standard HTML `<table>` markup. Three things to dig into.

The key prop and why it matters

Look at this line:

```

{teamsWithScores.map((team, i) => (
  <tr key={team.name}>

```

That `key={team.name}` is **not optional**. It's React's identity tracker for list items.

What is React doing?

When the data changes — say, a team's score goes up and the order shifts — React needs to figure out which DOM nodes to keep, which to update, and which to throw away. Without keys, React falls back to comparing by **position in the array**: the first row is the first row, the second is the second, etc.

That's a problem if the list re-orders. If "Invincibles" was rank 1 and "BBU" was rank 2, then BBU jumps to rank 1 — without keys, React thinks "the first row is BBU now (was

Invincibles); update its content. The second row is Invincibles now (was BBU); update its content.” It works, but **every row’s content is replaced**, even though the rows just swapped.

With keys, React thinks “the row with key=‘BBU’ moved from position 2 to 1. The row with key=‘Invincibles’ moved from position 1 to 2. Just rearrange them.” Much faster, no re-rendering of contents, animations work correctly.

What makes a good key?

Three rules:

1. **Stable** — the same item gets the same key across renders. `team.name` is stable. `Math.random()` is the worst possible key.
2. **Unique within the list** — no two siblings share a key. Two teams can’t have the same name (in our app), so `team.name` is fine. If you had duplicate-able names, use an `id` field.
3. **Predictable** — derived from the data, not from external state. `i` (the index) works **only** if the list never reorders or has items inserted/deleted. For a sortable list, `i` is wrong. **For our standings, `team.name` is right.**

The classic key bug

Beginners often write `key={i}` because the index is right there. It works **right up until the list reorders or has an item inserted in the middle**, at which point React applies updates to the wrong rows and you get bizarre bugs (form inputs lose their values, animations attach to the wrong row, etc.).

Rule of thumb: **use `i` only when the list is append-only and never sorted**. Otherwise use a stable id.

Interview question: “What does the `key` prop do?” The wrong answer is “it makes the warning go away.” The right answer is “it tells React the identity of each list item across renders, so React can reconcile the DOM correctly when the list changes.”

Embedding expressions inside JSX

```
<td>{i + 1}</td>
<td>{team.name}</td>
<td>{team.scores.total}</td>
```

The `{...}` lets you embed any JavaScript expression. Three different things appear:

- `i + 1` — arithmetic. We add 1 because `.map`'s `i` is 0-indexed but humans expect ranks starting at 1.
- `team.name` — property access on an object.
- `team.scores.total` — nested property access.

All three are JS expressions. Anything that evaluates to a value works inside `{}`.

What **doesn't** work: statements. You can't write `{if (foo) ...}` because `if` is a statement, not an expression. You'd use a ternary instead: `{foo ? a : b}`.

Returning JSX vs returning null

What if we want to render nothing in some condition? React lets you return:

- JSX
- A string or number (rendered as text)
- `null` (renders nothing)
- `false` (also renders nothing)
- An array of JSX (renders all of them — but every item needs a key)

```
return null;                // renders nothing
return <p>Hello</p>;         // renders a paragraph
return condition ? <p>Yes</p> : null;  // conditional
```

Returning null is normal. It's not an error — it just means “don't render anything for this component this time.”

This comes up later when, e.g., a `<Modal>` returns `null` if it's not open.

Step 3: visit your browser

After saving both files:

1. Make sure `npm run dev` is running.
2. Open `http://localhost:3000`.

3. You should see your stub heading and the table below it.

If you see something different:

- **Blank page or error?** Check the terminal — Next.js will print compilation errors. The most common is “Cannot find module ‘@/lib/teams’” (typo in path) or “TEAMS is not exported” (forgot export in `lib/teams.ts`).
- **Table renders but all totals are 0?** Likely the matches don’t have winner filled in, or the picks contain typos. Add a `console.log(teamsWithScores)` temporarily and inspect.
- **Hydration failed because the initial UI does not match?** Server and client rendered different things. Common cause: using `Math.random()` or `Date.now()` directly in render. Don’t.

Why this is “the right pace” for a beginner

You’ve now written **20 lines of React** that:

- Imports data from a typed module.
- Transforms it via `.map()` and `.sort()`.
- Renders it as a table with proper keys.
- Lives inside a Next.js client-component subtree.

If a friend asked you “*what’s React?*” tonight, you could legitimately say “*it’s a way to derive a new data structure from the original data and tell the browser to render it as HTML, with React managing the diffing.*” That’s a senior-level mental model, in 20 lines.

The rest of the chapter (and the rest of the course) is just **more of the same pattern, with more sophisticated data transforms and JSX**. There’s no new fundamental concept after this; everything else is variations on a theme.

Vibe prompt you would have used

“Create components/tabs/StandingsTab.tsx. Import TEAMS from @/Lib/teams and calculateTeamScores from @/Lib/scoring. Build a teamsWithScores array by mapping over TEAMS and adding scores: calculateTeamScores(t) to each, then sort descending by .scores.total. Render a plain <table> with thead Rank/Team/Total and a tbody mapping

over teamsWithScores. Each row needs key={team.name}. No styling, no inline styles, just structural HTML. Default export.”

The “no styling” line is the secret. Without it the LLM will give you 80 lines of beautiful CSS and you won’t be able to tell what’s broken when something is broken.

CHECK YOURSELF

1. `<tr key={team.name}>` – what would happen if you used `<tr key={i}>` instead? Walk through what React does when you change the sort order. (Hint: think about `<input>` fields if any of the rows had editable cells.)
2. Why is `'use client'` on `SnookerFantasyLeague.tsx` but not on `StandingsTab.tsx`? What determines which files need it?
3. `TEAMS.map(t => ({ ...t, scores: ... })).sort(...)` – does this mutate the original `TEAMS` array? Why or why not?
4. You add a `console.log` inside the component body, just before return. When does it run? What if the user clicks a button somewhere on the page that doesn’t affect this component – does the log run again?
5. Add a `<th>R1</th>` to the header and a `<td>{team.scores.r1}</td>` to each row. What’s the minimum change? Now the page shows R1 totals next to the grand total. (Don’t move on until you’ve actually done this.)

When you’ve answered #5 by typing the change and seeing it work, you’re ready for Lesson 3 – adding the rest of the columns and the visual polish.

Head to [03-progressive-enhancement.md](#).

Chapter 3 · Lesson 3 — Progressive Enhancement

Goal: turn the ugly v1 standings table into the polished version you see in the finished app – by adding one feature at a time. By the end you’ll have built `<LeagueTable>`, `<StatCard>`, `<FormBadge>`, and a beautiful, working standings tab.

What “progressive enhancement” means here

The single most useful workflow for a React beginner is **add one thing at a time, run the dev server, look at the screen, repeat**. Don't add five features at once and then try to debug the result.

In this lesson we'll add seven features to the standings tab. After each feature you should:

1. Type the change.
2. Save the file (hot reload triggers).
3. Look at the browser.
4. Confirm it looks right.
5. Move on.

If something looks wrong, the diff between the previous-working state and the broken state is small. **Small diffs = easy debugging**.

v2: extract the table into its own component

Right now the standings logic and the rendering both live in `StandingsTab.tsx`. As we add columns and styling, that file will balloon. Let's pre-emptively split the data computation (which stays in `StandingsTab`) from the rendering (which moves to a new `<LeagueTable>` component).

Create `components/standings/LeagueTable.tsx`:

```
import type { TeamWithScores } from '@/lib/types';
import { th, td } from '@/lib/constants';

interface LeagueTableProps {
  teams: TeamWithScores[];
  onTeamClick?: (team: TeamWithScores) => void;
}

export default function LeagueTable({ teams, onTeamClick }: LeagueTableProps) {
  return (
    <table style={{ width: '100%', borderCollapse: 'collapse' }}>
      <thead>
        <tr style={{ background: '#0F5132', color: '#FBBF24' }}>
```

```

        <th style={{ ...th, textAlign: 'center' }}>#</th>
        <th style={th}>Team</th>
        <th style={{ ...th, textAlign: 'center' }}>Total</th>
      </tr>
    </thead>
    <tbody>
      {teams.map((team, i) => (
        <tr key={team.name}
          onClick={() => onTeamClick?.(team)}
          style={{ cursor: onTeamClick ? 'pointer' : 'default' }}>
          <td style={{ ...td, textAlign: 'center', fontWeight: 700 }}>{i + 1}</td>
          <td style={td}>
            <span style={{ marginRight: 8 }}>{team.icon}</span>
            {team.name}
          </td>
          <td style={{ ...td, textAlign: 'center', fontWeight: 800, color: 'red' }}>
            {team.scores.total}
          </td>
        </tr>
      ))}
    </tbody>
  </table>
);
}

```

Three new ideas:

1. Props for data — teams is now passed in

Before, `StandingsTab` knew about `TEAMS` directly. Now `<LeagueTable>` is **decoupled** from where the teams come from. It's a pure rendering component: give it teams, it renders them. Anyone could reuse it.

This is the **dumb component pattern** (also called *presentational component*): the component takes data via props and renders it. It doesn't fetch data, doesn't compute scores, doesn't decide what to do on click. It just renders.

2. An optional callback prop — onTeamClick?

The `?` in `onTeamClick?`: makes the prop optional. The caller can pass it (and clicks become interactive) or not pass it (and clicks do nothing). The component handles both cases:

```
onClick={() => onTeamClick?.(team)}
```

The `?.` is **optional chaining**. If `onTeamClick` is undefined, `onTeamClick?.(team)` does nothing. If it's a function, it gets called with the team.

This is a small but powerful pattern: **make features optional via optional callbacks**. Components become more reusable, and the caller decides what behavior to wire up.

3. Style spreading

```
<td style={{ ...td, textAlign: 'center' }}>
```

We import the `td` style object from `lib/constants.ts` and spread it into a new style object, then override `textAlign`. This is JavaScript object spreading — same idea as array spreading. The result is `{ padding: '12px 8px', borderBottom: '1px solid #F3F4F6', fontSize: 15, textAlign: 'center' }`.

The point: **shared base styles + per-cell overrides**, all without CSS classes.

v3: update StandingsTab to use the new LeagueTable

Update `components/tabs/StandingsTab.tsx`:

```
import { TEAMS } from '@lib/teams';
import { calculateTeamScores } from '@lib/scoring';
import type { TeamWithScores } from '@lib/types';
import LeagueTable from '../standings/LeagueTable';

interface StandingsTabProps {
  onTeamClick?: (team: TeamWithScores) => void;
}

export default function StandingsTab({ onTeamClick }: StandingsTabProps) {
```

```

const teamsWithScores: TeamWithScores[] = TEAMS.map(t => ({
  ...t,
  scores: calculateTeamScores(t),
})).sort((a, b) => b.scores.total - a.scores.total);

return (
  <div>
    <LeagueTable teams={teamsWithScores} onTeamClick={onTeamClick} />
  </div>
);
}

```

Now StandingsTab does only data prep and delegates rendering. Save and check the browser — should look the same as before, just better-organized internally.

v4: medal-color the rank circle

The first visual feature we'll add. Change the rank cell in LeagueTable.tsx:

```

<td style={{ ...td, textAlign: 'center', fontWeight: 700 }}>
  {(() => {
    const rank = i + 1;
    const rankColor =
      rank === 1 ? '#FBBF24' :    // gold
      rank === 2 ? '#9CA3AF' :    // silver
      rank === 3 ? '#B45309' :    // bronze
      '#E5E7EB';                  // gray
    return (
      <div style={{
        width: 32, height: 32,
        borderRadius: '50%',
        background: rankColor,
        color: rank <= 3 ? 'FFFFFF' : '#374151',
        display: 'inline-flex',
        alignItems: 'center',
        justifyContent: 'center',

```



```

        fontWeight: 900,
      }}>
      {rank === 1 ? '👑' : rank}
    </div>
  );
})){}}
</td>

```

Whoa — what's that `((() => { ... })){}` thing?

IIFE inside JSX (immediately-invoked function expression)

JSX `{}` expects an **expression**, not a sequence of statements. We have several statements (`const rank = ...`, `const rankColor = ...`, then a `return`). To embed those in JSX, we wrap them in an arrow function and immediately invoke it:

```

{(() => {
  // statements here
  return <jsx />;
})()}

```

That's an IIFE — Immediately-Invoked Function Expression. It evaluates to whatever the inner function returns.

It's ugly. It works. It's a common React pattern when you have multiple `let/const` statements you want close to where they're used. The cleaner alternative is to **extract this into its own component** — which we'll do in Lesson 5 when it's time to refactor.

For now, the IIFE makes the rank logic local to this cell. **A small ugliness in service of a small feature is fine. Don't pre-refactor.**

A nested ternary for the colors

```

const rankColor =
  rank === 1 ? '#FBBF24' :
  rank === 2 ? '#9CA3AF' :
  rank === 3 ? '#B45309' :
  '#E5E7EB';

```

Three checks, four outcomes. Reads top-to-bottom: gold for 1st, silver for 2nd, bronze for 3rd, gray default. **Nested ternaries are fine if formatted vertically and have a clear reading order.** If you find yourself wrapping ternaries on a single line or your editor wraps them weirdly, refactor to a function `getRankColor(rank: number)`.

display: 'inline-flex' for centering

```
display: 'inline-flex',
alignItems: 'center',
justifyContent: 'center',
```

This is the canonical CSS recipe for **centering text inside a fixed-size circular badge**. `inline-flex` lays out the children with flexbox; `alignItems: center` is vertical centering; `justifyContent: center` is horizontal. Works for any container with a defined size.

Save. Refresh. The leader has a 🏆 in a gold circle, 2nd has silver, 3rd has bronze. It's starting to look real.

v5: per-round columns

Add columns for R1, R2, QF, SF, Final. Update the header and body of `LeagueTable.tsx`:

```
<thead>
  <tr style={{ background: '#0F5132', color: '#FBBF24' }}>
    <th style={{ ...th, textAlign: 'center' }}>#</th>
    <th style={th}>Team</th>
    <th style={{ ...th, textAlign: 'center' }}>R1</th>
    <th style={{ ...th, textAlign: 'center' }}>R2</th>
    <th style={{ ...th, textAlign: 'center' }}>QF</th>
    <th style={{ ...th, textAlign: 'center' }}>SF</th>
    <th style={{ ...th, textAlign: 'center' }}>Final</th>
    <th style={{ ...th, textAlign: 'center' }}>Total</th>
  </tr>
</thead>
```

And in the body, after the team-name cell:

```

<td style={{ ...td, textAlign: 'center' }}>
  <div style={{ fontWeight: 700, fontSize: 17, color: '#0F5132' }}>
    {team.scores.r1}
  </div>
  <div style={{ fontSize: 11, color: '#6B7280' }}>/ 48</div>
</td>
<td style={{ ...td, textAlign: 'center' }}>
  <div style={{ fontWeight: 700, fontSize: 17, color: '#0F5132' }}>
    {team.scores.r2}
  </div>
  <div style={{ fontSize: 11, color: '#6B7280' }}>/ 24</div>
</td>
{ /* ...QF (/12), SF (/6), Final (/3)... */ }

```

The “/ 48” “/ 24” lines show the **maximum possible score for the round**, computed at design time:

- R1: 16 matches × 3 points = 48
- R2: 8 × 3 = 24
- QF: 4 × 3 = 12
- SF: 2 × 3 = 6
- Final: 1 × 3 = 3

Hard-coded because they’re constants of the tournament structure. If we ever changed the bracket size, we’d want to compute these from `ROUND1_MATCHES.length * 3` etc.

Save. Refresh. The table now has 8 columns and feels like a real fantasy leaderboard.

v6: the Final Pick chip

Add another column showing each team’s Final Pick:

```

<th style={th}>Final Pick</th>

<td style={{ ...td, textAlign: 'center' }}>
  <div style={{
    display: 'inline-block',
    padding: '4px 10px',

```

```

    borderRadius: 12,
    background: team.final === 'Wu Yize' ? '#DCFCE7' : '#FEE2E2',
    color: team.final === 'Wu Yize' ? '#166534' : '#991B1B',
    fontWeight: 700,
    fontSize: 12,
  }}>
    🏆 {team.final}
  </div>
</td>

```

A pill-shaped badge with a 🏆 and the team’s pick.

The conditional coloring

```

background: team.final === 'Wu Yize' ? '#DCFCE7' : '#FEE2E2',
color:      team.final === 'Wu Yize' ? '#166534' : '#991B1B',

```

If the team picked the actual champion (Wu Yize), the chip is **green**. Otherwise it’s **red**. This is the post-tournament color rule we discussed in the original course’s Lesson 7 — hard-coding the champion’s name into the styling logic.

A more general version would compare against `FINAL_MATCH[0]?.winner`. The hard-coded version is faster to write but ties this code to the 2026 tournament. **For a single-tournament app, fine. For a reusable framework, not fine.** Most real codebases have a mix.

v7: the FormBadge

Last column: a “Form” indicator showing whether the team is improving or slipping between rounds.

Create `components/standings/FormBadge.tsx`:

```

import { TrendingUp, TrendingDown, Minus } from 'lucide-react';

interface FormBadgeProps {
  r1Pct: number;
  r2Pct: number;

```

```

}

export default function FormBadge({ r1Pct, r2Pct }: FormBadgeProps) {
  const trending = r2Pct > r1Pct ? 'up' : r2Pct < r1Pct ? 'down' : 'flat';
  const Icon = trending === 'up' ? TrendingUp : trending === 'down' ? TrendingDown : TrendingFlat;
  const color = trending === 'up' ? '#16A34A' : trending === 'down' ? '#DC2626' : '#666666';
  const label = trending === 'up' ? 'Rising' : trending === 'down' ? 'Falling' : 'Flat';

  return (
    <div style={{
      display: 'inline-flex',
      alignItems: 'center',
      gap: 4,
      color,
      fontSize: 13,
      fontWeight: 700,
    }}>
      <Icon size={16} strokeWidth={3} />
      {label}
    </div>
  );
}

```

Then in `LeagueTable.tsx`:

```

import FormBadge from './FormBadge';

// ...in the row:
<td style={{ ...td, textAlign: 'center' }}>
  <FormBadge
    r1Pct={(team.scores.r1 / 48) * 100}
    r2Pct={(team.scores.r2 / 24) * 100}
  />
</td>

```

Three new ideas:

Lucide icons as components

```
import { TrendingUp, TrendingDown, Minus } from 'lucide-react';
```

Lucide is a tree-shakeable icon library. We import three icon components and treat them like any other React component (`<TrendingUp size={16} />`).

The **tree-shakeable** part matters: the production bundle only includes the icons you actually import. If we import 3 icons, we ship 3, not all ~1,000 in the library.

Picking a component dynamically

```
const Icon = trending === 'up' ? TrendingUp : trending === 'down' ? TrendingDown : TrendingUp
// ...
<Icon size={16} strokeWidth={3} />
```

Icon is a variable holding a **component**. We then use it as a JSX tag. This works because JSX is just a function call; `<Icon />` compiles to `React.createElement(Icon, ...)` and the Icon reference can come from anywhere — a prop, a variable, a `useState` value, anything.

Recall the earlier rule: **JSX tag names must be capitalized to be treated as components**. We renamed our local variable Icon (capital I) so JSX recognizes it. If we'd used `icon`, JSX would have looked for an HTML `<icon>` tag.

Calculating percentages inline

```
r1Pct= {(team.scores.r1 / 48) * 100}
```

We pass percentages, not raw scores. Why? Because R1 has a max of 48 and R2 has a max of 24 — direct comparison would be misleading. Comparing **percentages** of perfect normalizes across rounds.

Save. Refresh. Each row now has a Form indicator showing whether they got better or worse from R1 to R2. Most teams will probably show “Rising” or “Falling” since percentages rarely tie exactly.

v8: stat cards above the table

The top of the standings tab in the finished app shows three stat tiles: **Leader**, **Total Matches**, **Avg Score**. Create components/standings/StatCard.tsx (this is the file we walked through in Lesson 1):

```
import type { LucideIcon } from 'lucide-react';

interface StatCardProps {
  icon: LucideIcon;
  label: string;
  value: string | number;
  color: string;
  bgColor: string;
}

export default function StatCard({ icon: Icon, label, value, color, bgColor }: StatCardProps) {
  return (
    <div style={{
      background: `linear-gradient(135deg, ${bgColor} 0%, ${bgColor}DD 100%)`,
      color: '#FFFFFF',
      padding: 24,
      borderRadius: 16,
      boxShadow: `0 8px 24px ${bgColor}40`,
    }}>
      <Icon size={28} strokeWidth={2.5} />
      <div style={{ fontSize: 13, letterSpacing: '1.5px', fontWeight: 600, opacity: 0.7 }}>
        {label}
      </div>
      <div style={{ fontSize: 32, fontWeight: 900, fontFamily: "'Roboto Slab'" }}>
        {value}
      </div>
    </div>
  );
}
```

Then update StandingsTab.tsx to render three cards above the table:

```

import { Trophy, Calendar, Target } from 'lucide-react';
import StatCard from '../standings/StatCard';
import { ROUND1_MATCHES, ROUND2_MATCHES, QF_MATCHES, SF_MATCHES, FINAL_MATCHES } from '../constants';

export default function StandingsTab({ onTeamClick }: StandingsTabProps) {
  const teamsWithScores: TeamWithScores[] = TEAMS.map(t => ({
    ...t,
    scores: calculateTeamScores(t),
  })).sort((a, b) => b.scores.total - a.scores.total);

  const totalMatches =
    ROUND1_MATCHES.length + ROUND2_MATCHES.length +
    QF_MATCHES.length + SF_MATCHES.length + FINAL_MATCHES.length;

  const leader = teamsWithScores[0];
  const avgScore = Math.round(
    teamsWithScores.reduce((sum, t) => sum + t.scores.total, 0) / teamsWithScores.length
  );

  return (
    <div>
      <div style={{
        display: 'grid',
        gridTemplateColumns: 'repeat(auto-fit, minmax(220px, 1fr))',
        gap: 16,
        marginBottom: 24,
      }}>
        <StatCard icon={Trophy} label="LEADER" value={leader.name} color="blue">
          {leader.name}
        </StatCard>
        <StatCard icon={Calendar} label="MATCHES" value={totalMatches} color="green">
          {totalMatches}
        </StatCard>
        <StatCard icon={Target} label="AVG SCORE" value={avgScore} color="orange">
          {avgScore}
        </StatCard>
      </div>

      <LeagueTable teams={teamsWithScores} onTeamClick={onTeamClick} />
    </div>
  );
}

```



```
}
```

Three new things to call out:

CSS Grid with auto-fit and minmax

```
gridTemplateColumns: 'repeat(auto-fit, minmax(220px, 1fr))',
```

This is the **modern way to do responsive card grids**. It says:

- Repeat as many columns as fit.
- Each column is **at least 220px** and **at most 1 fraction** of available space.

So if the screen is 660px wide, you get 3 columns of ~220px. If it's 480px, you get 2 columns. If it's 220px, you get 1. **Without media queries**. This single CSS line replaces what used to take a dozen breakpoints.

Memorize this pattern. It's interview gold.

Computed values inside the component

```
const totalMatches = ROUND1_MATCHES.length + ROUND2_MATCHES.length + ...;  
const leader = teamsWithScores[0];  
const avgScore = Math.round(teamsWithScores.reduce((sum, t) => sum + t.score,
```

We derive three values from the data and pass them as props to the StatCards. **Derived state stays close to where it's used** — no need to compute these elsewhere.

The `.reduce((sum, t) => sum + t.scores.total, 0)` is JavaScript's "sum an array" idiom. `0` is the initial value of `sum`. For each team `t`, we add `t.scores.total` to `sum`. End result: total points across all teams. Divide by 8 to get the average. `Math.round` for clean integer display.

Importing icons by name

```
import { Trophy, Calendar, Target } from 'lucide-react';
```

Three named imports. Each is a React component. We pass them as props to `<Stat-Card>`, which renders them.

Notice we **don't** instantiate them — we pass the component itself, not `<Trophy />`. The StatCard component receives the component-class and renders it inside its own JSX. This is how you build flexible, “give me an icon to render here” APIs.

What v8 looks like

Save everything and refresh. You should now see:

- Three colored stat cards at the top: green “LEADER” card with the leader’s name, red “MATCHES” card showing 31, purple “AVG SCORE” card.
- Below them, the full standings table with medal-color rank badges, per-round columns, Final Pick chips, Form badges.

It looks like a real fantasy app now.

What we did NOT do (yet)

Notice we still don’t have:

- The hero header (gradient banner).
- The tab nav bar.
- Multiple tabs (Matches, Predictions, Players, Analytics).

Those are Chapter 4. Right now we have **one tab, fully built, with no tab bar**. That’s intentional — building one screen completely before adding navigation lets you focus on each problem in turn.

The discipline you just practiced

Read back through this lesson. We added **eight versions** of the same component, each one a small addition:

- v1: plain table (Lesson 2)
- v2: extract LeagueTable
- v3: wire it up
- v4: medal-color rank
- v5: per-round columns
- v6: Final Pick chip
- v7: FormBadge

- v8: stat cards

Each version was 5–30 lines of new code. Each one was independently verifiable in the browser. **No version did three things at once.**

This is **the way professional React engineers build features**. Not all in one go — incrementally. The git history of any well-managed React project looks exactly like this: dozens of small commits, each adding one tiny thing. The big-bang “I built it all in one PR” approach is what produces unreviewable code and untraceable bugs.

Interview tip. When asked “how do you approach building a new feature?”, your answer should sound like “I start with the smallest version that proves the data path works, then I add visual polish one piece at a time, running the dev server and looking at the screen between every change. Small commits, easy to review, easy to roll back.” That answer signals you’ve shipped real software.

Vibe prompt you would have used (per feature)

Feature by feature, the prompts would have been:

v4: “In the rank cell of `<LeagueTable>`, replace the plain number with a 32x32 circle. Background: gold for rank 1, silver for 2, bronze for 3, gray otherwise. Replace the number with a 🏆 emoji for rank 1.”

v5: “Add R1, R2, QF, SF, Final columns to `<LeagueTable>`, between Team and Total. Show the round score as a big number with `/ {maxScore}` underneath in small gray text. Maxes: 48, 24, 12, 6, 3.”

v6: “Add a Final Pick column to `<LeagueTable>`. Show `team.final` inside a rounded chip with a 🏆 emoji. Background green if `team.final === 'Wu Yize'`, red otherwise.”

v7: “Create `<FormBadge>` taking `r1Pct` and `r2Pct` numbers. Show TrendingUp icon + ‘Rising’ (green) if `r2 > r1`, TrendingDown + ‘Falling’ (red) if `r2 < r1`, Minus + ‘Steady’ (gray) otherwise. Use Lucide icons. Then add a Form column to `<LeagueTable>` using it.”

Each prompt is **one feature, one component, one paragraph**. That’s the right grain. Compare to a bad prompt: “make the standings table nicer” — that produces an unmaintainable wall of CSS that you can’t reason about.

CHECK YOURSELF

1. The IIFE inside the rank cell `((() => { ... } )())` works but is ugly. What's the cleaner refactor? What's the cost of doing it now vs leaving it?
2. `onTeamClick?: (team) => void` is optional. What does the `?.` in `onTeamClick?.(team)` do? What would happen without the `?` if a parent didn't pass `onTeamClick`?
3. `gridTemplateColumns: 'repeat(auto-fit, minmax(220px, 1fr))'` — explain in plain English what this does. What replaces this in old-style CSS?
4. `<StatCard icon={Trophy} />` passes the *Trophy component itself*, not `<Trophy />`. Why? What would go wrong if you wrote `icon={<Trophy />}`?
5. The Final Pick chip hard-codes `'Wu Yize'` for its color choice. Write the one-line edit that makes it tournament-agnostic by reading from `FINAL_MATCH[0]?.winner` instead.

Once you've answered (and ideally typed #5), you've completed Chapter 3. The standings tab is fully built. Onward to [Chapter 4 — State & Hooks](#), where we add the tab bar and finally meet `useState`.

Chapter 3 — Your First React Component

Now we write React. From a 15-line hello-world component to a fully styled standings table with sorted rows, medal-color rank badges, per-round columns, and a form indicator.

What you'll learn

- The **anatomy** of a React component: the function, the return, the JSX, the props.
- How **JSX** is just syntactic sugar for `React.createElement` calls (and why that matters).
- **Lists and keys** — why React needs a key prop on every item in a `.map()` loop.
- **Conditional rendering** with `&&`, ternaries, and early returns.
- **Inline styles** with TypeScript's `React.CSSProperties`.
- **Component props** typed with TypeScript interfaces.
- The discipline of **progressive enhancement** — building a feature one column / one widget at a time.

Lessons in this chapter

1. [01-anatomy-of-a-component.md](#) — What a component IS, what JSX IS, and what makes a component “render”. The mental model that powers everything else.
2. [02-rendering-the-standings.md](#) — Building `<StandingsTab>` v1: a plain table of 8 teams with totals. The smallest version that proves the data flows.
3. [03-progressive-enhancement.md](#) — Iterating on the standings: medal-color rank circles, per-round columns, the Final Pick chip, the Form badge. Six small features added one at a time.

Files you'll create in this chapter

```
components/  
├── SnookerFantasyLeague.tsx      ← stub orchestrator that renders <StandingsTab>  
├── tabs/  
│   ├── StandingsTab.tsx        ← the standings tab  
└── standings/  
    ├── LeagueTable.tsx         ← the table itself (extracted in Lesson 3)  
    ├── StatCard.tsx            ← summary stat tiles above the table  
    └── FormBadge.tsx           ← rising/falling/steady arrow indicator
```

New React/Next.js concepts introduced

- **Functional components** — the only kind of component you'll write in 2026.
- **Props** — how parent components pass data to children.
- **Lists with `.map()`** and the key prop.
- **Conditional rendering** in JSX.
- **Inline styles** vs class names.
- **The 'use client' directive** — when (and only when) to use it.
- **Component composition** — passing JSX through props (children).

How long it should take

- 30 min reading per lesson
- 1.5–2 hours typing the code
- ~3.5 hours total

What you'll see in your browser at the end

Open `http://localhost:3000` after Lesson 3 and you should see:

- A dark green hero header with “World Snooker Championship” in gold serif type and “ WU YIZE — WORLD CHAMPION 2026 (18-17)” in a red-gold badge.
- Below it, a sticky tab bar with five tabs (only Standings will work for now — clicking other tabs may error or do nothing).
- The Standings table: 8 rows, ranked by total, gold/silver/bronze rank circles for the top 3, per-round columns, a Final Pick chip, a Form badge, hover effect on each row.

If your browser shows that, you've nailed Chapter 3.

Before you start

Make sure Chapter 2 is done. You should have:

- `lib/types.ts`, `lib/scoring.ts`, `lib/matches.ts`, `lib/teams.ts`, `lib/players.ts`, `lib/constants.ts` all populated with real data.
- `app/layout.tsx`, `app/page.tsx`, `app/globals.css` all in place.
- `npm run dev` boots without crashing (it'll error on the missing `<SnookerFantasyLeague>` component — that's fine, you're about to create it).

Open [01-anatomy-of-a-component.md](#).

Chapter 4 · Lesson 1 — The `useState` Mental Model

Goal: completely demystify `useState`. By the end of this lesson you can answer “what does `useState` do?” in a way that gets you hired, and you can predict exactly what happens to your component when state changes.

What is “state”?

In a React app, every piece of data falls into one of two categories:

- **Props** — data passed in from a parent component. The component cannot change props directly. They flow downward.
- **State** — data the component owns and can change. When state changes, the component re-renders.

Then there's a third bucket that beginners sometimes confuse:

- **Derived data** — data computed from props or state. Not state itself; just a function of state. (Example: `teamsWithScores` in Chapter 3 — computed from `TEAMS`, not stored separately.)

Senior rule: if a piece of data can be **derived** from existing state or props, **don't put it in state**. Compute it on every render. Storing it duplicates the source of truth and creates synchronization bugs. State is for the *minimum* set of changeable values.

The interview question: *"How do you decide what should be state?"* Answer: *"I ask 'is this data the source of truth, or can it be computed from something else?' Only sources of truth become state."*

What is `useState`?

`useState` is a **React hook** — a function with a special property that React tracks per-component-instance.

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

Six lines, complete React component with state.

Anatomy of `useState`

```
const [count, setCount] = useState(0);
```

Three things in this line:

1. **useState(0)** – call the hook with an *initial value* (0). React allocates a “state cell” for this component instance.
2. **useState returns an array of two items** – [currentValue, setterFunction].
3. **Array destructuring** – `const [count, setCount]` pulls the two items out and names them.

By convention, the setter is named `set` + capitalized state name. `setCount` for `count`. `setUser` for `user`. Stick with this convention; everyone reading your code expects it.

What does the setter do?

When you call `setCount(5)`, two things happen:

1. React **updates the state cell** to 5.
2. React **schedules a re-render** of this component.

That’s it. It doesn’t immediately mutate `count` (the variable). It doesn’t return the new value. It schedules React to re-execute your component function with the new state.

The classic gotcha:

```
function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
    console.log(count);    // logs the OLD value, not the new one!
  }

  return <button onClick={handleClick}>{count}</button>;
}
```

Why? Because `count` is a `const` captured from the current render. The setter changes the underlying state cell, but the variable in this closure still holds the old value until the **next** render produces a new closure with the new value.

The mental model: **state is not a variable you mutate; it’s a value you read this render and request a new value for next render.**

The re-render cycle

When state changes, React **re-renders the component** — it calls the function again from the top.

```
function Counter() {  
  console.log('rendering Counter');  
  const [count, setCount] = useState(0);  
  return <button onClick={() => setCount(count + 1)}>{count}</button>;  
}
```

Initial render: 'rendering Counter' logs once. Browser shows "0".

User clicks: `setCount(0 + 1)` is called. React schedules a re-render.

Re-render: 'rendering Counter' logs again. `useState(0)` runs **again** but returns the *new* state value (1, not 0). The button now shows "1".

User clicks again: `setCount(1 + 1)` schedules re-render. Function re-runs. `useState` returns 2. Button shows "2".

Every state change triggers a re-render. Every render runs the function from the top.

Variables inside the function are fresh every time. The persistent state is the value React holds for you.

The mental loop

1. React calls your function (render).
2. Your function returns JSX.
3. React updates the DOM to match the JSX.
4. User does something (clicks, types).
5. Event handler fires; calls `setSomething(...)`.
6. React queues a re-render.
7. Go to 1.

That's the whole React lifecycle in seven steps. Memorize it. **Every UI bug you ever encounter has its root in some step of this loop.** Knowing the loop = knowing where to look.

Hooks: the rules

There are exactly **two rules of hooks**, and they exist because of how React tracks state cells.

Rule 1: Call hooks at the top level. Never inside loops, conditions, or nested functions.

```
// BROKEN
function Foo({ show }) {
  if (show) {
    const [x, setX] = useState(0); // ❌ conditional hook
  }
}

// CORRECT
function Foo({ show }) {
  const [x, setX] = useState(0); // ✅ always called
  if (show) {
    return <div>{x}</div>;
  }
  return null;
}
```

Why? React tracks hooks **by call order**. Render 1: `useState` is the first hook. Render 2: `useState` is the first hook. They match — same state cell. If on render 3 the condition is false and `useState` isn't called, React loses track of which state cell maps to which `useState` call. Bugs ensue.

Rule 2: Call hooks only from React function components or other hooks.

You can call `useState` inside a function component (`function Foo()`) or inside a custom hook (`function useFoo()`). Not inside a regular utility function. The reason is the same — React's per-component tracker only runs during component rendering.

These rules are enforced by the `eslint-plugin-react-hooks` package, which is installed by default in `create-next-app`. If you violate them, ESLint yells at you. **Listen to ESLint.**

Hooks are detectable by name

The convention: hooks start with `use`. `useState`, `useEffect`, `useMemo`, `useReducer`, `useRef`, `useContext`, etc. **Custom hooks must also start with `use`** — that's how the linter knows to enforce the rules on them.

Initial value can be expensive — use a function

```
// Expensive initial value computed every render (wasteful)
const [data, setData] = useState(JSON.parse(localStorage.getItem('data')) ||

// Lazy initialization — function is called only on first render
const [data, setData] = useState(() => JSON.parse(localStorage.getItem('data'))
```

The first form re-evaluates `JSON.parse(...)` on every render (though `useState` ignores the result on subsequent renders). The second form is **lazy** — React only calls the initializer once.

For cheap initial values (`useState(0)`, `useState('')`), don't bother. For anything that involves computation, IO, or `JSON.parse`, use the function form.

Multiple state values

You can call `useState` as many times as you want in one component:

```
function Form() {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [age, setAge] = useState(18);
  // ...
}
```

That's totally fine. Each call gets its own state cell. **Don't try to combine them into one giant `useState({ name, email, age })` unless they truly belong together** — the multi-call form is more readable and lets each piece update independently.

When *should* you combine into one `useState`? When the values are tightly coupled and always update together. Example: `useState({ width: 0, height: 0 })` for a measured DOM element. They're always set at the same time, so one state cell is fine.

State updates are batched

If you call multiple setters in the same event handler, React batches them into one re-render:

```
function handleClick() {
  setCount(count + 1);
  setName('clicked');
  setMessage('hello');
  // Only ONE re-render happens, after all three setters resolve.
}
```

This is **automatic batching**, on by default in React 18+. It's a performance feature you barely notice – but if you're debugging strange behavior, it's good to know.

State updates can be functional

```
setCount(count + 1);           // value form – uses the count from this closure
setCount(prev => prev + 1);    // functional form – receives the latest state
```

When does it matter? When you're updating state based on previous state **AND** the update might happen multiple times before the re-render:

```
// PROBLEM: stale closure
function handleClick() {
  setCount(count + 1);
  setCount(count + 1);    // count is still the OLD value here, so this sets
  setCount(count + 1);    // same – final result is old+1, not old+3
}

// FIX: functional updates
function handleClick() {
  setCount(prev => prev + 1); // prev is the latest queued value
  setCount(prev => prev + 1);
  setCount(prev => prev + 1); // final result is old+3
}
```

Rule of thumb: **if the new state depends on the old state, use the functional form.** It's

safer in async / batching scenarios.

What state is in <SnookerFantasyLeague>?

Now apply this thinking to our app. From Chapter 1, the user stories are:

- See standings
- See matches
- See predictions
- See player drill-downs
- See analytics

Read those carefully. **What in our app changes over time as the user clicks?**

Two things:

1. **Which tab is active.** The user clicks “Predictions” → `activeTab` becomes `'predictions'`. That’s state.
2. **Which team is selected (for predictions/players drill-down).** The user clicks “Invincibles” in the standings → `selectedTeam` becomes the `Invincibles` object. That’s state.

Anything else? No. The match data doesn’t change (it’s typed into a file). The team picks don’t change. The scores are derived from the data. Nothing else changes from user interaction.

Two pieces of state. That’s our entire global state surface. Compare to a Redux app where you might have 50 slices — for *this* app, two `useState` calls suffice.

This is the senior answer to “how do you manage state?” — start by enumerating what *changes*. The answer is usually a smaller list than juniors expect.

A small example: the round filter

Some components have **local** state that doesn’t need to live at the top. Take the `<MatchesTab>` (which we’ll build properly in Chapter 5) — it has a tab strip for R1 / R2 / QF / SF / Final. Which round is showing?

That state could live in `<SnookerFantasyLeague>`, but **nothing else in the app cares which round `MatchesTab` is showing**. It’s a UI detail of `MatchesTab` itself. So it lives there:

```
'use client';
import { useState } from 'react';

export default function MatchesTab() {
  const [round, setRound] = useState<'r1' | 'r2' | 'qf' | 'sf' | 'f'>('r1');
  // ...
}
```

The TypeScript type `'r1' | 'r2' | 'qf' | 'sf' | 'f'` is a **string union** — only those five values are allowed. Pass `'foo'` and TS yells. **Use string unions instead of plain string whenever your state has a fixed set of values.** It catches typos, helps autocomplete, makes refactoring safe.

This is the **lowest common ancestor** rule from Chapter 1: state lives at the lowest component that needs to read or write it. For tab-internal state, that's the tab itself. For app-wide state, it's the orchestrator.

When state belongs in a parent (lifting state up)

The other situation: **two siblings need to share state**. Example: clicking a row in `<StandingsTab>` should switch the active tab to `<PredictionsTab>` and pre-select the team.

Three components need to know:

- `<SnookerFantasyLeague>` — owns `activeTab` to decide which tab to render.
- `<StandingsTab>` — needs to *trigger* the change.
- `<PredictionsTab>` — needs to *read* `selectedTeam`.

The rule: **state lives at the lowest common ancestor of every component that needs it.** Lowest common ancestor of those three is `<SnookerFantasyLeague>`. So state lives there. `<StandingsTab>` gets a callback prop (`onTeamClick`) that calls the parent's setter. `<PredictionsTab>` gets the value as a prop (`selectedTeam`).

Lesson 2 walks through this in detail. For now, internalize the principle: **state location is determined by the data, not by which component “owns” the feature.**

The hook isn't magic — it's a closure trick

You don't need to know how `useState` works internally to use it. But the explanation closes the mental gap for many beginners:

```
// Pseudocode for what React does internally
let stateCells: any[] = [];
let cellIndex = 0;

function useState<T>(initial: T): [T, (newVal: T) => void] {
  const i = cellIndex;
  if (stateCells[i] === undefined) stateCells[i] = initial;
  const setter = (newVal: T) => {
    stateCells[i] = newVal;
    scheduleRerender();
  };
  cellIndex++;
  return [stateCells[i], setter];
}

function render(component) {
  cellIndex = 0; // reset before each render
  return component();
}
```

That's a simplified version — React actually uses linked lists of “fiber” hooks per component instance — but the spirit is right. **State cells indexed by call order, reset on each render, looked up by position.** That's why hooks must be called in the same order every render. Now you know.

Vibe prompt you would have used

“I'm going to add interactivity to my app for the first time. Here's the data flow: clicking a row in <StandingsTab> should set a `selectedTeam` and switch the active tab to <PredictionsTab>. The state for `activeTab` and `selectedTeam` should live in the parent (<SnookerFantasyLeague>). Add `useState` for both, pass them down where needed, and pass setters as callbacks

to `<StandingsTab>`. **Don't use Context, don't use Redux** – just use `useState` in the parent and prop drilling. Mark the parent file with `'use client'.`

The “Don't use Context, don't use Redux” line is critical. LLMs love to over-engineer. Telling them what *not* to use is half the prompt.

CHECK YOURSELF

1. `const [count, setCount] = useState(0);` – what does each side of the destructuring do? What's the type of `count`? What's the type of `setCount`?
2. You write `setCount(count + 1); console.log(count);` – what does the console log? Why?
3. The two rules of hooks are: **top-level only**, and **React-functions-or-hooks only**. Why does the first rule exist? (Hint: it's about React's tracking mechanism.)
4. You're building a search input that filters a list. Should `searchQuery` be state? Should the *filtered list* be state? Why or why not?
5. In our snooker app, should `teamsWithScores` (the sorted, decorated team list) be state? Why or why not?

When you've answered these, head to [02-the-orchestrator-pattern.md](#) – where we lift state up and finally make the app feel like an app.

Chapter 4 · Lesson 2 – The Orchestrator Pattern

Goal: build the real `<SnookerFantasyLeague>` orchestrator – hero header, sticky tab bar, five tab buttons, conditional tab rendering, and the state that ties them together. By the end of this lesson, the app feels like an app.

What is an “orchestrator” component?

In real React apps you'll see a pattern called **the orchestrator** (also “container component”, “smart component”, “shell component” – same idea, different vocabulary).

The orchestrator's job:

1. **Own** the top-level state (`activeTab`, `selectedTeam`).
2. **Compute** the derived data (`teamsWithScores`).
3. **Decide** which child to render based on the state.

4. **Pass** state down as props and setters down as callbacks.

It typically does **not** render very much JSX itself — it just glues other components together.

This pattern shows up in literally every React codebase past five components. It's worth getting right early.

Step 1: rewrite `SnookerFantasyLeague.tsx` as an orchestrator

Replace the stub from Chapter 3 with this:

```
'use client';

import { useState, useMemo } from 'react';
import { Trophy, Calendar, Target, Users, BarChart3 } from 'lucide-react';
import { TEAMS } from '@/lib/teams';
import { calculateTeamScores } from '@/lib/scoring';
import type { TeamWithScores } from '@/lib/types';
import { BALL_COLORS } from '@/lib/constants';
import StandingsTab from './tabs/StandingsTab';
import MatchesTab from './tabs/MatchesTab';
import PredictionsTab from './tabs/PredictionsTab';
import PlayersTab from './tabs/PlayersTab';
import AnalyticsTab from './tabs/AnalyticsTab';

type TabId = 'standings' | 'matches' | 'predictions' | 'players' | 'analytics';

export default function SnookerFantasyLeague() {
  const [activeTab, setActiveTab] = useState<TabId>('standings');
  const [selectedTeam, setSelectedTeam] = useState<TeamWithScores | null>(null);

  const teamsWithScores = useMemo<TeamWithScores[]>(() => {
    return TEAMS.map(t => ({ ...t, scores: calculateTeamScores(t) }))
      .sort((a, b) => b.scores.total - a.scores.total);
  }, []);

  const tabs: { id: TabId; label: string; icon: typeof Trophy }[] = [
    { id: 'standings', label: 'Standings', icon: Trophy },
  ];
```

```

    { id: 'matches',      label: 'Matches',      icon: Calendar },
    { id: 'predictions', label: 'Predictions', icon: Target },
    { id: 'players',      label: 'Players',      icon: Users },
    { id: 'analytics',    label: 'Analytics',    icon: BarChart3 },
  ];

  return (
    <div style={{ /* ...big background gradient... */ }}>
      {/* Hero header */}
      <Hero />

      {/* Sticky tab bar */}
      <nav style={{ /* ... */ }}>
        {tabs.map(tab => {
          const Icon = tab.icon;
          const active = activeTab === tab.id;
          return (
            <button
              key={tab.id}
              onClick={() => setActiveTab(tab.id)}
              style={{
                /* active vs inactive styling */
                background: active ? '#FFF8E7' : 'transparent',
                color: active ? '#0F5132' : '#6B7280',
                borderBottom: active ? '4px solid #DC2626' : '4px solid tran
                /* ... */
              }}
            >
              <Icon size={20} />
              {tab.label}
            </button>
          );
        })}
      </nav>

```

```

    {/* Active tab content */}
    <main style={{ maxWidth: 1400, margin: '0 auto', padding: '32px 24px' }}
      {activeTab === 'standings' && (
        <StandingsTab
          teams={teamsWithScores}
          onTeamClick={(t) => {
            setSelectedTeam(t);
            setActiveTab('predictions');
          }}
        />
      )}
      {activeTab === 'matches' && <MatchesTab />}
      {activeTab === 'predictions' && (
        <PredictionsTab
          teams={teamsWithScores}
          selectedTeam={selectedTeam}
          setSelectedTeam={setSelectedTeam}
        />
      )}
      {activeTab === 'players' && <PlayersTab teams={teamsWithScores} />}
      {activeTab === 'analytics' && <AnalyticsTab teams={teamsWithScores} />}
    </main>
  </div>
);
}

```

I've abbreviated the Hero and outer styling (full version is in `components/SnookerFantasyLeague.tsx`). The skeleton is what matters. Walk through every part below.

Walking the orchestrator, top to bottom

Line 1: 'use client'

Required because we use `useState`. Without it, Next.js would try to render this as a server component and crash on the first `useState` call.

Lines 3–13: imports

Standard imports. Five tab components, five icons, the data and helpers. **Order matters slightly**: third-party (react, lucide-react), then internal (@/lib/...), then local (./tabs/...). Keeping that order makes the imports readable at a glance.

Lines 15: a string-union type for tab IDs

```
type TabId = 'standings' | 'matches' | 'predictions' | 'players' | 'analytics'
```

Five strings, no others. TypeScript will reject any other value.

This is **the cheapest, most underrated TypeScript pattern**: union types of string literals. They give you autocomplete and typo-protection for free. Compare to string everywhere — you’d have no protection against setActiveTab('predicions') (note the typo).

Use string unions for any value that has a fixed, small set of options. Round IDs ('r1' | 'r2' | 'qf' | 'sf' | 'f'), tab IDs, status enums ('idle' | 'loading' | 'success' | 'error'), etc.

Lines 18–19: the two state cells

```
const [activeTab, setActiveTab] = useState<TabId>('standings');  
const [selectedTeam, setSelectedTeam] = useState<TeamWithScores | null>(null)
```

Two useState calls. Each gets:

- An explicit type parameter (<TabId>, <TeamWithScores | null>) — useful when the initial value alone doesn’t pin down the type.
 - useState<TabId>('standings') — the initial value 'standings' could be inferred as 'standings' (a literal type), too narrow. Adding <TabId> widens it to “any tab ID”.
 - useState<TeamWithScores | null>(null) — without the type param, TS would infer null as the type forever, and you’d get a “can’t assign TeamWithScores to null” error when you tried to set it.
- An initial value ('standings', null).

Two pieces of state. That’s the whole app. Compare to the imagined Redux version with reducers, actions, slices, dispatchers, selectors. Two useStates, no library, no boiler-

plate.

Lines 21–24: the memoized derived data

```
const teamsWithScores = useMemo<TeamWithScores[]>(() => {  
  return TEAMS.map(t => ({ ...t, scores: calculateTeamScores(t) })))  
  .sort((a, b) => b.scores.total - a.scores.total);  
}, []);
```

useMemo is Lesson 3 of this chapter. For now, the short version: it caches a computed value across renders, only recomputing when the dependency array changes. The empty array [] means “never recompute” — teamsWithScores is computed once on mount and reused on every subsequent render.

Why memoize? Two reasons:

1. **Performance** — without useMemo, every tab click would re-run calculateTeamScores for all 8 teams. Wasteful.
2. **Reference stability** — without useMemo, the teamsWithScores array would be a *new array reference* on every render. That can cause downstream useMemo and useEffect re-runs in child components. Stable references are quietly important.

We’ll see both reasons play out in Lesson 3.

Lines 26–32: the tabs configuration

```
const tabs: { id: TabId; label: string; icon: typeof Trophy }[] = [  
  { id: 'standings', label: 'Standings', icon: Trophy },  
  { id: 'matches', label: 'Matches', icon: Calendar },  
  // ...  
];
```

A **data-driven nav bar**. Define an array of tab descriptors; map over it to render buttons. Adding a new tab is one line in this array (plus the corresponding tab component, of course). Removing one is also a one-line edit.

This is dramatically better than hard-coding five <button> elements with their labels. **Configuration over repetition** is a core principle.

The `icon: typeof Trophy` type — that’s TypeScript syntax for “*the same type as the value `Trophy`*”. Each Lucide icon has the same component type, so we just use one as the canonical example.

Lines 35–80: the JSX

The orchestrator returns three top-level pieces:

1. **The Hero** — a big gradient banner with the app name and the champion badge. (Pure decoration; could be its own component.)
2. **The nav** — sticky tab bar.
3. **The main** — switches on `activeTab` to render the right tab.

Walk the tab bar:

```
{tabs.map(tab => {
  const Icon = tab.icon;
  const active = activeTab === tab.id;
  return (
    <button
      key={tab.id}
      onClick={() => setActiveTab(tab.id)}
      style={{
        background: active ? '#FFF8E7' : 'transparent',
        color: active ? '#0F5132' : '#6B7280',
        borderBottom: active ? '4px solid #DC2626' : '4px solid transparent'
      }}
    >
      <Icon size={20} />
      {tab.label}
    </button>
  );
})}
```

`tab.icon` is renamed to local `Icon` (capitalized so JSX recognizes it). The `active` boolean controls every visual difference: background, color, border. Three branches in one component, all tied to one state value.

Click a tab ✕ **setter fires** ✕ **state changes** ✕ **orchestrator re-renders** ✕ **all five buttons re-**

render with new active values ✎ **DOM updates**. Six steps, one click. Memorize the loop.

The cross-tab interaction

This is the part that makes the app feel like an app:

```
{activeTab === 'standings' && (  
  <StandingsTab  
    teams={teamsWithScores}  
    onTeamClick={(t) => {  
      setSelectedTeam(t);  
      setActiveTab('predictions');  
    }}  
  />  
)}
```

When the user clicks a team row in `<StandingsTab>`, the callback runs **two state setters back-to-back**:

1. `setSelectedTeam(t)` — store the clicked team.
2. `setActiveTab('predictions')` — switch the visible tab.

React's automatic batching collapses these into **one re-render**. Both states change atomically. The user clicks ✎ both states update ✎ orchestrator re-renders ✎ predictions tab is shown with the right team selected.

That's the orchestrator pattern in action. Two setter calls in one click, multiple components affected, one consistent re-render. State at the top, behavior glued together at the top, leaves are pure.

The && short-circuit

```
{activeTab === 'standings' && <StandingsTab ... />}
```

This is JavaScript's logical `&&` operator. If the left side is **truthy**, the right side is evaluated and returned. If the left side is **falsy**, the right side is short-circuited and `&&` returns the left side (the falsy value).

For our use case: if `activeTab === 'standings'` is true, we get the `<StandingsTab />` element. If false, we get false itself, which React renders as nothing. **That's how**

conditional rendering works in JSX: render or don't render based on a condition.

The classic `&&` gotcha is when the left side is `0`:

```
{count && <Badge>{count}</Badge>}
```

If `count` is `0`, React renders... `0`. Because `0 && anything` is `0`, and React renders `0` as text! Use `{count > 0 && ...}` or a ternary `{count ? <Badge>{count}</Badge> : null}` instead.

Updating the `tab` components to receive props

`<StandingsTab>` from Chapter 3 used `TEAMS` directly. Now it needs to receive `teams` from the parent:

```
import type { TeamWithScores } from '@/lib/types';
import LeagueTable from '../standings/LeagueTable';

interface StandingsTabProps {
  teams: TeamWithScores[];
  onTeamClick?: (team: TeamWithScores) => void;
}

export default function StandingsTab({ teams, onTeamClick }: StandingsTabProps) {
  return (
    <div>
      {/* Stat cards above */}
      <LeagueTable teams={teams} onTeamClick={onTeamClick} />
    </div>
  );
}
```

What changed: `<StandingsTab>` no longer imports `TEAMS` or calls `calculateTeamScores`. It receives `teams` from the orchestrator. **The orchestrator is now the single source of truth.**

Stub the other four tabs

For the orchestrator to compile, all five tab files must exist. Create stubs for the four we haven't built:

```
// components/tabs/MatchesTab.tsx
export default function MatchesTab() {
  return (
    <div style={{ padding: 32, textAlign: 'center', color: '#6B7280' }}>
      <h2>Matches</h2>
      <p>Coming soon – Chapter 5.</p>
    </div>
  );
}
```

Repeat for PredictionsTab.tsx, PlayersTab.tsx, AnalyticsTab.tsx. Each takes whatever props the orchestrator passes (or none) and renders a placeholder. The compiler is happy; the runtime is happy; you can switch tabs and see “Coming soon” placeholders for the unimplemented ones. Save and refresh.

You should now be able to click between tabs. Standings shows the full table. The others show placeholders. The hero header is sticky, the tab bar is sticky, the main content scrolls.

Why does state live where it lives?

Three pieces of state in this app:

1. **activeTab** – used by the orchestrator (to decide which tab to render) and the tab buttons (to highlight the active one). Lowest common ancestor: the orchestrator. **Lives there.**
2. **selectedTeam** – set by <StandingsTab> (via onTeamClick) and read by <PredictionsTab>. Lowest common ancestor: the orchestrator. **Lives there.**
3. **round** (inside <MatchesTab>) – set by tab strip clicks inside MatchesTab, read by MatchesTab itself. Lowest common ancestor: MatchesTab. **Lives in MatchesTab.**

The pattern: **each piece of state lives at the lowest component that needs to read or write it.** Higher than that = unnecessary prop drilling. Lower than that = inability to share.

This is “**lifting state up**” – when two siblings need to share, you lift the state to their parent.

Lifting state is the canonical solution for sibling communication in React. Don't pass functions through siblings, don't use refs, don't reach for a global store. Just lift.

Why this is interview gold

If an interviewer hands you a multi-screen React problem on a whiteboard and you start by asking *"what state do we have, and where does it live?"* — you've already aced the first 60% of the interview. **The vast majority of senior React interviews are about state location,** not about hooks or lifecycle methods or rendering tricks.

Concretely, in interviews:

- *"How would you let one component trigger a change in another?"* ☞ "Lift the state to their common ancestor; pass the value down as a prop and a setter down as a callback."
- *"How would you avoid prop drilling for deeply nested data?"* ☞ "Three levels is fine; for more, consider Context or a state library. But I'd default to drilling first because it's explicit."
- *"How would you share state between two routes?"* ☞ "Move it above the router. URL params if it should be bookmarkable. Top-level state if not."

All of those answers reduce to *"state lives at the lowest common ancestor of its readers and writers."* Memorize that one rule and you've got most of state management licked.

Why we DON'T use Context yet

Context is React's built-in alternative to prop drilling. It looks like this:

```
const TeamsContext = createContext<TeamWithScores[]>([]);

// Provider in the orchestrator
<TeamsContext.Provider value={teamsWithScores}>
  {children}
</TeamsContext.Provider>

// Consumer anywhere below
const teams = useContext(TeamsContext);
```

We don't use it. Why?

- We have **3 levels** of nesting at most (orchestrator → tab → leaf). Three levels of teams props is fine.
- Context has its own pitfalls: every Provider value change re-renders **all consumers** unless you split it carefully. Easy to misuse.
- Context **hides** the data flow. With prop drilling you can Cmd+F "teams" and see exactly where it goes. With Context, you can't.

Context is for cross-cutting concerns — theme, auth user, locale — not for general state. The internet's love affair with Context is misplaced for many cases.

If we ever needed deeply-nested access, **Zustand** would be a better choice for our app size. But we don't, so we don't.

Senior interview answer: *"Context is great for theme/auth/locale. For application data, I default to prop drilling because it's explicit and Cmd+F-able. If drilling becomes painful, I'd reach for Zustand or Jotai before Context."*

Vibe prompt you would have used

*"Build the <SnookerFantasyLeague> orchestrator at components/SnookerFantasyLeague. Mark with 'use client'. Two useStates: activeTab: 'standings' | 'matches' | 'predictions' | 'players' | 'analytics' (default 'standings'), selectedTeam: TeamWithScores | null (default null). One useMemo that maps TEAMS through calculateTeamScores and sorts by .scores.total desc. Render: a hero header with the app title and champion badge, a sticky tab nav bar with five Lucide-icon buttons, then a <main> that conditionally renders the matching tab component. From <StandingsTab>'s onTeamClick, set selectedTeam AND switch activeTab to 'predictions'. Stub <MatchesTab>, <PredictionsTab>, <PlayersTab>, <AnalyticsTab> as one-line 'Coming soon' placeholders. **Don't use Context. Don't use Redux.** Just useState + props."*

The **"don't use X"** instructions are critical here. The default LLM reflex is to introduce Context or a state library "for cleanliness" — and you'll get back 200 lines of boilerplate. Forbid it.

CHECK YOURSELF

1. `activeTab` is state at the orchestrator level. `round` (inside `MatchesTab`) is state at the tab level. What's the principle that determines which level each lives at?
2. Walk through every render that happens when the user clicks a team row in `<StandingsTab>`. How many setter calls? How many re-renders? Which components re-render?
3. Why does the orchestrator have `'use client'` but `app/layout.tsx` doesn't? What's the boundary?
4. You add `console.log('orchestrator')` at the top of `<SnookerFantasyLeague>`'s function body. Then you click "Predictions". How many times does it log?
5. A teammate suggests moving `selectedTeam` into `<PredictionsTab>` "to keep state close to where it's used." Why is this the wrong move? What would break?

When you've answered these, head to [03-useMemo-and-derived-data.md](#) — the final lesson of this chapter.

Chapter 4 · Lesson 3 — `useMemo` and Derived Data

*Goal: understand `useMemo` deeply enough that you know not just **how** to use it, but **when not to**. By the end of this lesson, you can defend every memoization decision in this codebase.*

What is `useMemo`?

`useMemo` is a React hook that **caches the result of a computation across renders**. The function inside it runs only when its dependencies change.

```
const teamsWithScores = useMemo(() => {
  return TEAMS.map(t => ({ ...t, scores: calculateTeamScores(t) }))
    .sort((a, b) => b.scores.total - a.scores.total);
}, []);
```

Two arguments:

1. **A function** that produces the value.
2. **A dependency array** that tells React when to re-run.

If the dependency array is `[]` (empty), the value is computed once and reused forever. If it's `[someValue]`, React re-runs whenever `someValue` changes.

In our orchestrator, TEAMS is a module-level constant — it never changes during the app's lifetime. So `[]` is correct: compute the decorated team list once and reuse it.

What does “across renders” mean?

Recall from Lesson 1: every state change re-runs the component function from the top. Without `useMemo`, this:

```
const teamsWithScores = TEAMS.map(t => ({ ...t, scores: calculateTeamScores(t)
  .sort((a, b) => b.scores.total - a.scores.total);
```

...runs on **every render**. Click a tab → re-render → recompute scores. Type in a search box → re-render → recompute scores. Resize the window → re-render → recompute scores.

For 8 teams × 31 picks each, the wasted work is microseconds. **Performance isn't the main reason to memoize this.** The main reason is something subtler: **reference equality**.

Reference equality vs value equality

In JavaScript, object and array equality is **by reference**, not by content:

```
const a = [1, 2, 3];
const b = [1, 2, 3];
a === b;           // false! different references
a === a;           // true – same reference

const c = a;
c === a;           // true – same reference (just an alias)
```

Two arrays with the same content are NOT equal in JS. Only the literal same array (same memory address) is.

This matters because **React uses reference equality** to decide whether props or state changed. If you pass `teamsWithScores` as a prop:

```
<StandingsTab teams={teamsWithScores} />
```

...and `teamsWithScores` is a **new array on every render** (because we recomputed it), then React thinks the prop changed every render — even though the contents are identical. `<StandingsTab>` and everything below it re-renders unnecessarily.

`useMemo` with `[]` deps **freezes the reference**. The exact same array object is reused across renders. React's reference comparison sees "same reference, no change, skip re-render of children." (With `React.memo` / proper reconciliation, anyway. We'll talk about `React.memo` in a sec.)

Why this matters: the cascade

Without `useMemo`, here's what happens when the user clicks a tab:

1. Orchestrator re-renders.
2. `teamsWithScores = TEAMS.map(...)` — new array, new reference.
3. `<StandingsTab teams={teamsWithScores} />` — React sees the prop is "different".
4. `<StandingsTab>` re-renders.
5. Inside, `<LeagueTable teams={teams} />` — new prop reference cascading down.
6. `<LeagueTable>` re-renders.
7. Each `<tr>`'s child components re-render.

Every component below the orchestrator re-renders, on every state change, even when the data is identical. **Cascading wasted work.**

With `useMemo`, the reference is stable. React compares old prop to new prop, sees they're the same reference, and *can* skip re-rendering. (Whether it actually skips depends on whether components are wrapped in `React.memo` — see below.)

For our app's size, the actual performance difference is invisible. But the **mental model** is what matters: **stable references mean predictable rendering**. Senior engineers care about this even when it's not measurably slow.

When NOT to memoize

This is where most beginners go wrong. They learn `useMemo` and start wrapping every computation in it. **Don't.**

Three situations where `useMemo` is **wrong**:

1. The computation is cheap.

```
// ❌ Pointless – addition is faster than the memoization machinery
const total = useMemo(() => count + 1, [count]);

// ✅ Just compute it
const total = count + 1;
```

useMemo itself has overhead. React stores the value, the dependencies, compares dependencies on every render. For trivial computations, the memoization machinery costs **more** than the computation it's avoiding.

Rule of thumb: **don't memoize anything cheaper than calling a function**. Arithmetic, string concatenation, simple property access – leave them alone.

2. The result isn't being passed to a child or used in another hook's deps.

```
// ❌ Pointless – totalGames is only used in this render
function Component() {
  const games = useState(...)[0];
  const totalGames = useMemo(() => games.length, [games]); // why?
  return <div>{totalGames}</div>;
}

// ✅ Just compute it
function Component() {
  const games = useState(...)[0];
  return <div>{games.length}</div>;
}
```

Memoization protects against unnecessary work *somewhere else*. If the value never leaves the current render, there's nothing to protect.

3. The dependencies change every render anyway.

```
// ❌ The dep array contains a new object every render – useMemo never hits i
const config = useMemo(() => ({ apiKey, timeout: 5000 }), [{ apiKey, timeout
```

```
// ❗ same problem with inline functions
useEffect(() => fetchData(config), [config]); // re-runs every render
```

If your deps change every render, you've defeated the purpose of memoization.

The senior rule

Memoize when the result is referentially unstable AND that instability causes waste downstream. In other words: when the result is passed to a child component, used in another hook's deps, or put in Context. Otherwise leave it alone.

For our orchestrator, `teamsWithScores` is passed to **four different tab components** as a prop. That's clear downstream waste if the reference flickers. Memoize.

What about `React.memo`?

This is a separate but related tool. Different name, related job.

```
import { memo } from 'react';

const StandingsTab = memo(function StandingsTab({ teams, onTeamClick }) {
  // ...
});
```

`React.memo` wraps a component and tells React: *"only re-render this component if its props' references actually changed."* Without memo, every parent re-render re-renders the child even if props are the same reference.

`useMemo` keeps a value stable. `React.memo` makes a component skip re-renders. They're often used together: memoize the props, then memo the component, then the component skips re-renders when the parent re-renders for unrelated reasons.

In our codebase we don't use `React.memo` because the app is small enough that the unnecessary re-renders are imperceptible. But you'll see it in larger codebases — it's worth knowing the pattern.

Interview question: *"What's the difference between `useMemo` and `React.memo`?"* Answer: *"`useMemo` caches a **value** across renders. `React.memo` wraps a **component** and skips re-renders when props don't change. They're*

often used together: stable values via useMemo, plus component skipping via React.memo."

What about useCallback?

useCallback is useMemo for functions. It caches a function reference across renders.

```
// Without useCallback: new function every render
<StandingsTab onClick={(t) => { setSelectedTeam(t); setActiveTab('predic

// With useCallback: stable reference
const handleTeamClick = useCallback((t) => {
  setSelectedTeam(t);
  setActiveTab('predictions');
}, []);
<StandingsTab onClick={handleTeamClick} />
```

Same use case as useMemo: protect downstream children that compare props by reference. **Same rules apply** — only worth it if the resulting function is passed somewhere or used in another hook's deps.

In our orchestrator we don't bother with useCallback. Why? Because <StandingsTab> is rendered fresh on every active-tab match anyway (it only mounts when activeTab === 'standings'); the inline function reference doesn't ripple beyond it. Optimizing here would be theater, not a real win.

If we ever introduced React.memo on <StandingsTab> we'd want to wrap the callback in useCallback. Until then, no.

When the dependency array is wrong

The dependency array is **the most common bug source** in useMemo and useEffect. Two failure modes:

Missing a dependency

```
// BUG — depends on `prefix` but doesn't list it
const greeting = useMemo(() => `${prefix} ${name}`, [name]);
```

If prefix changes but name stays the same, the memo doesn't recompute and you get a stale value. ESLint's `react-hooks/exhaustive-deps` rule catches this — **don't disable it**.

Including a dependency that breaks memoization

```
// BUG – config is recreated every render, so this never hits the cache
function Component({ config }) {
  const result = useMemo(() => expensiveCalc(config), [config]);
}
```

If the parent passes `config={{ a: 1 }}` inline, `config` is a new object every render. The memo never caches. Either memoize `config` in the parent or pass primitives instead of objects.

The right way

The eslint rule is your safety net. Fix what it tells you. **Trust ESLint over your gut** for hooks-related rules — they encode subtle React behavior that's easy to miss.

Why we use `useMemo` for `teamsWithScores`

Recap of our orchestrator:

```
const teamsWithScores = useMemo<TeamWithScores[]>(() => {
  return TEAMS.map(t => ({ ...t, scores: calculateTeamScores(t) }))
    .sort((a, b) => b.scores.total - a.scores.total);
}, []);
```

Justification:

- **Passed to four tabs as a prop.** Stable reference helps any future `React.memo` we add.
- **Computed from a module-level constant** (`TEAMS`). Empty dep array `[]` is correct — never recompute.
- **Marginally cheaper** — saves $8 \times 31 = 248$ microsecond-y `scorePick` calls per render.

If TEAMS were loaded from an API and stored in `useState`, the `dep` array would need to include the state variable: `useMemo(() => ..., [teamsRaw])`. Whenever the API state updated, the memo would recompute.

Why we DON'T use `useMemo` for the `tabs` array

```
const tabs: { id: TabId; label: string; icon: typeof Trophy }[] = [
  { id: 'standings', label: 'Standings', icon: Trophy },
  // ...
];
```

This array is also recreated every render. Why don't we memoize it?

Three reasons:

1. **It's not passed as a prop anywhere** — it's only consumed within the orchestrator's own JSX. No downstream cascade.
2. **It's small** — five entries, no real computation cost.
3. **Memoizing it would add boilerplate without measurable benefit.**

This is the senior call: **memoize where it matters, leave where it doesn't**. The result is more readable code and no measurable performance hit.

A common pitfall: memoizing JSX

You may see this in the wild:

```
// Anti-pattern — memoizing JSX is rarely worth it
const heroSection = useMemo(() => (
  <div>...</div>
), []);
return <div>{heroSection}</div>;
```

JSX inside `useMemo` is a **code smell**. JSX is cheap to evaluate. The “memoization” doesn't actually skip re-renders of the inner components; React re-renders them based on their own props. You've added complexity and gained nothing.

If you want to skip re-rendering of a component, **use `React.memo` on the component itself**, not `useMemo` on its JSX.

What about state machines and useReducer?

useReducer is useState's big sibling — useful when state has complex transitions:

```
const [state, dispatch] = useReducer(reducer, initialState);
dispatch({ type: 'TEAM_SELECTED', team });
```

Pros: state transitions are explicit (each action type is documented). Cons: more boilerplate.

For our app, useState is the right call because we have two simple state cells. If selectedTeam updates ever became coupled to other state changes (e.g. “selecting a team also clears a filter and updates the URL”), useReducer would start to pay for itself.

Interview tip: “useState vs useReducer?” Answer: “useState for independent values. useReducer when state transitions become complex enough that an explicit action type makes the code clearer. They’re both stored as state — useReducer is just a different ergonomic interface.”

The whole orchestrator's hook list

Re-read our orchestrator with hooks in mind. The hook calls in order are:

1. useState<TabId>('standings') — first state cell.
2. useState<TeamWithScores | null>(null) — second state cell.
3. useMemo(() => ..., []) — first memo cell.

Three hook calls. **They run in this exact order on every render.** That's why “rules of hooks” exist — React indexes them by order.

If you moved any of them inside an if block, React would lose track of the indexing on renders where the condition was false. Hence the rule: **always at the top level of the function body.**

Vibe prompt you would have used

“Wrap the teamsWithScores computation in the orchestrator with useMemo. The dependency array is empty (TEAMS is module-level). Add the explicit type parameter <TeamWithScores[]>. Don't memoize the tabs array (it's only used inside this component). Don't add useCallback for the inline event handlers

(no `React.memo`'d children consume them). Add a one-line comment explaining why `useMemo` with `[]` deps."

The big picture

After this lesson you should be able to answer these in interviews:

- *"What does `useState` do?"* — Allocates a state cell. Returns the current value and a setter. Setter triggers a re-render.
- *"What does `useMemo` do?"* — Caches a computed value across renders. Re-runs only when deps change.
- *"What does `React.memo` do?"* — Wraps a component to skip re-renders when props haven't changed by reference.
- *"What does `useCallback` do?"* — Like `useMemo` but for functions.
- *"When should I memoize?"* — When a value is passed as a prop to many children, used in another hook's deps, or put in Context, AND its reference would otherwise be unstable.
- *"What's the difference between state and derived data?"* — State is the source of truth. Derived data is computed from state. Don't store derived data in state.
- *"Where should state live?"* — At the lowest common ancestor of every component that reads or writes it.
- *"Why are hooks ordered by call sequence?"* — Because React tracks them by call index. They must be called in the same order every render.

That list is **most of the React-knowledge surface area for a mid-level interview**. If you can answer all eight crisply with a real example from this codebase, you're hireable.

CHECK YOURSELF

1. **Why does `useMemo<TeamWithScores[]>(() => ..., [])` use an empty dependency array?** What would change if TEAMS were loaded from an API into a `useState` instead?
2. **What's the difference between `useMemo` and `React.memo`?** Give one scenario where you'd use one but not the other.
3. **A teammate wraps every computation in `useMemo` "for performance." What do you tell them?** Be specific about when memoization helps and when it just adds complexity.

4. You add `console.log('memo running')` inside the `useMemo` callback. After app boots and the user clicks 4 different tabs, how many times does it log?
5. The `tabs` array is recreated every render. Why is this fine? What would change if you started passing `tabs` as a prop to a memoized `<TabBar>` component?

When you've answered these, you've finished Chapter 4. Onward to [Chapter 5 – Composition](#) — where we break the big tabs into reusable components.

Chapter 4 — State & Hooks

State is what makes a static page interactive. This chapter introduces `useState`, `useMemo`, the orchestrator pattern, and the most-asked React interview concept of all: lifting state up.

What you'll learn

- What **state** is and why it's different from props.
- How `useState` works — the array-destructuring syntax, the setter, the re-render cycle.
- **Lifting state up** — the canonical React pattern for “two components share data.”
- The **orchestrator** component — where state lives in real apps.
- **`useMemo`** for derived data — when it's necessary, when it's not.
- **Cross-component navigation** — clicking in one tab moves to another.
- Why **state is a tree, not a graph** — and why that constraint is a feature, not a limitation.

Lessons in this chapter

1. [01-useState-mental-model.md](#) — Your first hook. What state is, what `useState` returns, the re-render cycle, the rules of hooks. The mental model that takes most beginners weeks to internalize.
2. [02-the-orchestrator-pattern.md](#) — Lifting state into the parent. Tab switching, cross-tab interactions, callbacks-down. This is the chapter where the app starts feeling like an app.
3. [03-useMemo-and-derived-data.md](#) — When (and when not) to memoize. Reference equality, identity stability, and when not optimizing is the right call.

Files you'll create or update

```
components/
├── SnookerFantasyLeague.tsx    ← UPDATED: orchestrator with state, tab nav, he
├── tabs/
│   ├── StandingsTab.tsx      ← UPDATED: receives teams via props, no longer sel
contained
│   ├── MatchesTab.tsx        ← NEW: stub for now
│   ├── PredictionsTab.tsx    ← NEW: stub for now
│   ├── PlayersTab.tsx        ← NEW: stub for now
│   └── AnalyticsTab.tsx      ← NEW: stub for now
```

The non-Standings tabs are stubs in this chapter (“Coming soon” placeholders). Chapters 5 and 6 fill them in.

New React/Next.js concepts introduced

- **useState** — the most-used hook in React.
- **The re-render cycle** — what triggers a component to re-execute.
- **Lifting state up** — moving state to a common ancestor.
- **Controlled components** — parent owns the data; child receives + dispatches updates.
- **useMemo** — caching expensive computations across renders.
- **Reference equality** vs **value equality**.
- **Rules of hooks** — and why they exist.

How long it should take

- 30–45 min reading per lesson
- 1–2 hours typing the code
- ~4 hours total

Why this chapter is the most important in the course

Honestly? Because **state is what gets you hired**.

Once you can answer “where does this state live?” correctly, every other React question gets easier. UI structure, data flow, performance, testing — they all derive from where the

state is. Junior interviews ask “*what’s a hook?*” — easy. Senior interviews ask “*why does the state live here and not there?*” — that’s this chapter.

If you only deeply read three chapters of this course, make it 1, 4, and 7. The rest is execution; these are the architecture.

Before you start

You should have:

- Chapter 3 complete: a fully working standings tab with stat cards, medal-color rank circles, per-round columns, Final Pick chip, FormBadge.
- `components/SnookerFantasyLeague.tsx` as a stub with `'use client'` and `<StandingsTab />` inside.
- `npm run dev` running and the page rendering correctly.

Open [01-useState-mental-model.md](#) when ready.

Chapter 5 · Lesson 1 — When to Split a Component

Goal: develop the senior judgement of when JSX deserves its own component file. Two extremes are wrong; the middle is craft.

The two failure modes

Beginners fail in opposite ways:

Failure mode 1 — the giant component. Everything in one file. 800 lines. One return statement that wraps a hero, a nav, three tabs, a footer, all inline. Hard to read, hard to test, hard to find anything.

Failure mode 2 — over-extraction. Every `<div>` is its own component. `<TeamNameLabel>`, `<TeamNameLabelInner>`, `<TeamNameLabelInnerSpan>`. Hard to read for the opposite reason — every variable name is a click-through to another file.

Senior engineers live in the middle. **Split when splitting buys you something.** Don’t split for splitting’s sake.

The three real reasons to split

Every legitimate component extraction passes one of these tests:

Test 1: The copy-paste test

If you're about to copy a chunk of JSX with minor variations, **extract first**.

Example from our codebase, inside `<MatchPickAnalysis>`:

```
<div>
  <div>BACKED {playerName.toUpperCase()}</div>
  {teamsBackedThis.map(t => <TeamChip team={t} ptsEarned={ptsForPicker} />)}
</div>
<div>
  <div>BACKED {opponent.toUpperCase()}</div>
  {teamsBackedOpponent.map(t => <TeamChip team={t} ptsEarned={ptsForOpponent} />)}
</div>
```

The two `<TeamChip>` map calls are **identical structure**, only the input list and the points value differ. If `<TeamChip>` were inlined as 20 lines of JSX, copy-pasting it twice would smell. Extracted, both call sites are one line.

The rule: **if you're about to copy-paste 10+ lines of JSX, extract first**.

Test 2: The “what vs how” test

If a chunk of JSX is conceptually about *one specific thing* and the rest of the component is about *something else*, the chunk should be its own component.

The parent says **what** (`<PathStep label="QF" status="won" />`). The child knows **how** (which colors, which icon, which sizes for each status).

Example: the tournament path indicator at the top of `<PlayerDetail>`:

```
<PathStep label="R1" status={r1Won ? 'won' : 'lost'} />
<PathArrow />
<PathStep label="R2" status={r2Won ? 'won' : 'lost'} />
<PathArrow />
<PathStep label="QF" status={qfFinished ? (qfWon ? 'won' : 'lost') : 'live'} />
<PathArrow />
```

```
<PathStep label="SF" status={...} />
<PathArrow />
<PathStep label="F" status={...} />
```

Imagine inlining this. Each `<PathStep>` is ~20 lines of JSX (status colors, icon, padding, border). Multiplied by 5 = 100 lines. Plus 4 inline `<PathArrow>` spans. The parent component would lose all sense of its **own** structure under the weight of the path-rendering code.

Extracted, the parent reads like a sentence: “*step, arrow, step, arrow, step.*” That clarity is the win. **The component name is documentation.**

Test 3: The reusability test

If a chunk of JSX will be used in 2+ places (now or soon), it deserves its own component.

`<StatCard>` from Chapter 3 is used 3 times in the standings (Leader, Matches, Avg Score). It’s used again in `<PlayerDetail>` for per-player stats. **Four uses. Obvious extraction.**

But: don’t pre-extract for *hypothetical* reuse. Wait until you have the second use case. Premature extraction often picks the wrong abstraction; you end up retro-fitting the component to the second use case and making it worse for both.

Rule of three. Some authors say: extract on the third use. The first time you write something, write it. The second time, copy-paste and feel a small pang of guilt. The third time, extract. By then you understand the abstraction well enough to design it right.

For our codebase, we extract on the **second** use because the codebase is small and TypeScript catches misuse. For larger projects, three is fine.

The case for tiny components

`<PathArrow>` is **one character**:

```
export default function PathArrow() {
  return (
    <span style={{ color: 'rgba(255,255,255,0.45)', fontSize: 18, fontWeight
```

```
);  
}
```

It has no props. It does no logic. It's just a styled →. Why is it a component?

Because **the parent's JSX reads like a sentence with it:**

```
<PathStep label="R1" status="won" />  
<PathArrow />  
<PathStep label="R2" status="won" />  
<PathArrow />  
<PathStep label="QF" status="lost" />
```

Inlined, the same code:

```
<PathStep label="R1" status="won" />  
<span style={{ color: 'rgba(255,255,255,0.45)', fontSize: 18, fontWeight: 700 }}>  
<PathStep label="R2" status="won" />  
<span style={{ color: 'rgba(255,255,255,0.45)', fontSize: 18, fontWeight: 700 }}>  
<PathStep label="QF" status="lost" />
```

The reader's eye loses the structure under the styled spans. The component **names the visual unit** and lets the structure breathe.

Tiny components like this aren't about reuse or DRY — they're about **readability of the parent**. The parent component is the customer; the tiny component serves it.

When NOT to split

Three signals that an extraction is wrong:

Signal 1: The component takes too many props.

```
// Smell – 8 props is too many  
<TeamRow  
  rank={i + 1}  
  name={team.name}  
  icon={team.icon}  
  total={team.scores.total}
```

```

    r1={team.scores.r1}
    r2={team.scores.r2}
    finalPick={team.final}
    isLeader={i === 0}
  />

```

Eight props mean the abstraction isn't right. Either:

- The component should just accept the whole team object: `<TeamRow team={team} rank={i+1} />`. That hides the data shape behind one prop.
- The component is doing too many jobs and should be split into smaller ones.

In our codebase, components rarely have more than 4–5 props. **5 props is the warning line.** 8+ is “rethink this.”

Signal 2: The component has logic that depends on its parent's context.

```

function TeamRow({ team, parentTabState, parentSelectedTeam, parentSetters }
  // ...
}

```

If a child component accepts a bag of “parent state” as a prop, it's leaking the parent's internals into the child. Either lift the **logic** to the parent (and pass primitives down), or restructure so the child genuinely only needs primitives.

Signal 3: Extraction makes the parent harder to read.

```

<TeamSection>
  <TeamSectionHeader>
    <TeamSectionHeaderTitle>{title}</TeamSectionHeaderTitle>
  </TeamSectionHeader>
  <TeamSectionBody>...</TeamSectionBody>
</TeamSection>

```

Five layers of components for what could be a single `<section>` with two `<div>`s. **Don't introduce abstractions you can't justify.**

If extraction makes the parent component longer and the new file barely justifies its own

existence, undo the extraction.

The composition pattern: parent says what, child knows how

This is the central pattern of React composition:

- **Parent components** are about **layout and orchestration**. They render lists, switch between views, decide which children show up. Their JSX is structural.
- **Child components** are about **rendering one thing well**. They take props, return JSX, don't know about the parent.

The classic test: can you delete the child component file and copy its JSX back inline, without touching the parent's logic? If yes, the split is clean. If you have to drag parent state down with you, the split was leaky.

Walkthrough: extracting <TeamChip>

Let's see this in action. Inside <MatchPickAnalysis> you have 8 team chips per match – 16 if you count both sides. The chip looks like:

```
<div style={{
  background: '#FFFFFF',
  borderRadius: 8,
  padding: '8px 12px',
  display: 'flex',
  alignItems: 'center',
  gap: 10,
  border: `2px solid ${team.accent}66`,
  boxShadow: `0 2px 6px ${team.accent}20`,
}}>
  <div style={{ fontSize: 22 }}>{team.icon}</div>
  <div style={{ flex: 1, fontWeight: 700, color: '#1F2937', fontSize: 14 }}>
    {ptsEarned !== null && (
      <div style={{
        background: ptsEarned === 3 ? '#16A34A' : '#92400E',
        color: '#FFFFFF',
        padding: '3px 10px',
        borderRadius: 6,
```

```

        fontSize: 12,
        fontWeight: 800,
      }}>
      +{ptsEarned}
    </div>
  )}
</div>

```

15 lines. Used 16+ times per match. Extract:

```

// components/players/TeamChip.tsx
import type { Team } from '@lib/types';

interface TeamChipProps {
  team: Team;
  ptsEarned: number | null;
}

export default function TeamChip({ team, ptsEarned }: TeamChipProps) {
  return (
    <div style={{ /* 5 styles */ }}>
      <div>{team.icon}</div>
      <div>{team.name}</div>
      {ptsEarned !== null && (
        <div style={{ background: ptsEarned === 3 ? '#16A34A' : '#92400E', /
          +{ptsEarned}
        </div>
      )}
    </div>
  );
}

```

The parent's call site is now one line:

```
{teamsBackedThis.map(t => <TeamChip key={t.name} team={t} ptsEarned={ptsForP
```

Cleaner. Reusable. The parent is shorter. The chip has a name. **All three benefits at once — that's a good extraction.**

Walkthrough: when NOT to extract <TwoColumnLayout>

Inside <MatchPickAnalysis>, the layout is:

```
<div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: 16 }}>
  <div>...left column with chips...</div>
  <div>...right column with chips...</div>
</div>
```

Could we extract <TwoColumnLayout>?

```
<TwoColumnLayout>
  <div>...left column...</div>
  <div>...right column...</div>
</TwoColumnLayout>
```

Maybe. But:

- It's used **once** in this codebase. **Rule of three** says wait.
- The grid template is 4 lines of CSS. Saving 4 lines isn't worth a new file.
- The name is generic — TwoColumnLayout is at the wrong level of abstraction (CSS), not domain (snooker fantasy).

So we **don't** extract. The CSS stays inline. **Discipline of saying no.**

The children prop — the most underused React feature

When you DO extract a layout component, you use the children prop:

```
function Card({ children, accent }: { children: React.ReactNode; accent: string }) {
  return (
    <div style={{ borderLeft: `4px solid ${accent}`, padding: 16, background: 'white' }}>
      {children}
    </div>
  );
}

// Usage:
<Card accent="#0F5132">
  <h3>Round 1 Picks</h3>
```

```
<PicksList details={...} />
</Card>
```

The `children` prop receives whatever JSX is between the opening and closing tags. Type it as `React.ReactNode` (the most permissive React type — accepts elements, strings, numbers, arrays, null, undefined, fragments).

Why this matters: `children` lets you build **layout components** that don't care what's inside them. The parent describes the contents; the layout component just wraps them.

In our codebase, `<ChartCard>` (which we'll see in Chapter 6) uses `children` exactly this way:

```
<ChartCard title="Points Breakdown" subtitle="...">
  <ResponsiveContainer width="100%" height={360}>
    <BarChart>...</BarChart>
  </ResponsiveContainer>
</ChartCard>
```

The card knows about title and subtitle. The chart inside is whatever the parent passes. **Layout-vs-content separation in five lines.**

A note on file structure

In this codebase we organize by **feature folder**:

```
components/
├─ standings/    ← things only used in standings
├─ matches/     ← things only used in matches
├─ predictions/ ← things only used in predictions
├─ players/     ← things only used in players
├─ analytics/   ← things only used in analytics
└─ tabs/        ← the top-level tab containers
```

Alternative would be by **type** (`/components/charts`, `/components/cards`, `/components/buttons`). For small apps, by-type works. For anything with more than ~20 components, **by-feature** scales better — you can find everything related to “the standings page” by opening one folder.

The `tabs/` folder is the exception; it's where the top-level tab containers live, since they

don't fit cleanly into one feature folder. They're more like "feature entry points."

This is interview gold too: "How do you organize React components?" Answer: "By feature, not by type. Each folder owns the leaf components for that feature, plus a top-level tab/page container."

Vibe prompt you would have used

*"This <MatchPickAnalysis> component is getting long. Extract two pieces. (1) The colored pill showing a team's icon + name + points-earned chip — call it <TeamChip>, props { team: Team, ptsEarned: number | null }. (2) The small status step in the tournament-path bar — call it <PathStep>, props { label: string, status: 'won' | 'lost' | 'live' | 'na' }, each mapping to colors and icon (🏠 🏡 —). Also extract the literal → arrow into a one-line <PathArrow> so the path JSX reads like 'step arrow step arrow step'. **Don't change any visuals — just move the JSX into the new files and import them back.**"*

That last sentence is the magic. Without it, the LLM "improves" the styling while moving the JSX, and the visual diff between before and after is bigger than expected. **"Don't change visuals — just move JSX"** is what turns a refactor back into a refactor.

CHECK YOURSELF

1. **<PathArrow> is a single styled span. What's the argument FOR keeping it as a component instead of inlining?**
2. **Why doesn't <MatchPickAnalysis> extract the two-column layout into its own component?** What three things would have to be true to justify it?
3. **A teammate proposes <TeamSection> containing <TeamSectionHeader> containing <TeamSectionTitle>. What questions do you ask? What would the smell be?**
4. **You see a 5-prop component: <Foo a b c d e />. Is that automatically too many? When is it fine?**
5. **Open components/players/PlayerDetail.tsx. List every component it imports.** For each, classify: copy-paste rule, what-vs-how rule, reusability rule, or readability rule.

When you've answered these, head to [02-prop-drilling-vs-context.md](#).

Chapter 5 · Lesson 2 — Prop Drilling vs Context

Goal: stop being scared of prop drilling. By the end of this lesson you can defend prop drilling in interviews, recognize when Context is genuinely the right tool, and know why most internet advice on this topic is overcooked.

What is “prop drilling”?

When the orchestrator at the top of the tree has data that a component three levels deep needs, you “drill” the data through every level in between:

```
<SnookerFantasyLeague>
├── <PlayersTab teams={teams}>      ← receives teams (level 1)
│   └── <PlayerDetail teams={teams}> ← receives teams (level 2)
│       └── <MatchPickAnalysis teams={teams}> ← receives teams (level 3, fine)
```

The middle components don’t use teams themselves — they just pass it through. That feels wasteful. It feels like ceremony. Beginners hate it.

It’s also fine. Three levels of explicit prop passing is one of the most readable patterns in React.

The internet’s bad advice

Search “prop drilling” and you’ll find articles screaming “*avoid prop drilling at all costs! Use Context!*” This is wrong for most apps.

The reason it’s wrong: **the alternative — Context — has costs the articles never mention.** Specifically:

1. **Context hides data flow.** With drilling you can Cmd+F “teams” and see exactly where it goes. With Context, you can’t — `useContext(TeamsContext)` could be called from anywhere.
2. **Context’s default behavior is to re-render every consumer when the value changes.** Even if a particular consumer doesn’t care about the part that changed.
3. **Context requires a Provider somewhere up the tree.** That’s a setup tax, plus a “what if the Provider is missing?” test surface.
4. **Context is contagious.** Once you start using it for one piece of data, it’s tempting to put everything in Context and end up with a global mess.

For 3-level drilling, **the costs of Context exceed the costs of drilling**. For 8-level drilling, maybe Context wins. Always ask: “what specifically am I trying to avoid?”

When prop drilling is fine (most of the time)

The honest answer: **prop drilling is fine until it’s painful**, and “painful” usually means more than 4–5 levels.

Symptoms of “actually too much drilling”:

- A component takes a prop and passes it unchanged to literally one child, and that’s its only purpose.
- You’re passing the same prop through 6+ component levels.
- Adding a new piece of data requires editing 7 files just to thread it through.
- You’re naming intermediate props things like `passThroughTeams` because “teams” already means something else at the intermediate level.

Until you hit those symptoms, **drill happily**. It’s explicit, debuggable, type-checked, and refactor-friendly.

In our codebase

Maximum drilling depth in this codebase is **3 levels**:

```
<SnookerFantasyLeague>           (level 0)
├── <PlayersTab teams={...}>      (level 1)
│   ├── <PlayerDetail>           (level 2)
│   │   └── <MatchPickAnalysis teams={...}> (level 3, uses)
```

Three levels of teams. Each component in between either uses it or passes it. Total cognitive cost: nearly zero. We do not need Context.

Why we DON’T use Context for teams

Let’s pretend we did. Context-based version:

```
// Set up Context at the orchestrator
const TeamsContext = createContext<TeamWithScores[]>([]);

function SnookerFantasyLeague() {
```

```

const teamsWithScores = useMemo(...);
return (
  <TeamsContext.Provider value={teamsWithScores}>
    <PlayersTab />
  </TeamsContext.Provider>
);
}

// Consume in MatchPickAnalysis
function MatchPickAnalysis() {
  const teams = useContext(TeamsContext);
  // ...
}

```

Pros: <PlayersTab> and <PlayerDetail> don't have a teams prop in their signatures.

Cons:

- A new file (contexts/TeamsContext.tsx) with a Provider definition and a hook.
- The Provider somewhere in the tree.
- Slightly more cognitive load — readers have to know that <MatchPickAnalysis> is not self-contained; it depends on a Provider being present.
- Testing <MatchPickAnalysis> in isolation requires wrapping it in a Provider.
- If we accidentally render <MatchPickAnalysis> outside the Provider, we get the default value ([]) and silent rendering bugs.

For 3 levels and one piece of data, **the drilled version is just better**. Three levels of teams props is more readable than the Context version.

When Context IS the right answer

Context excels at **cross-cutting concerns** — data that's needed by many components at unrelated parts of the tree. Three real cases:

1. Theme

```

const ThemeContext = createContext<'light' | 'dark'>('light');

```

Theme is consumed by **every** component (or close to it). Drilling theme through 50 components would be misery. Context is the right call.

2. Authenticated user

```
const UserContext = createContext<User | null>(null);
```

Many components need to know who's logged in: header (show name), nav (show admin links), forms (pre-fill email), etc. The set of consumers is broad and unrelated. **Drilling user through everything is misery; Context wins.**

3. Locale / i18n

```
const LocaleContext = createContext<Locale>('en');
```

Same logic. Every translated string needs the locale; drilling it would be ridiculous.

What these have in common

- Used by **many** components scattered across the tree.
- The data is **stable** (it doesn't change every interaction).
- The consumers are **disparate** — they don't share a parent above the orchestrator.

Compare to teams in our app:

- Used by **3** components in a single sub-tree (Players).
- Computed once and stable.
- All consumers are in one feature folder.

teams doesn't qualify. theme, user, locale do.

Context's performance footgun

Even when Context is the right architectural choice, it has a **performance** trap.

```
const TeamsContext = createContext({ teams: [], filter: '', setFilter: () =>
```

When you put multiple unrelated values in one Context, **every consumer re-renders when ANY of them changes**. If `setFilter` is called, components that only read teams re-render too.

The fix is **splitting Context** into smaller, single-purpose contexts:

```
const TeamsContext = createContext<TeamWithScores[]>([]);
const FilterContext = createContext<{ filter: string; setFilter: (s: string)
```

Or using **separate read and write contexts**:

```
const TeamsContext = createContext<TeamWithScores[]>([]);
const TeamsActionsContext = createContext<{ setSelectedTeam: (...) => void }
```

These are real production patterns. **If you reach for Context, plan for splitting it.**

What about Zustand, Jotai, and friends?

If Context is “too much” but drilling is “too painful,” there’s a middle layer: **lightweight state libraries**. The two leading options in 2026:

Zustand

```
import { create } from 'zustand';

const useStore = create<{ teams: TeamWithScores[]; setTeams: (t: TeamWithScores) => void }>({
  teams: [],
  setTeams: (teams) => set({ teams }),
});

// Anywhere in the app:
function MatchPickAnalysis() {
  const teams = useStore(s => s.teams);
  // ...
}
```

About 1 KB of code. No Provider required. Components subscribe to slices of state. Re-renders are tracked per-slice. **The selector pattern (`s => s.teams`) is the killer feature** — it ensures only components that read changed data re-render.

When to use: more shared state than `useState` can comfortably handle, less than would justify Redux. **Most production apps these days.**

Jotai

```
import { atom, useAtom } from 'jotai';

const teamsAtom = atom<TeamWithScores[]>([]);

function MatchPickAnalysis() {
  const [teams] = useAtom(teamsAtom);
  // ...
}
```

Similar scope, different mental model. State is decomposed into “atoms”; components subscribe to specific atoms. Computed atoms can derive from others. **Smaller, more compositional, slightly more learning curve.**

Redux

The classic. **Boilerplate-heavy.** Modern Redux Toolkit (RTK) makes it bearable, but for new projects in 2026, you reach for it only when you genuinely need:

- Time-travel debugging.
- Action logging / replay.
- Middleware (e.g. observability hooks).
- Strict, auditable state transitions.

For 95% of apps, Zustand or Jotai is enough.

Interview answer: *“For small apps, useState in the parent. Past that, Zustand for most cases — small, no Provider, selector pattern. Context for cross-cutting concerns only (theme, auth, locale). Redux only when you need its specific features (time-travel, middleware, action replay).”*

The senior decision tree

When considering “where should this state live?”:

Is this state used by ONE component?

- └ Yes → useState inside that component.
- └ No → continue.

Is it used by 2-3 nearby components (sibling-ish)?

- └─ Yes → Lift to common ancestor; prop drill.
- └─ No → continue.

Is it cross-cutting (theme, auth, locale)?

- └─ Yes → Context.
- └─ No → continue.

Is it medium-shared and changes often?

- └─ Yes → Zustand or Jotai.
- └─ No → continue.

Do you need time-travel debugging or middleware?

- └─ Yes → Redux.
- └─ No → reconsider – you may have over-thought it.

That tree gets you the right answer 95% of the time. Memorize it.

A real-world Context: the dark mode example

Pretend our app had dark mode. The implementation would look like:

```
// contexts/ThemeContext.tsx
'use client';
import { createContext, useContext, useState } from 'react';

type Theme = 'light' | 'dark';

interface ThemeContextValue {
  theme: Theme;
  toggle: () => void;
}

const ThemeContext = createContext<ThemeContextValue | null>(null);

export function ThemeProvider({ children }: { children: React.ReactNode }) {
```



```

const [theme, setTheme] = useState<Theme>('light');
const toggle = () => setTheme(t => t === 'light' ? 'dark' : 'light');
return (
  <ThemeContext.Provider value={{ theme, toggle }}>
    {children}
  </ThemeContext.Provider>
);
}

export function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error('useTheme must be used inside ThemeProvider');
  return ctx;
}

```

Three things to call out:

The “create + Provider + custom hook” trio

Every Context in production code follows this pattern:

1. `createContext` — defines the shape.
2. A `Provider` component that owns the state.
3. A custom hook (`useTheme`) that calls `useContext` AND throws if no `Provider` is present.

The custom hook is the small but huge insight. Without it, every consumer writes `useContext(ThemeContext)` directly and silently gets the default value if no `Provider` wraps them. With the hook, missing `Provider` throws a clear error.

Default value `null` + assertion in the hook

```

const ThemeContext = createContext<ThemeContextValue | null>(null);

export function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error(...);
}

```

```
    return ctx;
  }
```

The default value is `null`, which is invalid. The custom hook narrows the return type to `ThemeContextValue` (non-null) by throwing if it's null. **This means consumers don't have to handle null themselves** — the hook guarantees a real value or fails loudly.

The Provider pattern

```
<ThemeProvider>
  <App />
</ThemeProvider>
```

The Provider wraps the app (or a sub-tree). Its value prop is what consumers receive. **Wherever the Provider is in the tree, that's the boundary** — only descendants of the Provider can use the Context.

Most apps put global Providers in `app/layout.tsx` or right after.

What we DO use that's “globalish”

Even though we don't use Context, there are two pieces of “shared, app-wide” data in our codebase:

Module-level constants

```
// lib/teams.ts
export const TEAMS: Team[] = [...];

// lib/matches.ts
export const ROUND1_MATCHES: Match[] = [...];
```

These are just exported constants. Any file can import them. No React state, no Context. **They're “global” in the sense that they're available everywhere, but they don't change.** That's fine — it's just imported data.

For data that doesn't change at runtime, **module-level constants are the simplest possible “global state.”** No hooks, no Providers, no library. Just `import { TEAMS } from`

'@/lib/teams' and use it.

Style helper functions

```
// lib/constants.ts
export function tabStyle(active: boolean): React.CSSProperties { ... }
```

Same idea — a global helper, but it's a pure function, not state.

The senior takeaway: **a lot of what beginners think needs to be “state” is actually just “imported data” or “imported helpers.”** Don't put data in state if it doesn't change. Don't put helpers in Context if they're stateless.

The seven-level drilling test

Here's a thought experiment to know if you've drilled too far:

Imagine you're inserting a new component between an ancestor and a descendant. Do you have to add a prop to your new component just to thread something through that it doesn't use?

If the answer is “yes” and it's the third time you've done it, you're drilling too far. If the answer is “no” or “rarely,” drilling is fine.

For our codebase, the answer is rarely. Inserting `<MatchPickAnalysisCard>` between `<PlayerDetail>` and `<MatchPickAnalysis>` would require passing teams through it. That's once. Not enough to refactor.

Vibe prompt you would have used

“My app's teams array is being prop-drilled three levels deep: orchestrator → tab → detail → match-analysis. A teammate suggests using Context to avoid drilling. Should I? Lay out the trade-offs honestly. Default to NOT using Context unless drilling is causing real pain.”

The “default to NOT using Context” line is the trick. Without it the LLM will help you implement Context “for cleanliness” — and you'll regret it.

CHECK YOURSELF

1. A friend says “you should use Context for teams to avoid prop drilling.” What questions do you ask before agreeing?
2. What’s the difference between prop drilling** and passing a prop? When does the latter become the former?**
3. Why is the “create + Provider + custom hook” trio the standard pattern for Context? What does the custom hook protect against?
4. You’re building a settings page that needs the user’s theme, locale, and authenticated identity. Three Contexts or one? Justify.
5. TEAMS is a module-level export `const`. It’s available everywhere in the app. So is it “global state”? Why or why not?

When you’ve answered these, head to [03-building-player-detail.md](#) — the case study.

Chapter 5 · Lesson 3 — Building PlayerDetail (Case Study)

Goal: study a real, deeply nested feature view — the Players tab and its `<PlayerDetail>` drill-down — to see composition, prop drilling, conditional rendering, and back-and-forth navigation in action.

The problem

The user clicks a player card from a 32-player grid. They land on a deep “player detail” view: the player’s tournament path (R1 → R2 → QF → SF → F won/lost/live), four stat tiles, and a per-match breakdown showing which fantasy teams backed them and earned what points. The opponent’s name in each match analysis is **clickable** — clicking it navigates to *that* player’s detail view.

Now hit “Back” and you’re returned to the grid.

The component tree

```
<PlayersTab teams>
├─ if no player selected:
│   └─ grid of <PlayerCard /> (32 of them)
└─ if a player IS selected:
```

- └─ <PlayerDetail playerName teams onBack onSelectPlayer>
 - └─ header (gradient, name, status, path bar)
 - └─ grid of <StatTile /> (4 of them)
 - └─ for each match the player appeared in:
 - <MatchPickAnalysis analysis playerName onSelectPlayer>
 - └─ header (round, players, outcome)
 - └─ BACKED <playerName> column
 - └─ list of <TeamChip team ptsEarned />
 - └─ BACKED <opponent> column
 - └─ list of <TeamChip team ptsEarned />

Six components, four levels of nesting. **None of it more than ~70 lines.** Each component has one job.

Step 1: PlayersTab and the conditional render

```
'use client';

import { useState, useMemo } from 'react';
import type { TeamWithScores } from '@/lib/types';
import { PLAYER_INFO } from '@/lib/players';
import { QF_MATCHES, SF_MATCHES, FINAL_MATCH } from '@/lib/matches';
import PlayerCard from '../players/PlayerCard';
import PlayerDetail from '../players/PlayerDetail';

interface PlayersTabProps {
  teams: TeamWithScores[];
}

export default function PlayersTab({ teams }: PlayersTabProps) {
  const [selectedPlayer, setSelectedPlayer] = useState<string | null>(null);

  // ... (build allPlayers list, compute who's still in)

  if (selectedPlayer) {
    return (
```

```

        <PlayerDetail
          playerName={selectedPlayer}
          teams={teams}
          onBack={() => setSelectedPlayer(null)}
          onSelectPlayer={setSelectedPlayer}
        />
      );
    }

    // ... else render the grid
    return (
      <div>
        {filtered.map(p => (
          <PlayerCard key={p.name} player={p} onClick={() => setSelectedPlayer
        ))}
      </div>
    );
  }
}

```

Two key ideas here.

Local state for the detail view

```
const [selectedPlayer, setSelectedPlayer] = useState<string | null>(null);
```

`selectedPlayer` is local to `<PlayersTab>`. **No other tab cares which player is open.** So it doesn't need to live at the orchestrator level.

This is the **lowest common ancestor** rule from Chapter 4 in action: the readers (`PlayersTab`, `PlayerDetail`, `MatchPickAnalysis`) and writers (`PlayerCard` click, opponent name click) all live inside the `PlayersTab` subtree. So state lives there.

Compare with `selectedTeam` from Chapter 4 – that lives at the orchestrator because writers (`Standings` click) and readers (`Predictions` tab) span different sub-trees.

Selected by name, not by object

```
useState<string | null>(null);  
// ...  
onSelectPlayer={setSelectedPlayer}  
// passes a `string` (player name)
```

We store **the player's name** (a string), not the full player object. Why?

- **Stability across re-renders.** A string is primitive; React can compare it cheaply. An object reference would change if we ever re-derived the players list.
- **Simplicity.** The detail view can look up its own player metadata from `PLAYER_INFO[playerName]` and the matches the player appeared in via `.find()` calls. No need to pass the whole object around.
- **Matches the data shape.** `PLAYER_INFO` is keyed by name, so name-as-id is natural.

The trade-off: the detail view does a few `.find()` calls to resolve the name into rich data. Microseconds. Worth the simplicity.

The conditional return

```
if (selectedPlayer) {  
  return <PlayerDetail ... />;  
}  
return <div>{/* grid */</div>;
```

Two completely different views, gated on whether `selectedPlayer` is set. **Early return** is a clean way to handle “two-mode” components.

The alternative is a single JSX with `{selectedPlayer ? <PlayerDetail /> : <div>{grid}</div>}`. Both work; early return is more readable when the two branches are very different. Use early return when the difference is structural (different layout entirely); use ternary when the difference is a single element.

Step 2: PlayerCard — the leaf

`<PlayerCard>` is a 30-line button. The whole job: render one player's name, seed, country, eliminated/alive status. Click handler.

```

interface PlayerCardProps {
  player: { name: string; seed?: number; status: string; flag: string; still
  onClick: () => void;
  finalPicksCount?: number;
}

export default function PlayerCard({ player, onClick, finalPicksCount }: Play
  return (
    <button onClick={onClick} style={{ /* ... */ }}>
      {/* seed badge */}
      {/* champion / runner-up / out badge */}
      {/* name */}
      {/* country flag */}
      {/* hint chip with finalPicksCount if set */}
    </button>
  );
}

```

Three things to note:

A button, not a div

The card is a `<button>`. Why? **Accessibility**. Screen readers announce buttons differently from divs; keyboard users can Tab to a button and Enter on it. A `<div onClick>` traps both.

This is one of those small senior moves that interviewers love to spot. *“This div should be a button.”* — if you can articulate that distinction, you’ve leveled up.

Optional finalPicksCount prop

```
finalPicksCount?: number;
```

When set, the card shows a small chip like *“+3 each from 6 teams”*. When undefined, no chip. The component handles both cases with `{finalPicksCount !== undefined && <chip />}`.

This is the **optional-feature-via-optional-prop** pattern. The card stays simple in the no-chip case; callers can opt in by passing the prop.

Hover effects via inline `onMouseOver` / `onMouseOut`

```
onMouseOver={(e) => { e.currentTarget.style.transform = 'translateY(-3px)';  
onMouseOut={(e) => { e.currentTarget.style.transform = 'none'; }}
```

This is the inline-styles equivalent of a CSS `:hover`. Works fine; slightly verbose. If you had 100 cards needing the same hover, you'd extract it into CSS. For 32 cards on one screen, inline is acceptable.

A more "CSS-purist" version would use CSS Modules or Tailwind's `hover:` modifier. The codebase chose inline because every card already has dynamic colors that don't fit CSS classes. **Consistency within a component matters more than purity.**

Step 3: `PlayerDetail` — the big one

`<PlayerDetail>` is ~270 lines. Let's not paste it all — open `components/players/PlayerDetail` to follow along. Instead, let's see its **structure**:

```
export default function PlayerDetail({ playerName, teams, onBack, onSelectPl  
  // 1. Look up player metadata  
  const info = PLAYER_INFO[playerName] || {};  
  
  // 2. Find which matches the player appeared in (across all 5 rounds)  
  const r1Idx = ROUND1_MATCHES.findIndex(m => m.p1 === playerName || m.p2 ===  
  const r2Idx = ROUND2_MATCHES.findIndex(m => m.p1 === playerName || m.p2 ===  
  // ...  
  
  // 3. Build a `pickAnalyses` array — one entry per match-the-player-was-in  
  const pickAnalyses: PickAnalysis[] = matches.map(mp => {  
    const opponent = mp.match.p1 === playerName ? mp.match.p2 : mp.match.p1;  
    const teamsBackedThis: TeamWithScores[] = [];  
    const teamsBackedOpponent: TeamWithScores[] = [];  
    teams.forEach(t => {  
      const pick = mp.pickKey === 'final' ? t.final : t[mp.pickKey]?.[mp.mat
```

```

        if (pick === playerName) teamsBackedThis.push(t);
        else if (pick === opponent) teamsBackedOpponent.push(t);
    });
    return { ...mp, opponent, teamsBackedThis, teamsBackedOpponent };
});

// 4. Compute aggregate stats
const totalPicks = pickAnalyses.reduce((s, m) => s + m.teamsBackedThis.length);
// ...

// 5. Render header, stats, match-by-match analyses
return (
    <div>
        <button onClick={onBack}><- Back to All Players</button>
        <Header playerName={playerName} info={info} stillIn={...} />
        <StatTilesRow stats={...} />
        {pickAnalyses.map((pa, i) => (
            <MatchPickAnalysis key={i} analysis={pa} playerName={playerName} onS
        ))}
    </div>
);
}

```

Five phases: lookup, find matches, build analyses, compute aggregates, render. Each phase is a few lines. The whole thing is large because there are many phases, not because any single phase is complex.

Why doesn't this need `useMemo`?

You might think *"this component is computing a lot – should it be memoized?"*

The answer: **no**, because the computation runs once per render of `<PlayerDetail>`, and `<PlayerDetail>` only re-renders when its parent (`<PlayersTab>`) re-renders, which only happens when `selectedPlayer` changes. Each new player triggers a new computation; that's correct. There's no waste.

If `<PlayersTab>` re-rendered for unrelated reasons (e.g. a parent state change), then

`<PlayerDetail>` would recompute too — wasteful. We could `React.memo` it then. For this app's size, that's premature.

The `pickAnalyses.map` is the most important transform

This is the data shape the inner component (`<MatchPickAnalysis>`) needs:

```
interface PickAnalysis {
  round: string;
  roundFull: string;
  bestOf: string;
  matchIndex: number;
  match: Match;
  finished: boolean;
  pickKey: 'r1' | 'r2' | 'qf' | 'sf' | 'final';
  opponent: string;
  won: boolean | null;
  teamsBackedThis: TeamWithScores[];
  teamsBackedOpponent: TeamWithScores[];
}
```

Each entry is one match the player was in, decorated with everything the inner component will display. **All the heavy lifting happens in the parent**; the inner component is pure presentation.

This is a key composition pattern: **the parent does the data shaping; the child renders the shape**. Don't make children do their own data lookups — pass them the ready-to-render data.

Step 4: `MatchPickAnalysis` — the per-match section

About 150 lines. Receives one `PickAnalysis` and renders:

```
<div>
  {/* round badge + matchup title + outcome chip */}
  <Header />

  {/* "X of 8 teams backed Wu Yize · Y of 8 backed Murphy" */}
```

```

<Summary />

{/* two columns of TeamChips */}
<div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fit, minm
  <BackedColumn label={`BACKED ${playerName}`} teams={analysis.teamsBacked}
  <BackedColumn label={`BACKED ${analysis.opponent}`} teams={analysis.team
</div>
</div>

```

(Note `<BackedColumn>` is **inlined** here, not extracted — covered in Lesson 1’s “when not to extract” section.)

The clickable opponent name

The most interesting piece:

```

<button
  onClick={() => onSelectPlayer(analysis.opponent)}
  style={{ background: 'none', border: 'none', textDecoration: 'underline',
>
  {analysis.opponent}
</button>

```

Clicking the opponent name calls `onSelectPlayer(opponentName)` — the **same setter** that originally got us into `<PlayerDetail>`. This effectively:

1. Sets `selectedPlayer` in `<PlayersTab>` to the opponent’s name.
2. Triggers a re-render of `<PlayersTab>`.
3. `<PlayerDetail>` re-renders with the new `playerName` prop.
4. The whole detail view recomputes for the new player.

That’s **navigation by state change**, not by URL or routing. We jump from one detail view to another by changing one piece of state. The user sees a smooth transition; under the hood it’s React reconciling the new tree.

The callback chain

`onSelectPlayer` was passed:

- From <PlayersTab> to <PlayerDetail>.
- From <PlayerDetail> to <MatchPickAnalysis>.

Two levels of drilling. Lesson 2's pain threshold is 4–5 levels. Two is fine.

If we ever moved player navigation to a global URL-based system (e.g. /players/wu-yize), this drilling would disappear and the navigation would become a <Link> from next/link. **The drilling exists because the navigation is in-memory state.** Different design, different prop flow.

Step 5: TeamChip and StatTile — the leaves

We've already seen <TeamChip> from Lesson 1. <StatTile> is similar:

```
interface StatTileProps {
  label: string;
  value: string | number;
  sub?: string;
  icon: string;    // emoji
}

export default function StatTile({ label, value, sub, icon }: StatTileProps)
  return (
    <div style={{ /* card background */ }}>
      <div style={{ fontSize: 28 }}>{icon}</div>
      <div style={{ /* small label */ }}>{label}</div>
      <div style={{ /* big value */ }}>{value}</div>
      {sub && <div style={{ /* small sub */ }}>{sub}</div>}
    </div>
  );
}
```

Four props, ~20 lines, no logic. Pure presentation. Used 4 times in <PlayerDetail>.

These tiny presentation components are **the bedrock of a reusable codebase**. Each one is testable in isolation, swappable, and easy to refactor. The complexity lives at the orchestrator level; the leaves stay simple.

Step 6: PathStep and PathArrow — the readability components

We talked about these in Lesson 1. They exist purely to make the parent's path JSX read like a sentence:

```
<PathStep label="R1" status="won" />
<PathArrow />
<PathStep label="R2" status="won" />
<PathArrow />
<PathStep label="QF" status="lost" />
```

`<PathStep>` maps a 4-state union ('won' | 'lost' | 'live' | 'na') to four different visual treatments. The map lives inside `<PathStep>`:

```
const styles: Record<PathStatus, { bg, color, border, icon }> = {
  won: { bg: 'rgba(255,255,255,0.95)', color: '#0F5132', icon: '🏆' },
  lost: { bg: 'rgba(220, 38, 38, 0.85)', color: '#FFF', icon: '🏆' },
  live: { bg: '#FBBF24', color: '#0F5132', icon: '●' },
  na: { bg: 'rgba(0,0,0,0.18)', color: 'rgba(255,255,255,0.55)', icon: '—' }
};
```

A **lookup object** of styles, indexed by the status string. Cleaner than a chain of ternaries. **Use lookup objects whenever you have 3+ branches mapping a value to a style/config.**

What we just learned about composition

Read back through what `<PlayerDetail>` and its descendants accomplish:

- **270 lines of orchestration** (`<PlayerDetail>`).
- **Six leaf components**, each 15–60 lines (`<PlayerCard>`, `<MatchPickAnalysis>`, `<TeamChip>`, `<StatTile>`, `<PathStep>`, `<PathArrow>`).
- **No useMemo, no Context, no global state.** Just `useState` at the right level + prop drilling for two pieces of data (teams, `onSelectPlayer`).
- **Self-referential navigation** — clicking the opponent name in a detail view loads *their* detail view via the same state setter.

That's a fully-functional 32-player drill-down feature, built with **only the techniques in Chapters 3 and 4**. No new React concepts in this chapter — just **the discipline to apply them well**.

Vibe prompts for the Players tab

Building it from scratch would have been three or four prompts:

1. "Create `<PlayersTab>` at `components/tabs/PlayersTab.tsx`. Top-level filter buttons: *Still In, Eliminated, All*. Render a grid of `<PlayerCard>` for each player from `PLAYER_INFO`. Compute *stillIn* from `QF_MATCHES + SF_MATCHES + FINAL_MATCH` (a player is still in if they appear in QF and haven't lost since). Mark whoever won the Final as champion. State: `selectedPlayer: string | null`. Conditional render `<PlayerDetail>` if set."
2. "Create `<PlayerCard>` at `components/players/PlayerCard.tsx`. Button. Props: `player: PlayerCardData`, `onClick: () => void`, `finalPicksCount?: number`. Visuals: seed badge, status badge (champion/runner-up/out), name, country flag, optional chip showing how many teams picked them in the Final."
3. "Create `<PlayerDetail>` at `components/players/PlayerDetail.tsx`. Props: `playerName`, `teams`, `onBack`, `onSelectPlayer`. Look up info from `PLAYER_INFO`, find every match the player appeared in, build a `pickAnalyses` array decorating each match with which teams backed the player vs the opponent. Render header (gradient, name, status, tournament-path bar), 4 stat tiles, then map `pickAnalyses` through `<MatchPickAnalysis>`."
4. "Create `<MatchPickAnalysis>` at `components/players/MatchPickAnalysis.tsx`. Props: `analysis: PickAnalysis`, `playerName`, `onSelectPlayer`. Render the matchup header (round, players, outcome with score). Make the opponent's name a button calling `onSelectPlayer(opponent)`. Below: two columns of `<TeamChip>`s, one for each side. Color the points chip green for +3, brown for +1."

Each prompt is **one component, one paragraph, names every prop**. That's the right grain. **Compose by prompts the same way you compose by components.**

CHECK YOURSELF

1. `selectedPlayer` lives in `<PlayersTab>`. Why not in `<SnookerFantasyLeague>`?
2. The opponent's name in `<MatchPickAnalysis>` is a `<button>` calling `onSelectPlayer(opponent)`. What state changes? In which component? How many re-renders happen?
3. `<PlayerDetail>` does 5 phases of computation in its function body before returning JSX. Why isn't it wrapped in `useMemo`? What would have to be true to memoize it?
4. `<PathStep>` uses a lookup object `{ won: ..., lost: ..., live: ..., na: ... }` instead of `if/else if` chains. Why is this better?
5. Open `components/players/PlayerDetail.tsx`. Find the `pickAnalyses.map(...)` block. Walk through what it produces. Specifically: for each match the player appeared in, what does each `PickAnalysis` entry look like, and where does it get used downstream?

When you've answered these, you've finished Chapter 5. Onward to [Chapter 6 — Data Visualization](#) — the analytics tab and Recharts.

Chapter 5 — Composition

A 200-line component is a smell. So is splitting too early. This chapter teaches you the senior judgement of when to extract a component, when to leave one alone, and how to build deeply nested feature views without losing the plot.

What you'll learn

- The **copy-paste rule** — when a chunk of JSX deserves its own component.
- **Tiny components for readability** (`<PathArrow>`, `<PathStep>`) — when zero-prop components are *good* code.
- **Prop drilling** — how deep is too deep, and how to know.
- **Context** — when to reach for it, when not to.
- How to build a **deeply nested feature** — the Players tab — by composition, not by giant monolithic files.

- The two-way **back/forward navigation** pattern between sibling views.

Lessons in this chapter

1. **01-when-to-split-components.md** — The judgement call: when a chunk of JSX deserves a name. The copy-paste rule. The “what vs how” rule. Tiny components and the case for them.
2. **02-prop-drilling-vs-context.md** — When to keep drilling, when to introduce Context, why most React advice on this topic is wrong, and what senior engineers actually do.
3. **03-building-player-detail.md** — A real case study: the 270-line `<PlayerDetail>` component, the four sub-components it composes, and the design decisions behind each split.

Files you’ll create or update

```

components/
├── matches/                                ← MatchesTab fully built out
│   ├── MatchesList.tsx
│   ├── PlayerLine.tsx
│   ├── RoundButton.tsx
│   └── UpcomingMatchList.tsx
├── predictions/                            ← PredictionsTab fully built out
│   ├── PredictionMatrix.tsx
│   ├── TeamCardView.tsx
│   ├── PicksList.tsx
│   ├── RoundStat.tsx
│   └── Legend.tsx
├── players/                               ← PlayersTab + PlayerDetail
│   ├── PlayerCard.tsx
│   ├── PlayerDetail.tsx                  ← the big one
│   ├── MatchPickAnalysis.tsx
│   ├── PathStep.tsx
│   ├── PathArrow.tsx
│   ├── StatTile.tsx
│   └── TeamChip.tsx

```

```

└─ tabs/
    ├── MatchesTab.tsx          ← UPDATED: real implementation
    ├── PredictionsTab.tsx      ← UPDATED
    └── PlayersTab.tsx          ← UPDATED

```

That's a lot of files. Most are 30–60 lines each. The point of the chapter is **making lots of small files instead of a few big ones** — and learning to defend that choice.

New React/Next.js concepts introduced

- **Component extraction** — moving JSX into a sibling component file.
- **Composition** — building complex views by nesting simple components.
- **children prop** — passing JSX as a prop.
- **Optional callbacks for navigation** (onSelectPlayer, onBack).
- **Local state in tab components** (useState for the round filter, the player drill-down).
- **Conditional rendering of detail views** (if selectedPlayer return <PlayerDetail />).
- **Reusable pure-presentation components** with no business logic.

How long it should take

- 30–45 min reading per lesson
- 2–3 hours typing the code (lots of files, but each is short)
- ~5 hours total

What you'll see at the end

After this chapter:

- **Matches tab** with a round filter and cards for every match.
- **Predictions tab** with a comparison matrix and a single-team card view.
- **Players tab** with a 32-player grid and click-through to a deep detail view.

Three of the five tabs fully working. Only Analytics (Chapter 6) and final polish remain.

Before you start

You should have:

- Chapter 4 complete: orchestrator + state + tab nav.
- All five tabs at least stubbed.
- The Standings tab fully working.

Open [01-when-to-split-components.md](#).

Chapter 6 · Lesson 1 — Recharts Fundamentals

*Goal: understand the **mental model** behind Recharts so the actual API feels predictable. By the end of this lesson, you can read any Recharts code in the wild and describe what it draws without running it.*

The single insight that makes Recharts click

Recharts is a **declarative** chart library. Declarative means: you describe the *result you want*, not the steps to draw it. You hand Recharts an array of objects and a couple of `<Bar>/<Line>/<Pie>` declarations, and it figures out pixels, axes, scales, animations.

The key insight: **a chart is a function of the data array's shape.**

Specifically:

- The **rows** of the array become the X-axis categories (one bar per row, one slice per row, etc.).
- The **fields** of each row become the data series. Want one bar per row? Use one field. Want stacked bars? Use multiple fields.

Get the data shape right, and the chart writes itself. Get it wrong, and no amount of `<Bar>` props will save you. **90% of charting work is reshaping data.** This lesson sets up that mindset; Lessons 2 and 3 do it for real.

The shape of a Recharts chart

Open `components/tabs/AnalyticsTab.tsx`. The simplest chart in there:

```
import { BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer } from 'recharts'

<ResponsiveContainer width="100%" height={360}>
  <BarChart data={r1Data}>
```

```

<CartesianGrid strokeDasharray="3 3" stroke="#FDE68A" />
<XAxis dataKey="team" />
<YAxis />
<Tooltip />
<Legend />
<Bar dataKey="R1" stackId="a" fill="#0F5132" name="Round 1" />
<Bar dataKey="R2" stackId="a" fill="#DC2626" name="Round 2" />
<Bar dataKey="QF" stackId="a" fill="#FBBF24" name="Quarter-Finals" />
<Bar dataKey="SF" stackId="a" fill="#7C3AED" name="Semi-Finals" />
<Bar dataKey="Final" stackId="a" fill="#9F1239" name="Final" />
</BarChart>
</ResponsiveContainer>

```

Read it like a sentence: *“Inside a responsive container, draw a bar chart of r1Data. Use team as the X-axis. Five <Bar> series, all stacked together, each reading a different field of the data.”*

That’s the API. **The data is r1Data. The X-axis label is team. The Y values are R1, R2, QF, SF, Final.** Recharts handles the rest.

The data prop is a flat array of objects

r1Data looks like this (one row per team):

```

[
  { team: 'Invincibles', R1: 35, R2: 19, QF: 9, SF: 5, Final: 3, Total: 71 },
  { team: 'Uncredibles', R1: 29, R2: 15, QF: 6, SF: 3, Final: 1, Total: 54 },
  // ...
]

```

Six fields per row. team is the categorical (X-axis). The rest are numeric (Y-values).

When `<Bar dataKey="R1">` evaluates, Recharts walks every row and pulls row.R1 to draw a bar. Then `<Bar dataKey="R2">` does the same for R2 — but because they share `stackId="a"`, the R2 bar is stacked on top of the R1 bar. Same for QF, SF, Final.

One pass through the data per <Bar>. That’s the whole pattern.

The `<ResponsiveContainer>` is non-negotiable

```
<ResponsiveContainer width="100%" height={360}>
  <BarChart>...</BarChart>
</ResponsiveContainer>
```

Without `<ResponsiveContainer>`, Recharts charts have **no size**. They render as zero-pixel-wide invisible elements and you spend an hour wondering why your screen is blank.

`<ResponsiveContainer>` is React's "fill my parent" wrapper for SVG-based charts. It listens for resize events and tells the chart what dimensions to draw at. Always wrap your charts in it.

The two props:

- `width="100%"` — fills the parent's width.
- `height={360}` — fixed height (Recharts charts can't auto-size their own height; you have to specify).

You can also use `aspect={2}` instead of height, for a 2:1 width:height ratio. Either works.

The four "decoration" components

```
<CartesianGrid strokeDasharray="3 3" stroke="#FDE68A" />
<XAxis dataKey="team" />
<YAxis />
<Tooltip />
<Legend />
```

These show up on almost every Recharts chart. Walk through them.

`<CartesianGrid>` — the dotted/dashed grid

Draws faint lines across the chart for visual reference. The `strokeDasharray="3 3"` says "3px line, 3px gap" — i.e. dotted/dashed.

You can omit it on simple charts. Most production charts have one because it makes values easier to read.

<XAxis> – the X-axis

The `dataKey="team"` is the **categorical key** — Recharts looks at each row's team field to label the axis. If you forget `dataKey`, Recharts uses array indices (0, 1, 2, ...), which is rarely what you want.

You can also style ticks: `<XAxis tick={{ fontSize: 12, fill: '#1F2937' }}>` for smaller, darker labels.

<YAxis> – the Y-axis

Recharts auto-computes the domain (min/max) from your data. If you want to force it: `domain={[0, 100]}`. Useful for percentages where 100 is the natural max.

<Tooltip> – the hover tooltip

When the user hovers a bar, this floating box shows the values. Customizable via `<Tooltip contentStyle={{...}}>` or, for full control, a custom `content={MyCustomTooltip}` render prop.

The defaults are usually fine for a dashboard. Don't over-engineer the tooltip on day one.

<Legend> – the color key below

Shows which color = which series. Generated automatically from `<Bar name="Round 1">` props.

You can position it: `<Legend verticalAlign="top">`. Default is bottom.

Tree-shakable imports

```
import { BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveCont
```

You import **only the components you use**. The Recharts library has dozens of components — `LineChart`, `PieChart`, `AreaChart`, `RadarChart`, `ScatterChart`, etc. — but if you don't import them, they don't end up in your bundle.

This is why Recharts can have a “huge” surface area without producing a huge bundle. **Tree-shaking + named imports = small bundle**. Modern bundlers (the one Next.js uses, SWC) shake aggressively.

The chart card pattern

In our codebase, every chart is wrapped in a `<ChartCard>`:

```
// components/analytics/ChartCard.tsx
interface ChartCardProps {
  title: string;
  subtitle: string;
  children: React.ReactNode;
}

export default function ChartCard({ title, subtitle, children }: ChartCardProps) {
  return (
    <div style={{
      background: '#FFFFFF',
      borderRadius: 16,
      padding: 28,
      boxShadow: '0 4px 20px rgba(0,0,0,0.06)',
      border: '2px solid #FEF3C7',
    }}>
      <h3 style={{ fontFamily: "'Roboto Slab', serif", fontSize: 22, color:
        {title}
      }}>
      </h3>
      <div style={{ color: '#6B7280', fontSize: 14, marginBottom: 20 }}>{sub
        {children}
      </div>
    </div>
  );
}
```

Three props (title, subtitle, children). The chart goes inside via the children prop:

```
<ChartCard title="Points Breakdown by Round" subtitle="How each team has scored" >
  <ResponsiveContainer width="100%" height={360}>
    <BarChart data={r1Data}>
      <CartesianGrid />
      <XAxis dataKey="team" />
      <YAxis />
    </BarChart>
  </ResponsiveContainer>
</ChartCard>
```

```

    <Tooltip />
    <Legend />
    <Bar dataKey="R1" fill="#0F5132" />
  </BarChart>
</ResponsiveContainer>
</ChartCard>

```

This is the **layout-vs-content separation** from Chapter 5 in action. The card knows about title, subtitle, padding, shadow. The chart knows about bars and axes. **Separated concerns; reusable card.**

Custom tooltip styling

```

<Tooltip
  contentStyle={{
    background: '#FFFBEB',
    border: '2px solid #FBBF24',
    borderRadius: 8,
    fontSize: 14,
  }}
/>

```

You can pass a `contentStyle` object to style the default tooltip box. Works for most needs.

For more elaborate tooltips (custom layout, conditional content), pass a `content` render prop:

```

<Tooltip content={({ active, payload, label }) => {
  if (!active || !payload || !payload.length) return null;
  return (
    <div style={{ background: '#FFF', padding: 8, border: '1px solid #ccc' }}>
      <strong>{label}</strong>
      {payload.map(p => (
        <div key={p.name} style={{ color: p.color }}>{p.name}: {p.value}</div>
      ))}
    </div>
  )
}

```



```
);  
}} />
```

We don't bother in our codebase — defaults are fine.

Bar chart vs line chart vs area chart

Same data shape, different component:

```
<LineChart data={r1Data}>  
  <Line dataKey="R1" stroke="#0F5132" />  
  <Line dataKey="R2" stroke="#DC2626" />  
</LineChart>  
  
<AreaChart data={r1Data}>  
  <Area dataKey="R1" stroke="#0F5132" fill="#0F5132" fillOpacity={0.3} />  
  <Area dataKey="R2" stroke="#DC2626" fill="#DC2626" fillOpacity={0.3} />  
</AreaChart>
```

The data array is identical. The chart wrapper changes. **Chart type is a thin layer over the same data.** Once you've reshaped your data correctly, switching chart types is a one-line edit.

When NOT to use Recharts

Recharts is great for the bread-and-butter dashboards — bars, lines, pies, radar charts. It struggles with:

- **Highly custom interactions** (drag a region, lasso-select points, etc.). Use D3 directly or Visx.
- **Tens of thousands of data points** at once. Recharts isn't a perf monster; for canvas-rendered high-data charts, use [uPlot](#) or pull from D3 directly.
- **Animations beyond Recharts' built-ins.** The animation API is limited.
- **Geographic / map visualizations.** Use [react-leaflet](#) or [react-map-gl](#).

For our snooker app, Recharts is the right tool. We're drawing 8 bars, not 80,000.

A worked example: simplest possible chart

Suppose you want to render a single bar chart of total points per team. The reshape:

```
const data = teams.map(t => ({ name: t.name, total: t.scores.total }));
```

The chart:

```
<ChartCard title="Total Points" subtitle="">
  <ResponsiveContainer width="100%" height={360}>
    <BarChart data={data}>
      <CartesianGrid strokeDasharray="3 3" />
      <XAxis dataKey="name" />
      <YAxis />
      <Tooltip />
      <Bar dataKey="total" fill="#0F5132" />
    </BarChart>
  </ResponsiveContainer>
</ChartCard>
```

Eight rows, each with { name, total }. One <Bar> reading total. Done. **Eight lines of JSX, ~3 lines of data shaping.**

That's the baseline. Lessons 2 and 3 build on this with stacks, cell colors, and pivots.

A note on <RechartsCommonProps>

A small TypeScript wrinkle: most Recharts components accept dozens of props. The TS types reflect this – autocomplete shows you 40+ options. **Most are optional.** Stick to the ones you've seen in this lesson; the rest are advanced.

Vibe prompt you would have used (for the simplest chart)

"Create an Analytics tab at components/tabs/AnalyticsTab.tsx. First chart: a bar chart of total points per team using Recharts. Reshape teams.map(t => ({ name: t.name, total: t.scores.total })). Wrap in a <ChartCard> (which I'll write next) with title 'Total Points'. Use ResponsiveContainer (100% width, 360 height), CartesianGrid (3 3 dasharray, light yellow), XAxis dataKey='name', YAxis defaults, Tooltip with contentStyle

(cream bg, gold border), one <Bar dataKey='total' fill='#0F5132'>. Tree-shake imports from 'recharts'. Don't add any other charts yet."

Specific, scoped, names every visual decision. **You'll get exactly this chart and nothing else.**

CHECK YOURSELF

1. **Recharts charts render at 0 pixels without <ResponsiveContainer>. Why?** What is the container actually doing?
2. **Two <Bar> elements with the same stackId produce a stacked bar. With different stackIds, what do they produce?**
3. **<XAxis dataKey="team"> – what happens if your data array has a typo and the field is actually called teamName?** Walk through what shows on the X-axis.
4. **Why is <Tooltip> a separate component instead of a prop on <BarChart>?** What architectural pattern does this reflect?
5. **You decide to switch from a stacked bar chart to a line chart with the same data. What's the minimum change?**

When you've answered these, head to [02-reshaping-data.md](#).

Chapter 6 · Lesson 2 — Reshaping Data for Charts

Goal: master the three transforms that produce 90% of the charts you'll ever build – .map(), .filter(), .reduce() – applied to real data. Plus per-bar color overrides with <Cell>.

The transform pattern

Every chart in <AnalyticsTab> follows the same recipe:

```
const chartData = teams.map(t => ({ /* fields the chart needs */ }));
```

1. Start with the raw data (teams: TeamWithScores[]).
2. .map() it into the shape Recharts expects – one row per chart entry, fields named to match dataKey props.
3. Optionally .sort(), .filter(), or further transform.
4. Pass to <BarChart data={chartData}>.

This recipe is **the entire mental model**. Internalize it and every chart you build is a 5-minute job.

Reshape #1 — Stacked bar of points by round

The first chart in <AnalyticsTab>:

```
const r1Data = teams.map(t => ({
  team: t.name.length > 12 ? t.name.split(' ').map(w => w[0]).join('') : t.name,
  fullName: t.name,
  R1: t.scores.r1,
  R2: t.scores.r2,
  QF: t.scores.qf,
  SF: t.scores.sf,
  Final: t.scores.f,
  Total: t.scores.total,
}));
```

Let's break this `.map()` callback down.

Abbreviating long team names

```
team: t.name.length > 12 ? t.name.split(' ').map(w => w[0]).join('') : t.name
```

Read it in three pieces:

- `t.name.length > 12` — is the name long?
- `t.name.split(' ').map(w => w[0]).join('')` — if so, split on spaces, take the first letter of each word, and concatenate. *“Break Builders United”* → *“BBU”*.
- `: t.name` — otherwise, use the name as-is.

Why? Because the X-axis on a chart with 8 bars has limited space. Long names wrap, overlap, or get truncated by Recharts. Abbreviating long names is the smallest fix.

A more general version would render long names angled (`<XAxis angle={-45} />`), but that has its own problems (cramped labels, harder to read). Abbreviation is simpler and cleaner.

fullName: t.name is also stored — used by the tooltip's custom rendering if needed.

The numeric fields are the bar heights

```
R1: t.scores.r1,  
R2: t.scores.r2,  
QF: t.scores.qf,  
SF: t.scores.sf,  
Final: t.scores.f,
```

Five numeric fields, one per round. Each becomes a stacked <Bar>. Total is also stored even though we don't currently use it — handy for tooltip totals or future features.

The chart that consumes it

```
<BarChart data={r1Data}>  
  <Bar dataKey="R1" stackId="a" fill="#0F5132" name="Round 1" />  
  <Bar dataKey="R2" stackId="a" fill="#DC2626" name="Round 2" />  
  <Bar dataKey="QF" stackId="a" fill="#FBBF24" name="Quarter-Finals" />  
  <Bar dataKey="SF" stackId="a" fill="#7C3AED" name="Semi-Finals" />  
  <Bar dataKey="Final" stackId="a" fill="#9F1239" name="Final" radius={[4, 4,  
</BarChart>
```

Five <Bar>s, each stacked on top of the previous (stackId="a"). The radius={[4, 4, 0, 0]} on the Final bar rounds **only the top corners** — making the entire stack look like a single rounded-top bar.

Notice **the visual is encoded in the JSX, not in the data**. The data is just numbers. The chart turns numbers into colored rectangles. **Always keep the visual decisions in the chart, not in the data shaping.**

Reshape #2 — Pick accuracy %

Next chart: a horizontal bar chart of “what percentage of picks each team got right per round.”

```
const accuracyData = teams.map(t => {  
  const r1Hits = t.scores.r1Details.filter(d => d.correct).length;  
  const r2Hits = t.scores.r2Details.filter(d => d.correct).length;  
  const qfHits = t.scores.qfDetails.filter(d => d.correct).length;
```

```

const sfHits = t.scores.sfDetails.filter(d => d.correct).length;
const fHits = t.scores.fDetails.filter(d => d.correct).length;
return {
  team: t.name.length > 12 ? t.name.split(' ').map(w => w[0]).join('') : t.name,
  R1: Math.round((r1Hits / 16) * 100),
  R2: Math.round((r2Hits / 8) * 100),
  QF: Math.round((qfHits / 4) * 100),
  SF: Math.round((sfHits / 2) * 100),
  F: Math.round((fHits / 1) * 100),
};
});

```

Two new tools.

.filter() to count hits

```

const r1Hits = t.scores.r1Details.filter(d => d.correct).length;

```

`.filter()` takes a predicate function and returns a new array of items where the predicate is true. We then take `.length` to count them.

`d.correct` is true | false | null (Lesson 1.2's tri-state). `.filter(d => d.correct)` keeps only the true items. `null` and `false` are both falsy, so they're excluded.

This is a **much cleaner way to count** than:

```

let r1Hits = 0;
for (const d of t.scores.r1Details) {
  if (d.correct === true) r1Hits++;
}

```

Same logic, more verbose. **Prefer functional methods (`.filter`, `.map`, `.reduce`) over `for` loops** when you're transforming data. They read closer to the intent.

```
Math.round((hits / max) * 100)
```

```
R1: Math.round((r1Hits / 16) * 100),
```

Convert (hits, max) to a percentage integer. The `Math.round` keeps the number tidy for display (75 instead of 74.81818...). For chart axes, integer percentages just look better.

The max values (16, 8, 4, 2, 1) are the number of picks in each round — hard-coded because they're tournament constants. A more flexible version would use `ROUND1_MATCHES.length`, but for a fixed-bracket app, the literals are fine.

The horizontal-bar chart that consumes it

```
<BarChart data={accuracyData} layout="vertical">
  <XAxis type="number" domain={[0, 100]} unit="%" />
  <YAxis dataKey="team" type="category" />
  <Bar dataKey="R1" fill="#16A34A" />
  <Bar dataKey="R2" fill="#F59E0B" />
  <Bar dataKey="QF" fill="#0EA5E9" />
  <Bar dataKey="SF" fill="#7C3AED" />
  <Bar dataKey="F" fill="#9F1239" />
</BarChart>
```

Two changes from the previous chart:

- **layout="vertical"** — flips the chart on its side. Bars run horizontally now.
- **<XAxis type="number" domain={[0, 100]} unit="%">** — explicit numeric axis with a fixed 0-100 range and % suffix on tick labels.
- **<YAxis dataKey="team" type="category">** — categorical Y-axis.

Notice we **removed** `stackId="a"` — these bars are NOT stacked. Each round shows as a separate bar per team, grouped together. **One prop change, completely different chart layout.** That's the magic of declarative chart APIs.

Reshape #3 — The Final pick distribution

```
const finalPickCount: Record<string, number> = {};
teams.forEach(t => {
```

```

    if (t.final) finalPickCount[t.final] = (finalPickCount[t.final] || 0) + 1;
  });
const finalPickData = Object.entries(finalPickCount)
  .map(([name, count]) => ({ name, count })))
  .sort((a, b) => b.count - a.count);

```

Three new tools.

Counting with `.forEach()` and a dictionary

```

const finalPickCount: Record<string, number> = {};
teams.forEach(t => {
  if (t.final) finalPickCount[t.final] = (finalPickCount[t.final] || 0) + 1;
});

```

We're tallying: how many teams picked each player as Final winner? Output is a dict: { 'Wu Yize': 6, 'Shaun Murphy': 2 }.

The loop:

- For each team `t`, look at `t.final` (their pick).
- `finalPickCount[t.final] || 0` – start at 0 if not seen, else use the existing count.
- `+ 1` – increment.
- Assign back.

This is JavaScript's idiomatic “increment a counter in a dict” pattern. **Memorize it; it comes up constantly.**

`Object.entries()` to convert dict → array

```

.map(([name, count]) => ({ name, count })))

```

`Object.entries({ a: 1, b: 2 })` returns `[['a', 1], ['b', 2]]` – an array of [key, value] pairs.

We then `.map` each pair (destructured as `[name, count]`) into an object `{ name, count }`. Now we have:


```
[{ name: 'Wu Yize', count: 6 }, { name: 'Shaun Murphy', count: 2 }]
```

Recharts wants this exact shape: array of objects, X-axis-categorical-key on name, value on count.

.sort() for descending count

```
.sort((a, b) => b.count - a.count);
```

Sort by count descending. Wu Yize (6) comes first. **The chart order matches the array order**, so sorting the data sorts the chart.

If you wanted ascending: `(a, b) => a.count - b.count`. Memorize both.

Reshape #4 — Per-bar colors with <Cell>

Now the magic. The Final pick distribution chart needs to highlight the actual champion's bar in green; everyone else in gray.

```
<Bar dataKey="count" radius={[6, 6, 0, 0]} name="Teams who picked"
  label={{ position: 'top', fontSize: 16, fontWeight: 800, fill: '#1F2937' }}
  {finalPickData.map((d, i) => (
    <Cell key={i} fill={d.name === 'Wu Yize' ? '#16A34A' : '#9CA3AF'} />
  ))}
</Bar>
```

The <Bar> has **children**: one <Cell> per row.

What a <Cell> does

A <Cell> overrides the <Bar>'s default fill for **one specific row**. The map produces one <Cell> per data row, in order. Recharts uses cell N for the N-th bar.

```
<Cell key={i} fill={d.name === 'Wu Yize' ? '#16A34A' : '#9CA3AF'} />
```

Green if Wu Yize, gray otherwise. **Per-row conditional coloring**.

Why this exists

Without `<Cell>`, every bar in a `<Bar>` gets the same fill. You'd have to render multiple `<Bar>`s with different `dataKeys` to get different colors — but that's a different chart structurally.

`<Cell>` is the bridge. **One series, but one of its bars is special.** Pattern: highlight a champion, mark a current period, flag an outlier.

The same pattern works for `<Pie>` (per-slice colors) and `<Line>` doesn't really need it because lines have one color per series.

The `label` prop on `<Bar>`

```
label={{ position: 'top', fontSize: 16, fontWeight: 800, fill: '#1F2937' }}
```

Adds a value label on top of every bar — the count rendered above each bar. Without this prop, you'd have to hover to see values; with it, they're permanent.

For dashboards where the chart is the primary view, **always show labels**. Tooltips are for “more detail on hover”; labels are for “see the value at a glance.”

The `InsightCard` — separating data from presentation

Below the charts, three colored summary cards:

```
<InsightCard
  icon={Trophy}
  title="The Final Pick Was Right"
  body={`Wu Yize won 18-17 in a final-frame thriller. The 6 teams who backed
  bg="#9F1239"
/>
```

`<InsightCard>` is a tiny component:

```
interface InsightCardProps {
  icon: LucideIcon;
  title: string;
  body: string;
  bg: string;
```

```

}

export default function InsightCard({ icon: Icon, title, body, bg }: InsightCardProps) {
  return (
    <div style={{ background: `linear-gradient(135deg, ${bg} 0%, ${bg}DD 100%)` }}>
      <Icon size={28} strokeWidth={2.5} />
      <div>{title}</div>
      <div>{body}</div>
    </div>
  );
}

```

Same `linear-gradient(... ${bg} 0%, ${bg}DD 100%)` recipe we've used everywhere. **Consistent visual language, applied via component.**

The body text is constructed inline:

```
body={`Wu Yize won 18-17 in a final-frame thriller. The ${wuYizeCount} teams`}
```

Where `wuYizeCount` is computed from `finalPickData` higher in the component:

```
const wuYizeCount = finalPickData[0]?.name === 'Wu Yize' ? finalPickData[0].wuYizeCount : 0;
```

This is **derived display data** — not state, not prop. Just a value computed in the parent, passed to a presentation component. Same as `teamsWithScores` from Chapter 4 but at a smaller scale.

The grid layout

Three insight cards in a row:

```

<div style={{
  display: 'grid',
  gridTemplateColumns: 'repeat(auto-fit, minmax(320px, 1fr))',
  gap: 20,
}}>
  <InsightCard ... />
  <InsightCard ... />
  <InsightCard ... />

```

</div>

The auto-fit, `minmax(320px, 1fr)` recipe again — three columns on wide screens, two on medium, one on narrow. **Same pattern from Chapter 3's stat cards.** Once you know it, you use it everywhere.

Recap: the transforms you've learned

You can now do all four of these in your sleep:

1. **`.map()` to reshape** — change one shape to another row-by-row.
2. **`.filter()` to subset** — keep only items matching a predicate.
3. **`.reduce()/.forEach()` to aggregate** — sum, count, group.
4. **`Object.entries()` + `.map()` to convert dict to array** — for chart-friendly shapes.

These four are **the fundamental data-shaping toolkit** of all React (and JavaScript) data work. Master them and dashboards become trivial.

Vibe prompt you would have used

"Build the second chart in <AnalyticsTab>: pick accuracy by round per team. Reshape: `teams.map(t => ({ team, R1: round((r1Hits/16)*100), R2: ..., QF: ..., SF: ..., F: ... }))` where `r1Hits` etc. come from `t.scores.r1Details.filter(d => d.correct).length`. Render a horizontal bar chart (`Layout='vertical'`), XAxis numeric 0-100 with % unit, YAxis categorical team names, 5 separate <Bar>s for R1/R2/QF/SF/F (no stacking), each in a distinct color. Wrap in <ChartCard> titled 'Pick Accuracy %' subtitled 'What share of picks each team got right per round'."

The recipe pattern: name the reshape explicitly, name the chart layout explicitly, name the colors explicitly. **Specificity = predictability.**

CHECK YOURSELF

1. `teams.map(t => ({ ... }))` produces a new array. What's the size of the new array? What's the type of each item?
2. `<Bar dataKey="R1" stackId="a">` and `<Bar dataKey="R2" stackId="a">` — same `stackId`. What if you give them different `stackIds` instead?

3. **<Cell> overrides the <Bar>'s default fill for one row. What other Recharts components accept <Cell> children?** (Hint: think about charts where each rendered piece has identity.)
4. **Walk through the data flow for the Final pick distribution chart.** Start from TEAMS, end with the bars on screen. Name every transform.
5. **You want to add a 6th chart: matches won per nationality (e.g. England has 4 of 32 players, won 12 of 31 matches). Sketch the reshape function.** What does the data array look like?

When you've answered these, head to [03-pivots-and-internal-state.md](#) for the toughest chart in the codebase.

Lesson 6.3 – Pivots and Internal State

Some state belongs to a single component. This lesson is about a chart that owns its own UI state — and about pivoting data so the *match* (not the *team*) becomes the unit of analysis.

By the end of this lesson you will:

- Understand the difference between **pivoting** and **summarizing** data.
- Build `<LeagueAgreementChart>` — a chart with **internal tab state** (useState inside the chart, not lifted up).
- Know precisely when to lift state vs keep it local.
- Render **conditional <Bar> elements** based on whether matches are finished.
- Convert the long, ugly transform inside `AnalyticsTab` into a clean, named helper function.

This is the case study that ties the entire chapter together. It also happens to be one of the most asked-about patterns in senior React interviews: “*When do you lift state?*”

1. What is a pivot?

A **pivot** is a transform where you change *what each row represents*.

- Before: team -> picks (each row is one team).
- After: match -> backers (each row is one match, plus how the league split on it).

The same underlying data, viewed through a different lens.

If you've ever used Excel pivot tables, you've already done this. In React, we do it with `.map()`, `.filter()`, and sometimes `.reduce()`.

Why pivot?

Because **the chart's x-axis dictates the data shape**.

- Want a chart with one bar per *team*? You need an array where each element is a team. That's the shape we used in Lesson 2.
- Want a chart with one bar per *match*? You need an array where each element is a match. That's a pivot.

The chart can't pivot for you. The data has to arrive in the shape the chart expects.

2. The question this chart answers

Look back at your standings. You can see who's winning. But you can't see **where the league agreed and where it disagreed**.

For each match in Round 1, ask:

- *How many of the 8 teams picked the actual winner?*
- *How many got it wrong?*

That tells you which matches were **upsets** (most teams got it wrong) and which were **chalk** (everyone agreed and was correct).

For unfinished matches, ask a different question:

- *Of the 8 teams, how many backed player 1? How many backed player 2?*

That tells you where consensus lies *before* the match has been played.

This is the chart we're building.

3. Plan the data shape first

Always plan the data shape before touching the chart code.

For each match, we need an object like this:

```
{
  match: 'Williams v Wakelin', // x-axis label
  finished: true,
  correct: 5,
  wrong: 3,
  pending: 0,
}
```

Or, for an unfinished match:

```
{
  match: 'Murphy v Wu',
  finished: false,
  p1Name: 'Murphy',
  p2Name: 'Wu',
  p1Backers: 2,
  p2Backers: 6,
  pending: 8,
}
```

Two shapes — one for finished, one for pending — but with **enough overlap that the same chart can render both** (we just toggle which `<Bar>` components mount based on finished).

This is a pattern you'll see often: design *one* row shape that satisfies *all* render branches.

4. Define the type contract

Open `components/analytics/LeagueAgreementChart.tsx`. Top of the file:

```
'use client';

import { useState } from 'react';
import {
  BarChart,
```

```

    Bar,
    XAxis,
    YAxis,
    CartesianGrid,
    Tooltip,
    ResponsiveContainer,
    Legend,
  } from 'recharts';

export interface AgreementDatum {
  match: string;
  finished: boolean;
  correct?: number;
  wrong?: number;
  pending: number;
  p1Name?: string;
  p2Name?: string;
  p1Backers?: number;
  p2Backers?: number;
}

export interface AgreementRound {
  id: string;
  label: string;
  subtitle: string;
  data: AgreementDatum[];
}

```

Why these types?

- **AgreementDatum** describes *one bar* on the chart.
- **AgreementRound** describes *one tab* — Round 1, Round 2, etc. — bundling the round's name, subtitle, and the array of bars.

Why are correct, wrong, p1Name, p1Backers, p2Name, p2Backers all optional?

Because each row uses *one set or the other*, never both:

- A finished row uses correct and wrong.
- A pending row uses p1Backers and p2Backers.

TypeScript would let us model this as a **discriminated union** (a stricter type), and that's what a senior engineer might do. We're using optional fields here because it keeps the data builder simple. **Both are valid – pick the one that fits your team's style.**

Interview answer: "I'd use a discriminated union if the consumer of the type does a lot of branching on `finished`. I went with optional fields here because the chart is the only consumer, and the branching happens once."

Why export interface?

So that **other files can import AgreementDatum** and use it as the return type of helper functions. Concretely, `AnalyticsTab.tsx` will do:

```
import LeagueAgreementChart, { type AgreementDatum } from '../analytics/LeagueAgreementChart'
```

The `type` keyword in the import is a TypeScript signal: *"This is a type-only import – it disappears at compile time."* That keeps your runtime bundle clean.

5. The component shell

Add the props interface and the component skeleton:

```
interface LeagueAgreementChartProps {
  rounds: AgreementRound[];
}

export default function LeagueAgreementChart({ rounds }: LeagueAgreementChartProps) {
  const [active, setActive] = useState<string>('r1');
  const round = rounds.find((r) => r.id === active) || rounds[0];
  const isPendingRound = round.data.every((d) => !d.finished);

  return (
```

```

    <div /* ... container styles ... */>
      {/* tab buttons */}
      {/* chart */}
    </div>
  );
}

```

Read this carefully – there are three big ideas here.

5.1. `useState` is local. The active state – *which round tab is selected* – lives inside this component. The orchestrator at the top of the app **does not know it exists**.

That’s deliberate.

Lifting state up is a real React principle, but the rule is: **lift state only when something else needs it**. Nothing outside this chart cares which tab is open. So it stays here.

If, three months from now, the URL needs to reflect the active tab (so a user can share a link to “League Agreement ☒ Final”), we’d lift this into a route param. Until then: local.

```
const round = rounds.find((r) => r.id === active) || rounds[0];
```

5.2. The `find()` fallback. If `active` somehow points to a round that doesn’t exist (typo, removed round), we fall back to the first round. **Defensive programming.** The user sees something instead of a crash.

```
const isPendingRound = round.data.every((d) => !d.finished);
```

5.3. Derived booleans live above the JSX. Compute once at the top of the render. Use the boolean inside JSX. Don’t write `round.data.every((d) => !d.finished)` twice – that’s a smell.

6. Build the tab buttons

Inside the container `<div>`:

```
<div style={{ display: 'flex', gap: 8, flexWrap: 'wrap', marginBottom: 20 }}
  {rounds.map((r) => {
    const isActive = r.id === active;
    return (
      <button
        key={r.id}
        onClick={() => setActive(r.id)}
        style={{
          padding: '8px 16px',
          borderRadius: 8,
          border: isActive ? '2px solid #0F5132' : '2px solid #E5E7EB',
          background: isActive ? '#0F5132' : 'FFFFFF',
          color: isActive ? '#FBBF24' : '#374151',
          fontWeight: 700,
          fontSize: 13,
          cursor: 'pointer',
          fontFamily: 'inherit',
          letterSpacing: '0.5px',
          transition: 'all 0.15s',
        }}
      >
        {r.label}
      </button>
    );
  })}
</div>
```

Why a simple button row instead of a `<select>` or `radio`?

Because we have a small, fixed set of options (5 rounds) and we want each option to be a **prominent target**. Buttons in a row are touch-friendly, glanceable, and don't hide options behind a click.

Why one `onClick={() => setActive(r.id)}` per button?

Each button needs its own handler that knows its `id`. The arrow function creates a closure that captures `r.id`. This is standard React.

You may have read that creating arrow functions in render is “expensive” or “bad for performance.” For a list of 5 buttons that re-renders only when `active` changes? **Don’t worry about it.** Optimize when measurements say to, not before.

7. Build the chart

After the tab row, add:

```
<ResponsiveContainer width="100%" height={Math.max(280, round.data.length * 10)}>
  <BarChart data={round.data} margin={{ top: 10, right: 20, bottom: 80, left: 20}}>
    <CartesianGrid strokeDasharray="3 3" stroke="#FDE68A" />
    <XAxis
      dataKey="match"
      tick={{ fontSize: 11, fontWeight: 600, fill: '#1F2937' }}
      angle={-45}
      textAnchor="end"
      interval={0}
      height={90}
    />
    <YAxis tick={{ fontSize: 12, fill: '#1F2937' }} domain={[0, 8]} />
    <Tooltip contentStyle={{ background: '#FFFBEF', border: '2px solid #FBBF6F' }} />
    <Legend wrapperStyle={{ fontSize: 13, fontWeight: 600 }} />

    {isPendingRound && (
      <Bar dataKey="p1Backers" stackId="a" fill="#0F5132" name="Backed first" />
    )}
    {isPendingRound && (
      <Bar dataKey="p2Backers" stackId="a" fill="#B45309" name="Backed second" />
    )}
    {!isPendingRound && (
      <Bar dataKey="correct" stackId="a" fill="#16A34A" name="Correct picks" />
    )}
  </BarChart>
</ResponsiveContainer>
```

```

    })
    {!isPendingRound && (
      <Bar dataKey="wrong" stackId="a" fill="#DC2626" name="Wrong picks" />
    )}
  </BarChart>
</ResponsiveContainer>

```

Three details worth circling.

```
height={Math.max(280, round.data.length * 30 + 120)}
```

7.1. Dynamic height. Some rounds have 16 matches (R1), some have 1 (Final). A fixed height of 280px would crush 16 bars together. We compute the height from the data length: 30px per row plus padding, with a floor of 280.

This is a tiny detail that **separates beginner React from production React**. Layouts that adapt to data, not the other way around.

```

{isPendingRound && <Bar dataKey="p1Backers" ... />}
{!isPendingRound && <Bar dataKey="correct" ... />}

```

7.2. Conditional <Bar> rendering. Recharts traverses its children to figure out which series to draw. If a <Bar> is not rendered, that series doesn't appear. So our toggle between *finished* and *pending* visualizations is just **conditional JSX**.

This is React composition at its best: you don't reach for an imperative API to "add a bar" or "remove a bar." You just describe which <Bar>s should exist for the current state.

7.3. stackId="a". All bars in the same stack share an `id`. Recharts treats them as one stacked column. For a finished match, correct (green) is stacked on top of wrong (red), and the column total is always 8. For a pending match, p1Backers (green) is stacked on p2Backers (orange), and again the total is 8. **Visually consistent.**

8. Build the pivot helper in AnalyticsTab.tsx

Open components/tabs/AnalyticsTab.tsx. We need a helper that turns a list of matches into the chart-ready array.

```
const buildAgreementData = (
  matches: Match[],
  pickKey: 'r1' | 'r2' | 'qf' | 'sf' | 'final'
): AgreementDatum[] => {
  matches.map((m, i) => {
    const matchLabel = `${m.p1.split(' ').slice(-1)[0]} v ${m.p2.split(' ').slice(-1)[0]}`;
    if (m.winner) {
      const correct = teams.filter((t) => {
        const pick =
          pickKey === 'final' ? t.final : t[pickKey] ? t[pickKey][i] : null;
        return pick === m.winner;
      }).length;
      return {
        match: matchLabel,
        correct,
        wrong: 8 - correct,
        pending: 0,
        finished: true,
      };
    } else {
      const p1Backers = teams.filter((t) => {
        const pick =
          pickKey === 'final' ? t.final : t[pickKey] ? t[pickKey][i] : null;
        return pick === m.p1;
      }).length;
      const p2Backers = teams.filter((t) => {
        const pick =
          pickKey === 'final' ? t.final : t[pickKey] ? t[pickKey][i] : null;
        return pick === m.p2;
      }).length;
      return {
        match: matchLabel,
```

```

    p1Name: m.p1.split(' ').slice(-1)[0],
    p2Name: m.p2.split(' ').slice(-1)[0],
    p1Backers,
    p2Backers,
    pending: 8,
    finished: false,
  };
}
});

```

This is the messiest function in the entire project. Let's break it down piece by piece.

8.1. Why is this a function inside the component, not a lib/ helper?

Two reasons:

1. **It needs teams from the props.** Putting it in lib/ would force us to pass teams in every call and would gain us nothing because teams is already in scope here.
2. **It's only used here.** No other component will ever build agreement data. Premature extraction is a real cost.

Interview answer: "I extract to lib/ when (a) the function is pure and (b) more than one component needs it. This one fails (b) for now, so I keep it local."

8.2. What is pickKey doing?

pickKey is a **string parameter** that tells the function *which array on the team to read*: `t.r1`, `t.r2`, `t.qf`, `t.sf`, or — special-cased — `t.final`.

This is how we avoid copy-pasting the function five times (one per round). One function, parametrized by which slot on the team we read.

The TypeScript type `'r1' | 'r2' | 'qf' | 'sf' | 'final'` is a **string literal union**. It restricts the caller to exactly those five strings. If you pass `'foo'`, TypeScript errors.

8.3. Why the special case for 'final'?

Because the team type has `t.final: string` (a single player name), not `t.final: string[]` (an array of picks). So:

- For rounds, the team's pick for match *i* is `t[pickKey][i]`.
- For the final, the pick is just `t.final` (no array index).

Hence the ternary:

```
pickKey === 'final' ? t.final : t[pickKey] ? t[pickKey][i] : null
```

Read this top to bottom:

1. Is `pickKey` literally `'final'`? Yes → use `t.final`.
2. Otherwise, does `t[pickKey]` exist? Yes → grab the *i*th element.
3. Otherwise → `null` (the team didn't pick anything for this round).

The middle check (`t[pickKey] ?`) is defensive — early in the season, some teams might not have R2 picks yet.

8.4. The label trick.

```
const matchLabel = `${m.p1.split(' ').slice(-1)[0]} v ${m.p2.split(' ').slice
```

```
'Mark Williams'.split(' ') → ['Mark', 'Williams']. .slice(-1) → ['Williams']. [0] → 'Williams'.
```

Net effect: keep the **last word** (the surname) of each player's full name.

Why? Because chart x-axes have very little horizontal space. `'Mark Williams v Lei Peifan'` is a label that gets crushed into nothing. `'Williams v Peifan'` actually reads.

8.5. The vote count.

```
const correct = teams.filter((t) => /* pick equals winner */).length;
```

This is the **counting-by-filtering** idiom. Count how many array elements satisfy a predicate.

Could you use `.reduce((acc, t) => pick === winner ? acc + 1 : acc, 0)`? Yes. But `.filter().length` reads better here.

Interview answer: “I use `.filter().length` when readability beats one extra pass over the array. For arrays of millions of elements, `.reduce()` is faster because it avoids the intermediate array. For 8 teams? It doesn't matter.”

8.6. The implicit invariant.

We have **8 fantasy teams**. Every team picks for every match. So `correct + wrong = 8`. Always.

That's why we can write:

```
return { correct, wrong: 8 - correct, ... };
```

When the **invariant is structural** (it can't change without changing the whole app), it's safe to hardcode the 8. If you wanted to be extra-defensive, you could write `wrong: teams.length - correct` — both are fine. The hardcoded 8 is a tradeoff for clarity in a UI label that says “out of 8 teams”.

9. Wire it up

Below the helper definition (still inside `AnalyticsTab`), build one array per round:

```
const r1Universal = buildAgreementData(ROUND1_MATCHES, 'r1');
const r2Universal = buildAgreementData(ROUND2_MATCHES, 'r2');
const qfUniversal = buildAgreementData(QF_MATCHES, 'qf');
const sfUniversal = buildAgreementData(SF_MATCHES, 'sf');
const finalUniversal = buildAgreementData(FINAL_MATCH, 'final');
```

Then render the chart, passing all five rounds at once:

```
<LeagueAgreementChart
  rounds={
    [
      { id: 'r1', label: 'Round 1', subtitle: 'Last 32 – Best of 19', data: r1 },
      { id: 'r2', label: 'Round 2', subtitle: 'Last 16 – Best of 25', data: r2 },
      { id: 'qf', label: 'Quarter-Finals', subtitle: 'Last 8 – Best of 25', data: qf },
      { id: 'sf', label: 'Semi-Finals', subtitle: 'Last 4 – Best of 33', data: sf },
      { id: 'final', label: 'Final', subtitle: 'Murphy v Wu – Best of 35', data: final }
    ]
  }
/>
```

Click around the tabs. The chart's internal `useState` flips `active`, and the right round renders.

10. The big question: lift state, or keep it local?

This is the question senior interviewers love.

When to keep state local

- Only this component reads it.
- Only this component writes it.
- The state doesn't need to survive remounting.
- The state doesn't need to be shared with the URL or persistence.

The chart's active tab passes all four tests. **Local.**

When to lift state up

- A *sibling* component needs to read it.
- A *parent* component needs to react to it.
- The state should persist (e.g., URL param, localStorage).
- Two pieces of state must stay in sync.

For example, `selectedTeam` in our orchestrator — that one *had* to be lifted, because both the standings tab (which sets it via clicking) and the predictions tab (which reads it via the team selector) need to share the same value.

The diff is roughly:

Concern	Local state	Lifted state
Component count needing it	1	2+
Persistence required	No	Sometimes
URL reflects it	No	Sometimes
Resets on remount	OK	Often not OK
Test isolation	Easier	Harder

Interview-grade answer: “I default to local. I only lift state when the absence of lifting causes a bug — typically when two components need to be in sync,

or when state needs to survive a route change. Lifting state too eagerly turns every parent into a state monolith, and that's the start of every 'too much prop drilling' complaint I've ever heard."

11. Common mistakes I want you to avoid

11.1. Building the data inside the chart

This is wrong:

```
function LeagueAgreementChart({ teams, matches }: Props) {  
  const data = matches.map(m => /* pivot logic */);  
  // ...  
}
```

The chart now knows *too much*. It knows what a Team is, what a Match is, how scoring works. Reuse just died.

The right shape: the chart accepts **already-pivoted data**. The orchestrator does the pivot. Loose coupling.

11.2. Storing the pivoted data in useState

This is also wrong:

```
const [data, setData] = useState(buildAgreementData(...));
```

Why? Because `buildAgreementData` is **derived** from `teams`. If `teams` changes, `data` is stale. `useState` is for state that the *user* mutates. **Derived data does not belong in state.**

The right approach: just compute it on every render. (Or wrap in `useMemo` if profiling shows it's slow.)

11.3. Lifting active into the orchestrator "just in case"

A team I worked with did this once. Every chart's "active tab" lived in a top-level reducer. Why? *"In case we want to share active tabs across charts."*

We never did. The reducer ballooned. Every chart re-rendered when *any* tab changed. We eventually pulled it all back down to the chart level.

Don't optimize for hypothetical future requirements. Optimize for now. Refactor when "now" actually demands it.

12. Vibe prompt you would have used

"I have an array of fantasy teams, each with picks for 5 rounds, and I have arrays of matches per round. I want a Recharts bar chart where each bar is one match, stacked: green = how many teams picked the winner, red = how many got it wrong. For unfinished matches, swap to green = backed player 1, orange = backed player 2. Use a button row to switch rounds, and put the active-round state inside the chart component (don't lift it). Type everything strictly. Show me how to pivot the team-data into match-data with a single helper."

Notice the structure:

1. **Inputs** described precisely (teams shape, matches shape).
2. **Output** described visually (stacked bar, color rules).
3. **State location specified** (inside the chart, don't lift).
4. **Type discipline asserted** (strict typing).
5. **One specific request** (pivot helper).

This is the kind of prompt that produces high-quality code. The *vague* version — "*make a chart of who agreed with each match*" — produces a mess.

13. CHECK YOURSELF

Don't proceed until you can explain each of these:

- ☐ What does **pivot** mean? Give an example.
- ☐ Why is `useState` for the active tab kept inside the chart and not in the orchestrator?
- ☐ What is a **string literal union** type? Why use it for `pickKey`?
- ☐ What's the difference between **derived data** and **state**? Where does each belong?

- ☐ Why are `correct`, `wrong`, `p1Backers`, `p2Backers` optional fields on `AgreementDatum`?
- ☐ What would change if you used a **discriminated union** instead?
- ☐ Why use **conditional `<Bar>` rendering** instead of one `<Bar>` whose `dataKey` is computed?
- ☐ Why is the chart's height computed from `round.data.length`?
- ☐ What's the rule for *when to lift state up*?
- ☐ Why don't we put `buildAgreementData` in `lib/`?

If any of these is fuzzy, scroll back. The next chapter (Mastery + Interview Prep) assumes all of them are solid.

14. Where you are now

You've finished Chapter 6. Take stock:

- **Chapter 1** taught you the problem and the data model.
- **Chapter 2** scaffolded the project and typed the data.
- **Chapter 3** built your first component.
- **Chapter 4** taught you state, hooks, and the orchestrator pattern.
- **Chapter 5** taught you composition and component extraction.
- **Chapter 6** taught you data visualization, transforms, and where state lives.

You can now *build* the app. Chapter 7 will teach you how to *talk about it* — testing, deployment, and what to say in an interview when someone asks “walk me through your project.”

What's next

Open `chapter-07-mastery-and-interviews/README.md`.

It's the closing chapter. We tighten everything, add the safety nets you haven't built yet (loading states, accessibility, error handling), and rehearse the interview narrative.

Chapter 6 — Data Visualization

Charts in React are about data shape, not chart libraries. This chapter teaches you Recharts — but more importantly, it teaches you the *transform-then-render* pattern that applies to every charting library you'll ever touch.

What you'll learn

- The mental model that makes Recharts feel easy: **the chart is a function of the data shape**.
- How to **reshape teams into chart-ready arrays** with `.map()`, `.filter()`, `.reduce()`.
- **Stacked vs grouped bars** — how a single prop (`stackId`) flips between them.
- `<Cell>` — per-bar color overrides for highlighting one row.
- **Pivot transforms** — going from “team 🏠 picks” to “match 🏠 backers” without losing your mind.
- When **internal state** (component-local `useState`) is correct vs lifting it.
- The **chart card pattern** — wrap every chart in a consistent container.

Lessons in this chapter

1. [01-recharts-fundamentals.md](#) — What Recharts gives you, the components you actually use, the responsive container pattern, and the chart card wrapper.
2. [02-reshaping-data.md](#) — Three concrete transforms: stacked-bar totals, accuracy %, finalist pick distribution. Plus per-bar `<Cell>` colors.
3. [03-pivots-and-internal-state.md](#) — The `<LeagueAgreementChart>` case study. Pivoting from team-data to match-data. Why the chart's tab state is local, not global.

Files you'll create or update

```
components/  
├── analytics/  
│   ├── ChartCard.tsx           ← container wrapper for every chart  
│   ├── InsightCard.tsx        ← summary tiles below the charts  
│   └── LeagueAgreementChart.tsx ← the pivot chart with internal state  
└── tabs/
```

New React/Next.js concepts introduced

- **Render-prop / children-as-elements** with Recharts.
- **<ResponsiveContainer>** — the only way to size a chart in a flexible layout.
- **Component-local state** for UI-only toggles inside a chart.
- **Tree-shakable Recharts imports** (only the bits you use).
- **Conditional <Bar> rendering** based on data state.

How long it should take

- 30–45 min reading per lesson
- 1.5–2 hours typing the code
- ~4 hours total

Why charting deserves its own chapter

Three reasons:

1. **Every senior interview eventually asks “build me a dashboard.”** Knowing how to wire a charting library to live data without copy-paste-driven panic is a core senior skill.
2. **Most beginners learn one chart library by rote and then crumble when they switch.** The mental model in this chapter (“data shape, not library”) transfers directly to D3, Victory, Visx, Plotly, anything.
3. **Charts force you to confront *transforms* — .map(), .reduce(), .filter() — at scale.** The data-shaping muscles built here are useful far beyond charting.

Before you start

You should have:

- Chapter 5 complete: Players tab and PlayerDetail working.
- All tabs except Analytics rendering real content.
- recharts already in package.json (Chapter 2 set it up).

Open [01-recharts-fundamentals.md](#).

Lesson 7.1 — Production Polish

The features work. Now make the app robust. This lesson is the bag of small, high-leverage improvements that separate “demo-quality” from “ship-quality.”

By the end of this lesson you will:

- Add **accessibility** to the standings table, tab bar, and player navigation — the parts that block screen-reader users today.
- Implement a **React error boundary** so one broken component doesn’t kill the whole page.
- Use Next.js **loading.tsx** for instant skeleton UIs.
- Audit your bundle size with **next build** and read the output critically.
- Run **Lighthouse** and act on its three highest-leverage suggestions.

You’ll write maybe 60 lines of code in this lesson. The conceptual change is bigger: you start *seeing* the rough edges that beginners blow past.

1. The mindset shift

Up to now you’ve been building features. Each lesson asked: “*What does the user need next?*” and you answered with components.

Production polish is a different mindset. The questions are:

- *What happens when this fails?*
- *Who can’t use this right now?*
- *How long does this take to load?*
- *What does the URL bar do when something crashes?*

These questions don’t add new features. They harden the features you already shipped.

Junior engineers ship features. Senior engineers ship resilient features.

If you do nothing else from this lesson, internalize that mindset shift. The rest is mechanics.

2. Accessibility (a11y) – the part that gets you hired

If you Google “React accessibility,” you’ll drown in 90-page documents. Most teams don’t need 90 pages. They need **five rules** applied consistently. Those rules:

2.1. Use semantic HTML, not `<div>` soup.

A `<button>` is not a `<div onClick={...}>`. The browser gives you keyboard support, focus rings, and screen-reader announcements *for free* when you use the right element.

In our app, we already use `<button>` for the tab bar. Good. But the **round selectors in MatchesTab** were styled `<div>`s in some early drafts. If you find any, swap them to `<button type="button">`.

2.2. Every image must have alt text.

We don’t use `<img>` heavily – most icons come from `lucide-react` (which renders SVG with `aria-hidden` by default). But if you ever add ``, write `alt="Trophy"` next to it. Decorative images get `alt=""` (empty string, not missing).

2.3. Tab interfaces need `role="tablist"`, `role="tab"`, `role="tabpanel"`.

Open `components/SnookerFantasyLeague.tsx`. Find the tab nav. Right now it looks like:

```
<nav style={{ ... }}>
  {tabs.map((tab) => (
    <button key={tab.id} onClick={() => setActiveTab(tab.id)}>
      {tab.label}
    </button>
  ))}
</nav>
```

Upgrade to:

```
<nav role="tablist" aria-label="App sections" style={{ ... }}>
  {tabs.map((tab) => (
    <button
```

```

      key={tab.id}
      role="tab"
      aria-selected={activeTab === tab.id}
      aria-controls={`panel-${tab.id}`}
      id={`tab-${tab.id}`}
      onClick={() => setActiveTab(tab.id)}
    >
      {tab.label}
    </button>
  )})
</nav>

```

And wrap the tab body:

```

<main role="tabpanel" id={`panel-${activeTab}`} aria-labelledby={`tab-${activeTab}`}
  {/* tab content */}
</main>

```

Why bother? A screen reader user lands on your page. Without these attributes, they hear: *"button, button, button, button."* They have no idea those buttons are tabs.

With these attributes, they hear: *"App sections tab list, Standings tab, selected, 1 of 5."* That's the entire app surfacing in audio.

Why isn't this automatic? Because HTML can't read your design. The browser doesn't know that those four buttons are *related* and *mutually exclusive*. ARIA is how you tell it.

2.4. Every interactive element must be keyboard-reachable.

Try this: open your app in a browser and press Tab repeatedly. Can you get to every button? Every link? Every team row in the standings?

If not, you've got a problem. Common culprits:

- A `<div onClick={...}>` (not focusable). Fix: use `<button>` or add `tabIndex={0}` and an `onKeyDown` handler.
- A custom dropdown that traps focus. Fix: handle Escape to close, Tab to move out.

In our app, the **player cards in PlayersTab** are buttons (good). The **team selector in PredictionsTab** is a button (good). The **standings rows** — currently — are not clickable. If we ever made them clickable, they'd need to be `<button>` elements, not `<tr onClick={...}>`.

2.5. Color contrast: text must be readable.

Open Chrome DevTools ☞ Lighthouse ☞ Accessibility audit. It will flag any text whose contrast ratio drops below **4.5:1** (3:1 for large text).

The dark green #0F5132 on white passes. The yellow #FBBF24 on dark green passes. The light gray #9CA3AF on white *barely* passes for medium text. If you ever have to lighten text further for design reasons, push it to a heavier weight or a larger size to compensate.

2.6. The five-rule checklist

Run through this before shipping:

- ☐ **Semantic elements:** every interactive thing is a `<button>`, `<a>`, `<input>`, etc.
- ☐ **Alt text:** every `<img>` has one.
- ☐ **ARIA roles:** tab interfaces, dialogs, alerts, and lists are correctly labeled.
- ☐ **Keyboard reachable:** Tab walks through every interactive element.
- ☐ **Color contrast:** Lighthouse a11y audit passes.

This checklist is the answer to “*How do you handle accessibility on your team?*” in an interview. Memorize it.

3. Error boundaries — when one component crashes

By default, **a single thrown error inside a React component takes down the entire tree**. The whole page goes white. Users see nothing.

That's a terrible UX. Worse, it's a *silent* failure in production — your error tracker (Sentry, Datadog, etc.) might catch it, but the user just sees a broken page with no explanation.

Error boundaries fix this.

3.1. The class-component reality

Error boundaries are one of the **two places React still requires a class component** (the other is the `componentDidCatch` lifecycle that error boundaries themselves use). There is no hook for this. Yet.

Create `components/ErrorMessage.tsx`:

```
'use client';

import { Component, type ReactNode } from 'react';

interface Props {
  children: ReactNode;
  fallback?: ReactNode;
}

interface State {
  hasError: boolean;
  error?: Error;
}

export default class ErrorMessage extends Component<Props, State> {
  constructor(props: Props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError(error: Error): State {
    return { hasError: true, error };
  }

  componentDidCatch(error: Error, info: { componentStack: string }) {
    console.error('Error boundary caught:', error, info.componentStack);
    // In production, send to Sentry / Datadog / etc.
  }
}
```

```

render() {
  if (this.state.hasError) {
    return (
      this.props.fallback ?? (
        <div
          role="alert"
          style={{
            padding: 24,
            background: '#FEE2E2',
            border: '2px solid #DC2626',
            borderRadius: 12,
            color: '#7F1D1D',
          }}
        >
          <strong>Something went wrong.</strong>
          <div style={{ marginTop: 8, fontSize: 13 }}>
            {this.state.error?.message ?? 'Unknown error'}
          </div>
          <button
            onClick={() => this.setState({ hasError: false, error: undefined })}
            style={{ marginTop: 12, padding: '6px 14px', borderRadius: 6 }}
          >
            Try again
          </button>
        </div>
      )
    );
  }
  return this.props.children;
}
}

```

3.2. Wrap things that might break

In `app/page.tsx`:

```
import ErrorBoundary from '@components/ErrorBoundary';
import SnookerFantasyLeague from '@components/SnookerFantasyLeague';

export default function Page() {
  return (
    <ErrorBoundary>
      <SnookerFantasyLeague />
    </ErrorBoundary>
  );
}
```

For finer-grained control, wrap individual tabs:

```
<ErrorBoundary fallback={<div>Analytics failed to load.</div>}>
  <AnalyticsTab teams={teamsWithScores} />
</ErrorBoundary>
```

3.3. The interview question this answers

Q: “How do you handle errors in a React app?”

A: “Three layers. (1) Error boundaries around any component subtree that could crash, with a friendly fallback UI and a logger that ships the error to Sentry. (2) Try/catch around any async work that could reject — fetch failures, JSON parse errors. (3) Defensive null checks in the render path for any data that might be missing. Error boundaries don’t catch errors in event handlers or async code, so the three layers complement each other.”

That’s a senior-grade answer. It hits the **what** (boundaries, try/catch, null checks), the **why** (boundaries don’t catch async), and the **so what** (Sentry).

4. Loading states with `loading.tsx`

Next.js App Router has a built-in primitive for “show this while a route segment is loading.”

Create `app/loading.tsx`:

```

export default function Loading() {
  return (
    <div
      role="status"
      aria-live="polite"
      style={{
        minHeight: '60vh',
        display: 'grid',
        placeItems: 'center',
        fontFamily: "'Roboto', sans-serif",
      }}
    >
      <div>
        <div
          style={{
            width: 60,
            height: 60,
            border: '4px solid #FEF3C7',
            borderTopColor: '#0F5132',
            borderRadius: '50%',
            animation: 'spin 0.8s linear infinite',
            margin: '0 auto 16px',
          }}
        />
        <div style={{ color: '#6B7280', fontWeight: 600 }}>Loading league da
      </div>
      <style>{`
        @keyframes spin {
          to { transform: rotate(360deg); }
        }
      `}</style>
    </div>
  );
}

```

4.1. When does this fire?

Whenever a route's server component is doing async work (`await`, dynamic imports). Right now our app is fully static — all data is in `lib/`. So `loading.tsx` will rarely fire in this app.

That's still worth setting up. The day you migrate to a real backend (fetching tournament data from Sportradar's API, say), the loading state is already in place. Future-you will thank present-you.

4.2. Why `aria-live="polite"`?

Screen readers announce the loading state to the user, but in a non-disruptive way. `aria-live="assertive"` interrupts whatever the user is hearing — too aggressive for a spinner.

5. Bundle audit with `next build`

Run:

```
npm run build
```

You'll see output like:

Route (app)	Size	First Load JS
└─ ○ /	312 kB	407 kB
└─ ○ /_not-found	872 B	88 kB
+ First Load JS shared by all	87.4 kB	

5.1. What to look at

- **Size** — the JS for *just this route's* code.
- **First Load JS** — the JS the browser downloads when it first hits this route, including shared chunks.
- **Shared chunks** — code split out so that navigating between routes doesn't re-download it.

5.2. The numbers to worry about

For a public-facing app, **First Load JS under 200 KB** is excellent. **Under 500 KB** is fine. **Over 1 MB** is a red flag.

Our app is heavy because **Recharts is heavy** (~150 KB minified, gzipped). That's a known trade-off — we picked Recharts for ease of use, knowing it costs us bundle size. If we ever needed to slim this down:

1. **Dynamic-import the Analytics tab** so its bundle only loads when the user clicks it.
2. **Switch to a lighter charting library** (Visx, custom SVG).
3. **Server-render the charts** (Recharts has experimental SSR; not always worth it for fantasy leagues).

You don't have to do this today. You *do* have to **know to do it** when an interviewer asks.

5.3. Dynamic import in 30 seconds

In components/SnookerFantasyLeague.tsx:

```
import dynamic from 'next/dynamic';

const AnalyticsTab = dynamic(() => import('./tabs/AnalyticsTab'), {
  loading: () => <div>Loading charts...</div>,
  ssr: false,
});
```

The component is loaded *only when it first renders*. The Recharts JS is no longer in your initial bundle.

This is a one-line optimization. Use it sparingly — it adds a network round-trip when the user finally clicks the tab. The trade-off is worth it for heavy components used by a minority of visitors.

6. Lighthouse — the report card

Open Chrome DevTools ☞ Lighthouse ☞ click *Analyze page load*. You'll get four scores:

- **Performance** — how fast does the page render?

- **Accessibility** — did you follow the rules?
- **Best Practices** — HTTPS, no console errors, etc.
- **SEO** — meta tags, mobile-friendly, etc.

6.1. The three highest-leverage fixes

Almost every React app has the same three issues on the first Lighthouse run:

1. **Missing meta description.** Add one in `app/layout.tsx`:

```
export const metadata: Metadata = {
  title: 'Snooker Fantasy League',
  description: 'Live standings, picks, and analytics for our 8-team fant
};
```

2. **Color contrast on a button.** Pick a darker text color on light backgrounds.
3. **Missing lang attribute.** Already in `app/layout.tsx` if you used the boilerplate (`<html lang="en">`). Verify it.

A 100/100 Lighthouse score is rare and not the goal. **A 90+ across the board** is the realistic target. Anything you can fix in 5 minutes and gain a point on, you should.

7. Performance: the React-specific wins

Lighthouse measures *page-load* performance. There's another category: *interaction* performance — how snappy the app feels after it's loaded.

7.1. The `useMemo` you already wrote

Remember `useMemo` from Chapter 4? You used it for `teamsWithScores`. That's the kind of thing that helps interaction performance. Without it, every re-render recomputes scores for every team. With it, that work is done once.

7.2. `React.memo` for expensive children

If you ever have a chart that re-renders on every parent state change, wrap its export:

```
export default React.memo(LeagueAgreementChart);
```

Now React only re-renders the chart when its **props** change. Don't reach for this until profiling shows you need it.

7.3. The React DevTools Profiler

Install React DevTools (browser extension). Open the Profiler tab. Click record. Click around the app. Stop recording. You'll see a flame graph of what re-rendered and how long it took.

This is the tool that turns “the app feels slow” into “the standings table re-renders 12 times when I switch tabs.”

You don't have a perf problem in this app. You should still know how to use the profiler — it's an interview-grade skill.

8. Defensive rendering

Two patterns to internalize:

8.1. Guard against missing data

```
{team.scores ? <TeamCard team={team} /> : <div>No scores yet.</div>}
```

The `&&` shortcut also works:

```
{team.scores && <TeamCard team={team} />}
```

But beware: `0 && <Foo />` renders `0`, not nothing. Use `team.scores !== null && <TeamCard />` if you might have a numeric-zero value.

8.2. Empty states

When an array is empty, **render an empty state, not a blank gap**:

```
{matches.length === 0 ? (  
  <div role="status">No matches yet for this round.</div>  
)}
```

```
) : (  
  matches.map(/* ... */)   
)}
```

Empty states are the difference between a broken-looking app and a polished one. Apply this everywhere a list might be empty.

9. Vibe prompt you would have used

“Audit my Next.js 14 app for production polish. Specifically: (1) add `role="tablist"`, `role="tab"`, `role="tabpanel"`, and `aria-selected` to my tab bar. (2) Generate a class-based `ErrorBoundary` component with a friendly fallback UI and a `componentDidCatch` that logs to console. Wrap the root component in it. (3) Add `app/loading.tsx` with a polite `aria-live` spinner. (4) Add a metadata export with title and description. Don't change my existing styling — just layer the a11y and resilience on top.”

That prompt is **specific**. It calls out exactly which ARIA attributes, which React feature, which file paths. The LLM produces clean code instead of generic boilerplate.

10. CHECK YOURSELF

Don't proceed until you can explain each of these:

- ☐ Why does a tab bar need `role="tablist"` and `role="tab"` and `role="tabpanel"`?
- ☐ What's the difference between `aria-live="polite"` and `aria-live="assertive"`?
- ☐ Why is an error boundary still a class component in 2024?
- ☐ What does `getDerivedStateFromError` do that `componentDidCatch` doesn't?
- ☐ Why doesn't an error boundary catch errors thrown in event handlers?
- ☐ What's the practical effect of wrapping a tab in `dynamic(() => import(...), { ssr: false })`?
- ☐ What contrast ratio does Lighthouse demand for body text?
- ☐ When would you use `React.memo`?
- ☐ What's the danger of something `&& <Foo />` when something could be 0?

- Why does the “five-rule a11y checklist” matter more than memorizing all of WCAG?

If any answer is fuzzy, scroll back. The next lesson covers deployment and testing — both of which assume your app is shippable.

11. Where you are now

Your app is now **resilient**:

- Screen-reader users can navigate it.
- A crash in one tab doesn’t kill the page.
- Loading states are in place for the day you fetch live data.
- The bundle is reasonable; the heavy pieces are flagged for future optimization.
- Lighthouse won’t embarrass you.

Move on to [02-deployment-and-testing.md](#).

Lesson 7.2 — Deployment and Testing

Code that isn’t deployed isn’t real, and code that isn’t tested isn’t trusted. This lesson teaches you to ship the app to the public internet and to write the small set of tests that catch the bugs that actually happen.

By the end of this lesson you will:

- Deploy this Next.js app to **Vercel** through GitHub, with preview deployments on every PR.
- Understand the difference between **build-time** and **run-time** in Next.js.
- Configure **environment variables** safely (you don’t have any yet, but you will).
- Set up **Vitest + React Testing Library** for unit tests.
- Write the three tests that actually matter for this app.
- Set up **Playwright** for one end-to-end smoke test.
- Add a **GitHub Actions CI** workflow that runs tests on every push.

This is a long lesson because it covers two big topics. Both are tightly tied together — *you can’t deploy with confidence without tests, and there’s no point writing tests if you can’t ship.*

Part A — Deployment

1. The deployment story for Next.js

Next.js was built by Vercel. They host the framework, they host the docs, they host the deployment platform. **The path of least resistance is Vercel.** Use it for personal projects. Use it for the interview-portfolio version of this app.

For a corporate gig, you might end up on AWS, GCP, Cloudflare, or self-hosted Kubernetes. Those all work — Next.js exports a Node.js server (`next start`) or static files (`next export`). The principles transfer. We're going to use Vercel because it's:

- **Free for hobby projects.**
- **Zero-config** — point it at a GitHub repo and you're done.
- **Preview deployments** — every PR gets its own URL.
- **Built-in CDN** — your global users see fast loads.

Per the project rules, deployment goes through **GitHub** → **Vercel automatically**. We are *not* using `vercel deploy` from the CLI. We are *not* using Netlify. We push to GitHub; Vercel pulls from GitHub.

2. Connect the repo to Vercel

You only do this once. Walkthrough:

1. Go to vercel.com and sign in with your GitHub account.
2. Click **Add New** → **Project**.
3. Vercel will list your GitHub repos. Pick `reverse-understanding`.
4. Vercel auto-detects Next.js. Leave the build command (`next build`) and output directory (`.next`) at defaults.
5. Click **Deploy**.

Two minutes later you'll have a live URL like `https://reverse-understanding.vercel.app`. Open it. Click around. Confirm the standings render, the analytics charts appear, etc.

If anything is broken on the live site but works locally, **read the build logs first**. Vercel surfaces them in the deployment detail page. The most common causes:

- A `lib/` constant that imports from a Node-only module (e.g., `fs`).
- A component using `'use client'` incorrectly (or *not* using it when it should).
- A TypeScript error that `npm run dev` ignored but `next build` rejected.

3. Preview deployments

Once connected, **every push to a non-main branch** triggers a preview deployment. Vercel comments on the PR with a link.

Try it:

```
git checkout -b polish/error-boundary
# make changes
git add .
git commit -m "Add error boundary"
git push -u origin polish/error-boundary
gh pr create --title "Add error boundary" --body "Wraps the root component."
```

Within ~90 seconds, the PR will have a comment from Vercel: “Deployed to <https://reverse-understanding-git-polish-error-boundary-yourname.vercel.app>”.

You and your reviewers can now click through that URL and verify the change *visually* — without anyone running it locally. This is the workflow that makes design review possible at scale.

4. Environment variables

Right now, this app has no secrets. All data is hardcoded in `lib/`. The day you add a real backend (e.g., fetching tournament data from a Sportradar API), you’ll need to handle:

- An API key.
- An API base URL.
- Maybe a feature-flag value.

In Next.js, env vars come in two flavors:

Prefix	Where it’s available	Example
<code>NEXT_PUBLIC_*</code>	Client <i>and</i> server. Bundled into the JS sent to browsers. Use only for <i>non-secret</i> values.	<code>NEXT_PUBLIC_API_BASE</code>

Prefix	Where it's available	Example
(no prefix)	Server only. Available in server components, API routes, and <code>getServerSideProps</code> -equivalents.	<code>SPORTRADAR_API_KEY</code>

Rule of thumb: any value the browser must not see goes without the prefix. Anything the browser *does* see — e.g., a public CDN URL — gets the prefix.

4.1. Local `.env.local`

Create `.env.local` in the project root (it's already gitignored):

```
NEXT_PUBLIC_API_BASE=https://api.example.com
SPORTRADAR_API_KEY=sk_test_abc123
```

Reference these in code:

```
const apiBase = process.env.NEXT_PUBLIC_API_BASE;
const apiKey = process.env.SPORTRADAR_API_KEY; // server only
```

4.2. Vercel environment variables

In the Vercel dashboard: **Project** → **Settings** → **Environment Variables**. Add the same keys, but with the production values. Choose which environments they apply to: Production, Preview, Development.

The day you onboard another developer, they don't need your `.env.local`. They run `vercel env pull` and Vercel fills it in. (We're not using the CLI for deployment. We *are* allowed to use it for env-var convenience.)

5. Build-time vs. run-time

This trips up most React developers moving to Next.js. Internalize it now.

- **Build time** — when `next build` runs, either locally or on Vercel. The output is a `.next/` directory with prerendered HTML, JS bundles, and a manifest.
- **Run time** — when a user hits the deployed app. Vercel serves the prebuilt assets and runs the Node server for any dynamic pages.

Pages in App Router come in three flavors:

1. **Static** (the default for our app) — rendered at build time, served as static HTML. Fastest possible load.
2. **Dynamic SSR** — rendered on every request from a server component. Slower but always fresh.
3. **Client-only** — `'use client'` at the top means the component runs in the browser only.

Our app is fully static. The `lib/` data is baked into the bundle at build time. To make it *dynamic* — e.g., to fetch the latest match results from an API — we'd convert `app/page.tsx` to be async and await a fetch. That moves it from category 1 to category 2.

Why does this matter for an interview? Because someone *will* ask: “Where does this app get its data?” and “What would you change to support live data?” — and you need to be able to answer with the build-time / run-time vocabulary.

Part B — Testing

6. The testing pyramid (Trophy)

The classical *test pyramid* says: many unit tests, some integration tests, few E2E tests.

The modern *testing trophy* (Kent C. Dodds) says: many integration tests, some unit tests, a small E2E layer, and static checking (TypeScript) underneath everything. We're using the trophy mental model:

- **Static** — TypeScript already catches a huge category of bugs. **Free testing.** This is why we paid the upfront type-discipline tax in Chapter 1.
- **Unit** — pure functions in `lib/scoring.ts`. Cheap, fast, lots of cases.
- **Integration** — render a component, simulate user clicks, assert what changed. The **highest-value** tier.
- **E2E** — open a real browser, navigate the deployed app, click through. Slow, brittle, but irreplaceable for confidence.

For this app, we're going to write:

- **3 unit tests** for `scorePick` and `calculateTeamScores`.
- **1 integration test** for the standings tab rendering.
- **1 E2E test** that confirms tab switching works.

That's five tests total. **It's enough.** Senior engineers know the value curve drops off fast — your time is better spent writing more *features* than more *tests of features*.

7. Install Vitest + React Testing Library

Vitest is the modern Jest. It's fast, ESM-native, and integrates with Vite/Next out of the box.

```
npm install -D vitest @vitejs/plugin-react @testing-library/react @testing-library
```

Add a `vitest.config.ts` at the root:

```
import { defineConfig } from 'vitest/config';
import react from '@vitejs/plugin-react';
import path from 'path';

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    globals: true,
    setupFiles: ['./vitest.setup.ts'],
  },
  resolve: {
    alias: {
      '@': path.resolve(__dirname, '.'),
    },
  },
});
```

And a `vitest.setup.ts`:

```
import '@testing-library/jest-dom/vitest';
```

Add to `package.json` scripts:

```
"test": "vitest run",
"test:watch": "vitest"
```

Run `npm test`. You should see “No test files found, exiting with code 1.” Good — the

framework is wired up.

8. Unit tests for scorePick and calculateTeamScores

Create lib/scoring.test.ts:

```
import { describe, it, expect } from 'vitest';
import { scorePick, calculateTeamScores } from './scoring';
import type { Match, Team } from './types';

describe('scorePick', () => {
  it('returns 3 when the pick matches the winner', () => {
    const match: Match = { p1: 'A', p2: 'B', winner: 'A', score: '10-5' };
    expect(scorePick('A', match)).toBe(3);
  });

  it('returns 1 when the pick is wrong', () => {
    const match: Match = { p1: 'A', p2: 'B', winner: 'A', score: '10-5' };
    expect(scorePick('B', match)).toBe(1);
  });

  it('returns null when the match has no winner yet', () => {
    const match: Match = { p1: 'A', p2: 'B' };
    expect(scorePick('A', match)).toBeNull();
  });
});

describe('calculateTeamScores', () => {
  it('returns a total of zero for a team with no completed picks', () => {
    const team: Team = {
      name: 'Test Team',
      color: '#000',
      icon: '🏠',
      r1: [],
      r2: [],
      qf: [],
    };
  });
});
```

```

    sf: [],
    final: '',
  };
  const scores = calculateTeamScores(team);
  expect(scores.total).toBe(0);
});

it('contains a details array for every round', () => {
  const team: Team = {
    name: 'Test Team',
    color: '#000',
    icon: '🏆',
    r1: [],
    r2: [],
    qf: [],
    sf: [],
    final: '',
  };
  const scores = calculateTeamScores(team);
  expect(scores).toHaveProperty('r1Details');
  expect(scores).toHaveProperty('r2Details');
  expect(scores).toHaveProperty('qfDetails');
  expect(scores).toHaveProperty('sfDetails');
  expect(scores).toHaveProperty('fDetails');
});
});

```

What's worth noting

- We test the **pure functions** because they're cheap and high-confidence. Pure-function tests are the highest ROI tests in any codebase.
- We test the **shape of the return value**, not specific scores from real data. Tying tests to real-world results is brittle — when the tournament resets next year, your tests break for the wrong reason.
- We test **boundary conditions**: empty arrays, undefined winners. Bugs hide in the

boundaries.

Run `npm test`. All three should pass. (Adjust the type imports if your `Team` interface uses slightly different field names.)

9. Integration test for the standings tab

Create `components/tabs/StandingsTab.test.tsx`:

```
import { describe, it, expect } from 'vitest';
import { render, screen } from '@testing-library/react';
import StandingsTab from './StandingsTab';
import { TEAMS } from '@/lib/teams';
import { calculateTeamScores } from '@/lib/scoring';

describe('StandingsTab', () => {
  it('renders every team name', () => {
    const teams = TEAMS.map((t) => ({ ...t, scores: calculateTeamScores(t) })
      .sort((a, b) => b.scores.total - a.scores.total);

    render(<StandingsTab teams={teams} onTeamClick={() => {}} />);

    for (const t of TEAMS) {
      expect(screen.getByText(t.name)).toBeInTheDocument();
    }
  });
});
```

This test does what a real user does:

1. **Renders the component** with realistic props.
2. **Asks the DOM**: “Is the team name on screen?”

If you change the styling, this test still passes. If you accidentally `.filter()` out half the teams, this test fails. **It’s testing behavior, not implementation.**

This is the React Testing Library philosophy in two sentences:

The more your tests resemble the way your software is used, the more confidence they can give you.

— Kent C. Dodds

Memorize that. It's also a great interview answer to *"How do you decide what to test?"*

10. End-to-end test with Playwright

Install:

```
npm install -D @playwright/test
npx playwright install --with-deps
```

Create e2e/smoke.spec.ts:

```
import { test, expect } from '@playwright/test';

test('user can switch tabs', async ({ page }) => {
  await page.goto('http://localhost:3000');

  await expect(page.getByText('Standings', { exact: false })).toBeVisible();

  await page.getByRole('tab', { name: /matches/i }).click();
  await expect(page.getByText(/Round 1/i)).toBeVisible();

  await page.getByRole('tab', { name: /analytics/i }).click();
  await expect(page.getByText(/points breakdown/i)).toBeVisible();
});
```

Add a playwright.config.ts:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  testDir: './e2e',
  use: {
    baseURL: 'http://localhost:3000',
  },
  webServer: {
    command: 'npm run dev',
    url: 'http://localhost:3000',
  },
});
```

```
    reuseExistingServer: !process.env.CI,
  },
});
```

Add to `package.json`:

```
"test:e2e": "playwright test"
```

Run `npm run test:e2e`. Playwright will spin up the dev server, open a real Chromium browser, and walk through the test. Take a moment to watch it; it's satisfying.

What this test catches

- **Hydration errors** — if the server-rendered HTML doesn't match the client tree, Playwright would see a console error.
- **Routing bugs** — if a tab click stops working, the test fails.
- **Missing accessibility roles** — `getByRole('tab')` only finds elements with the right ARIA. Half of writing E2E tests is being forced to add proper a11y, which is a feature.

11. CI with GitHub Actions

Create `.github/workflows/ci.yml`:

```
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
```

```
node-version: 20
cache: npm
- run: npm ci
- run: npm run lint
- run: npm test
- run: npm run build
```

Push that to GitHub. Open the **Actions** tab. Within ~2 minutes you'll see a green check next to your commit.

Why include `npm run build`?

Because a passing test suite isn't worth much if the production build is broken. The build is itself a giant smoke test — it type-checks, bundles, and lints everything.

Why not run E2E in CI yet?

Playwright in CI requires a Linux runner with browsers preinstalled, plus more orchestration. It's worth doing eventually. For a personal portfolio project, run E2E locally before pushing big changes.

12. The five-test rule

If you take one thing from this lesson:

A small project deserves a small test suite. Don't aim for 100% coverage; aim for the five tests that catch the bugs that actually happen.

For *this* app, those are:

1. `scorePick` returns the right number for the right inputs. (Unit)
2. `calculateTeamScores` doesn't crash on missing data. (Unit)
3. The standings tab renders all teams. (Integration)
4. Tab switching works. (E2E)
5. The build succeeds. (CI)

Anything beyond this is bonus. Ship and iterate.

13. Vibe prompt you would have used

“I have a Next.js 14 app deployed on Vercel via GitHub. Set up Vitest + React Testing Library. Write three unit tests for my pure scoring functions in `lib/scoring.ts` and one integration test that renders `<StandingsTab>` with realistic props and asserts every team name is in the DOM. Then set up Playwright with a single E2E test that opens the home page, clicks the Matches tab, and asserts that ‘Round 1’ is visible. Finally, write a GitHub Actions workflow that runs lint, test, and build on every push. Use TypeScript everywhere.”

Notice:

- **One LLM, four deliverables.** A focused prompt produces focused output.
- **Specific assertions.** Saying “every team name is in the DOM” gives the LLM a target instead of generating vague test scaffolding.
- **Toolchain named.** Vitest *and* RTL *and* Playwright *and* Actions. The LLM doesn’t have to guess.

14. CHECK YOURSELF

- ☐ What’s the difference between `NEXT_PUBLIC_*` and an unprefixed env var?
- ☐ What’s a *preview deployment* and why are they valuable?
- ☐ What’s the difference between build-time and run-time in Next.js?
- ☐ Why is testing pure functions the highest-ROI testing you can do?
- ☐ What does Kent C. Dodds mean by “*the more your tests resemble the way your software is used, the more confidence they can give you*”?
- ☐ Why does Playwright’s `getByRole( 'tab' )` reward you for writing accessible code?
- ☐ What’s the testing trophy, and how does it differ from the testing pyramid?
- ☐ Why include `npm run build` in CI?
- ☐ What are the 5 tests this app needs, and why does it not need more?
- ☐ How would you switch from build-time data to live data fetched from an API?

15. Where you are now

- The app is live on the public internet.
- Every PR gets a preview URL.
- Five tests guard the things that matter.
- CI runs on every push.

You can confidently say *“the app is shipped”* in an interview. Move on to [03-interview-narrative.md](#) — the closing lesson, where we turn this entire project into a 90-second story you can tell from memory.

Lesson 7.3 — The Interview Narrative

Building the app got you in the room. Talking about it gets you the offer. This is the lesson that turns 1,654 lines of code into a 90-second story.

By the end of this lesson you will:

- Have a memorized **90-second walkthrough** of the project that hits architecture, decisions, and trade-offs.
- Know how to answer the **ten interview questions** that are asked about React projects 95% of the time.
- Have a tight **vibe-coding prompt library** distilled from this entire course.
- Know how to handle the dreaded *“What would you do differently?”* question.
- Be ready to share the GitHub link with confidence.

This is the closing lesson. It’s the most important. The strongest engineer in the world loses the interview to a slightly-less-strong engineer who *talks better* about their work. We’re going to fix that for you.

1. The 90-second walkthrough

When the interviewer says *“Tell me about a project you’ve worked on,”* you have ninety seconds before they zone out. Use them like this:

1.1. The script (memorize this shape, swap your own words)

*"This is **Snooker Fantasy League** — a Next.js 14 app I built for an 8-team fantasy league my friends and I run. The data is the picks each team made for each round of the World Snooker Championship, and the app scores those picks against actual results.*

Architecturally, it's a single-page app with five tabs: standings, matches, predictions, players, and analytics. The state lives in one orchestrator component at the top — `useState` for the active tab and the selected team, `useMemo` to derive scored teams from raw team data so I don't recompute on every render. Pure scoring functions in `Lib/` make the whole thing trivial to unit-test.

*The interesting decisions were two: first, picks are stored as **arrays where index matches the match-index** instead of using IDs, which keeps the scoring code five lines but enforces an invariant the type system can't catch. Second, the **analytics tab pivots team-data into match-data** to answer "where did the league agree?" — that's a cleaner mental model than trying to make the chart understand teams.*

I deployed it on Vercel through GitHub. Vitest covers the pure logic, Playwright covers the tab navigation, and Lighthouse comes back at 95+ across the board.

*If I were starting over I'd probably use a **discriminated union** for the agreement-data type — I went with optional fields and it works, but the union would be stricter."*

That's around 200 words, ~75 seconds at a normal pace. Practice it out loud until it feels conversational.

1.2. Why this script works

It hits the **five things every interviewer wants to hear**:

1. **What is it?** (Snooker Fantasy League, fantasy sports app.)
2. **What technology?** (Next.js 14, TypeScript.)
3. **What's the architecture?** (Single orchestrator, `useState` + `useMemo`, pure functions.)
4. **What was hard / what did I think about?** (Index-based picks, pivot pattern.)
5. **How is it run?** (Vercel via GitHub, tested, audited.)

And it **invites the next question** with the closing "if I were starting over..." line. Interviewers

love when you bring up your own trade-offs — it signals you've thought past *"it works on my machine."*

1.3. The variant for non-technical interviewers

If you're talking to a recruiter or a non-engineer hiring manager:

"It's a fantasy sports site for an 8-team snooker league. You log in, see who's leading, drill into one team's picks, and look at charts of who agreed with whom. It's hosted free on Vercel. Took me about two weekends. I built it because I wanted a hands-on excuse to get fluent in Next.js and TypeScript, and a real spreadsheet of friends' picks was the perfect data set."

No jargon. Outcome-first. Suitable for the first 5 minutes of any interview.

2. The ten questions you will be asked

Every React/Next.js interview asks variations of these. We're going to answer each one *using this app as the example*. Memorize the structure.

2.1. *"Walk me through your folder structure."*

"Top-level: `app/` for routes, `components/` for UI, `lib/` for data and pure logic, `course/` for documentation. Inside `components/`, I split by *role*, not by *type*: a `tabs/` folder for top-level tab components, a `standings/` folder for everything that's only used inside the standings tab, and so on. The advantage is that when I want to delete the analytics tab, I delete one folder and don't have to chase down stragglers in a generic `widgets/` directory."

2.2. *"How does state management work in your app?"*

"I deliberately *don't* use Redux or Zustand. The app has two pieces of shared state — the active tab and the selected team — and both live in the top-level `SnookerFantasyLeague` component as plain `useState`. Tab-local state, like which round is selected inside the matches tab, lives inside that tab. The rule I follow: lift state only when *something else needs it*. State libraries are

tools for the day a `useState` tree becomes painful — and that day hasn't come for an 8-team app."

2.3. "How do you decide when to extract a component?"

"Two triggers. First, when I copy-paste the same JSX twice — the second time I write a function. Second, when a piece of JSX has a *name*: 'this is a stat card,' 'this is a path step.' Naming forces extraction. I avoid premature extraction because over-componentized React is just as painful as under-componentized — you end up chasing props through ten files instead of two."

2.4. "How do you handle data fetching?"

"Right now this app's data is hardcoded — eight teams, ~60 matches, all in `lib/`. So technically there's no fetching. The way I'd add it: convert `app/page.tsx` to an `async` server component, `await` `fetch` from a sports-results API at request time, and pass the data down. The pure scoring functions in `lib/scoring.ts` would not change at all. That's the value of separating data from logic from UI."

2.5. "What's the trade-off between server and client components?"

"Server components run only on the server. They can read environment variables and talk to databases, but they can't use `useState` or event handlers. Client components run in the browser. They have hooks and interactivity, but they bloat the JS bundle. The default in App Router is server, and I push 'use client' down to leaves — only the orchestrator and any tab that uses `useState` get that directive. The static parts above stay server-rendered."

2.6. "How do you optimize performance in React?"

"Three layers. First, the *render layer* — `useMemo` for derived data, `React.memo` for expensive components, but only after profiling says I need them. Second, the *bundle layer* — `next/dynamic` for code-splitting heavy tabs like analytics with Recharts. Third, the *paint layer* — `loading.tsx` for instant skeletons during transitions. I never optimize before measuring; the React DevTools Profiler is my first stop."

2.7. *“How do you handle errors?”*

“An error boundary at the root catches anything thrown during render and shows a friendly fallback. Async errors don’t bubble through error boundaries, so I wrap fetches in try/catch and surface the message in the UI. In production I’d ship componentDidCatch output to Sentry. And I lean hard on TypeScript to make ‘this object might be undefined’ a compile-time error rather than a runtime one.”

2.8. *“What’s your testing strategy?”*

“Five tests for this app. Three Vitest unit tests on the pure scoring functions — those are the highest ROI tests in the codebase. One React Testing Library integration test that renders the standings tab and asserts every team name is in the DOM. One Playwright E2E that confirms tab switching works. Plus the full TypeScript build, which is itself a giant static test. I aim for confidence, not coverage percentage.”

2.9. *“How would you scale this to 1,000 teams?”*

“Three things change. First, the data moves out of lib/ into a database — Postgres for relational picks-vs-matches, with a real foreign key instead of array-index correspondence. Second, the orchestrator’s useMemo over all teams becomes a paginated server-side query — you don’t render 1,000 rows in a table. Third, the Recharts visualizations either get aggregated server-side or switch to a canvas-based library because SVG can’t draw 1,000 bars. None of those changes touch the UI layer’s component shape — that’s a deliberate result of the data/logic/UI separation.”

2.10. *“What would you do differently?”*

This is the one that trips most candidates. **They say “nothing.”** Wrong answer. The right answer:

“Three things. First, I’d type the agreement-data shape as a discriminated union instead of optional fields — stricter, less room for `if (datum.correct)` mistakes. Second, I’d add a tiny custom hook called `useTabState` that wraps `useState<TabId>` plus history-pushing so the URL reflects the active tab —

right now reloading drops you to standings. Third, I'd extract the inline styles into Tailwind once the app stabilized — I went with inline styles to move fast, and that decision is now paying interest."

Three concrete, modest, technically literate trade-offs. Not "I'd rewrite it in Rust." Not "nothing."

3. The vibe-coding prompt library

This entire course was built around the idea that *the prompt is the source of truth*. Here's the distilled prompt library that produced this codebase. Steal these.

3.1. The *project-shaping* prompt

"I want to build a [DOMAIN] app. The data is [DESCRIBE THE DATA SHAPE IN ONE PARAGRAPH]. The user can [DESCRIBE 3-5 USER ACTIONS]. Don't write code yet — just confirm you understand the data and the actions, and propose a 5-tab UI. Push back on anything that's ambiguous."

This is **Lesson 0 distilled**. The "don't write code yet" line is the magic ingredient. It forces the LLM to surface ambiguities you didn't see.

3.2. The *data-modeling* prompt

"Define TypeScript interfaces for [the entities]. Be strict — no any, no implicit optional fields. Make every invariant explicit with a comment. If two interfaces share fields, use composition rather than inheritance."

This is **Lesson 1 distilled**. Strict types, named invariants. Output is interfaces you can almost commit verbatim.

3.3. The *pure-logic* prompt

"Write [N] pure functions that take [INPUTS] and return [OUTPUTS]. No side effects, no React, no state. The functions should be testable in isolation. Add JSDoc comments explaining each one."

This is **Lesson 2 distilled**. Pure functions are the parts that survive every refactor. Get them right first.

3.4. The *first-component* prompt

“Render [the simplest possible thing] using my types from `lib/types.ts`. Use a functional component with explicit props. No `useState`. No `useMemo`. Just JSX over the data. Use inline styles for now.”

Lesson 3. Smallest thing first. Inline styles to move fast. State and abstraction earn their way in later.

3.5. The *progressive-enhancement* prompt

“Add [ONE FEATURE] to [EXISTING COMPONENT]. Don’t redesign anything. Don’t refactor unrelated code. If the change touches more than three files, stop and tell me which files, before changing them.”

Lesson 3.3 distilled, and the most useful prompt in the library. It prevents the LLM from helpfully rewriting half your project.

3.6. The *state-and-tabs* prompt

“Refactor [SINGLE-COMPONENT FILE] into an orchestrator pattern. The orchestrator owns [STATE PIECES] with `useState`. Tab components receive props and a callback. Use `useMemo` for [DERIVED VALUE]. Keep all styling identical.”

Lesson 4. Names the pattern, names the state, names the derivation, locks the styling. Output is clean.

3.7. The *composition* prompt

“Extract a [COMPONENT NAME] from [PARENT FILE]. Move only the JSX that has a single responsibility — [DESCRIBE]. Pass props for [INPUTS]. Don’t introduce Context. Don’t change behavior.”

Lesson 5. Pinpoints the unit of extraction. Forbids accidental Context introduction. Forbids behavior drift.

3.8. The *charting* prompt

“Generate a Recharts [CHART TYPE] from this data shape: [PASTE A SAMPLE ROW]. Bars stacked by [KEY1] and [KEY2]. Use <ResponsiveContainer> and the <ChartCard> wrapper from components/analytics/ChartCard.tsx. Match the existing color palette.”

Lesson 6. Data shape *first*. The chart-library specifics get filled in around it.

3.9. The *production-polish* prompt

“Audit my [COMPONENT] for: (1) ARIA roles, (2) keyboard reachability, (3) loading and empty states, (4) defensive null checks. Add them in-place without changing the visual design.”

Lesson 7.1. Specific list of polish items. *In-place* is the keyword that prevents the LLM from rewriting the file.

3.10. The *testing* prompt

“Write [3 unit + 1 integration + 1 E2E] tests. Unit tests cover [PURE FUNCTION] with [3 BOUNDARY CASES]. Integration test renders [COMPONENT] with [REAL DATA] and asserts [VISIBLE BEHAVIOR]. E2E test opens the app and [USER ACTION]. Use Vitest, RTL, Playwright.”

Lesson 7.2. Counts the tests. Names the tools. Names the assertions. Output runs first try most of the time.

4. The pattern behind every prompt

Look at all ten prompts above. They share a structure:

1. **Verb first.** “Render”, “Refactor”, “Extract”, “Add”, “Audit”, “Generate”, “Write”.
2. **One clear deliverable.** Not a wishlist.
3. **Constraints named.** Don’t introduce X. Match Y. Use Z.
4. **Existing context referenced by file path.** components/analytics/ChartCard.tsx, not “you know that chart wrapper I have.”
5. **Failure modes anticipated.** “If this touches more than three files, stop.”

This is the **vibe-coding contract**. Whenever an LLM-generated mess shows up in your codebase, look back at the prompt — it almost always violated one of those five rules.

5. The hardest interview questions, rehearsed

A few real ones I've been asked. Use these to practice.

5.1. *"Show me the worst code in this project. Why is it the worst?"*

Pick one. Be specific.

"The `buildAgreementData` function in `AnalyticsTab.tsx` is the worst piece of code in the project. It's a 40-line nested function with a ternary inside a `.filter()` callback that special-cases the final round. It works, and the test suite covers it, but readability is poor. If I had another hour I'd extract it to `lib/analytics.ts`, split the finished and pending paths into two helpers, and add a discriminated-union return type."

What this signals: **you can name your own technical debt**. That's a senior trait.

5.2. *"Why didn't you use Redux / Zustand / TanStack Query?"*

"Because the app has two pieces of shared state and zero remote data. Redux solves a problem this app doesn't have yet. TanStack Query solves a problem this app *will* have the day I add live API data — and that's the day I'd pull it in. Premature library adoption is a cost: extra learning, extra surface area for bugs, extra bundle size. I add libraries when local code starts to suffer, not preemptively."

5.3. *"How would I know if a feature you added broke something?"*

"The CI pipeline runs lint, type-check, unit tests, integration tests, and the production build on every PR. The Playwright E2E covers the tab-switching happy path. Vercel preview deployments give a clickable URL on every PR for visual review. Beyond that, I'd want a real user-tracking and error-tracking layer in production — Sentry for crashes, PostHog or similar for usage. None of that

is in the project yet because it's a personal app, but I'd add it on day one of a paid project."

5.4. "What's the most React-specific bug you've fixed?"

Tell a story. The structure is **STAR**: Situation, Task, Action, Result.

"Situation: I had a list of teams that re-sorted by score on every keystroke in an unrelated search box. **Task:** figure out why the chart was re-rendering when the search-box state changed. **Action:** opened React DevTools Profiler, saw that the parent re-rendered the chart's parent because its props identity changed every render — even though the underlying data was equal. I wrapped the derived data in `useMemo`. **Result:** chart re-renders dropped from once-per-keystroke to only when scores actually changed. The lesson is reference equality matters more than value equality in React's diffing."

This is **the** interview structure. Memorize STAR.

6. Closing the interview strong

When the interviewer asks *"Do you have any questions for me?"* — and they will — have three ready.

6.1. About the codebase

- *"What's the one part of your codebase that everyone on the team agrees needs a rewrite, but nobody has time for?"*

This question is a goldmine. It tells you exactly where the technical debt is and how the team handles it.

6.2. About the team

- *"How do you decide what's worth a unit test vs. an integration test on this team?"*

This one signals you care about testing without forcing them to ask.

6.3. About the role

- *"What does success look like in the first 90 days?"*

Forces them to articulate expectations in a way you can hold them to later.

7. The portfolio link

Your README is the second thing the interviewer looks at after the live URL. Make sure it has:

- A **screenshot** of the standings tab (saved as docs/screenshot.png).
- A **two-paragraph summary** of what the app does.
- A **bullet list of tech**: Next.js 14, TypeScript, Tailwind, Recharts, Vitest, Playwright.
- A **link to the deployed app** at the top, in bold.
- A **link to the course folder** with the line *"I wrote a 7-chapter course teaching React from scratch using this app."*

That last bullet is **massive**. It signals: *I don't just code. I teach.* That's a senior trait. Hiring committees notice.

8. The final vibe prompt

"I have completed a Next.js 14 portfolio project. Help me write a 90-second elevator pitch and a README.md for the GitHub repo. The pitch should hit (1) what the app is, (2) the tech stack, (3) one interesting architectural decision, (4) one trade-off I'd revisit. The README should have a hero screenshot, two-paragraph summary, tech list, deployed-URL link, and a 'lessons learned' section. Tone: confident but not arrogant. Length: under 400 words for the README intro."

Everything we built in this lesson, captured in one prompt. Save it. Run it on your *next* portfolio project. The output will have the shape we just memorized.

9. CHECK YOURSELF

- ☐ Can you deliver the 90-second walkthrough from memory, twice in a row, without notes?
- ☐ Can you answer all ten common questions in section 2 with concrete references to *this* codebase?
- ☐ Do you have three “what would you do differently” answers ready, all modest and technically real?
- ☐ Have you memorized at least five of the ten vibe-coding prompts?
- ☐ Can you describe the difference between **server components** and **client components** without checking the docs?
- ☐ Can you tell a STAR-format story about a React bug you fixed?
- ☐ Have you prepared three questions to ask the interviewer at the end?
- ☐ Is your GitHub README in the shape described in section 7?

If yes to all eight, you are ready. Genuinely.

10. Where you are now

You have:

- A working Next.js 14 + TypeScript + Tailwind + Recharts app, deployed on Vercel.
- A separation of data, logic, and UI that scales beyond this project.
- Five tests covering the things that matter.
- A polished, accessible, resilient UI.
- A 90-second pitch and ten interview answers ready to roll.
- A vibe-coding prompt library you can use on the next project to move 3× faster.

You also have a 7-chapter course in your course/ folder that **demonstrates you can teach React**, not just write it. That’s the differentiator at most senior hires.

11. Final words

This course taught you React, Next.js, TypeScript, and a way of thinking about code that survives any framework change. The frameworks will move. The principles — **separate**

data from logic from UI, lift state only when something else needs it, write pure functions first, polish before you ship, test the things that break — those will outlast every JavaScript meta-framework that comes next.

Go to interviews. Tell the story you just rehearsed. Build the next app in a third of the prompts.

You've got this.

— *End of course.*

Re-read [course/README.md](#) any time you want a refresher on the chapter map. The original single-file walkthrough lives at [course/SNOOKER_FANTASY_LEAGUE_COURSE.](#) for one-stop reading.

Chapter 7 — Mastery + Interview Prep

You can build it. Now learn to ship it and talk about it. This chapter is the difference between a junior developer who *finished a tutorial* and an engineer who can stand behind their work in an interview.

What you'll learn

- The **production polish** every senior engineer adds: loading states, error boundaries, accessibility, performance.
- A **realistic testing strategy** for a project this size — what to test, what *not* to test, and which tools to reach for.
- How to **deploy to Vercel** through GitHub, set up previews, and reason about build-time vs. run-time.
- The **interview narrative**: a 90-second walkthrough of the project that hits architecture, decisions, and trade-offs.
- The **vibe-coding prompt library**: the prompt templates that *demonstrably produced this codebase*.

By the end of this chapter you'll be able to:

1. Open this repo on a hiring manager's screen and explain *why* every directory exists.
2. Answer "what would you do differently?" with confidence (because you've already thought about it).

3. Reach for the right testing tool when someone says “add a test for that.”
4. Reproduce this style of work on a new project at maybe a third of the prompt count.

Lessons in this chapter

1. **01-production-polish.md** — Accessibility, loading states, error boundaries, and the performance audits that show you care.
2. **02-deployment-and-testing.md** — Vercel deployment via GitHub, environment variables, the testing pyramid, and what the first three tests should cover.
3. **03-interview-narrative.md** — The 90-second walkthrough, the common interview questions answered with this app, and the vibe-coding prompt library distilled.

New concepts introduced

- **Error boundaries** in React (class components — yes, still!).
- **<Suspense> and loading UI** in Next.js App Router.
- **Lighthouse / Web Vitals** as a quality bar.
- **Semantic HTML** and **ARIA** essentials (no more `<div>` soup).
- **Vitest + React Testing Library** for unit tests.
- **Playwright** for end-to-end smoke tests.
- **Vercel preview deployments** triggered by every PR.
- **STAR-format interview answers** (Situation, Task, Action, Result).

How long it should take

- 30–45 min reading per lesson
- 2–3 hours typing the polish code (a11y + tests)
- 30 min setting up Vercel (mostly waiting)
- ~6 hours total

Why this chapter exists separately

Most React courses end at “you built the feature.” That’s the moment a senior engineer’s job *starts*. The polish, the tests, the deployment, and the ability to talk about your trade-offs are the things that get you hired. We isolated them here so you can binge them as a single concentrated unit before your next interview.

Before you start

You should have:

- All previous chapters complete.
- The app running locally (`npm run dev`).
- A GitHub account with the repo pushed.
- A Vercel account (free tier is fine — you'll use it in Lesson 2).

What this chapter is *not*

- It is **not** a complete testing course. That's a book. We're teaching you what an 8-team fantasy app actually needs.
- It is **not** a complete a11y course. We're teaching the 80% that catches 95% of issues.
- It is **not** a Vercel sales pitch. We picked Vercel because it's the path of least resistance for Next.js. Cloudflare Pages, Netlify, Render, AWS Amplify all work — the principles transfer.

Open [01-production-polish.md](#) when you're ready.

Learn React + Next.js by Building a Snooker Fantasy League

A structured, beginner-friendly course that teaches you React from scratch by **rebuilding** a real, working fantasy-sports app — one component at a time, in the order a senior engineer would actually build it. Designed to take you from zero to interview-ready.

Who this course is for

You. Specifically, you if you:

- Have written a tiny bit of JavaScript or another language (Python, Java, etc.) but **never built a real React app**.
- Want to understand **why** React is structured the way it is — not just memorize syntax.
- Want to prepare for **React interviews** by building something you can show off and explain.
- Want to get good at **vibe coding** — directing an AI to produce clean, maintainable code instead of accepting whatever it spits out.

If you've already built three React apps and just want a quick reference, the single-file course at `SNOOKER_FANTASY_LEAGUE_COURSE.md` (preserved untouched in this folder) is a denser, faster read. This expanded multi-chapter version is the one to read if you actually want to **learn the craft**.

What you'll build

By the end of this course you will have built — by hand, line by line, with full understanding — a complete Next.js + TypeScript application:

- A **standings table** ranking 8 fantasy teams by points
- A **matches view** with cards for every match across 5 rounds ($32 \times 16 \times 8 \times 4 \times 1 = 31$ matches)
- A **predictions matrix** comparing 8 teams' picks side-by-side
- A **players directory** with deep per-player drill-downs (32 player profiles)
- An **analytics dashboard** with five Recharts visualizations

The finished version of all of this lives at `components/`, `lib/`, and `app/` in the project root. **You'll rebuild it from scratch as you learn.** The course tells you exactly what to type, why to type it, and what each line does.

What you'll learn (the React inventory)

By the time you finish Chapter 7, you will have used and **understood** every one of these React concepts:

Concept	Where you'll meet it
Functional components	Chapter 3
Props & prop types	Chapter 3
<code>useState</code>	Chapter 4
<code>useMemo</code>	Chapter 4
Lifting state up	Chapter 4
Conditional rendering	Chapter 3, 4
Lists & keys	Chapter 3
Event handlers	Chapter 4
Component composition	Chapter 5
Prop drilling vs Context	Chapter 5

Concept	Where you'll meet it
Controlled components	Chapter 4
TypeScript with React	Throughout
Inline styles vs Tailwind vs CSS	Chapter 3
Pure functions in UI code	Chapter 1
Data transforms for charts	Chapter 6
Server vs client components (Next.js App Router)	Chapter 2, 7
useState inside a child vs at the top	Chapter 6
Re-renders, references, and identity	Chapter 4

You'll also pick up the **mental models** that experienced React developers carry around — when to split a component, when to memoize, what state lives where, why prop drilling is fine for three levels and a problem at six. These don't show up in tutorials, but they show up in every senior interview.

How the course is structured

Seven chapters. Each chapter has three lessons. Each lesson is around 1,500–2,500 words and ends with a **CHECK YOURSELF** box of questions you should be able to answer before moving on.

```

course/
├── README.md                                ← you are here
├── SNOOKER_FANTASY_LEAGUE_COURSE.md        ← original single-
file version (reference)
├──
├── chapter-01-foundations/                  ← UNDERSTAND THE PROBLEM
│   ├── 01-the-problem.md
│   ├── 02-data-modeling.md
│   └── 03-pure-functions-and-scoring.md
├──
├── chapter-02-project-setup/                ← FROM EMPTY FOLDER TO RUNNING DEV SERV
│   ├── 01-why-nextjs-typescript.md
│   ├── 02-scaffolding-the-project.md
│   └── 03-creating-data-files.md

```

└─ chapter-03-first-component/	← YOUR FIRST REACT COMPONENT
└─ 01-anatomy-of-a-component.md	
└─ 02-rendering-the-standings.md	
└─ 03-progressive-enhancement.md	
└─ chapter-04-state-and-hooks/	← INTERACTIVITY & SHARED STATE
└─ 01-useState-mental-model.md	
└─ 02-the-orchestrator-pattern.md	
└─ 03-useMemo-and-derived-data.md	
└─ chapter-05-composition/	← BIG COMPONENTS, SMALL COMPONENTS
└─ 01-when-to-split-components.md	
└─ 02-prop-drilling-vs-context.md	
└─ 03-building-player-detail.md	
└─ chapter-06-data-visualization/	← CHARTS WITH RECHARTS
└─ 01-recharts-fundamentals.md	
└─ 02-reshaping-data.md	
└─ 03-pivots-and-internal-state.md	
└─ chapter-07-mastery-and-interviews/	← INTERVIEW PREP & VIBE CODING
└─ 01-production-polish.md	
└─ 02-deployment-and-testing.md	
└─ 03-interview-narrative.md	

Each chapter folder also has its own `README.md` with a chapter-level overview, a list of new concepts introduced, and a list of files you'll have created by the end of the chapter.

How to use this course

The recommended workflow

1. **Read the lesson** in your IDE or browser.
2. **Open a second editor window** showing the *finished* version of whatever file the lesson is teaching (in `components/`, `lib/`, etc.). Use it as the answer key.
3. **Type the code yourself** as you go. Don't copy-paste. Typing is how the patterns

become muscle memory.

4. **Run `npm run dev`** at the end of each lesson and look at the result in the browser. If the screen looks wrong, you skipped something.
5. **Answer the CHECK YOURSELF questions out loud** (or in writing) before moving on. They're not optional. They're the difference between *reading* React and *knowing* React.
6. **Commit at the end of every lesson** with a message like `Chapter 3 Lesson 2: render the standings table`. Building a real git history is part of the point.

Pace yourself

The full course is about 35,000 words plus typing the code. Realistically:

- **One lesson per evening (1–2 hrs)** = comfortable pace, ~3 weeks total.
- **One chapter per weekend** = aggressive but doable, ~3 weekends.
- **Binge** = don't. You'll forget. Spaced repetition wins.

Don't skip Chapter 1

Chapter 1 has zero React in it. It's about **the problem** and **the data model**. Beginners always want to skip to "the React part." Don't. Half of senior interview questions are "*How would you model X?*" and "*What state is owned where?*" — and those are answered before any JSX gets typed. Chapter 1 is where you build the muscle for that.

Reading vs building

Each lesson has two layers:

- **Concepts** — the *why*. Why do components return JSX? Why is `useState` a hook? Why does this data shape work for this chart?
- **Code** — the *what*. The exact lines you'll type, copy-pasted from the real codebase.

Both matter. If you only read the concepts, you'll be a React philosopher who can't ship. If you only type the code, you'll be a React typist who can't debug. The course alternates deliberately.

What's in the rest of the repo

Beyond the course folder, the repo contains the **finished application** so you can compare your work to the answer key at any point:

```
e:\reverse-understanding\
├─ _source/
│   └─ SnookerFantasyLeague.jsx    ← the original 1,654-line single-
file build
│
│                               (preserved verbatim, the artifact this course
│                               was reverse-engineered from)
├─ _briefs/
│   ├── PROMPT_1_NEXTJS_REFACTOR.md ← the spec that generated the refactor
│   └─ PROMPT_2_REVERSE_ENGINEERING_COURSE.md
│
│                               ← the spec that generated this course
├─ app/                          ← Next.js App Router entry points
├─ components/                   ← all the React components, organized by feature
│   ├── SnookerFantasyLeague.tsx   ← the top-level orchestrator
│   ├── tabs/                      ← StandingsTab, MatchesTab, PredictionsTab, ...
│   ├── standings/                 ← LeagueTable, StatCard, FormBadge
│   ├── matches/                  ← MatchesList, PlayerLine, RoundButton
│   ├── predictions/              ← PredictionMatrix, TeamCardView, PicksList
│   ├── players/                  ← PlayerCard, PlayerDetail, MatchPickAnalysis
│   └─ analytics/                 ← LeagueAgreementChart, ChartCard, InsightCard
├─ lib/                          ← pure data + logic (no React)
│   ├── types.ts                  ← TypeScript interfaces
│   ├── matches.ts                ← all 31 matches
│   ├── teams.ts                  ← all 8 fantasy teams + their picks
│   ├── players.ts                ← player metadata
│   ├── scoring.ts                ← scorePick + calculateTeamScores
│   └─ constants.ts               ← shared style helpers + ball colors
├─ package.json
├─ tsconfig.json
├─ tailwind.config.ts
└─ next.config.mjs
```

You'll re-create most of these files yourself by following the lessons. The originals stay

there as your reference.

Prerequisites

You **need**:

- Node.js 20+ (for Next.js 14)
- npm (comes with Node)
- A code editor (VS Code, Cursor, Windsurf — anything)
- A browser
- ~15–20 hours of focused time

You **don't need**:

- Prior React experience
- A computer-science degree
- Mac or Linux (this course was written on Windows; PowerShell commands are noted explicitly)
- Any paid services

Conventions used in this course

- **backticks** for variable, function, file, and component names.
- **Code blocks** with a language tag (````tsx`, ````ts`, ````bash`, ````json`).
- **CHECK YOURSELF boxes** at the end of each lesson. Don't skip them.
- **Real code only.** Every code snippet in this course is taken from the actual finished app in this repo. There are no toy examples or simplified pretend-code.
- **TypeScript everywhere.** If you've never touched TS, don't worry — the lessons explain types as they appear.

A note on AI / vibe coding

The original build of this app was done with AI assistance (“vibe coding”). This course teaches you both the **React fundamentals** and the **prompting craft** that produced the codebase. Chapter 7 is dedicated to making you a better vibe coder: when an LLM gives you React, what do you accept and what do you reject? When do you ask for “extract this into a component” versus “rewrite this”?

Interviewers in 2026 don't ask "Do you use AI?" — they ask "**How** do you use AI?" If you can articulate why a particular component should be split, why a particular piece of state lives where it does, why a particular memoization is or isn't worth it — you sound like a senior, regardless of whether you typed the code or prompted it.

Ready?

Start at [Chapter 1, Lesson 1: The Problem](#).

You won't write a single line of JavaScript in Chapter 1. By the end of Chapter 7 you'll have built a complete Next.js application from scratch and you'll be able to explain every architectural decision in it. **That's how you pass interviews.**

Snooker Fantasy League: A Reverse-Engineering Course

A nine-lesson walk-back from a finished 1,654-line React app to the first commit it would have come from. Read alongside `_source/SnookerFantasyLeague.jsx` open in another window.

Confirming the source

I read `SnookerFantasyLeague.jsx` end-to-end. The five top-level **data constants** are:

1. `ROUND1_MATCHES` — 16 first-round matches
2. `ROUND2_MATCHES` — 8 last-16 matches
3. `QF_MATCHES` — 4 quarter-finals
4. `SF_MATCHES` — 2 semi-finals
5. `FINAL_MATCH` — 1 final

...with two more big data structures that the app leans on: `TEAMS` (the eight fantasy teams and their picks) and `PLAYER_INFO` (per-player metadata, keyed by name).

The five **tabs** are:

1. **Standings** — leaderboard + headline stats
2. **Matches** — match cards per round, finished/not-finished
3. **Predictions** — comparison matrix + single-team card
4. **Players** — 32-player grid + a deep player-detail view

5. **Analytics** — Recharts dashboards (points, accuracy, agreement, insights)

All set. Course starts below. Each lesson ends with a **Vibe prompt you would have used** section and a **CHECK YOURSELF** box.

Lesson 0 — The Problem (no code)

Imagine the pub conversation

You and seven mates watch the World Snooker Championship every May. Sheffield, Crucible, two weeks of green-baize warfare. Someone — probably you — says: “*We should make a fantasy league out of this.*” Everyone agrees. Pints are raised. Now what?

A snooker tournament has a fixed **bracket of 32 players**. They play through five rounds: Round 1 (last 32), Round 2 (last 16), Quarter-Finals (last 8), Semi-Finals (last 4), and the Final. The bracket is published before a ball is potted. So **every match is known in advance** — only the winners are unknown.

That’s the whole hook. If every match is known, then every team can submit picks for every match in every round, **before the tournament starts**. As results come in, the league scores those picks.

The 8 teams

There are eight fantasy teams: *Invincibles*, *Uncredibles*, *Hopeless*, *Clueless*, *Break Builders United*, *The Untouchables*, *Selbies*, *One Four Sevens*. Each one is a person (or a couple) sitting at home with their bracket, picking who they think wins each match. Each team needs:

- 16 picks for Round 1 (one per match)
- 8 picks for Round 2
- 4 picks for the Quarter-Finals
- 2 picks for the Semi-Finals
- 1 pick for the Final

That’s **31 picks per team, times 8 teams = 248 picks total**, all locked in before the first break-off.

The scoring rules (3 bullets)

- **3 points** if your pick wins the match.
- **1 point** (a “consolation”) if your pick loses — you still showed up.
- **null / pending** if the match hasn’t been played yet — neither right nor wrong, just not scored.

That last point matters more than it looks. Lesson 2 is built around it.

What the app needs to do

When the tournament is live, you want to be able to: see who’s leading, see how each round shook out, drill into one team’s picks, drill into one player’s appearances, and see charts of who agreed and who didn’t. That’s it. **No login, no database, no sharing.** It’s a static page; the data is edited by hand as results come in.

That gives the architecture away already: it’s a single-page React app whose entire “database” is a few JavaScript constants. Lesson 1 starts there.

Vibe prompt you would have used

“I want to build a single-page React app for a fantasy snooker league. Eight teams pick winners across five rounds (R1: 16 picks, R2: 8, QF: 4, SF: 2, Final: 1). Scoring: 3 points if the pick wins, 1 if it loses, null while the match is unplayed. No database — the bracket and team picks are edited by hand in code. The page should have tabs for Standings, Matches, Predictions, Players, and Analytics. Don’t write any code yet — just confirm you understand the problem and ask me anything that’s ambiguous.”

That last sentence (“don’t write any code yet — just confirm”) is the single most useful trick in this lesson. It forces the LLM to surface ambiguities you didn’t think of.

CHECK YOURSELF

1. Why does scoring use **null** for unplayed matches instead of **0**?
2. Why are there exactly **31 picks per team**, no more and no less?
3. If a 9th friend wanted to join, what’s the minimum data change required? (Hint: it’s a one-line addition.)

Lesson 1 — The Shape of the Data (data modeling, no UI yet)

Before any pixel renders, you need to decide what shapes you're pushing around. In a Python data-engineering background this is the part that feels comfortable — it's a schema design problem, just expressed in JS objects instead of pydantic models.

What's a Match?

Every match in the tournament is the same shape:

```
{ id: 1, p1: 'Zhao Xintong', p2: 'Liam Highfield', winner: 'Zhao Xintong', score: '10-7', seed1: 1 }
```

Five fields, three of which are obvious (id, p1, p2) and two of which are the interesting ones:

- **winner** is **either a string or undefined**. Undefined means *the match hasn't been played*. As soon as a result comes in, you set winner to one of the two player names. This is the field that drives the scoring logic.
- **score** is a display-only string like '10-7' or '17-18'. The app never parses it for points; it only splits it on '-' to render two big numbers next to the players.
- **seed1** is on R1 only, because that's the only round where the bracket is constructed by seed; later rounds the seed is implied by the path.

The app declares the matches up-front in five flat arrays — one per round — in match order:

```
const ROUND1_MATCHES = [
  {
    id: 1,
    p1: 'Zhao Xintong',
    p2: 'Liam Highfield',
    winner: 'Zhao Xintong',
    score: '10-7',
    seed1: 1,
  },
  { id: 2, p1: 'Judd Trump', p2: 'Gary Wilson', winner: 'Judd Trump', score: '10-7', seed1: 2 },
  // ... 14 more
];
```

The match order is **the bracket order**, top to bottom. R1 has 16 entries, R2 has 8, QF has 4, SF has 2, Final has 1 — the exact halving you'd expect from a knockout draw.

What's a Team?

A team is a name + a vibe + five arrays of picks:

```
{
  name: 'Invincibles',
  color: '#0F5132', accent: '#10B981',
  icon: '🏆', motto: 'Unstoppable & Unbeatable',
  r1: ['Zhao Xintong', 'Judd Trump', /* 14 more */], // 16 picks
  r2: ['Zhao Xintong', 'Xiao Guodong', /* 6 more */], // 8 picks
  qf: ['Zhao Xintong', 'Mark Allen', 'John Higgins', 'Wu Yize'], // 4 picks
  sf: ['Shaun Murphy', 'Wu Yize'], // 2 picks
  final: 'Wu Yize', // 1 pick (no array)
}
```

The color / accent / icon / motto are pure cosmetics — they drive every gradient header and team chip in the UI. The five pick arrays are where the actual game lives.

final is a single string instead of a one-element array. That asymmetry is on purpose: there's only one Final, and it's nicer to write `team.final === 'Wu Yize'` than `team.final[0] === 'Wu Yize'`. The cost is that the scoring code has one branch for final and another for everything else (you'll see this in `calculateTeamScores`).

What's PLAYER_INFO and why is it an object keyed by name?

```
const PLAYER_INFO = {
  'Zhao Xintong': { country: '🇨🇳', seed: 1, status: 'Defending Champion', flag: 'CN' },
  'Judd Trump': { country: '🇬🇧', seed: 2, status: 'World No. 1', flag: 'ENG' },
  // ... 30 more
};
```

It's an object (a dictionary, a map) keyed by **player name** — not an array. Why? Because every other place in the app already has a player's name as a string (from a match's p1/p2, or from a team's pick), and the most common operation is “given a name, get the metadata for that player.” That's a hash lookup: `PLAYER_INFO['Zhao Xintong']`. If it were an array of { name, country, seed, ... } objects, you'd be writing `.find(p => p.name === name)` everywhere — $O(n)$ instead of $O(1)$, and uglier to read.

The Python equivalent is the same instinct that makes you reach for `dict[str, Play-`

erInfo] instead of list[PlayerInfo].

The most important rule in the codebase

Here is the invariant that the entire scoring engine depends on:

For each round, the i -th element of team.r1 (or r2, or qf, or sf) corresponds to the i -th element of ROUND1_MATCHES (or ROUND2_MATCHES, ...).

Index 0 of team.r1 is the team's pick for ROUND1_MATCHES[0]. Index 1 is the pick for match 1. And so on. There is no matchId field on the picks. There is no key. **Order is the contract.**

That's why scorePick can be the trivial five-line function it is in Lesson 2 — it just takes a pick and a match and asks “did the pick equal the match's winner?” If you ever re-order ROUND1_MATCHES without re-ordering all eight teams' r1 arrays in lockstep, every score in Round 1 silently goes wrong. There is no warning, no test, no type system catching this — it's a convention enforced by the file.

If you grew up writing SQL, this feels barbaric (“where's the foreign key?”). It's barbaric. It's also fine for an 8-team league where one person owns the file. The moment a 9th team or 33rd player shows up, this is the first thing that needs schema-ing.

Vibe prompt you would have used

“Here's the data model for my fantasy snooker league. Don't generate any UI yet. I want to define the constants. Match: { id, p1, p2, winner?, score?, seed1? }. Team: { name, color, accent, icon, motto, r1[16], r2[8], qf[4], sf[2], final } where each pick array is in the same order as the matching round's match array. PlayerInfo: an object keyed by player name with { country, seed, status, flag }. Generate empty constants for ROUND1_MATCHES through FINAL_MATCH (5 arrays), an empty TEAMS array of 8 with placeholder picks, and a PLAYER_INFO object stub with the 16 seeded players. Add a comment above the team picks reminding the reader: 'pick array index i corresponds to match index i of the matching round.'”

CHECK YOURSELF

1. If you swapped match 7 and match 8 in `ROUND1_MATCHES` but forgot to swap them in every team's `r1` array, what would visibly go wrong in the UI?
 2. Why is `PLAYER_INFO` keyed by name and not by seed?
 3. `team.final` is a string but `team.sf` is an array. What's the smallest code change that would make them consistent, and what would it cost in readability?
-

Lesson 2 — The Pure Logic (scoring, before any React)

This is the smallest, most important file in the project. It's about 40 lines and it's the **heart** of the app — every UI screen is just a different lens on its output.

Why scoring lives outside React

The original file declares two functions before the first JSX tag:

```
function scorePick(pick, match) {  
  if (!match || !match.winner) return null;  
  if (pick === match.winner) return 3;  
  return 1;  
}  
  
function calculateTeamScores(team) {  
  /* ... */  
}
```

Neither of them imports React. Neither knows what a component is. They take plain data in and return plain data out. That's deliberate. **Pure functions are the part of your codebase that survives every refactor.** When Phase 1 of this project moves the app to Next.js, these two functions are copied into `lib/scoring.ts` essentially unchanged — the components around them are rewritten, but `scorePick` is the same.

The Python instinct here is right: this is the “business logic” layer. Keep it free of framework concerns. UIs come and go; the rule that picking-the-winner = 3 points doesn't.

Walking through scorePick line by line

```
function scorePick(pick, match) {  
  if (!match || !match.winner) return null; // pending  
  if (pick === match.winner) return 3; // correct  
  return 1; // wrong (consolation)  
}
```

Three branches:

1. `!match || !match.winner` → `null`. If the match doesn't exist (defensive) or hasn't been played yet (winner is undefined), the pick can't be scored. The function returns `null`, **not 0** and **not false**. That distinction is the heart of the round.
2. `pick === match.winner` → `3`. Right pick.
3. **Else** → `1`. Wrong pick, but you participated, so 1 point.

Why correct is null, not false

Inside `calculateTeamScores` you'll see this pattern repeated for every round:

```
const r1Details = team.r1.map((pick, i) => {  
  const pts = scorePick(pick, ROUND1_MATCHES[i]);  
  r1 += pts || 0;  
  return { match: ROUND1_MATCHES[i], pick, points: pts, correct: pts === null  
});
```

Look at the last expression: `correct: pts === null ? null : pts === 3`.

`correct` has **three states**, not two:

- `true` — the match is finished and the pick was right.
- `false` — the match is finished and the pick was wrong.
- `null` — the match isn't finished yet.

The UI uses this directly. In `<PicksList>`:

```
const pickPending = d.points === null;  
// ...  
background: pickPending ? '#F9FAFB' : d.correct ? '#DCFCE7' : '#FEE2E2',
```

Three colors: gray for pending, green for correct, red for wrong. If correct were just a

boolean, “pending” and “wrong” would render as the same red badge — and during the tournament, when most matches are still upcoming, the screen would look catastrophic.

This is the first lesson the brief tells you to internalize: **tri-state booleans aren’t a code smell, they’re a feature**. The other place this comes up is `match.winner` itself — undefined means “not played”, a string means “played and winner is this person”. Those are not the same as `null` would be in some languages; the absence of a winner is a real first-class state.

`r1 += pts || 0` — the running total trick

That line is doing two things at once:

```
r1 += pts || 0;
```

If `pts` is 3 or 1, you add it. If `pts` is `null` (pending), `null || 0` evaluates to 0 and you add nothing. So `r1` accumulates only finished, scored points. The total never goes “wrong” while a round is in progress; it just lags reality until results land.

What `calculateTeamScores` returns

The return shape is wide on purpose:

```
return {  
  r1,  
  r2,  
  qf,  
  sf,  
  f,  
  total, // 6 numbers  
  r1Details,  
  r2Details,  
  qfDetails,  
  sfDetails,  
  fDetails, // 5 arrays of {match, pick, points, correct}  
};
```

Six numbers (the per-round totals + the grand total) plus five arrays of the per-pick detail objects. The numbers drive the leaderboard column totals. The detail arrays drive the

predictions matrix, the team card view, the player detail page — anywhere the app says “here’s pick X against match Y, was it right, how many points.” **Compute once, render five different ways.**

Inside the orchestrator component you’ll see exactly that:

```
const teamsWithScores = useMemo(() => {
  return TEAMS.map((t) => ({ ...t, scores: calculateTeamScores(t) })).sort(
    (a, b) => b.scores.total - a.scores.total
  );
}, []);
```

Every team is decorated with its scores object once, and that decorated array is then handed down to every tab. The five tabs share the same computation; none of them re-invoke `calculateTeamScores` themselves.

Vibe prompt you would have used

*“I have my data constants set up (ROUND1_MATCHES through FINAL_MATCH, plus a TEAMS array with r1[16]/r2[8]/qf[4]/sf[2]/final picks). Write me two pure functions, no React, no JSX. `scorePick(pick, match)` returns 3 if `pick === match.winner`, returns 1 if the match has a winner but the pick was wrong, and returns null if `match.winner` is undefined. Then `calculateTeamScores(team)` walks through each round’s pick array and the matching match array in parallel by index, sums the points (treating null as 0 for the running total), and returns { r1, r2, qf, sf, f, total, r1Details, r2Details, qfDetails, sfDetails, fDetails } where each *Details is an array of { match, pick, points, correct } and correct is null if pending, otherwise points === 3.”*

CHECK YOURSELF

1. If `scorePick` returned 0 instead of null for pending matches, what’s the first thing that would visibly break in the UI? (Hint: `.filter(d => d.correct).length` is used a lot.)
2. Why is `calculateTeamScores` called inside `useMemo` instead of inline in the JSX?
3. Which property of the return value of `calculateTeamScores` do the **standings columns** read, and which property do the **predictions matrix cells** read? Why are

both needed?

Lesson 3 — The First Render (a single component, no tabs yet)

This is the part that vibe coders get wrong on day one: trying to build the whole app in the first prompt. Don't. **A vibe-coded app is grown one feature at a time, starting with the smallest thing that proves the data is real.** For Snooker Fantasy League, that thing is a list of team names with their total points.

v1: just team names and totals

If you go back to the very beginning of the project, the first version of `<StandingsTab>` would have been about 15 lines:

```
function StandingsTab({ teams }) {
  return (
    <table>
      <thead>
        <tr>
          <th>Rank</th>
          <th>Team</th>
          <th>Total</th>
        </tr>
      </thead>
      <tbody>
        {teams.map((team, i) => (
          <tr key={team.name}>
            <td>{i + 1}</td>
            <td>{team.name}</td>
            <td>{team.scores.total}</td>
          </tr>
        ))}
      </tbody>
    </table>
  );
}
```

```
}
```

That's it. No styles, no icons, no gradients. The point is to **prove that the data flow works**: `calculateTeamScores` runs, the totals compute, `.sort()` orders the rows, and React renders eight rows. If any one of those is wrong, you find out in the first 30 seconds, not in hour three after you've added a hero header and tab bar.

Once that's on screen you have your foothold. Everything afterwards is "now add a column."

v2: per-round columns

The next thing a human builder would notice is *"I can't tell which team is doing well in which round."* So you add R1 and R2 columns:

```
<th>Rank</th><th>Team</th>
<th>R1</th>
<th>R2</th>
<th>Total</th>
```

```
<td>{team.scores.r1}</td>
<td>{team.scores.r2}</td>
```

This is a one-line-prompt change ("add a column for R1 score next to the team name"). You don't redesign the table; you just add cells. The fact that `team.scores` already has `r1` and `r2` waiting is exactly why Lesson 2's "wide return shape" was worth it.

v3: progressive enhancement of the same column

Then the tournament progresses and you add `qf`, `sf`, and `final`. By the end of the tournament, the row in the production code looks like the one in the file:

```
<td style={{ ...td, textAlign: 'center' }}>
  <div style={{ fontWeight: 700, fontSize: 17, color: '#0F5132' }}>{team.score}
  <div style={{ fontSize: 11, color: '#6B7280' }}>/ 48</div>
</td>
```

Two divs, not just a number — the team's R1 score over the maximum possible 48 (16 matches × 3 points). That `/48` line was almost certainly added in a separate prompt:

“under the R1 number, show ‘/ 48’ in a smaller gray font so I can see how close they are to a perfect round.” This is the right grain for a vibe prompt — small, specific, low-risk.

v4: the special “Final Pick” cell

The Final column is different from the others. It’s not a number — it’s a **chip** showing who the team picked, color-coded by whether the actual champion matched:

```
<td style={{ ...td, textAlign: 'center' }}>
  <div
    style={{
      display: 'inline-block',
      padding: '4px 10px',
      borderRadius: 12,
      background: team.final === 'Wu Yize' ? '#FEE2E2' : '#DBEAFF',
      color: team.final === 'Wu Yize' ? '#991B1B' : '#1E40AF',
      fontWeight: 700,
      fontSize: 12,
    }}
  >
    {team.final}
  </div>
</td>
```

Notice that the colors here are **hard-coded** to “if the pick is Wu Yize, red — otherwise blue.” That’s a tell that this code was written **after the final result was known**. During the tournament, when the champion wasn’t yet decided, this branch would have looked completely different — probably just neutral coloring. The hard-coded ‘Wu Yize’ is a tournament-end snapshot. A more general version would compare against `FINAL_MATCH[0]?.winner`. The code is one prompt away from being made tournament-agnostic.

This is a normal pattern in vibe-coded codebases: **branches that were once dynamic become hard-coded after the data is final**. Not technically wrong, but worth knowing it’s there.

v5: the leader highlight

The leader row deserves a crown. So somewhere mid-project a prompt added:

```
const rankColor = rank === 1 ? '#FBBF24' : rank === 2 ? '#9CA3AF' : rank ===
// ...
<div style={{ ... background: rankColor, ... }}>
  {rank === 1 ? '👑' : rank}
</div>
```

Gold for 1st, silver for 2nd, bronze for 3rd, gray otherwise. The leader gets a crown emoji instead of “1”. That’s two visual rules added in one prompt; you can see how a single sentence (“medal-color the rank circle and replace 1st place’s number with a crown emoji”) becomes those four lines.

v6: the form badge

Last, the rightmost column tracks form — whether the team is doing better in R2 than they did in R1. That’s `<FormBadge>`:

```
function FormBadge({ r1Pct, r2Pct }) {
  const trending = r2Pct > r1Pct ? 'up' : r2Pct < r1Pct ? 'down' : 'flat';
  const Icon = trending === 'up' ? TrendingUp : trending === 'down' ? TrendingDown : TrendingFlat;
  // ... renders an arrow + label
}
```

It’s a tiny presentational component. Notice it’s a **separate function**, not inlined into the table row. That’s because you’ll want it again in player cards, in summary tiles, anywhere you want to express “this is going up / down / sideways.” Lesson 5 will come back to this idea: when a thing has a name, give it a function.

Vibe prompt you would have used (at each step)

v1: “Render the eight teams as a table sorted by `team.scores.total`. Three columns: Rank, Team, Total. Plain HTML, no styles yet, just verify the totals are right.”

v2: “Add R1 and R2 columns next to Team showing `team.scores.r1` and `team.scores.r2`.”

v3: “Under each round score in the table, show the maximum possible like ‘/ 48’ for R1 and ‘/ 24’ for R2 in a smaller gray font.”

v4: “Add a *Final Pick* column. Render *team.final* inside a rounded chip with a 🏆 emoji. If the pick equals the eventual champion (Wu Yize), use a red background; otherwise blue.”

v5: “Make the rank circle gold for 1st, silver for 2nd, bronze for 3rd. Show a 🥇 emoji instead of the number for 1st place.”

v6: “Add a *Form* column at the right that compares the team’s R1 percentage to their R2 percentage. If they’re improving show a green up-arrow and ‘Rising’; if dropping, red down-arrow ‘Falling’; if flat, gray circle ‘Steady’. Extract it to a `<FormBadge>` component.”

The shape is always the same: **one column, one widget, one prompt**. Never “redesign the standings table” — that’s the kind of prompt that produces 800 lines of speculative CSS and breaks something invisible.

CHECK YOURSELF

1. The Final Pick chip currently hard-codes 'Wu Yize' to choose its color. What’s the one-line edit to make it tournament-agnostic?
2. Why is `<FormBadge>` extracted into its own function instead of inlined into the table cell?
3. If you wanted to add a new “% correct” column between R2 and Total, what minimum changes would the prompt need to specify? (Hint: it’s two cells per row plus one header.)

Lesson 4 — State & Tabs (introducing `useState`)

Up until now we’ve talked about the app like it’s one screen. It isn’t — it’s five tabs that share data. The moment you have two screens that show the *same* data with *different* lenses, you need a top-level component that owns the state and feeds it down. This is the React mental model that breaks the most for newcomers, so it’s worth getting right.

Two pieces of state, one home

The orchestrator component is `<SnookerFantasyLeague>`. Its top three lines:

```
export default function SnookerFantasyLeague() {
  const [activeTab, setActiveTab] = useState('standings');
  const [selectedTeam, setSelectedTeam] = useState(null);
}
```

Two pieces of state. **activeTab** is a string like 'standings' or 'predictions' — which of the five tabs is currently visible. **selectedTeam** is either null (nobody clicked yet) or one of the team objects — used by the Predictions tab to know which team's deep card to show.

These two pieces of state cannot live inside the individual tab components, because **clicking a row in <StandingsTab> needs to switch the visible tab to <PredictionsTab> and remember which team to highlight**. That cross-tab effect needs a place where both tabs can read and write the same state. That place is the parent.

The pattern is:

State up, callbacks down. The state lives at the highest component that needs to see all of it. The leaf components don't own state; they receive props and call back when they want a change.

useMemo for the decorated team list

The next line is the unsung hero of the whole file:

```
const teamsWithScores = useMemo(() => {
  return TEAMS.map((t) => ({ ...t, scores: calculateTeamScores(t) })).sort(
    (a, b) => b.scores.total - a.scores.total
  );
}, []);
```

Read it inside-out:

- `TEAMS.map(t => ({ ...t, scores: calculateTeamScores(t) }))` — for each of the 8 teams, spread its fields (name, color, r1[], ...) and add a fresh scores object computed from `calculateTeamScores`.
- `.sort((a, b) => b.scores.total - a.scores.total)` — descending by total points so index 0 is the leader.
- Wrapped in `useMemo(..., [])` — compute this **once** on first render and reuse forever.

Why useMemo? Because **without it, every time the user clicks a tab, React would recompute all 8 teams' scores from scratch**. That's $8 \times 31 = 248$ scorePick calls per click. It's not slow enough to feel, but it's wasteful, and more importantly it would mean a new teamsWithScores array reference on every render — which can cascade into spurious re-renders of every tab that receives it as a prop.

The empty dependency array [] says: *“the inputs don't change during the lifetime of the component — TEAMS is a module-level constant — so cache the result once.”* If TEAMS were ever loaded from an API and stored in state, that array would have to include the state variable.

This is the same reflex as memoizing an expensive .transform() step in a Pandas pipeline. The data layer's free; the UI layer should look it up, not recompute it.

Wiring the tabs

After the state and the memoized data, the component renders three things:

1. The hero header (decorative — no state).
2. A horizontal nav bar that maps over a tabs array and renders five buttons. Clicking any button calls setActiveTab(tab.id).
3. The content area, which **switches on activeTab** and renders the right tab component:

```
{
  activeTab === 'standings' && (
    <StandingsTab
      teams={teamsWithScores}
      onTeamClick={(t) => {
        setSelectedTeam(t);
        setActiveTab('predictions');
      }}
    />
  );
}
{
  activeTab === 'matches' && <MatchesTab />;
}
```

```

{
  activeTab === 'predictions' && (
    <PredictionsTab
      teams={teamsWithScores}
      selectedTeam={selectedTeam}
      setSelectedTeam={setSelectedTeam}
    />
  );
}
{
  activeTab === 'players' && <PlayersTab teams={teamsWithScores} />;
}
{
  activeTab === 'analytics' && <AnalyticsTab teams={teamsWithScores} />;
}

```

Three things to notice:

- **<MatchesTab /> takes no props.** It only uses the static match constants from `lib/matches.ts`. There's no team data flowing in.
- **<StandingsTab> receives a callback `onTeamClick`** that does *two things at once*: it sets the selected team **and** switches the tab. That's the cross-tab interaction the parent had to coordinate. A child component couldn't have done this on its own.
- **<PredictionsTab> receives both the read (`selectedTeam`) and the write (`setSelectedTeam`).** This is the pattern that gets called "controlled component": the parent owns the source of truth; the child renders against it and asks the parent to update it.

Why not put `selectedTeam` inside `<PredictionsTab>`?

Because then the `<StandingsTab>`'s row click couldn't reach it. The state has to live at the **lowest common ancestor** of every component that needs to read or write it. For `selectedTeam`, the readers are `<PredictionsTab>` and the writers are both `<StandingsTab>` (when you click a row) and `<PredictionsTab>` (when you click a team chip inside it). Their lowest common ancestor is `<SnookerFantasyLeague>` itself, so that's where the state lives.

Vibe prompt you would have used

"Wrap my single <StandingsTab> in a top-level <SnookerFantasyLeague> component. Add a useState for activeTab (default 'standings') and a sticky horizontal nav bar with five tabs: Standings, Matches, Predictions, Players, Analytics, each with a Lucide icon. Render the matching tab component below. Add a second useState for selectedTeam (default null). Pass teamsWithScores (a useMemo'd sort of TEAMS.map(t => ({ ...t, scores: calculateTeamScores(t) }))) by descending total) to every tab that needs it. From <StandingsTab>, when a row is clicked, set selectedTeam and switch activeTab to 'predictions'."

CHECK YOURSELF

1. If you removed useMemo and just wrote `const teamsWithScores = TEAMS.map(...)` directly, what specifically would re-run on every tab switch?
 2. Why does setSelectedTeam get passed down to <PredictionsTab> as a prop instead of being created with its own useState inside that tab?
 3. Why does <MatchesTab /> not need any props at all, while <StandingsTab> needs two?
-

Lesson 5 — Composition & Prop Drilling (when to split a component)

A 200-line component is a smell. So is splitting too early. The right time to extract a sub-component is when **either** of these is true:

- You're about to copy-paste a chunk of JSX and tweak one variable.
- A section is conceptually about a different "thing" than the rest of the component.

Both of those happen all over this codebase. The clearest example is the chain <PlayersTab> → <PlayerDetail> → <MatchPickAnalysis> → <TeamChip>.

The chain, explained

<PlayersTab> (top of the players folder) has two jobs: render the 32-player grid, and — when a player is clicked — render <PlayerDetail> for that player. The rule "two jobs" is

fine in a parent component; that's literally what an orchestrator does. The actual rendering of those two jobs is delegated:

```
if (selectedPlayer) {  
  return <PlayerDetail playerName={selectedPlayer} teams={teams} onBack={...}  
}  
// ... otherwise render the grid of <PlayerCard> buttons
```

<PlayerDetail> (about 270 lines of computation + JSX) does **all** the per-player math: which matches did this player appear in, who picked them, who picked against them, did they win, what's their tournament path. That's a lot. So inside it, the per-match analysis is delegated:

```
{  
  pickAnalyses.map((pa, i) => (  
    <MatchPickAnalysis  
      key={i}  
      analysis={pa}  
      playerName={playerName}  
      onSelectPlayer={onSelectPlayer}  
    />  
  ));  
}
```

<MatchPickAnalysis> renders one match (e.g. "Murphy vs Wu Yize, Final, Wu won 18-17") and the two columns of teams that backed each side. Inside it, the team chips are delegated:

```
{  
  teamsBackedThis.map((t) => <TeamChip key={t.name} team={t} ptsEarned={ptsFor}  
}
```

<TeamChip> is **15 lines**. It's a single colored pill showing the team icon, name, and the points its picker earned. Why does it deserve its own file? Because it's drawn **twice on every single match analysis** — once in the "Backed Murphy" column and once in the "Backed Wu" column — and copy-pasting 15 lines of JSX twice would be an obvious smell. Extract.

When to extract: the copy-paste rule

If you find yourself writing the same JSX twice with minor tweaks, that's the signal. Inside `<MatchPickAnalysis>`:

```
<div>
  <div>BACKED {playerName.toUpperCase()}</div>
  {teamsBackedThis.map(t => <TeamChip team={t} ptsEarned={ptsForPicker} />)}
</div>
<div>
  <div>BACKED {opponent.toUpperCase()}</div>
  {teamsBackedOpponent.map(t => <TeamChip team={t} ptsEarned={ptsForOpponent} />)}
</div>
```

Two columns, identical structure, different inputs. The chip is extracted; the **column wrapper is not** — because it's only used twice and only in this file, and the difference (color theme, points value) is enough that a third level of abstraction would be premature. **The right amount of abstraction is the smallest amount that keeps you from copy-pasting.**

Tiny components that exist for readability, not reuse

`<PathStep>` and `<PathArrow>` are a different breed.

```
<PathStep label="R1" status={r1Won ? 'won' : 'lost'} />
<PathArrow />
<PathStep label="R2" status={r2Won ? 'won' : 'lost'} />
<PathArrow />
<PathStep label="QF" status={qfFinished ? (qfWon ? 'won' : 'lost') : 'live'} />
<PathArrow />
<PathStep label="SF" status={...} />
<PathArrow />
<PathStep label="F" status={...} />
```

`<PathArrow>` is literally one character: `→`. Why is it a component? **Because the JSX above reads like a sentence.** If you inlined the arrow's styling, you'd see five identical `<span style={{ color: ..., fontSize: 18 }}>→</span>` blocks interrupting the flow. With the component, the markup tells you the structure at a glance: step, arrow, step, arrow, step. That's the "tournament path" widget, and the JSX makes that obvious. Extracting `<PathArrow>` cost nothing and bought one of the easier-to-read snippets in

the file.

`<PathStep>` is the same idea at slightly larger scale: the four visual states (won / lost / live / na) are mapped from a status string to a small style object inside the component. That logic could have been inlined, but then the parent would carry five copies of a `if (status === 'won') ... else if (status === 'lost') ...` block. By moving it inside `<PathStep>`, the parent declares **what it wants** (“step labelled R1, status ‘won’”) instead of **how to render it**. That’s the canonical reason to make a component: separating *what* from *how*.

Prop drilling: when it’s fine

teams gets passed from `<SnookerFantasyLeague>` to `<PlayersTab>` to `<PlayerDetail>` to `<MatchPickAnalysis>`. Three levels. That’s “prop drilling” and at some point newcomers ask “shouldn’t this be a Context or a Redux store?”

For an app with one shared array and three levels of nesting? **No**. Context exists to spare you from passing the **same** prop through five-plus levels of unrelated components. Three levels of clearly named props is **cheaper to reason about** than introducing a global. The day you have ten teams’ worth of state plus user auth plus filters plus a settings panel, you re-evaluate. Until then, `<Component teams={teams}>` is fine and the file will tell you.

Vibe prompt you would have used

“This `<MatchPickAnalysis>` component is getting long. Extract two pieces. First, the colored pill that shows a team’s icon, name, and points-earned chip – call it `<TeamChip>`, props { team, ptsEarned }. Second, the small status step in the tournament-path bar – call it `<PathStep>`, props { label, status } where status is 'won' | 'lost' | 'live' | 'na', each mapping to its own colors and icon (🟩 🟦 ● →). Also extract the literal → arrow into a one-line `<PathArrow>` component so the path JSX reads like ‘step arrow step arrow step’. Don’t change any visuals – just move the JSX into the new files and import them back.”

That last sentence (“don’t change any visuals, just move the JSX”) is the prompt that turns a refactor from a rewrite back into a refactor. Vibe coders have to add it explicitly; LLMs love to redesign while they’re moving things.

CHECK YOURSELF

1. `<PathArrow>` has zero props and renders one character. What's the argument *for* keeping it as its own component, even though it could trivially be inlined?
 2. Why is the **column wrapper** inside `<MatchPickAnalysis>` not extracted, even though it appears twice with similar structure?
 3. At what nesting depth would you reach for Context instead of prop drilling? Why isn't this app there yet?
-

Lesson 6 — Charts & Data Transforms (Recharts)

The trick most React-charting tutorials skip is the part that actually matters: **charts in React are about data shape, not chart libraries**. Recharts (or Victory, or any other) takes an array of plain objects with a known schema and draws bars/lines/cells from it. Your job, 90% of the time, is to *reshape your existing data into the schema the chart wants*. The other 10% is choosing axes and colors.

Look at the Analytics tab. Almost every chart in it begins with a `teams.map(...)` that produces a brand new array. That array is the chart's "data."

Reshape #1: stacked-bar totals

```
const r1Data = teams.map((t) => ({
  team:
    t.name.length > 12
      ? t.name
        .split(' ')
        .map((w) => w[0])
        .join('')
      : t.name,
  fullName: t.name,
  R1: t.scores.r1,
  R2: t.scores.r2,
  QF: t.scores.qf,
  SF: t.scores.sf,
```

```

    Final: t.scores.f,
    Total: t.scores.total,
  }));

```

Each row is one team. The team field is the X-axis label (abbreviated to initials if the name is too long). Then four numeric fields — R1, R2, QF, SF, Final — that become the **stacks**. Recharts' `<Bar dataKey="R1" stackId="a" />` reads the R1 field of every row; the `stackId="a"` tells it to stack on top of the previous bar with the same `stackId`. Same `stackId` across multiple `<Bar>` elements = a single stacked bar per row:

```

<Bar dataKey="R1"      stackId="a" fill="#0F5132" name="Round 1" />
<Bar dataKey="R2"      stackId="a" fill="#DC2626" name="Round 2" />
<Bar dataKey="QF"      stackId="a" fill="#FBBF24" name="Quarter-Finals" />
<Bar dataKey="SF"      stackId="a" fill="#7C3AED" name="Semi-Finals" />
<Bar dataKey="Final"   stackId="a" fill="#9F1239" name="Final" radius={[4, 4, 0, 0]} />

```

If you change `stackId` to a different value (or remove it), the same data renders as **grouped** bars (R1, R2, QF side-by-side per team) instead of stacked. **Same data, different stackId, different chart.** That's the pattern: choose the shape your data fits naturally, then let small props turn it into different visualizations.

Reshape #2: per-bar colors with `<Cell>`

The “Pick Distribution” chart for the Final has a special twist: each bar should be **green if the player picked is Wu Yize, gray otherwise**. Recharts lets you do that by mapping `<Cell>` children:

```

<Bar dataKey="count" radius={[6, 6, 0, 0]} ...>
  {finalPickData.map((d, i) => (
    <Cell key={i} fill={d.name === 'Wu Yize' ? '#16A34A' : '#9CA3AF'} />
  ))}
</Bar>

```

A `<Cell>` is a per-row override. The `<Bar>` defines the default behavior; the `<Cell>` for row `i` overrides the fill. There's one `<Cell>` per data row, in the same order. Without `<Cell>`, every bar is the same color. With `<Cell>`, every bar is whatever you want.

That conditional `fill={d.name === 'Wu Yize' ? '#16A34A' : '#9CA3AF'}` is the entire mechanism by which the chart highlights “Wu Yize was right.” Same pattern

works for any “highlight one thing” chart.

Reshape #3: pivot from “team picks” to “match agreement”

The most interesting chart on the page is `<LeagueAgreementChart>`. The question it answers is: “For each match in this round, of the 8 teams, how many picked the actual winner?” That’s a **pivot** — the source data is per-team-per-round, but the chart needs per-match-per-round.

```
const buildAgreementData = (matches, pickKey) => matches.map((m, i) => {
  // ...
  if (m.winner) {
    const correct = teams.filter(t => {
      const pick = pickKey === 'final' ? t.final : (t[pickKey] ? t[pickKey][i] : null);
      return pick === m.winner;
    }).length;
    return { match: matchLabel, correct, wrong: 8 - correct, pending: 0, finished: true };
  } else {
    // pre-tournament: split picks between p1 and p2
    const p1Backers = teams.filter(t => /* same dance */).length;
    const p2Backers = teams.filter(t => /* same dance */).length;
    return { match: matchLabel, p1Backers, p2Backers, pending: 8, finished: false };
  }
});
```

Two important things:

- **Same function returns two different shapes** depending on whether the match is finished. If finished: { match, correct, wrong, finished: true }. If pending: { match, p1Backers, p2Backers, finished: false }. The chart conditionally renders different `<Bar>` series based on `isPendingRound = round.data.every(d => !d.finished)`.
- **The pivot is index-based.** `t[pickKey][i]` reaches into team `t`’s pick array for round `pickKey` at index `i`. That works because of Lesson 1’s invariant — the `i`-th pick is for the `i`-th match in that round. Without that invariant this whole function would need a `find()` against `pick.matchId`, and it would have been twice as long.

Internal tab state inside a chart

`<LeagueAgreementChart>` is the only chart that has its own `useState`:

```
function LeagueAgreementChart({ rounds }) {  
  const [active, setActive] = useState('r1');  
  const round = rounds.find(r => r.id === active) || rounds[0];  
  // ...  
}
```

Why is its tab state local rather than lifted to `<SnookerFantasyLeague>`? Because **nothing outside this chart cares which round is active inside it**. It's a UI-internal toggle, not a piece of cross-cutting application state. The rule from Lesson 4 still holds: state lives at the lowest common ancestor of its readers and writers. Here both are inside `<LeagueAgreementChart>` itself, so the state is local. **Tab state isn't always app state**. Knowing the difference is half the React mental model.

Vibe prompt you would have used

"Add an analytics tab. Three charts at the top: (1) a stacked bar chart of points per team across R1/R2/QF/SF/Final using Recharts, X-axis is team name, stack each round in a different color. (2) A horizontal bar chart of pick accuracy %, one bar per round per team. (3) A bar chart of how many of 8 teams picked each Finalist; highlight the bar for the actual champion in green using `<Cell>`. Below those, a single 'League Agreement' chart with its own internal round-toggle (R1, R2, QF, SF, Final) – for each match in that round, show two stacked bars: green for teams who picked correctly, red for teams who picked wrong. Pull the team data into a fresh array shaped exactly how Recharts expects (`teams.map(...)` returning `{ team, R1, R2, ... }`); don't try to pass the team objects directly. Three insight cards beneath summarising leader, best round, champion."

CHECK YOURSELF

1. If you remove the `stackId="a"` prop from every `<Bar>` in the points-breakdown chart, what changes visually?
2. Why does `<LeagueAgreementChart>` keep its own `useState` instead of putting `active` in the top-level `<SnookerFantasyLeague>`?
3. The agreement-data builder has two return shapes (finished vs pending). What does

the chart code use to decide which `<Bar>` series to render — and what would happen if a single round had a mix of finished and unfinished matches?

Lesson 7 — The Evolution (what the git log would have shown)

Vibe-coded apps grow in layers, not waves. The current 1,654-line file didn't appear in one prompt. It accreted over (probably) a few dozen sessions across the two-week tournament. Here's what the git log would have looked like, reconstructed from the artifacts in the file. For each milestone, the **vibe prompt** that would have triggered the change.

Milestone 1 — “Just give me the standings”

Pre-tournament. R1 hasn't started. You have the bracket. You have eight friends' picks. You want a single screen that shows team totals.

“Give me one React component called `<StandingsTab>`. Render a table of 8 teams sorted by total points. Hard-code the `TEAMS` array and a `ROUND1_MATCHES` array (16 entries) above the component. Total points are 3 per correct R1 pick, 1 per wrong. Just R1 for now — the other rounds don't exist yet. White background, big readable numbers, no styling beyond a green header.”

The codebase at this point is **maybe 200 lines**. One file, one component, two constants, one `scorePick` function, one inline `.reduce()` for totals. There is no Matches tab, no Predictions tab, no `useState`, no `nav`. **This is the version you ship to your eight friends so they stop asking you “who's winning.”**

Milestone 2 — “I want to see the matches themselves”

Round 1 is half-finished. People want to know which matches are done and what the scores were.

“Add a second screen called Matches. It shows match cards, one per match, in a grid: player names, the score ('10-7' etc.), and a 'FINISHED' badge if the match has a winner. If there's no winner yet, show 'NOT FINISHED' with a dashed gray border. Make a top-of-page tab bar with two buttons: Standings and Matches. Use `useState` to switch between them.”

This is the prompt that introduces `useState` to the codebase. It also introduces the *concept of two screens that share data*, which is the single biggest jump in React complexity for a vibe coder. After this milestone the file is around **400 lines** and has its first orchestrator pattern.

Milestone 3 – “I want to compare predictions side by side”

End of Round 1. The standings tab tells you the *total*, but doesn’t show *whose pick was wrong*. You add a third screen.

“Add a third tab called Predictions. Two views: a comparison matrix showing every match as a row and every team as a column, with the team’s pick in each cell, green if correct and red if wrong. And a single-team card view, where clicking a team in the standings drops you here with that team selected. Round buttons inside the predictions tab to filter R1/R2/QF/SF.”

This is the prompt that introduces the *cross-tab interaction* pattern from Lesson 4 (clicking a team row in Standings switches to Predictions). It also introduces nested view state (`view: 'matrix' | 'team'` lives inside `<PredictionsTab>`, not at the top).

Milestone 4 – “Round 2 happened, the bracket shrunk”

Round 1 is done. Round 2 results trickle in. You add `ROUND2_MATCHES` and a `r2` array on every team. The whole `calculateTeamScores` function gets duplicated for R2:

“Round 2 is happening. Add `ROUND2_MATCHES` (8 entries) above `ROUND1_MATCHES`. Add `r2: [ 'Zhao Xintong', 'Shaun Murphy', ... ]` (8 picks) to every team. Update `calculateTeamScores` to also walk `r2` and return `{ r1, r2, total: r1 + r2, r1Details, r2Details }`. Add R2 columns to the standings table next to R1, and a Round 2 button to the Matches tab and the Predictions matrix.”

This is the prompt that creates the **duplication-by-round** pattern that runs through the whole file: every round needs a constant, a pick array, a totals field, a details field, a tab button, a matrix column. By the time you get to the Final, that pattern has fired five times. A more-architected codebase would parameterize it (`ROUNDS = [ 'r1', 'r2', 'qf', 'sf', 'final' ]` with a single loop). The vibe-coded version copies the pattern five times. Both are fine for an 8-team league.

Milestone 5 – “Show me a player’s tournament journey”

Mid-tournament, QF in flight. The Standings + Matches + Predictions trio is the league-level view. But you want to be able to ask, “How is Wu Yize doing? Which teams backed him? Did they get points?” That’s the Players tab.

“Add a Players tab. Grid of 32 cards, one per player from `PLAYER_INFO`. Filter buttons at the top: Still In, Eliminated, All. Click a card to open a deep player detail view: header with the player’s name + flag + status, a 5-step tournament path (`R1` → `R2` → `QF` → `SF` → `F`) with each step colored by won/lost/live/na, a stats row, and a per-match analysis showing which 8 teams backed this player vs the opponent in each round. The opponent’s name should be clickable so you can hop to their detail view too.”

This is the most ambitious single prompt in the project. It introduces **navigation by name** (selecting a player by string instead of by index), the **stillStanding set computed from QF + SF + Final results**, and the **PathStep + PathArrow** components from Lesson 5. After this milestone the file is ~1,200 lines.

Milestone 6 – “I want pretty charts”

Late tournament. With most matches in, you want trend visuals — points by round, accuracy by round, agreement on each match.

“Add an Analytics tab using Recharts. Three charts up top in a grid: stacked bars for total points (`R1+R2+QF+SF+Final`), horizontal bars for accuracy %, vertical bars for the Finalist pick distribution with the actual champion’s bar in green. Below them a ‘League Agreement’ chart with its own round-toggle showing per-match correct-vs-wrong stacks. Three colored insight cards at the bottom summarising leader, best R1, champion.”

This is where the file picks up Recharts as a dependency. The data-reshape pattern from Lesson 6 starts here.

Milestone 7 – “Wu Yize won. Crown him.”

Tournament ends. The Final result is in: Wu Yize 18, Murphy 17. The codebase needs to acknowledge that the champion is decided and the league is done.

“Wu Yize won the Final 18-17. Update `FINAL_MATCH` to set winner: ‘Wu Yize’. In

the hero header, replace the ‘tournament in progress’ badge with ‘🏆 WU YIZE – WORLD CHAMPION 2026 (18-17)’. In the standings, the Final Pick chip should be red for teams who picked Wu Yize and blue for teams who didn’t. In the Players tab, the player card for Wu Yize gets a 🏆 WORLD CHAMPION badge; Murphy gets 🥈 RUNNER-UP. The other Crucible-eliminated players say OUT. Tournament-complete language everywhere it says ‘tournament’ so I can put it in past tense.”

That last sentence is what lands the hard-coded 'Wu Yize' string we noted in Lesson 3. It's the trade-off: the prompt was easier to write because it referenced the actual champion by name; the cost was that the file is now tournament-specific. A purely time-symmetric version of this prompt would say *"compare against FINAL_MATCH[0].winner"* — identical UI, future-proof code. **A vibe prompt that uses a literal value is faster to write and produces faster code; the cost is portability.**

Putting the timeline together

If you read the file with this timeline in mind, you can almost see the seams between sessions. The five `*Details` arrays returned by `calculateTeamScores` each got added in their own session. The `<RoundButton>` and `<PicksList>` components were added at slightly different times — the round button has a status prop with literal English (`'FINISHED'`, `'🏆 WU WINS 18-17'`), which is unmistakably end-of-tournament wording, while `<PicksList>` is generic (it would render the same on day 1). **The newer code is more specific to “the tournament is over;” the older code is more generic to “any tournament.”**

Vibe prompt you would have used (the meta-pattern)

There isn't a single prompt for the whole evolution — that's the point. There's one prompt per milestone, and **each prompt is shaped like an addition, not a redesign:**

“Add X to Y showing Z.”

Read back over Milestones 1–7. Every one starts with “Add” or “Update” — never “Refactor everything” or “Make it nicer.” That's the discipline that keeps the file growing instead of churning.

CHECK YOURSELF

1. The Final Pick chip's red/blue coloring is a Milestone 7 artifact. What earlier file evidence (in any other component) tells you the codebase wasn't always tournament-end-aware?
 2. Why is each round's logic copy-pasted (one block for r1, one for r2, ...) instead of looped over ['r1', 'r2', 'qf', 'sf', 'final']? What would looping cost in readability for a vibe coder reading this file?
 3. The Players tab is by far the biggest single milestone in the timeline. Why is that a smell *only* if you have to ship by Friday — and why is it fine for this project?
-

Lesson 8 — The Shape of Your Prompts (meta-lesson on vibe coding)

You've now seen eight lessons of "the prompt I would have used." The whole point of those was to give you a private library of prompt templates that **demonstrably produced this codebase**. Here's the pattern, distilled.

The seven shapes of a good vibe prompt

After two weeks of building this app, your prompt library should look like this:

1. **The "Add a feature" prompt.** *"Add a [component / column / tab] to [location] that [does Z when X]." Single new thing, single insertion point, single behavior. Example: "Add a Form column at the right of the standings table that compares a team's R1 percentage to their R2 percentage and shows Rising / Falling / Steady with an arrow icon."*
2. **The "Tweak the visual" prompt.** *"In [component], change [X] when [Y]." Surgical visual edits. No structural changes. Example: "In the Standings table, make the rank circle gold for 1st, silver for 2nd, bronze for 3rd; replace 1st place's number with a 🥇 emoji."*
3. **The "Extract a sub-component" prompt.** *"This <Foo> is getting long. Extract [X piece] into a <Bar> component with props { a, b }. Don't change any visuals — just move the JSX." The "don't change any visuals" clause is mandatory. Without it the LLM redesigns. Example: see Lesson 5's prompt about <TeamChip>, <Path-Step>, <PathArrow>.*

4. **The “Wire two screens together” prompt.** *“When [component A] does X, set [parent state Y] and switch to [component B].”* Names the cross-tab interaction explicitly. The parent component is the one that has to be edited, even though the *description* is about the children. Example: *“When a row in <StandingsTab> is clicked, set `selectedTeam` and switch the active tab to Predictions.”*
5. **The “Reshape data for a chart” prompt.** *“Build a chart of X using Recharts. The data is `[teams.map(t => ({ ... }))]`. Bars / lines / cells map to fields `[a, b, c]`; color the bar for [special row] differently using `<CeLL>`.”* You spell out the data shape inside the prompt; that becomes the chart spec. Example: see Lesson 6’s analytics prompt.
6. **The “Add a column to an existing array of objects” prompt.** *“Add a `[fieldName]` field to every entry in `[constant]` and update `[scorePick / calculateTeamScores / something]` to read it.”* Common when the data model grows. The literal phrasing names the constant + the field name + the consumer that needs updating, all in one sentence.
7. **The “End-of-feature polish” prompt.** *“In `[view]`, when `[end-state condition is true]`, change `[strings / colors / badges]` to past tense / champion / closed-out wording.”* This is the Milestone 7 prompt shape. It bakes the final state into the UI.

Three bad prompts and why they’re bad

You’re going to write some of these. Recognise them:

- **“Make the standings better.”** Too vague. The LLM has to invent what “better” means. You’ll get bigger fonts, drop shadows, animation, maybe a redesigned card layout, and likely something visually nicer that subtly broke the data flow. If you can’t name *what* about the standings should change and *why*, don’t prompt yet — go look at it for another minute and decide.
- **“Use Redux for state.”** Premature abstraction. This app has **two** pieces of state at the top level. Redux’s overhead (actions, reducers, dispatch wiring, dev tools setup) costs more than it buys until your state is contended by a dozen-plus components. If a prompt names a *technology* before naming a *problem*, it’s almost always going to wreck the file. Useful version: *“teams is being prop-drilled into 3 levels and I want to keep it accessible without passing it through <MatchesTab> which doesn’t use it. What’s the smallest change?”* (The honest answer: don’t pass it through <Match-

esTab>. You're already there.)

- **“Refactor the whole codebase to use Tailwind classes instead of inline styles.”** A redesign disguised as a refactor. The visual richness in this app depends on dynamically computed styles (gradients with team-specific accent colors, conditional backgrounds, dynamic shadows). Tailwind can do most of that, but in JIT-compiled arbitrary-value mode that's noisier than the inline style is, and the conversion has to be done file-by-file with visual diffing. Useful version: *“In <StatCard>, replace the inline style on the outer div with Tailwind classes. Keep the dynamic gradient (bg-CoLoR) as inline because it's a runtime prop. Don't touch any other component.”* One file, one prompt, one verifiable diff.

The prompt-tree of this app

If you imagine the file as a tree of prompts, the trunk is Milestone 1's *“Give me the standings”* prompt. Every subsequent prompt is a branch off an existing branch — never the trunk. **You never re-prompt the standings table from scratch.** You always say *“in the standings, change X.”* That discipline is what keeps a vibe-coded app from becoming a churning rewrite machine.

The shape, again:

One feature. One file (or a small set of named files). One sentence per change. Avoid technology names. Avoid superlatives. Mention specific data fields and component names. Refer to behavior, not “polish.”

Final word

The thing this course was reverse-engineering wasn't just a 1,654-line file. It was the **prompting strategy** that grew that file without rewrites. If you internalize Lesson 4's “state up, callbacks down,” Lesson 5's “extract when you copy-paste twice,” Lesson 6's “data shape, not chart library,” and Lesson 8's “Add X to Y showing Z,” you can build the next app — pickleball ladder, World Cup pool, whatever — in maybe a third of the prompts.

That's the course. Now open `course/SNOOKER_FANTASY_LEAGUE_COURSE.md` next to `_source/SnookerFantasyLeague.jsx` and `components/SnookerFantasyLeague.tsx`, and read the three side-by-side. The `.jsx` is what you built. The course is how you would have built it. The `.tsx` is where it lives now.

CHECK YOURSELF

1. Look back at any vibe prompt you've written this month. Which of the seven shapes does it match? If none, what's missing — a target component name? a behavior? a data field?
2. Why is *"Use Redux for state"* almost always the wrong prompt for an app of this size, and what is the **right** prompt to write when you feel that itch?
3. The seven prompt shapes in this lesson all share one structural feature. What is it? (Hint: count how many of them name a specific component, file, or constant by name.)

21-Day Study Plan — React + Snooker Fantasy League

A day-by-day schedule to finish all 7 chapters and have the deployed app on your portfolio, in three weeks at ~60 min/day.

Use the in-app progress tracker at `/study` to check off lessons as you go (it persists in your browser via `localStorage`).

Week 1 — Foundations (Days 1-7)

Goal by Sunday: data model in your head, scoring engine memorized, project scaffolded, first component rendered.

Day 1 (60 min) — Orientation

- Read `course/README.md` end to end.
- Skim the **table of contents** of `course/SNOOKER_FANTASY_LEAGUE_COURSE.md` to get the full arc.
- Open the live deployed app (link in `README.md`) and click around for 10 minutes.
- *Output*: a one-paragraph note in `progress.txt` saying what you expect each chapter to teach.

Day 2 (60 min) — Chapter 1.1

- Read `chapter-01-foundations/01-the-problem.md`.

- Review the 5 corresponding flashcards (flashcards/react-snooker.csv, filter by 01-the-problem).
- *Output*: explain the user / data / scoring rule trio out loud, untimed.

Day 3 (60 min) — Chapter 1.2

- Read chapter-01-foundations/02-data-modeling.md.
- Sketch the Player, Match, Pick types on paper before reading the answer.
- Review the cheat sheet cheat-sheets/react-ch01.md.
- *Output*: list the three invariants you'd enforce at the type system level.

Day 4 (75 min) — Chapter 1.3

- Read chapter-01-foundations/03-pure-functions-and-scoring.md.
- Open lib/scoring.ts in this repo and read it alongside.
- Write down the test cases you would add for the scoring engine.
- *Output*: list 3 properties you would prove with property-based tests.

Day 5 (60 min) — Chapter 2

- Read all three lessons of chapter-02-project-setup/.
- Run `npm install` and `npm run dev`; verify the app boots locally.
- *Output*: a screenshot of the local app in your notes.

Day 6 (60 min) — Chapter 3.1-3.2

- Read chapter-03-first-component/01-anatomy-of-a-component.md and 02-rendering-the-standings.md.
- Open components/StandingsTable.tsx and lib/scoring.ts; trace one render top-to-bottom.
- *Output*: list the props the table component receives and where each comes from.

Day 7 (60 min) — Chapter 3.3 + review

- Read chapter-03-first-component/03-progressive-enhancement.md.
- Review week 1 cheat sheets back to back.
- 15-minute flashcard session: cards tagged chapter-01 and chapter-02.
- *Output*: write a 1-paragraph weekly retro.

Week 2 — Composition & Hooks (Days 8-14)

Goal by Sunday: confident with hooks, can talk about the orchestrator pattern, understand prop drilling vs context.

Day 8 (60 min) — Chapter 4.1

- Read `chapter-04-state-and-hooks/01-useState-mental-model.md`.
- Write 3 wrong-and-right code snippets for `useState`.
- *Output*: explain *why* state updates are queued, in your own words.

Day 9 (60 min) — Chapter 4.2

- Read `02-the-orchestrator-pattern.md`.
- Diagram the data flow: which component is the orchestrator, which are leaves?
- *Output*: list one alternative to the orchestrator pattern and its trade-off.

Day 10 (60 min) — Chapter 4.3

- Read `03-useMemo-and-derived-data.md`.
- Open `components/SnookerFantasyLeague.tsx` and find every `useMemo`; explain *why* each one exists.
- *Output*: identify one `useMemo` you would remove (premature optimization is real).

Day 11 (60 min) — Chapter 5.1

- Read `chapter-05-composition/01-when-to-split-components.md`.
- Pick one big component in this repo; sketch how you would split it.
- *Output*: list the four “smells” that suggest a component should be split.

Day 12 (60 min) — Chapter 5.2

- Read `02-prop-drilling-vs-context.md`.
- Find one place in the codebase where prop drilling is fine and one where context would be justified.
- *Output*: write 2-sentence reasoning for each.

Day 13 (60 min) – Chapter 5.3

- Read `03-building-player-detail.md`.
- Open `components/PlayerDetailPanel.tsx`; trace where its state lives and how it's controlled.
- *Output*: name one prop you would lift up and one you would push down.

Day 14 (60 min) – Week 2 review

- Re-read the cheat sheets for chapters 4 and 5.
 - 30-minute flashcard session, tag filter composition and hooks.
 - *Output*: weekly retro.
-

Week 3 – Visualization, Polish, Deployment (Days 15-21)

Goal by Sunday: app deployed, README is portfolio-quality, you can give a 5-minute architecture walkthrough.

Day 15 (60 min) – Chapter 6.1

- Read `chapter-06-data-visualization/01-recharts-fundamentals.md`.
- Open `components/ScoreHistoryChart.tsx`; tweak one prop and reload.
- *Output*: name the three things Recharts gives you for free.

Day 16 (75 min) – Chapter 6.2

- Read `02-reshaping-data.md`.
- Manually trace one match through the data array transformations.
- *Output*: write a test for one transformation.

Day 17 (75 min) – Chapter 6.3

- Read `03-pivots-and-internal-state.md`.
- Open `components/ResidualHistogram.tsx`; identify the pivot operation.
- *Output*: describe what the chart shows in one sentence to a non-engineer.

Day 18 (60 min) — Chapter 7.1

- Read `chapter-07-mastery-and-interviews/01-production-polish.md`.
- Apply one polish item from the list to your local copy.
- *Output*: a one-line PR title for the change.

Day 19 (75 min) — Chapter 7.2

- Read `02-deployment-and-testing.md`.
- Deploy your fork to Vercel (free).
- *Output*: shareable deployed URL in `progress.txt`.

Day 20 (60 min) — Chapter 7.3

- Read `03-interview-narrative.md`.
- Draft your 90-second project pitch using the template.
- *Output*: a recording of yourself delivering it (private OK).

Day 21 (90 min) — Capstone

- Walk through the full app end-to-end and explain it out loud as if presenting to a panel.
 - Run through *all* the cheat sheets at speed.
 - 45-minute flashcard session, full deck.
 - *Output*: portfolio-ready README update + LinkedIn-friendly summary.
-

Daily habits (every day, ~10 min on top)

- 10 minutes flashcard review (Anki / Mochi / RemNote — load flashcards/`react-snooker.csv`).
- Listen to that day's audio narration MP3 (`audio/react-snooker-course/...`) while walking or cooking.

What to skip if you're short on time

- Days 18-20 polish/deployment if you only care about the technical content. The deployed app is the credibility item, but the technical depth is in chapters 1-6.

What to add if you have more time

- Re-implement `lib/scoring.ts` from scratch without looking.
- Re-implement `components/StandingsTable.tsx` from scratch.
- Write Playwright tests for the deployed app.

When you finish

Move directly to the AI interview prep course ([ai-interview-course/](#)), which uses *this* project as its portfolio example throughout.